BoviCierre

Manual integral del software

Arquitectura, código, datos y operación local

Una guía para comprender el negocio, seguir las peticiones
y mantener el sistema desde sus fuentes reales

Preparado para Aldo y Jared

Revisión del repositorio 92d3d41
12 de octubre de 2026

PostgreSQL · Java · React · Docker Compose

Índice de contenido

1. Cómo leer y entender BoviCierre **1**
- 1.1. Propósito y revisión de este manual . . . 1
- 1.2. Ruta recomendada de lectura . . . 1
- 1.3. Qué problema resuelve . . . 2
- 1.4. Conceptos que no debes confundir . . . 2
- 1.5. Una historia completa antes de ver el código . . . 2

2. Mapa general de arquitectura y comunicación **4**
- 2.1. Las capas y sus responsabilidades . . . 4
- 2.2. Diseño inicial e implementación real . . . 4
- 2.3. Quién escribe y quién sólo consulta . . . 5
- 2.4. Qué pasa al pulsar confirmar . . . 5

3. Guía completa para ejecutar en local **7**
- 3.1. Requisitos y versiones . . . 7
- 3.2. Comprobar el checkout y el entorno . . . 7
- 3.3. Arranque académico integrado . . . 7
- 3.4. Puertos y direcciones . . . 8
- 3.5. Comprobar servicios sin modificar datos . . . 8
- 3.6. Detener y volver a iniciar . . . 8
- 3.7. Desarrollo con Java y Vite en primer plano . . . 9
- 3.8. Variables de configuración que debes conocer . . . 9
- 3.9. Verificación y su alcance . . . 10
- 3.10. Respaldo y recuperación aislada . . . 10

4. Infraestructura explicada desde sus archivos **12**
- 4.1. Docker Compose y separación de servicios . . . 12
- 4.2. Pools y presupuesto de conexiones . . . 12
- 4.3. Proxy, ETag y origen común . . . 12

5. Pruebas y evidencia que explican la confianza disponible **14**
- 5.1. Qué significa una prueba y qué no . . . 14
- 5.2. Oráculos económicos y concurrencia . . . 14
- 5.3. Rendimiento medido . . . 15
- 5.4. Cómo reproducir una medida sin alterar la demo . . . 15

6. Mantenimiento y diagnóstico con criterio **16**
- 6.1. Si no abre la interfaz . . . 16
- 6.2. Si un comando devuelve un conflicto . . . 16
- 6.3. Si la réplica no converge . . . 16
- 6.4. Si falta un documento . . . 16
- 6.5. Cómo extender el sistema sin romper sus fronteras . . . 17

7. Herramientas de desarrollo, trazabilidad y responsabilidades **18**

8. Preparación comercial y límites del alcance **19**

9. Cómo interpretar variables, cifras y fórmulas **20**
- 9.1. Tres clases de variables . . . 20

9.2. Notación y unidades . . . 20
9.3. Precio y reconocimiento . . . 21
9.4. Saldo, disponible y margen . . . 21
9.5. Precisión decimal y redondeo . . . 21
9.6. Distribución y conservación del total . . . 22
9.7. Tiempo y resultado histórico . . . 22

10. Diagramas de hechos y estados 23
10.1. Identidad y ciclos . . . 23
10.2. Venta, pago y entrega son ejes separados . . . 23
10.3. Mensaje de muerte entre dueños . . . 23

11. La base de datos de BoviCierre: modelo, comandos y reconocimiento 25
11.1. 1. Propósito y forma de estudiar el sistema . . . 25
11.2. 2. Qué DDL manda y por qué existen dos recuentos . . . 25
11.3. 3. Leer PK, FK, NN y UK como mecanismos distintos . . . 27
11.4. 4. Mapa de las42 tablas lógicas y las cuatro técnicas adicionales . . . 28
11.5. 5. El protocolo de un comando SQL . . . 32
11.6. 6. Aritmética decimal, unidades y materialización . . . 33
11.7. 7. Recorridos reales de comandos . . . 37
11.8. 8. Dinero aplicado, reversos y correcciones sin borrar originales . . . 39
11.9. 9. Gastos, costo tardío, baja y anulación . . . 40
11.10. 10. Periodos: elegir reconocimiento y conservar historia . . . 41
11.11. 11. Privilegios, SECURITY DEFINER y frontera de confianza . . . 42
11.12. 12. Propietarios fijos, replicación y fallos . . . 43
11.13. 13. Muerte B→A: outbox, inbox y lease durable . . . 44
11.14. 14. FDW sin transacción financiera remota . . . 44
11.15. 15. Consultas canónicas: qué responde cada pregunta . . . 45
11.16. 16. Ejemplos seguros para estudiar sin modificar hechos . . . 46
11.17. 17. Bootstrap, despliegue local y evidencia de rendimiento . . . 47
11.18. 18. Glosario SQL orientado a esta implementación . . . 47
11.19. 19. Qué quedó comprobado y qué no debe prometerse . . . 48
11.20. 20. Cuaderno de oráculos fijos para repasar . . . 49

12. Backend, comunicación y seguridad: cómo funciona el programa Java real 51
12.1. Alcance de este capítulo y forma de leerlo . . . 51
12.2. Qué está implementado y qué corresponde al diseño . . . 51
12.3. Dependencias, configuración y entrada del proceso . . . 52
12.4. Comunicación HTTP y forma de los datos . . . 54
12.5. Autenticación y autorización explicadas clase por clase . . . 55
12.6. JsonSupport: frontera de datos, ETag y huella de intención . . . 58
12.7. CommandService: enrutamiento, transacción local y observabilidad . . . 59
12.8. CoreController: un adaptador HTTP compartido . . . 61
12.9. ApiException y ApiErrors: del fallo interno al error público . . . 63
12.10. Archivos privados: puerto, adaptador, controlador y limpieza . . . 65
12.11. Integraciones: tarifas, outbox y estado de operación . . . 67
12.12. CSV, QR y PDF: producir representaciones sin reconocer otra operación . . . 68
12.13. Tres secuencias completas para reconstruir la comunicación . . . 70
12.14. Inventario del contrato HTTP y sus llamadores Java . . . 72
12.15. Cobertura, evidencia y límites de este capítulo . . . 73
12.16. Fuentes primarias y localización en el repositorio . . . 74

13. Frontend de BoviCierre: manual didáctico del software implementado 76
13.1. 1. Qué hace este capítulo y cómo leerlo . . . 76
13.2. 2. Mapa de archivos, dependencias y ejecución . . . 77
13.3. 3. HTML inicial y montaje de React . . . 78
13.4. 4. Tipos del dominio y representación exacta . . . 78
13.5. 5. Dos auxiliares que preservan la identidad comercial . . . 80
13.6. 6. La frontera HTTP: solicitar, reconocer errores y recuperar comandos . . . 80
13.7. 7. La biblioteca pequeña de interfaz compartida . . . 83
13.8. 8. App: acceso, sede y navegación . . . 85

13.9. 9. Operaciones, lotes, pesajes y acuerdos . . . 87
13.10. 10. Liquidaciones, dinero y correcciones económicas . . . 88
13.11. 11. Gastos, tarifas aceptadas y periodos . . . 89
13.12. 12. Archivos privados y usuarios sensibles . . . 90
13.13. 13. Expediente, movimientos, reportes y simulador . . . 91
13.14. 14. Recorrido completo con cuerpos de solicitud didácticos . . . 92
13.15. 15. Estados, borrador y resultado incierto: cómo pensar una falla . . . 95
13.16. 16. CSS, accesibilidad y adaptación de tamaños . . . 96
13.17. 17. Pruebas: qué verifican y qué límites conservan . . . 97
13.18. 18. Guía de estudio y lectura del código sin alterar datos . . . 98

14. Atlas de scripts Python y sus variables 100
14.1. scripts/doctor.py . . . 100
14.2. scripts/academic.py . . . 101
14.3. scripts/dev.py . . . 101
14.4. scripts/recovery.py . . . 101
14.5. scripts/sync-task-tables.py . . . 103
14.6. tools/esquema/comparar_catalogo.py . . . 103
14.7. tools/dev/check_api_contract.py . . . 104
14.8. db/bootstrap/bootstrap.py . . . 104
14.9. db/seed/demo.py . . . 105
14.10. db/tests/benchmark.py . . . 105

15. Atlas de funciones SQL y declaraciones locales 108
15.1. bovi.archivo_referenciado . . . 108
15.2. bovi.capturar_tarifas_proveedor . . . 108
15.3. bovi.cmd_ejecutar . . . 109
15.4. bovi.confirmar_entrega . . . 110
15.5. bovi.consultar . . . 110
15.6. bovi.consultar_tarifas_proveedor . . . 110
15.7. bovi.consumir_muerte . . . 110
15.8. bovi.despachar_outbox . . . 111
15.9. bovi.fallar_entrega . . . 111
15.10. bovi.fn_calcular_liquidacion . . . 111
15.11. bovi.fn_codigo . . . 111
15.12. bovi.fn_json_decimal . . . 112
15.13. bovi.fn_liquidacion_json . . . 112
15.14. bovi.fn_nodo . . . 112
15.15. bovi.fn_normalizar_payload . . . 112
15.16. bovi.fn_periodo . . . 113
15.17. bovi.fn_periodo_payload . . . 113
15.18. bovi.fn_pesaje_json . . . 113
15.19. bovi.fn_repartir . . . 113
15.20. bovi.fn_reparto . . . 114
15.21. bovi.fn_reporte . . . 114
15.22. bovi.fn_scope . . . 115
15.23. bovi.fn_validar_conciliacion_propietario . . . 115
15.24. bovi.leer_tarifas_proveedor . . . 115
15.25. bovi.reclamar_entregas . . . 116
15.26. bovi.trg_acuerdo_completo . . . 116
15.27. bovi.trg_aprobado . . . 116
15.28. bovi.trg_borrador . . . 116
15.29. bovi.trg_inmutable . . . 117
15.30. bovi.trg_pesaje_composicion . . . 117
15.31. bovi.trg_propietario . . . 117
15.32. bovi.trg_snapshot_tarifa . . . 117

16. Referencia completa del contrato HTTP 118
16.1. POST /sesion/login . . . 118
16.2. POST /sesion/logout . . . 118
16.3. GET /sesion/actual . . . 119

16.4. GET /sedes/{sedeId}/contrapartes . . . 119
16.5. POST /sedes/{sedeId}/contrapartes . . . 120
16.6. POST /sedes/{sedeId}/animales . . . 120
16.7. GET /sedes/{sedeId}/animales . . . 121
16.8. GET /sedes/{sedeId}/animales/{animalId} . . . 121
16.9. GET /sedes/{sedeId}/operaciones . . . 122
16.10. POST /sedes/{sedeId}/operaciones . . . 123
16.11. GET /sedes/{sedeId}/operaciones/{operacionId} . . . 123
16.12. PATCH /sedes/{sedeId}/operaciones/{operacionId} . . . 124
16.13. POST /sedes/{sedeId}/operaciones/{operacionId}/confirmar . . . 124
16.14. POST /sedes/{sedeId}/operaciones/{operacionId}/pesajes . . . 125
16.15. POST /sedes/{sedeId}/pesajes/{pesajeId}/aprobar . . . 126
16.16. POST /sedes/{sedeId}/pesajes/{pesajeId}/correcciones . . . 126
16.17. POST /sedes/{sedeId}/operaciones/{operacionId}/reservas . . . 127
16.18. POST /sedes/{sedeId}/reservas/{reservaId}/liberar . . . 128
16.19. GET /sedes/{sedeId}/inventario/posiciones . . . 128
16.20. POST /sedes/{sedeId}/inventario/movimientos . . . 129
16.21. GET /sedes/{sedeId}/liquidaciones/{liquidacionId} . . . 129
16.22. POST /sedes/{sedeId}/liquidaciones/{liquidacionId}/ajustes . . . 130
16.23. GET /sedes/{sedeId}/liquidaciones/{liquidacionId}/comprobante . . . 130
16.24. POST /sedes/{sedeId}/gastos . . . 131
16.25. GET /sedes/{sedeId}/gastos . . . 132
16.26. GET /sedes/{sedeId}/gastos/{gastoId}/reparto . . . 132
16.27. POST /sedes/{sedeId}/gastos/{gastoId}/reparto . . . 133
16.28. POST /sedes/{sedeId}/pagos . . . 133
16.29. GET /sedes/{sedeId}/pagos . . . 134
16.30. POST /sedes/{sedeId}/pagos/{pagoId}/aplicaciones . . . 134
16.31. POST /sedes/{sedeId}/aplicaciones/{aplicacionId}/reverso . . . 135
16.32. POST /sedes/{sedeId}/pagos/{pagoId}/reembolsos . . . 136
16.33. GET /sedes/{sedeId}/reportes/cartera . . . 136
16.34. GET /sedes/{sedeId}/reportes/margen . . . 137
16.35. POST /sedes/{sedeId}/periodos/{periodoId}/cierre . . . 137
16.36. POST /sedes/{sedeId}/documentos . . . 138
16.37. GET /sedes/{sedeId}/documentos . . . 139
16.38. GET /sedes/{sedeId}/documentos/{documentoId}/contenido . . . 139
16.39. GET /sedes/{sedeId}/integraciones/proveedor/tarifas . . . 140
16.40. GET /sesion/csrf . . . 140
16.41. GET /sedes/{sedeId}/liquidaciones . . . 141
16.42. POST /sedes/{sedeId}/pagos/{pagoId}/validar . . . 141
16.43. GET /sedes/{sedeId}/comandos/{comando}/{clave} . . . 142
16.44. GET /sedes/{sedeId}/usuarios . . . 142
16.45. POST /sedes/{sedeId}/usuarios . . . 143
16.46. GET /sedes/{sedeId}/periodos . . . 144
16.47. POST /sedes/{sedeId}/periodos . . . 144
16.48. GET /sedes/{sedeId}/predios . . . 145
16.49. POST /sedes/{sedeId}/predios . . . 145
16.50. PATCH /sedes/{sedeId}/usuarios/{usuarioId} . . . 146
16.51. POST /sedes/{sedeId}/usuarios/{usuarioId}/membresias . . . 147
16.52. POST /sedes/{sedeId}/periodos/{periodoId}/reabrir . . . 147
16.53. GET /sedes/{sedeId}/periodos/{periodoId}/versiones . . . 148
16.54. POST /sedes/{sedeId}/operaciones/{operacionId}/acordar . . . 149
16.55. POST /sedes/{sedeId}/operaciones/{operacionId}/reabrir . . . 149
16.56. POST /sedes/{sedeId}/operaciones/{operacionId}/cancelar . . . 150
16.57. POST /sedes/{sedeId}/operaciones/{operacionId}/anular . . . 151
16.58. GET /sedes/{sedeId}/operaciones/{operacionId}/simulador . . . 152
16.59. GET /sedes/{sedeId}/operaciones/{operacionId}/expediente . . . 152
16.60. GET /sedes/{sedeId}/pagos/{pagoId} . . . 153
16.61. GET /sedes/{sedeId}/gastos/{gastoId} . . . 153
16.62. GET /sedes/{sedeId}/documentos/{documentoId} . . . 154
16.63. POST /sedes/{sedeId}/documentos/{documentoId}/revisiones . . . 154
16.64. GET /sedes/{sedeId}/pesajes/comparacion . . . 155

16.65. GET /sedes/{sedeId}/animales/{animalId}/qr . . . 155
16.66. GET /sedes/{sedeId}/reportes/{tipo}/csv . . . 156
16.67. GET /sedes/{sedeId}/sincronizacion . . . 156
16.68. POST /sedes/{sedeId}/integraciones/proveedor/tarifas/aceptaciones . . . 157
16.69. GET /sedes/{sedeId}/integraciones/diagnostico . . . 158
16.70. GET /salud . . . 158

17. Diccionario completo de DTO y tipos API **159**
17.1. Decimal2 . . . 159
17.2. Decimal3 . . . 159
17.3. Decimal4 . . . 159
17.4. Dinero . . . 159
17.5. Peso . . . 160
17.6. Error . . . 160
17.7. Pagina . . . 160
17.8. SesionActual . . . 160
17.9. ContraparteNueva . . . 160
17.10. Contraparte . . . 161
17.11. Animal . . . 161
17.12. FichaAnimal . . . 161
17.13. EstadoOperacion . . . 161
17.14. EstadoPosicion . . . 161
17.15. OperacionResumen . . . 162
17.16. Operacion . . . 162
17.17. Version . . . 162
17.18. EdicionBorrador . . . 162
17.19. PesajeNuevo . . . 162
17.20. PesajeCorreccion . . . 163
17.21. Pesaje . . . 163
17.22. Reserva . . . 163
17.23. Posicion . . . 163
17.24. Liquidacion . . . 164
17.25. GastoNuevo . . . 164
17.26. Asignacion . . . 164
17.27. PagoNuevo . . . 165
17.28. Pago . . . 165
17.29. Documento . . . 165
17.30. ReporteCartera . . . 165
17.31. ReporteMargen . . . 166
17.32. TarifaExterna . . . 166
17.33. Csrf . . . 166
17.34. Reconocimiento . . . 167
17.35. LoteCompraInput . . . 167
17.36. LoteVentaInput . . . 167
17.37. DistribucionInput . . . 168
17.38. Gasto . . . 168
17.39. ResultadoComando . . . 168
17.40. OperacionNueva . . . 168
17.41. Usuario . . . 168
17.42. UsuarioNuevo . . . 169
17.43. Periodo . . . 169
17.44. Predio . . . 169
17.45. PredioNuevo . . . 170
17.46. CierreVersion . . . 170
17.47. CalculoLote . . . 170
17.48. TarifaCapturada . . . 170
17.49. DiagnosticoProceso . . . 170

18. Definiciones canónicas completas **172**
18.1. Reglas de negocio RN-001..RN-045 . . . 172
18.2. Glosario . . . 176

18.3. Diccionario de datos . . . 179
18.4. Dependencias funcionales y normalización . . . 187
18.5. Casos de uso . . . 193
18.6. ADR-001: Monolito modular frente a microservicios . . . 195
18.7. ADR-002: SQL explícito, cálculo canónico y migraciones versionadas . . . 195
18.8. ADR-003: Animal permanente y posición de inventario por ciclo . . . 196
18.9. ADR-004: Reconocimiento gerencial y costo por posición . . . 196
18.10. ADR-005: Maestro de identidad en A y propiedad de escritura inmutable . . . 196
18.11. ADR-006: Replicación bidireccional, bootstrap y recuperación . . . 197
18.12. ADR-007: Pgpool-II en modo raw y presupuesto de conexiones . . . 197
18.13. ADR-008: Autenticación, permisos y aislamiento del piloto . . . 197
18.14. ADR-009: Documentos privados, evidencia y revisión administrativa . . . 198
18.15. ADR-010: Despliegue, respaldos y límites de disponibilidad . . . 198
18.16. ADR-011: Graphify y mantenimiento del contexto . . . 198
18.17. ADR-012: Alcance aplazado y condiciones para comercializar . . . 198
18.18. ADR-013: Sesiones centrales y disponibilidad . . . 199
18.19. ADR-014: Muerte en B e identidad en A . . . 199
18.20. ADR-015: Reconocimiento y cierres históricos . . . 199
18.21. ADR-016: Autoridad del esquema físico . . . 200
18.22. ADR-017: Despliegue inicial preparado . . . 200

19. Estado de tareas y alcance de la entrega 201

20. Matriz de pruebas por escenario 204

21. Recorrido visual de la aplicación 209
21.1. Bandeja de operaciones . . . 209
21.2. Recorrido de operación . . . 210
21.3. Corrección de pesajes . . . 211
21.4. Cartera . . . 212
21.5. Margen . . . 213
21.6. Periodos y cierres . . . 214
21.7. Operación a390 px con teclado . . . 215

22. Láminas de arquitectura de datos 216

23. Fuentes y trazabilidad del manual 229

Capítulo 1

Cómo leer y entender BoviCierre

1.1 Propósito y revisión de este manual

Este manual explica BoviCierre desde el problema del negocio hasta sus archivos ejecutables. Está dirigido a Aldo, Jared y a una persona que necesite aprender a usar, mantener y explicar el sistema en una exposición académica. No exige conocer Spring, React o PostgreSQL de antemano: introduce sus responsabilidades antes de entrar a las firmas de métodos y estructuras de datos. Una lectura completa permite seguir una petición desde el clic hasta el registro que queda en PostgreSQL, identificar quién puede modificarlo y entender por qué una cifra cambia o permanece histórica.

La revisión documentada es `92d3d4103a174eb11f4ccf21eff2deed8cc77682`, publicada en la rama main de `https://github.com/ALDOOGOAT/bovicierre`. La fecha de elaboración es el 12 de octubre de 2026. Los resultados de implementación y pruebas reproducidos corresponden a sus ejecuciones registradas del 11 de octubre; elaborar este PDF no significa haber repetido todas las suites. Las fuentes principales son el código versionado, las migraciones, OpenAPI y la evidencia guardada. Los archivos del futuro piloto y los datos comerciales reales no existen en esta revisión.

El manual distingue tres planos. El **negocio** describe qué significado tiene comprar, entregar o pagar. La **implementación** identifica archivos y métodos que realizan esas acciones. La **evidencia** indica qué escenarios se ejecutaron, con qué resultados y qué límites conservan. Tener una documentación o un endpoint descrito no equivale a probar todas sus combinaciones. Un escenario DEMO es un dato sintético persistido en el backend; no es información de un productor real.

1.2 Ruta recomendada de lectura

Si primero necesitas arrancarlo, lee la guía de ejecución local y luego el recorrido de compra y venta. Si deseas entender el código, estudia la visión de arquitectura, la comunicación HTTP, el backend y el frontend en ese orden. Si debes explicar la asignatura de bases de datos, comienza con identidad y ciclo, continúa con modelos, funciones SQL, roles, transacciones, replicación y r estauración. Para revisar una fórmula, consulta su explicación y su caso GOLD: el caso contiene esperados fijos que sirven como oráculo.

Los anexos son material de consulta, no una segunda aplicación. El catálogo de API permite ubicar una ruta concreta; el diccionario reúne campos y relaciones; las reglas numeradas conservan los identificadores del repositorio. Las referencias de la forma `ruta:línea` apuntan al corte versionado. Una línea puede cambiar en commits posteriores, por lo que debe contrastarse siempre con el nombre del método y el archivo. Los nombres técnicos aparecen en tipografía monoespaciada, por ejemplo `posicion_inventario`.

1.3 Qué problema resuelve

Una operación ganadera tiene varias dimensiones que rara vez ocurren a la vez. Se puede pactar una compra hoy, recibir los animales mañana, pagar parte después y recibir una factura de flete la semana siguiente. También puede venderse una parte del lote antes de conocer todos sus costos. Si se utiliza una única etiqueta como “completado”, se pierde la posibilidad de responder cuánto dinero falta, quién tiene físicamente los animales y cuánto dejó cada ciclo.

BoviCierre conserva esas dimensiones separadas. La compra o venta tiene condiciones y una versión. La posición representa la adquisición concreta de un animal. La recepción y entrega son movimientos físicos. La liquidación representa una obligación. El pago representa dinero validado; su aplicación reduce una obligación. El gasto puede aumentar el costo de inventario, el costo de venta o la pérdida, según el estado del ciclo. La evidencia conserva documentos, pesajes y revisiones que explican el resultado.

El cliente objetivo es un comercializador o acopiador que compra y vende bovinos por cuenta propia, con una o dos sedes. El dueño o administrador compra el software; operaciones, caja y supervisión lo utilizan. El producto no es un marketplace, no custodia dinero y no emite una autorización sanitaria. Registrar un identificador o revisar un archivo administrativamente no certifica su autenticidad oficial.

1.4 Conceptos que no debes confundir

Concepto	Significado	Ejemplo
Animal	Identidad permanente.	El mismo animal puede aparecer en una recompra.
Posición o ciclo	Una adquisición con costo y cierre propios.	Primera compra de 1100 y segunda compra de 1300.
Operación	Compra o venta con selección y condiciones.	Una venta de cuatro posicion es.
Liquidación	Obligación y conceptos que explican su importe.	El comprador debe 6400 antes de aplicaciones.
Pago o cobro	Movimiento monetario registrado y validado.	Cobro de 1200 que puede tener 200 disponibles.
Aplicación	Vínculo de dinero validado a una obligación.	Se aplican 1000 a una deuda de 1000.
Entrega	Hecho físico distinto del reconocimiento económico.	La venta ya se confirmó pero aún no salió el animal.
Margen	Resultado gerencial de venta, costo y gastos directos.	Venta 6400 menos costo 4400 menos gasto 200.
Periodo	Intervalo mensual de reconoc imiento.	Un costo viejo se reconoce en el mes abierto actual.

1.5 Una historia completa antes de ver el código

Imagina una compra DEMO de diez animales a 1000 cada uno. Confirmar la compra reconoce 10000 y crea diez posiciones. Un gasto de adquisición de 1000 distribuido por cabeza agrega 100 a cada posición: el costo unitario pasa a 1100. La deuda del ganado y el gasto tienen su propia procedencia; el gasto no cambia silenciosamente el precio pactado con el vendedor.

Después se reciben los animales. Se seleccionan cuatro posiciones disponibles para vender a 1600 cada una y se reservan para impedir que otra venta las comprometa simultáneamente . Confirmar la venta reconoce 6400, consume reservas, transfiere 4400 del inventario al costo

de venta y deja los animales vendidos pendientes de entrega. Un gasto directo de venta de 200 produce margen 1800. Quedan seis posiciones con inventario de 6600.

Un cobro posterior reduce cartera mediante aplicaciones, pero no crea otra venta. Una entrega posterior cierra físicamente los ciclos vendidos, pero no vuelve a pasar el costo a resultados. Un flete tardío de 500, distribuido entre los diez animales originales, agrega 50 al costo de cada ciclo: 200 corresponden a los cuatro vendidos y 300 a los seis todavía en inventario. El margen actual baja a 1600 y el inventario aumenta a 6900. Si ya existía un cierre histórico, ese cierre no se reescribe para esconder el cambio.

La utilidad del sistema aparece al abrir el expediente: cada importe remite al acuerdo, a sus animales, a un gasto, una aplicación o un ajuste. El operador no tiene que deducir retrospectivamente qué ocurrió a partir de un total mutable.

Capítulo 2

Mapa general de arquitectura y comunicación

2.1 Las capas y sus responsabilidades

El navegador ejecuta React. React presenta formularios, conserva el estado de interacción y envía HTTP. El navegador no tiene la contraseña de PostgreSQL ni puede ejecutar SQL; tampoco es la autoridad del cálculo monetario. Caddy recibe la conexión del usuario en localhost:15180. Para `/api/*` utiliza el backend Java; para las páginas y archivos estáticos utiliza el servidor del frontend.

Java recibe las solicitudes, valida su forma, comprueba sesión y autorización, elige el dueño de la operación y ejecuta el caso de uso. Los pools JDBC proporcionan conexiones a PostgreSQL A, B y autenticación. Una transacción económica utiliza solamente el PostgreSQL propietario. PostgreSQL conserva las restricciones e implementa los comandos canónicos con SQL y PL/pgSQL. El resultado comprometido vuelve como JSON a Java y luego a React.

Dos bases de negocio son clústeres distintos con volúmenes distintos. A administra maestros y sus hechos locales; B administra los hechos de su sede. Las publicaciones y suscripciones distribuyen filas entre A y B. La replicación es asíncrona: una confirmación local puede responder antes de que la copia remota la reciba. El tercer PostgreSQL es un proveedor simulado para demostrar FDW; no es un organismo oficial ni una integración comercial.

2.2 Diseño inicial e implementación real

El proyecto contiene una arquitectura objetivo con diez módulos y nombres de interfaces como `OwnerDatabaseRouter` o `BranchTransactionRunner`. Esos nombres describen una separación planeada, no un permiso para afirmar que existen clases con ese nombre. En la revisión documentada, las responsabilidades ejecutables se concentran en los paquetes `acceso`, `operaciones`, `evidencia`, `reportes`, `integraciones` y `shared`. El capítulo Java identifica la clase concreta que realiza cada paso. El controlador central y el servicio de comandos comparten validación y ejecución; PostgreSQL reúne reglas persistentes en funciones canónicas.

Esta concentración reduce código duplicado, pero hace importante comprender el despacho de comandos y consultas. Añadir una función al SQL sin autorizarla en el backend no crea automáticamente una acción pública. Añadir una pantalla tampoco concede un permiso. La arquitectura se comprende por responsabilidades y fronteras, no contando carpetas con nombres ideales.

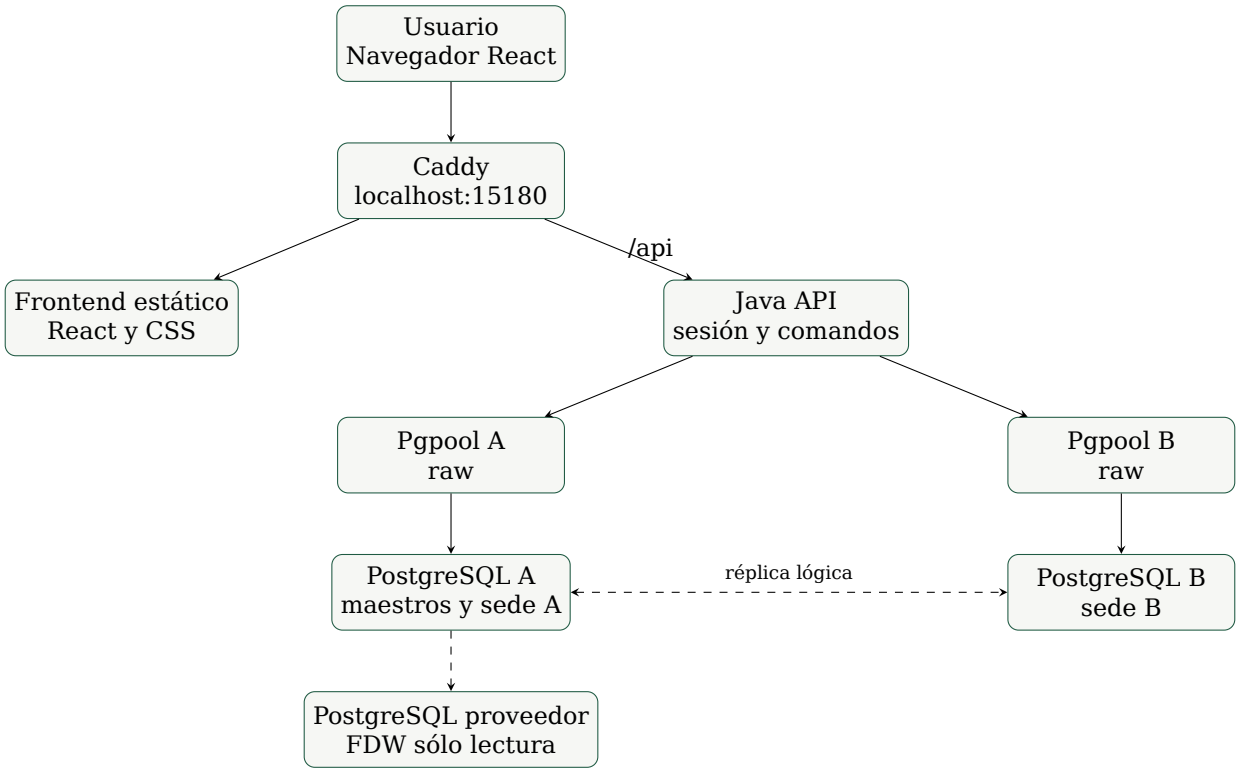


Figura 2.1: Comunicación del perfil académico. Autenticación accede a A; archivos privados se almacenan fuera del frontend.

2.3 Quién escribe y quién sólo consulta

La regla “un escritor por agregado” significa que no se actualiza la misma fila comercial indistintamente en A y B. El propietario está fijado por el tipo de dato y la sede. A conserva los maestros centrales; cada sede produce sus hechos locales en su dueño. Pgpool no toma esta decisión y no promociona B cuando A falla. Cada Pgpool raw tiene un único backend fijo.

Una consulta puede leer una copia replicada, pero esa copia tiene una fecha y una procedencia. Un consolidado A/B no es una transacción global atómica. Por eso el reporte debe indicar fuente, corte y antigüedad conocida o desconocida. Un `null` en antigüedad no debe convertirse en cero sólo para aparentar sincronización.

La sesión y la autorización vigentes son centrales en A. Si cae A, las nuevas solicitudes autenticadas a ambas sedes reciben un error controlado. Una transacción B que ya había sido autorizada puede terminar. Esto es una dependencia decidida del MVP y debe explicarse en la exposición: dos escritores académicos sobre conjuntos disjuntos no equivalen a disponibilidad continua de toda la aplicación.

2.4 Qué pasa al pulsar confirmar

Primero React conserva una intención con su clave idempotente y la versión revisada. Envía el body tipado, `Idempotency-Key`, `If-Match` cuando corresponde y el token CSRF. Caddy conserva el ETag de la API: la compresión gzip se aplica al frontend estático, no a la respuesta API que participa en la precondición.

Java autentica, valida la sede y los permisos, reconoce el comando admitido y determina el nodo. La transacción del dueño reclama la clave, resuelve un replay terminado antes de exigir una versión nueva, bloquea periodo y recursos en orden y valida las precondiciones persistentes. Si todo procede, registra liquidación, inventario o dinero, auditoría, outbox y resultado idempot

ente. Si algo falla antes del commit, se revierte todo ese conjunto. La respuesta aparece sólo después del resultado de la transacción.

Si la respuesta se pierde después del commit, el navegador no sabe todavía si se registró la opera ción. Mostrar “no se guardó” sería incorrecto. La UI conserva la intención, consulta el resultado o reintenta exactamente la misma clave y contenido. Una nueva clave podría crear un segundo hecho; por eso se distingue un error de validación de un resultado pendiente de comprobar.

Capítulo 3

Guía completa para ejecutar en local

3.1 Requisitos y versiones

La aplicación usa Java 21, Maven Wrapper 3.9.12, Spring Boot 4.1.1, React 19.2.4, TypeScript 5.9.3, Vite 8.0.3, PostgreSQL servidor 18.6 y Pgpool 4.7.3. El cliente `psql` del host puede tener versión menor distinta; en la evidencia original era 18.3. El frontend exige Node compatible con `^20.19.0||>=22.12.0`, según su manifiesto. Docker y Compose deben funcionar para el usuario que ejecuta los comandos.

El wrapper evita depender de la versión global de Maven. Para las herramientas Python se utilizan entornos privados y los requisitos fijados; no se recomienda instalar librerías de pruebas dentro del runtime de la aplicación. Las imágenes de infraestructura están fijadas por etiqueta y digest donde el Compose lo especifica. Las imágenes Java/frontend se construyen localmente.

3.2 Comprobar el checkout y el entorno

Ejecuta desde la raíz del repositorio. No copies rutas privadas de otra persona. Si estás en la computadora original, puedes entrar así:

```
cd /home/aldo/Descargas/PROYECTO_ZACA_TOPICOS_AVANZADOS
git status --short --branch
make doctor
```

`makedoctor` consulta versiones, CPU, memoria y puertos. Un puerto ocupado no demuestra un error: puede pertenecer a BoviCierre ya iniciado. Identifica primero el servicio. El script no mata procesos, no borra volúmenes y no muestra archivos de secretos. Si Java o Docker faltan, resuelve ese requisito antes de interpretar los errores siguientes como fallos del dominio.

No existe un instalador universal de Docker/Java para todos los sistemas en este proyecto. Las instrucciones que siguen corresponden al entorno Linux probado. En otra máquina deben verificarse permisos de Docker, disponibilidad de imágenes y puertos. El motor PostgreSQL vive en contenedores: instalar `psql` sirve como cliente, no sustituye el servidor de Compose.

3.3 Arranque académico integrado

```
make academic
make demo
```

`makeacademic` ejecuta `scripts/academic.py`. Verifica que no exista una API manual concurrente en18080, prepara el directorio de documentos y comprueba propietario/permisos. Después llama al bootstrap de BD, aplica migraciones, configura las relaciones académicas y construye/levanta backend, frontend y Caddy. Los ocho servicios pertenecen al proyecto `bovicierre-academic`. El arranque normal conserva datos y no contiene un `down-v`.

`makedemo` instala las dependencias Python de BD en el entorno privado y ejecuta `db/seed/demo.py`. La semilla utiliza comandos de negocio y claves reproducibles; no inserta totales financieros a mano ni borra una BD ajena. Se comprueban el proyecto, base y empresa DEMO. Ejecutar de nuevo la semilla debe recuperar los resultados anteriores sin duplicar compras o pagos. El modo de producción rechaza bootstrap y seed académicos.

Abre `http://localhost:15180`. El acceso comienza obteniendo un CSRF anónimo, realizando login y seleccionando una sede permitida. Los usuarios demo son `demo.admin`, `demo.operador`, `demo.caja` y `demo.auditor`. Las contraseñas se generan en archivos locales privados; este manual no contiene sus valores.

Para consultar tu propia contraseña de ADMIN, usa únicamente la terminal local y no pegues el resultado en capturas, chats o Git:

```
cat "$HOME/.local/share/bovicierre-academic/secrets/demo_password"
```

Los otros archivos son `demo_operador_password`, `demo_caja_password` y `demo_auditor_password` dentro del mismo directorio. El directorio debe tener permisos0700 y los archivos sensibles0600. En producción futura no se reutilizan estas credenciales ni datos DEMO.

3.4 Puertos y direcciones

Servicio	Dirección en el host	Función
Interfaz integrada	http://localhost:15180	Entrada recomendada del usuario.
PostgreSQL A	127.0.0.1:15432	Maestros, acceso y hechos sede A.
PostgreSQL B	127.0.0.1:15433	Hechos sede B.
Proveedor simulado	127.0.0.1:15434	Datos FDW sintéticos.
Pgpool A	127.0.0.1:19999	Canal JDBC propietario A.
Pgpool B	127.0.0.1:19998	Canal JDBC propietario B.
API manual	http://127.0.0.1:18080	Alternativa de desarrollo.
Vite manual	http://127.0.0.1:15173	Alternativa de desarrollo.

Los puertos publicados son loopback. Dentro de la red de Compose se emplean nombres como `pg-a`, `pgpool-a` y `backend`; el puerto interno puede ser distinto del publicado. `0.0.0.0` dentro del contenedor permite recibir al proxy, pero no implica que se haya expuesto PostgreSQL a Internet.

3.5 Comprobar servicios sin modificar datos

```
docker compose -p bovicierre-academic \
  -f infra/compose/compose.academic.yml \
  -f infra/compose/application.yml ps
curl --fail http://localhost:15180/api/v1/salud
```

El endpoint de salud diferencia el proceso y las dependencias. La información de replicación y mensajes debe consultarse con el alcance autorizado; no se deduce de que el HTML de login aparezca. Una caída de B no debe confundirse con una caída de A: las consecuencias sobre comandos, sesiones y lecturas son distintas.

3.6 Detener y volver a iniciar

```
make stop
make academic
```

`makestop` detiene únicamente los servicios del proyecto declarado, sin eliminar volúmenes. Si elegiste la alternativa de desarrollo manual, sus procesos se detienen desde la terminal que ejecuta `makedev`, mediante Ctrl+C. No utilices `killalljava`, `dockersystemprune` o borrados de volúmenes para resolver una ocupación de puerto: la computadora puede contener otros proyectos.

3.7 Desarrollo con Java y Vite en primer plano

Primero detén el perfil integrado para evitar API simultáneas y el puerto15180 ocupado. Mantén o vuelve a iniciar la infraestructura de BD, instala el frontend y ejecuta el ciclo manual:

```
make stop
sh db/bootstrap/start.sh
npm --prefix frontend ci
make dev
```

`makedev` ejecuta `scripts/dev.py`, que crea dos procesos hijos: `backend/run-local.sh` y `npmrun dev` en frontend. Rechaza puertos18080/15173/15180 ocupados y conserva la relación con sus hijos. Ctrl+C los termina gradualmente; una espera vencida sólo afecta a esos hijos identificad os. Vite sirve15173 y envía las peticiones API al backend local conforme a su configuración.

El script `backend/run-local.sh` lee los secretos externos, exporta las variables que requiere Spring y escribe PID/registro en un directorio privado. No se utiliza una contraseña fija en `appl ication.yml`. No debes iniciar otra instancia Java encima del backend integrado: ambos podrían despachar outbox y competir por el presupuesto de conexiones.

3.8 Variables de configuración que debes conocer

Una variable de entorno es un dato que el proceso recibe al iniciarse. No es una columna de BD ni una constante TypeScript. Las variables `BOVI_DB_*_PASSWORD` son secretos; los nombres y funciones pueden documentarse, sus valores no. El Compose utiliza archivos montados en `/run /secrets` y el entrypoint los transforma para el proceso Java.

Variable	Propósito	Valor o tratamiento
`BOVI_SECRET_DIR`	Directorio de secretos de scri pts/Compose.	Por defecto fuera del checkout.
`BOVI_PRIVATE_DIR`	Raíz privada de documentos y respaldo.	`~/.local/share/bovicierre-academic`.
`BOVI_DOCUMENTS_DIR`	Almacenamiento del adaptador documental.	Volumen privado, no webroot.
`BOVI_UID/BOVI_GID`	Identidad de ejecución de con tenedores.	Usuario local; no root por comodidad.
`BOVI_API_PORT`	Puerto API manual o restaura da.	18080 normal;18180 ensayo HTTP aislado.
`BOVI_BIND_ADDRESS`	Interfaz de escucha de Java.	Loopback manual; interfaz interna en Docker.
`BOVI_DB_A_URL/BOVI_DB_B_UR L`	JDBC de los dueños.	Pgpool A/B; nunca intercambiar por fallo.
`BOVI_AUTH_URL`	JDBC dedicado de acceso/sesi ones.	PostgreSQL A central.
`BOVI_OUTBOX_ENABLED`	Activación del worker.	Configurable; no altera dueño ni deduplicación.
`BOVI_RUNTIME_DIR`	PID/registro de API manual.	Directorio separado por orige n/restauración.
`BOVI_TEST_VENV`	Entorno Python auxiliar.	Caché privada de pruebas SQL.

Variable	Propósito	Valor o tratamiento
`BOVI_API_IT`	Activación de pruebas HTTP reales.	`true` sólo con API preparada.
`BOVI_API_TEST_URL`	Base API de pruebas HTTP.	localhost15180/api/v1 en evidencia final.
`BOVI_TEST_PASSWORD_FILE`	Archivo de contraseña de fixture.	Se lee localmente, no se versiona.
`SPRING_PROFILES_ACTIVE`	Perfil Spring.	Producción exige Secure; local permite HTTP.

`.env.example` es una orientación, no una lectura automática por todos los programas. El propio comentario pide exportar las variables para make o usar `--env-file` con Compose. No supongas que copiar un archivo `.env` cambia un proceso Java ya arrancado. Algunas variables son de scripts y otras de Spring; su consumidor exacto aparece en el atlas de configuración.

3.9 Verificación y su alcance

```
make verify-docs
make verify
make verify-api
make verify-sql
npm --prefix frontend run typecheck
npm --prefix frontend run build
npm --prefix frontend test
npm --prefix frontend run generate:api
```

`verify-docs` comprueba IDs documentales, referencias del esquema lógico, cinco tests de herramientas de diagramas y OpenAPI. No confirma compras ni carreras. `verify` agrega Maven, frontend y comparación de catálogo A/B. `verify-api` activa los tests HTTP contra una API viva, utilizando la contraseña externa. `verify-sql` prepara namespaces QA y ejecuta el runner SQL: incluye casos de negocio, guardas y pesajes/reportes. Debes distinguir sus bases de prueba de la base de negocio principal.

El generador frontend produce tipos desde OpenAPI. Después de modificar el contrato, genera de nuevo y comprueba tipos/build; el navegador conserva su adaptador HTTP propio. Tener un tipo generado no realiza una llamada ni prueba que Java devuelva ese formato.

Los E2E requieren el navegador y los fixtures configurados por `frontend/playwright.config.ts` y `tests_e2e/runtime.ts`. El script es `npm--prefixfrontendruntest:e2e`; revisa primero sus requisitos de API, contraseña y estado del periodo. Algunos casos provocan incidencias controladas; no ejecutes indiscriminadamente pruebas de caída mientras alguien usa la demo. La evidencia final separó15 casos integrados de una prueba física adicional del proveedor, no una única suite invisible de16 casos concurrentes.

3.10 Respaldo y recuperación aislada

```
make backup BACKUP_DIR="$HOME/.local/share/bovicierre-academic/backups/ensayo-nuevo"
make restore-check BACKUP_DIR="$HOME/.local/share/bovicierre-academic/backups/ensayo-nuevo"
```

El directorio de respaldo debe ser nuevo y externo al checkout. El procedimiento pausa la API y sus workers, espera que no queden transacciones/mensajes en vuelo y comprueba convergencia A/B. Copia archivos y secretos con hashes, crea dumps de A/B/proveedor y excluye sesiones/publicaciones/suscripciones que podrían apuntar al origen. Finalmente reanuda la fuente; la prueba de restauración no la deja pausada durante todo el ensayo.

La restauración crea un proyecto distinto `bovicierre-restore-<UTC>`, con otros puertos y volúmenes. Verifica integridad antes de configurar suscripciones sin copia inicial, reconstruye FDW

hacia su propio proveedor y conserva inbox/idempotencia. Se invalidan sesiones. El ensayo HTTP requiere una API adicional únicamente18180, con runtime/documentos/secrets propios y JDBC del destino; `verify-http` no arranca esa API por sí solo.

```
python3 scripts/recovery.py verify-http \
  --directory "$HOME/.local/share/bovicierre-academic/backups/ensayo-nuevo"
```

La restauración V13 registrada pausó8.038 segundos el origen y tardó28.616 segundos en el ensayo de restauración. Se comprobaron archivos, datos e intención económica nueva por HTTP. Son medidas de una recuperación local en ese equipo; no representan un SLA, un RTO contractual ni resistencia a perder la laptop.

Capítulo 4

Infraestructura explicada desde sus archivos

4.1 Docker Compose y separación de servicios

La topología se describe en dos archivos: `infra/compose/compose.academic.yml` contiene PostgreSQL y Pgpool; `infra/compose/application.yml` agrega Java, frontend y Caddy. El nombre de proyecto identifica redes, contenedores y volúmenes propios. No basta con que un contenedor se llame PostgreSQL: los scripts comprueban etiquetas de proyecto para evitar administrar un servicio ajeno.

El backend se construye desde `backend/Dockerfile` y el frontend desde `frontend/Dockerfile`. Los secretos se montan como archivos, no se incorporan en las capas de una imagen. Caddy utiliza configuración local de sólo lectura y directorios temporales acotados. El montaje de documentos permite que Java escriba en un directorio privado del usuario; el frontend no sirve ese directorio.

En el perfil académico inicial, A/B tienen1GiB de límite cada uno, el proveedor512MiB y Java1GiB. Frontend y Caddy tienen128MiB. Esas cifras son límites de contenedor, no consumo medido de toda la aplicación. El host también ejecuta otras herramientas. La asignación debe contrastarse con muestras de memoria, conexiones y carga antes de prometer capacidad.

4.2 Pools y presupuesto de conexiones

`DatabaseConfiguration.pool` crea un HikariDataSource a partir del prefijo `bovi.database.<nombre>`. Los pools de dueños y autenticación tienen máximo4 conexiones; cada worker tiene2. `minimumIdle=0` permite no reservar conexiones ociosas obligatorias. `connectionTimeout=3000` y `validationTimeout=1000` acotan esperas iniciales; `initializationFailTimeout=-1` evita exigir toda la infraestructura en la construcción del bean. Esa última opción no hace que una consulta a una BD caída tenga éxito.

Cada Pgpool tiene16 procesos y `max_pool=1`, apuntando a un solo dueño. PostgreSQL tiene `max_connections=50`. Al presupuestar deben incluirse los clientes directos de auth, workers de replicación, administración, migración y observación. No se cuentan dos veces una misma conexión lógica, pero tampoco se omiten las conexiones físicas que Pgpool mantiene abiertas. El informe `connection-budget.json` guarda la observación concreta del entorno probado.

4.3 Proxy, ETag y origen común

Caddy maneja `/api/*` con `reverse_proxybackend:18080`. El resto utiliza el frontend estático y compresión gzip. Esta separación evita que la compresión de una respuesta JSON modifique un

ETag fuerte de borrador y haga inválido el `If-Match` enviado por el cliente. Es un ejemplo de cómo una optimización de infraestructura puede afectar una regla de concurrencia HTTP.

El navegador trabaja con el mismo origen para páginas y API. Por ello no se recurre a CORS universal con cookies. La configuración local usa HTTP loopback, mientras que la preparación futura de producción requiere HTTPS y cookies Secure. Un archivo de ejemplo con dominio `.invalid` no implica que exista un certificado emitido o un servidor público contratado.

Capítulo 5

Pruebas y evidencia que explican la confianza disponible

5.1 Qué significa una prueba y qué no

Una prueba unitaria ejecuta una función o un grupo pequeño de objetos. Es rápida y útil para transformación, validación o serialización. Una prueba de SQL utiliza PostgreSQL real y puede proteger constraints, funciones, locks y efectos persistentes. Una prueba HTTP atraviesa autenticación, filtros, controlador y acceso a BD. Una prueba E2E maneja la interfaz en un navegador y observa el comportamiento para el usuario.

Estas capas son complementarias. Un test de React que verifica un formato decimal no demuestra que dos aplicaciones simultáneas respeten el saldo. Una captura visual no acredita persiste ncia. Una comprobación de IDs RN/TEST/TASK demuestra cobertura documental, no ejecución del dominio. El manual utiliza la matriz de pruebas para vincular cada afirmación con su nivel y límites.

El backend final registra24 pruebas:15 unitarias,8HTTP contra Caddy15180 y una integración Testcontainers PostgreSQL18.6 con migracionesV1-V13. El frontend registra13 unitarias y16 recorridos UI, separados en15 integrados y una prueba adicional de caída física del proveedor. La SQL registró157 intentos GOLD,22 guardas de versión y66 comandos de pesajes/reportes. Estas cuentas no deben sumarse para presentar un porcentaje de cobertura sin denominador.

5.2 Oráculos económicos y concurrencia

Un esperado fijo evita que la prueba comparta el mismo error de cálculo que el código. En GOLD-02 se escribe explícitamente que cuatro posiciones de1100 producen costo4400 y margen1800; no se obtiene el esperado llamando a la misma función que confirma la venta. GOLD-03 fija0.3 4/0.33/0.33 y suma1.00, incluyendo el orden estable.

Para una carrera se utilizan dos conexiones independientes y barreras que coordinan el momento de competencia. El estado final debe demostrar un solo ganador o un rechazo sin efecto. Dormir unos milisegundos y observar un mensaje no es una prueba fiable de exclusión mutua. También se comprueba el reintento tras pérdida de respuesta: la clave previa recupera el resultado y los registros no aumentan.

La matriz TEST-01..35 conserva los escenarios diseñados y registra evidencia SQL/API/UI por caso. Donde un caso quedó acotado, la limitación no se transforma en una aprobación universal. La parte humana del modelo y la rúbrica sigue pendiente aunque las verificaciones automáticas pasen.

5.3 Rendimiento medido

El benchmark QA se preparó con1901 comandos canónicos y repitió1901 sin duplicación. El dataset propio contiene200 animales,200 compras,100 ventas,300 gastos,100 pagos y100 entregas; la base QA completa conserva además fixtures anteriores. La medida utiliza800 lecturas y100 calentamientos, con conexión única y cuatro conexiones persistentes. Los tiempos incluyen consulta, transferencia y decodificación psycopg, pero excluyen HTTP, Java y React.

Consulta	p50/p95 una conexión en ms	p50/p95 cuatro conexiones en ms
Inventario	9.580 /14.088	9.478 /13.322
Cartera	6.866 /10.825	10.025 /12.372
Liquidación	1.534 /3.718	1.616 /2.813
Expediente	0.244 /0.970	0.444 /1.075

La evidencia pertenece a un i7-13620H con16CPU lógicos y aproximadamente15724.6MiB de memoria. Las consultas se ejecutaron con caché caliente y sin escritores. Los EXPLAIN del wrapper PL/pgSQL no desglosan todos los planes internos. Es una observación útil del MVP; no permite inferir el número de clientes de producción ni garantizar un p95 HTTP.

5.4 Cómo reproducir una medida sin alterar la demo

`db/tests/benchmark.py --prepare` y `--measure` están fuera del runner ordinario porque conservan datos QA y exigen una ventana coordinada. La base autorizada es `bovicierre_release_replay`, no la principal. El propio script valida perfil y contexto. Antes de medir, se coordina una pausa de API/escritores; después se reanudan. La reproducción no se incluye como un paso automático del arranque de usuarios.

La prueba visual se realizó con Chrome visible y GPU Intel integrada, incluyendo390/768/1440 px y un flujo sólo con teclado. No se ha certificado WCAG ni se ha probado teléfono físico. Para una nueva animación costosa o efecto GPU, deben repetirse medidas específicas sobre el hardware más débil del usuario; las capturas actuales no acreditan los FPS de un efecto que aún no existe.

Capítulo 6

Mantenimiento y diagnóstico con criterio

6.1 Si no abre la interfaz

Verifica el estado de los servicios propios, el puerto15180 y la URL. Si Caddy no inició, consulta sus logs sin imprimir secretos. Si el HTML aparece pero los datos fallan, comprueba `/api/v1/salud`: un problema de backend o autenticación puede ser distinto de un problema de frontend. Si `makeacademic` rechaza18080, identifica la API manual y termina su proceso gradualmente; no mates procesos de otros proyectos.

6.2 Si un comando devuelve un conflicto

Un409 puede indicar una reserva incompatible, una regla o una clave reutilizada con contenido diferente. Lee `codigo`, `mensaje` y los campos públicos de la respuesta. Un412 indica versión revisada obsoleta; un428 indica falta de la precondición requerida. No soluciones ambos generando automáticamente una versión o una clave nueva. Primero actualiza el detalle y comprende si ya existe un resultado para la intención anterior.

Un422 expresa una regla de dominio, como composición de pesaje incorrecta o asignaciones que no concilian. El usuario debe corregir las entradas permitidas. Un503 expresa una dependencia necesaria no disponible; no prueba por sí solo que un comando ya enviado hizo rollback. Si el resultado puede haberse comprometido, se conserva la intención y se consulta su estado.

6.3 Si la réplica no converge

Consulta suscripciones, apply, slots y WAL retenido. Un error de apply no se resuelve saltando transacciones al azar. Verifica las migraciones, constraints, propietarios y las referencias de origen. Recuerda que DDL, roles y secuencias no se replican como filas. La ausencia de actividad no permite afirmar retraso cero: puede significar que no hubo nada nuevo o que falta una medida.

Antes de volver a configurar `copy_data=false`, debes probar una base compatible común. Usarlo porque “parece que ya están iguales” puede omitir datos. Si faltó un slot o los datos divergieron, el runbook de recuperación explica la investigación y reconstrucción. No cambies el propietario para que B reciba escrituras que corresponden a A.

6.4 Si falta un documento

El archivo privado y PostgreSQL no comparten una transacción ACID. Por eso existe staging, metadata y limpieza controlada. Comprueba existencia, hash, estado y autorización. Un archivo huérfano no debe borrarse sólo por no verlo en la pantalla: puede estar en staging o corresponder

a una versión antigua todavía referenciada. La limpieza respeta edad/estado y falla de forma conservadora ante dudas.

Una descarga403/404 puede ser una protección de acceso, no un archivo perdido. El UUID no es una contraseña ni concede derecho a descargar. El QR lleva a una ruta autenticada. Nunca conviertas un directorio privado en una ruta estática de nginx para “hacer que abra”.

6.5 Cómo extender el sistema sin romper sus fronteras

Para una nueva acción, primero identifica la regla, el dueño, el periodo y el efecto persistente. Decide si es una consulta o un comando económico. Actualiza el contrato con entradas/respues tas/errores y luego el SQL canónico y el adaptador Java. El frontend debe conservar las claves y versiones pertinentes. Añade una prueba de comportamiento para el riesgo concreto; no copies cada línea de implementación como si eso garantizara corrección.

Una nueva columna exige migración numerada centralmente y sincronización con el diccionario y modelo lógico. No edites retrospectivamente una migración aplicada para esconder un upgrade: crea la siguiente versión. Coordina A/B antes de publicar cambios dependientes. Si la columna es técnica y local, documenta por qué no se replica. Si agrega datos sensibles, revisa permisos, logs y exportaciones.

Una nueva integración externa comienza por la operación requerida y una API oficial mantenida. API Anything puede ayudar en lecturas públicas autorizadas cuando no existe una opción suficiente, pero un MCP es una herramienta del agente, no el runtime automático de BoviCierre. Para que el producto consuma un servicio se necesita su propio adaptador backend, validación, dominios autorizados, timeout, errores y credenciales privadas. Mejorar la UI no obliga a copiar el backend de una referencia visual.

Capítulo 7

Herramientas de desarrollo, trazabilidad y responsabilidades

Las herramientas auxiliares ayudan a construir y comprobar el producto, pero no forman parte de la experiencia diaria del operador. Chrome DevTools MCP sirve para inspeccionar DOM/CSS, consola, red y capturas. UI/UX Pro Max fundamenta jerarquía, espaciado, componentes y estados. API Anything ofrece operaciones concretas de lectura cuando corresponden. Playwright es el runner reproducible de pruebas frontend; no debe confundirse con un servidor MCP que controla un navegador en una sesión de trabajo.

Graphify conecta reglas, tablas, comandos, endpoints, pantallas y pruebas. El corte final registra2509 nodos,4881 aristas y248 comunidades, con53 documentos reinterpretados y102 fuentes contrastadas. Los documentos históricos sin cambios mantienen extracción anterior. Una arista conceptual no demuestra una FK ni una prueba aprobada: se comprueba contra SQL, catálogo o evidencia. Sus salidas permanecen excluidas de Git.

Ponytail orienta una implementación pequeña pero completa: reutilizar piezas y evitar abstracciones sin frontera real, sin recortar controles monetarios, seguridad o accesibilidad. ECC aporta skills y mecanismos de trabajo según lo instalado; no existe una obligación de invocar un comando `/ecc` universal. Las instalaciones locales se documentan por versión y los perfiles/caches no se incluyen en el bundle de React.

Aldo conserva responsabilidad humana de backend, BD e infraestructura; Jared de frontend. Los agentes ejecutaron trabajo técnico, pero no suplantan la revisión de ambos. Las55 tareas originales tienen52 verificaciones técnicas locales, una pendiente externa y dos diferidas. Las55 revisiones humanas siguen pendientes. El commit MVP9282 b75 y el ajuste de wrapper92 d3 d41 están publicados en main; las notas antiguas de “sin publicar” describen el corte anterior, no el estado GitHub actual.

Capítulo 8

Preparación comercial y límites del alcance

La hipótesis inicial es3000MXN por sede/mes, hasta cinco usuarios,5GB de documentos y dos horas mensuales de soporte. Implantación y capacitación se cotizarían aparte; un piloto posterior tendría30 días de alcance acordado. Es una oferta a validar, no precio demostrado. El valor del ganado pertenece al negocio del cliente: no se contabiliza como ingreso SaaS.

Las entrevistas todavía no se realizaron. La cifra pública histórica de bovinos en Chiapas contextualiza la actividad regional; no determina el inventario actual ni el número de negocios com pradores. El snapshot geográfico de INEGI conserva claves y procedencia; los datos sintéticos permanecen separados. Una referencia SNIIM exige categoría, unidad, presentación, mercado y periodo, y no sustituye un precio pactado.

OCR, captura offline avanzada, marketplace, custodia de dinero, CFDI y autorizaciones sanitarias quedan fuera. La apertura de inventario real previo al piloto exige un procedimiento conciliado; no se inventan compras para sustituirlo. Traspasos que cambien dueño escritor y devoluciones físicas después de entrega también requieren evolución del alcance.

La preparación para una VM Linux con Compose y Caddy conserva archivos de ejemplo, redes, secretos y runbooks. No se contrató servidor, no se cambió DNS y no se emitió TLS público. Dos PostgreSQL en el mismo host son una demostración académica con escritores disjuntos; no redundancia física contra pérdida de la máquina. El siguiente paso comercial requiere revisión, usuarios reales, datos iniciales, recuperación externa y medición en el servidor elegido.

Capítulo 9

Cómo interpretar variables, cifras y fórmulas

9.1 Tres clases de variables

En este software “variable” puede significar tres cosas diferentes. Una **variable del negocio** representa una medida o importe: bruto, tara, precio, saldo o costo. Una **variable del programa** es un nombre que referencia un objeto durante la ejecución, como el resultado de una consulta o el AbortController de una petición. Una **variable de configuración** selecciona una dirección, puerto o recurso antes de arrancar, como `BOVI_DB_A_URL`. Ninguna de esas tres es intercambiable con las otras.

La columna `importe` de un asiento conserva un hecho persistente. La variable local `result` de Java puede contener el JSON de ese hecho. El estado React con ese JSON es una copia para mostrarlo. Si el operador cambia la presentación a formato monetario con separadores, no está modificando la columna. La autoridad económica sigue en PostgreSQL.

9.2 Notación y unidades

En las fórmulas de este manual, B representa peso bruto, T tara y N peso neto, todos en kilo gramos. p es precio por kilogramo, con unidad MXN/kg. n es el número entero de animales y p_h precio por cabeza. V es venta neta reconocida, C costo de posiciones vendidas y G_v gasto directo de venta. M es margen. Nunca debe sumarse una cantidad de kg a un importe en MXN.

$$N = B - T, \qquad 0 \le T \le B.$$

Por ejemplo, bruto5200.000 kg y tara200.000 kg producen neto5000.000 kg. Una tara mayor que el bruto es inválida, no un neto negativo que se arregla tomando su valor absoluto. Un peso individual desconocido se representa como ausencia, no como cero medido.

$$\Delta W = W_i - W_f, \qquad \Delta W_{\%} = 100 \frac{W_i - W_f}{W_i}, \quad W_i > 0.$$

Los símbolos W_i y W_f representan pesos de conjuntos comparables, inicial y final. Si el peso final es mayor, la variación es negativa y se conserva: no se convierte en merma cero. Si los animales de los dos tickets difieren, la aritmética puede producir un número pero el resultado carece de significado para esa comparación. Primero se valida la composición exacta.

9.3 Precio y reconocimiento

$$\text{Base}_{kg} = N\,p, \qquad \text{Base}_{cabeza} = n\,p_h.$$

En la primera expresión kg multiplicado por MXN/kg produce MXN; en la segunda, cabezas multiplicadas por MXN/cabeza también produce MXN. Esa comprobación dimensional ayuda a detectar una condición aplicada al lote equivocado. Las condiciones comerciales aceptadas determinan la base, no una referencia externa de mercado.

$$L = \text{Redondear}_2\left(\text{Base} + \text{Cargos} - \text{Bonificaciones}\right).$$

L representa el total de la obligación según las reglas de materialización. Los conceptos pueden materializarse y redondearse individualmente conforme al SQL canónico; esta expresión resume su significado, no autoriza una segunda implementación en Java o React. En particular, mostrar una previsualización no registra reconocimiento. La confirmación persistente produce el hecho económico una sola vez.

9.4 Saldo, disponible y margen

$$S = L + \sum_{j} A_j^{cargo} - \sum_{k} A_k^{abono} - \sum_{m} P_m^{neto}.$$

S es saldo de una liquidación. A^{cargo} incrementa la obligación y A^{abono} la reduce. P^{neto} incluye aplicaciones y sus reversos pertinentes; sumar una aplicación histórica revertida como si todavía existiera reduciría el saldo dos veces. Un pago registrado pero no validado no se puede aplicar para eludir esta regla.

$$D = I_{validado} - \sum_{m} P_m^{neto} - \sum_{r} R_r.$$

D es dinero disponible de un pago validado; $I_{validado}$ es su importe y R son reembolsos vincul ados. Disponible y saldo son dos límites distintos. Una aplicación sólo procede cuando respeta ambos y los ámbitos de empresa, sede, contraparte, moneda y sentido. Un disponible de200 no significa otra venta de200.

$$M = V - C - G_v.$$

En GOLD-02, $V = 6400$, $C = 4400$ y $G_v = 200$, por lo que $M = 1800$. Cobrar únicamente1200 no convierte el margen en un resultado de caja de1200 menos costos: margen y dinero cobrado son ejes diferentes. Un cierre o reporte debe explicar qué hechos y periodo utiliza.

9.5 Precisión decimal y redondeo

PostgreSQL utiliza NUMERIC, que almacena valores decimales de precisión fijada. `NUMERIC(18,2)` permite18 dígitos en total y2 decimales; tiene16 dígitos enteros. El límite finito positivo es $10^{16} - 10^{-2}$. `NUMERIC(14,3)` para peso permite11 dígitos enteros y tres decimales. El dominio incorpora cotas y validación de no finitos: disponer de un tipo numérico no reemplaza las reglas de entrada.

Java interpreta entradas monetarias con herramientas decimales cuando valida, mientras los importes confirmados provienen del SQL. El JSON utiliza cadenas para no introducir una representación binaria aproximada. Una cadena `"1000.00"` conserva significado monetario; `"400.000`

" conserva escala de kg. TypeScript puede dar formato visual, pero no recomputa la liquidación con `parseFloat`.

Para $x \geq 0$, una descripción matemática del redondeo HALF_UP a centavos es:

$$\text{Redondear}_2(x) = \frac{\lfloor 100x + 0.5 \rfloor}{100}.$$

Así,12.345 se materializa12.35 y12.344 se materializa12.34. Esta fórmula ilustrativa se limita a no negativos. Los efectos contrarios se representan con clases de asiento o registros de reversión según el dominio, no introduciendo libremente un importe negativo para saltar restricciones.

9.6 Distribución y conservación del total

Un gasto total G se asigna a n posiciones por pesos relativos w_i válidos. Por cabeza se toma $w_i = 1$. Por peso individual se usa una medición individual existente, nunca el promedio de un ticket grupal. Para importe individual la entrada declara cada importe y debe conciliar directa mente.

$$a_i^{exacto} = G \frac{w_i}{\sum_{j=1}^{n} w_j}, \qquad r = G - \sum_{i=1}^{n} \text{Redondear}_2(a_i^{exacto}).$$

r es el residuo después de redondear. La distribución corrige centavos en orden estable de posición, sin inventar o perder dinero. GOLD-03 exige que1.00 entre tres posiciones iguales ter mine0.34,0.33 y0.33. Si tres importes redondeados inicialmente dan0.99, queda un centavo que se asigna a la primera posición del orden. Si la suma inicial excede el total, la política corrige el residuo negativo respetando la validez de cada asignación.

$$\sum_{i=1}^{n} a_i^{final} = G.$$

Esta igualdad es una invariante: debe permanecer verdadera independientemente de cómo se presente la tabla. La implementación concreta y las validaciones de costo no negativo se explican en el capítulo SQL y se contrastan con esperados fijos.

9.7 Tiempo y resultado histórico

La fecha del documento identifica cuándo ocurrió o se fechó la evidencia. La fecha de reconocimiento determina el periodo gerencial del hecho. El timestamp de registro indica cuándo el servidor lo grabó. Un flete de septiembre registrado y reconocido en octubre conserva las tres nociones; no se modifica silenciosamente la fecha para reabrir un cierre pasado.

Si un costo compite con el cierre, ambos bloquean la misma fila del periodo. El costo entra antes del corte o el cierre gana y el costo se rechaza sin efecto en ese periodo. Reconocerlo en otro periodo exige una intención nueva explícita. El reintento de red conserva los parámetros anteriores; no puede convertirse de forma automática en un costo del mes siguiente.

Capítulo 10

Diagramas de hechos y estados

10.1 Identidad y ciclos

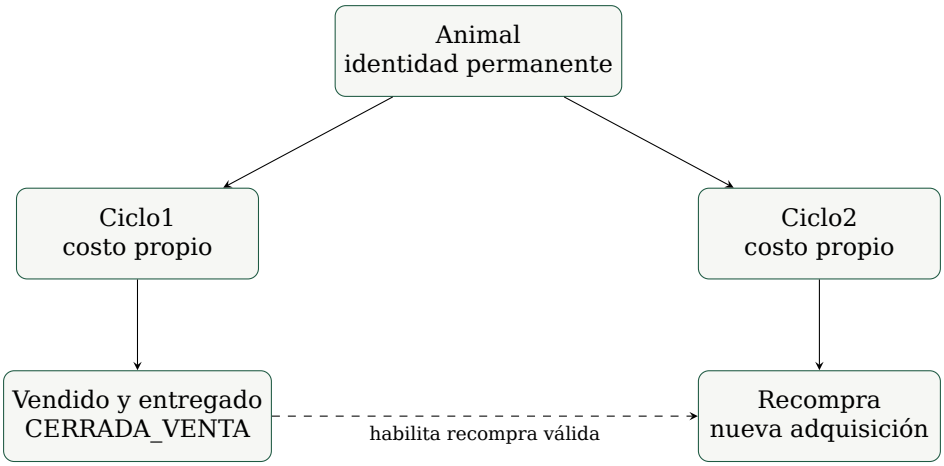


Figura 10.1: Un nuevo ciclo conserva el anterior. Nunca se habilitan dos posiciones activas para la misma identidad en el ámbito definido.

10.2 Venta, pago y entrega son ejes separados



Figura 10.2: Pagar y entregar no vuelven a reconocer la venta. Cada hecho conserva su propia fecha y evidencia.

10.3 Mensaje de muerte entre dueños

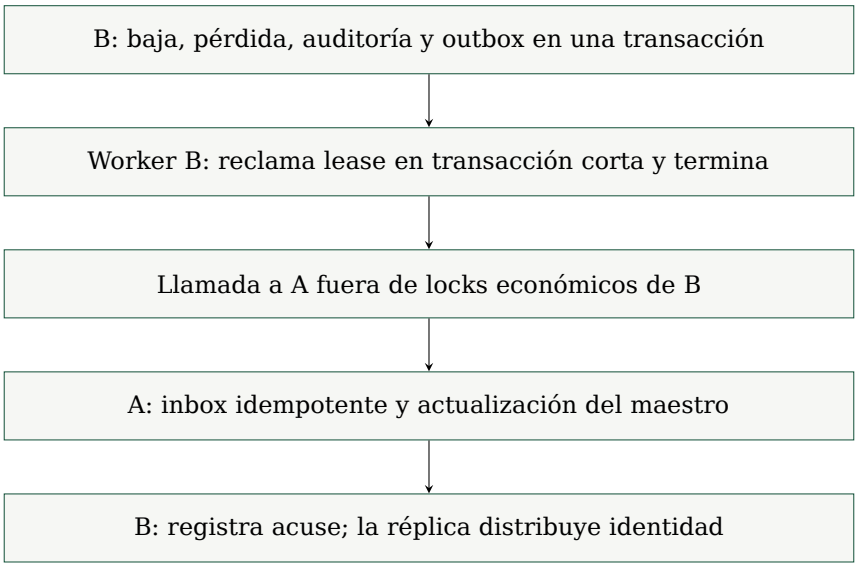


Figura 10.3: Acuse perdido se recupera con inbox. No existe una transacción distribuida A/B ni una segunda pérdida.

Capítulo 11

La base de datos de BoviCierre: modelo, comandos y reconocimiento

11.1 1. Propósito y forma de estudiar el sistema

Este capítulo explica la implementación de base de datos del checkout real de BoviCierre. Se estudian las migraciones V1-V13, las funciones vigentes, el modelo lógico y el catálogo físico observado. Su finalidad es didáctica: seguir una identidad, una medición, una obligación y un importe a través de sus transformaciones, entendiendo qué asegura cada restricción y qué necesita una transacción. No se presenta una lista de sentencias para ejecutar indiscriminadamente sobre la base principal. Las consultas ilustrativas de catálogo son de lectura; los ejemplos de escritura representan escenarios sintéticos que deben ensayarse únicamente en QA propia.

Hay cinco conceptos que deben distinguirse desde el primer ejercicio: identidad, intención, evidencia, reconocimiento y dinero. El animal tiene una identidad que permanece; seleccionar animales para una compra expresa una intención; aprobar un ticket aporta evidencia; confirmar la compra reconoce una obligación y un ciclo de inventario; pagar y aplicar dinero modifica caja y saldo. Recepción y entrega agregan hechos físicos. Cada paso explica una parte del trato, pero no debe reconocer otra compra o venta simplemente porque el usuario avanzó a otra pantalla.

El modelo evita una tabla única de animales que mezcle su nombre, el precio actual, la deuda pendiente, el pago y la fecha de entrega. Esa mezcla parecería sencilla en una demostración pequeña, pero impediría conservar un ciclo anterior después de recompra y explicaría mal una venta pagada que todavía no se entrega. En BoviCierre, cada eje tiene relaciones propias. Las consultas construyen una explicación conjunta, mientras las escrituras mantienen la identidad y la historia de cada hecho.

Las fuentes se citan con ruta y línea de inicio. Una función puede continuar muchas líneas después, por lo que la referencia orienta al bloque y no sustituye su lectura completa. Los valores GOLD son ficticios y fijos. No se incluyen contraseñas, cookies, conninfo, hashes de usuarios reales, contenido de sesiones ni datos comerciales privados. Los tiempos de pruebas se atribuyen a sus artefactos; no se vuelven a ejecutar aquí para producir una narración más cómoda.

11.2 2. Qué DDL manda y por qué existen dos recuentos

La autoridad física reside en db/migrations. V1 crea tablas, claves y parte del soporte; las migraciones siguientes añaden o sustituyen objetos de manera controlada. tools/esquema/esquema.py es la representación lógica para diagramas y comparación. No debe transformarse automáticamente en DDL sin revisar semántica. Un diagrama o una lista de columnas no contienen necesariamente todos los CHECK, los índices parciales, los triggers, las columnas generadas o los privilegios.

Las consultas de sólo lectura realizadas en A y B el 12 de octubre de 2026 confirmaron39 tablas ordinarias en bovi y7 en infra:46 tablas ordinarias y86FK en esos dos esquemas. La representación lógica documenta42 tablas y83FK. No se trata de una contradicción ni de tres relaciones artificiales para llegar a un número esperado. Las cuatro tablas físicas adicionales son infra.tar ifa_snapshot, infra.spring_session, infra.spring_session_attributes e infra.flyway_schema_histor y. La captura agrega dos FK a usuario y sucursal; los atributos agregan una FK hacia su sesión. Así,42 más4 da46 y83 más3 da86.

Ese alcance excluye vistas, catálogos internos de PostgreSQL, las tablas extranjeras del esquema ext_proveedor y las tablas ordinarias del servidor proveedor. Tampoco incorpora proyectos ajenos del equipo. El modelo lógico de42 ya incluye soportes técnicos como folio_contador, inbox e idempotencia; por tanto,42 no significa42 entidades exclusivamente ganaderas. El MER conceptual y el catálogo relacional persiguen objetivos distintos y deben defenderse como tales.

El conteo no demuestra que todas las45RN estén probadas en todas las combinaciones. Demuestra que un catálogo con un alcance explícito corresponde a la implementación observada. La comparación lógica contra A/B verifica columnas, tipos, nulabilidad y claves declaradas; las pruebas de permisos, triggers, carreras y oráculos económicos verifican mecanismos que el conteo no expresa.

Fuentes: tools/esquema/esquema.py:1; db/migrations/V1__esquema.sql:1; db/README.md:3; docs/database/relational-model.md:1.

11.2.1 2.1 La historia de migración no debe inventarse

En la instancia principal existente, V1-V3 fueron adoptadas mediante baseline3 después de comparar su catálogo con una reproducción limpia. El historial declara adopción auditada de catálogo, no ejecución original de esos archivos por Flyway. V4-V13 sí aparecen como migraciones ejecutadas. Una base completamente vacía aplica la secuencia V1-V13. Ambas historias pueden terminar en una estructura equivalente, pero no son la misma historia operativa.

El backend no lleva Flyway para crear objetos al iniciar una petición o al arrancar el servidor. Un trabajo Java21/Flyway separado aplica y valida migraciones con una identidad administrativ a. Las cuentas de negocio no necesitan DDL. Esta separación impide que una credencial usada por una solicitud HTTP modifique triggers o cree una tabla para eludir una validación.

CREATE OR REPLACE FUNCTION reemplaza la definición de una función existente con la misma firma. No significa que V2, V4, V8 y V13 sean cuatro algoritmos monetarios activos simult áneamente. La última definición de bovi.cmd_ejecutar es V13. La consulta bovi.consultar y los auxiliares de pesaje y reporte vigentes son V12. Cálculo, reparto, normalización y validación de periodos conservan auxiliares definidos en V2. Lectura FDW y captura local quedan según V9.

Migración	Aporte	Razón que debe explicarse
V1	Estructura, claves, restricciones e índices	Establece integridad de filas y relaciones.
V2	Comandos, cálculo, reparto, periodos y triggers	Define reconocimiento atómico y auxiliares.
V3	Consultas, sesiones, grants y trabajador	Separa interfaces técnicas y de negocio.
V4	Captura de proveedor y DTO completos	Materializa procedencia y respuestas persistibles.
V5	Corte por periodo de reconocimiento	Evita que documento posterior altere un cierre.
V6	Dinero a centavos y paginación	Conserva precisión original sin romper DTO monetario.
V7	Filtros de consulta	Evita búsqueda ignorada y pagina con orden estable.

Migración	Aporte	Razón que debe explicarse
V8	Borrador KG incompleto y reloj real	Capturar pendiente no autoriza confirmar sin medición.
V9	Lector A, captura dueña y fecha PATCH	Saca red de locks y preserva fecha omitida.
V10	Posiciones disponibles para venta	Estado solo no basta si hay reserva o baja.
V11	Guarda de versión al cancelar /anular	Un lock no detecta por sí solo intención obsoleta.
V12	Tickets y reportes canónicos	Conserva composición propia y margen histórico.
V13	Igualdad de conjunto en corrección	Rechaza omisiones/adiciones antes de tocar original.

11.3 3. Leer PK, FK, NN y UK como mecanismos distintos

PK identifica la fila y obliga a que sus columnas no sean NULL. NN, NOT NULL, exige un valor. FK obliga a que la combinación referenciada exista. UK, UNIQUE, impide duplicados de una co mbinación. Una UK que contiene la PK y un atributo redundante puede servir como destino de una FK que también comprueba tipo, pero no es necesariamente una clave candidata mínima. La minimalidad exige que no sobre ninguna columna.

En los hechos locales se utiliza con frecuencia la identidad compuesta E,S,id: empresa_id, suc ursal_id e identificador. Una FK local repite empresa y sede además del UUID. Esto evita que un pago de una sede encuentre una liquidación de otra simplemente porque su UUID existe. Un UUID difícil de adivinar tampoco concede autorización: estructura y permisos son garantías distintas.

Las entidades dependientes usan claves parciales. numero_lote identifica un lote dentro de una operación; numero_version identifica una propuesta dentro de un contrato o una versión dentro de un periodo. El mismo número puede existir en otro propietario. Presentar sólo lote1 sin operación es perder una parte esencial de la identidad.

Una FK opcional normalmente utiliza MATCH SIMPLE. E y S pueden ser obligatorios mientras d ocumento_id es NULL; al faltar el identificador opcional, la referencia se omite. Imponer MATCH FULL indiscriminadamente exigiría todos nulos o todos completos y rompería ese caso normal. Cuando el negocio exige exactamente una fuente, se aplica un CHECK XOR, como en asignacio n_costo, no una nulabilidad improvisada.

Los índices únicos parciales aplican a un subconjunto, por ejemplo posiciones activas. Los CHECK examinan valores de una fila. Los triggers controlan estados de padres o transicione s. Los comandos con FOR UPDATE protegen condiciones agregadas concurrentes. No se inter-cambian sin perder garantías: un CHECK de la aplicación nueva no puede verificar por sí solo la suma de otras aplicaciones concurrentes.

Las columnas obligatorias no vuelven obligatorio un dato distinto del que representan. Una fe-cha de creación NN no sustituye una fecha de reconocimiento elegida. Un actor NN no autoriza a utilizar un UUID inventado. Un documento opcional puede seguir siendo privado. En el diccio-nario íntegro de anexos deben leerse las claves completas y acciones referenciales, además de las marcas por columna.

11.4 4. Mapa de las42 tablas lógicas y las cuatro técnicas adicionales

Esta sección explica por qué existe cada tabla y qué columnas deben reconocerse al recorrer el sistema. El anexo de diccionario conserva todas las columnas y declaraciones; aquí se sigue su función. Se utiliza E para empresa_id y S para sucursal_id. Los nombres de módulos del modelo lógico no son esquemas físicos separados: las tablas de negocio están en bovi y el soporte explícito indicado en infra.

11.4.1 4.1 Organización, acceso y maestros centrales

empresa. Su PK es id; razon_social, estado, nodo_origen y creado_en completan organización y autoría, mientras rfc admite ausencia declarada. Es raíz del scope, no una etiqueta que pueda cambiarse para habilitar datos de otra empresa. La instalación configura una empresa; disponer de claves empresariales no acredita RLS multiempresa comercial.

sucursal. PK E,id y UK E,codigo distinguen identidad técnica de código humano. nodo_dueno fija A o B; zona_horaria fija contexto de calendario. La FK empresa conecta sede con organizac ión. Es maestro escrito en A aunque describa una sede cuyo dueño financiero sea B. Dueño no significa custodio físico del ganado.

rol_aplicacion. PK codigo y descripcion forman catálogo de roles humanos. ADMIN, OPERADOR, CAJA y AUDITOR no equivalen a cuentas PostgreSQL. La descripción permite comunicar responsabilidades; los permisos se materializan cuando una membresía vigente conecta usuario y sede con ese código.

usuario. PK E,id y UK E,login_normalizado mantienen identidad de acceso. nombre y estado explican presentación y vigencia. hash_contrasena es privado y contiene bcrypt, no texto legible. La autenticación tiene cuenta técnica separada para leerlo. Desactivar usuario limita solicitudes nuevas, pero no cambia actor de auditorías históricas.

membresia_usuario. PK E,S,usuario_id,rol_codigo expresa una relación ternaria. Sus tres FK conectan sede, persona y catálogo de roles. vigente permite retirar autorización sin borrar usuario. La misma persona puede tener varios roles o permisos diferentes entre sedes. El comando de asignación exige ADMIN y no permite elegir otra S en el cuerpo.

parte. PK E,id identifica contraparte. tipo_persona, nombre, telefono y es_sintetico ayudan a explicar declaración y DEMO. El RFC tiene unicidad parcial cuando existe. Parte no se divide en proveedor y cliente excluyentes: puede vender y comprar. Una liquidación conserva snapshots para que un renombre posterior no reescriba el comprobante.

predio. PK E,id identifica lugar declarado. nombre es obligatorio; clave_upp_declarada, municipio, entidad_federativa y parte_responsable_id aportan información opcional. La FK al responsable conecta declaración comercial. Capturar una UPP no demuestra consulta o certificación oficial; el término declarada evita atribuir una validación que no existe.

animal. PK E,id mantiene identidad permanente y UK E,sede_operadora_id,id permite comprobar pertenencia operativa. sexo y situacion son datos obligatorios; raza y fecha de nacimiento son declaradas. La FK hacia sucursal fija sede autorizada. VIGENTE, IDENTIDAD_EN_REVISIO N y MUERTO tienen efectos distintos sobre nuevos compromisos.

animal_identificacion. PK E,id y UK E,sistema,codigo_normalizado resuelven colisiones. cod igo_declarado conserva lo escrito; el normalizado facilita comparación. Las FK enlazan animal y evidencia documental opcional con su sede. La normalización no acepta símbolos arbitrarios: primero valida y luego elimina separadores. Escritura central A hace competir las solicitudes en una misma unicidad.

11.4.2 4.2 Intenciones comerciales y condiciones versionadas

operacion. PK E,S,id, UK de tipo/folio y una UK adicional id/tipo describen el agregado. parte_id y referencias opcionales a predios conectan contexto. tipo, moneda, fecha_operacion, estado y estado_entrega no mezclan dinero con entrega. version_lock controla concurrencia; numero_ve rsion_aceptada enlaza la versión contractual mediante FK diferible.

folio_contador. PK E,S,tipo,anio identifica un contador interno. ultimo cambia mediante UP-SERT transaccional. Su FK hacia sede mantiene contexto. Este soporte aparece en el modelo lógico aunque no sea entidad ganadera. Un folio puede tener huecos y no constituye folio fiscal autorizado.

lote_operacion. PK E,S,operacion_id,numero_lote expresa dependencia. descripcion es una etiqueta opcional; FK identifica la operación. El lote agrupa selección y condiciones. Cambiar su ordinalidad cuando existe historia puede aplicar el precio de un grupo a otro, por lo que identidad de lote y composición deben conservarse al editar.

contrato_version. PK E,S,operacion_id,numero_version distingue propuestas. estado, politica _redondeo y requiere_recepcion definen decisión contractual; documento_id es evidencia opcio-nal. aceptada_por y aceptada_en son nulos antes de aceptar. La FK a operación y la documental conservan contexto. ACEPTADA se protege contra edición o borrado.

condicion_lote. Su PK combina operación, versión y lote. FK hacia contrato, lote y pesaje unen precio y evidencia. unidad_precio, precio_unitario, deduccion_pct, cargos_pactados y bonific aciones_pactadas explican fórmula. pesaje_base_id puede faltar en borrador KG desde V8; las guardas de acuerdo y confirmación exigen completarlo, no simulan un total cero.

11.4.3 4.3 Selección, ciclos y compromisos físicos

seleccion_compra. PK incorpora operación, lote y animal; UK adicional impide repetir animal entre lotes del mismo trato. La FK con tipo_operacion comprueba COMPRA y la FK hacia animal incluye sede operadora. Seleccionar aún no crea inventario: representa intención cuya adquisi-ción se reconoce al confirmar.

posicion_inventario. PK E,S,id y UK de animal/ciclo mantienen historia de adquisición. Otra clave vincula compra, lote y animal. El índice parcial impide dos posiciones activas por animal en empresa. estado distingue pendiente de recepción, disponible, vendida pendiente de entrega, cerrada y baja. El costo es derivado, no un saldo manual editable en la fila.

seleccion_venta. PK incluye operación, lote y posicion_id. La UK impide repetir posición dentro de una venta y la FK de tipo exige VENTA. Se vende un ciclo concreto, no una identidad animal ambigua. Una venta puede reunir posiciones de varias compras sin perder el costo de origen de cada una.

reserva_venta. PK E,S,id y FK a selección exacta vinculan compromiso con venta/lote/posició n. creada_en, vence_en, cerrada_en y estado explican duración. Un índice único parcial limita ACTIVA por posición. El índice no utiliza now: vencer requiere transición/comprobación explíci ta. Reservada se deriva, no es otro estado persistido de posición.

movimiento_inventario. PK E,S,id y FK hacia posición, operación, predio, custodio, documento y original de reverso conservan hecho físico. ocurrido_en expresa cuándo ocurrió, no cuándo se reconoció una venta. Recepción y entrega no generan deuda ni costo adicional. Baja crea pérdida mediante comando y evidencia, conservando identidad y ciclo.

11.4.4 4.4 Mediciones y expediente documental

pesaje. PK E,S,id y FK hacia lote/documento/original conservan ticket. bruto_kg, tara_kg y ne-to_kg tienen precisión de tres decimales; CHECK impide valores incoherentes. modo, fuente, bascula, ticket_folio y pesado_en explican procedencia. APROBADO congela; CORREGIDO con-serva original y otro ticket apunta a corrige_a_id.

pesaje_animal. PK E,S,pesaje_id,animal_id y dos FK materializan el conjunto medido. peso_individual_kg admite NULL porque una báscula grupal no midió necesariamente cada individuo. Ese NULL no se rellena dividiendo el neto. La aprobación congela composición y la corrección conserva conjunto exacto.

documento. PK E,S,id, unicidad de clave de almacenamiento y enlace al documento previo conservan versiones. SHA256, tamaño y MIME describen bytes privados almacenados fuera de BD. Fechas, emisor, folio, origen y destino son datos declarados. Una revisión de v1 no se transforma en aprobación de v2.

operacion_documento. PK E,S,operacion_id,documento_id evita relación duplicada. rol explica función de evidencia y ambas FK mantienen scope. Asociar archivo no confirma importe, deuda ni movimiento. El servidor controla descarga; conocer una clave interna no autoriza obtener los bytes.

revision_administrativa. PK E,S,id y CHECK XOR exigen operación o identificación, exactamente una. criterio, resultado, responsable_id y revisado_en explican revisión. revision_anterior_id conserva continuidad. El encabezado no acredita certificación sanitaria ni permite editar una liquidación confirmada.

revision_documento. PK E,S,revision_id,documento_id enlaza versión exacta. resultado y observacion guardan juicio sobre lo revisado. Las dos FK sostienen asociación; una versión documental nueva deja intacta esa referencia. Es historia administrativa protegida, no una bandera mutable que siempre apunta al último archivo.

11.4.5 4.5 Obligaciones, costos y calendario

periodo_cierre. PK E,S,id, FK a sede, fechas inclusivas, estado y version_lock definen mes. EXCLUDE GiST impide superposición; política complementaria mantiene un abierto y meses consecutivos. La fila es mutex común para cierre y reconocimiento, evitando que un costo se cuele después del corte bajo fecha documental antigua.

cierre_periodo_version. PK E,S,periodo_id,numero_version y FK al periodo conservan CIERRE o REAPERTURA. actor_id, motivo, corte_en, hash_conciliacion y totales explican decisión. V12 añade snapshot de reporte_margen al JSON de cierres nuevos. Reabrir no borra versiones previas ni inventa un desglose inexistente de un cierre antiguo.

liquidacion. PK E,S,id y UK por operación permiten una obligación principal. FK al contrato aceptado y periodo conservan reconocimiento. total, direccion y snapshots son inmutables; vence_el puede ser NULL. El saldo se deriva de ajustes y aplicaciones; generar un PDF o recibir dinero no modifica el total original.

concepto_liquidacion. PK E,S,liquidacion_id,numero_concepto identifica desglose dependiente. cantidad, unidad, precio_unitario e importe explican BASE, CARGO o BONIFICACION. Lote y pesaje aportan evidencia. Una bonificación pactada puede materializar importe negativo según tipo; los signos de valoración continúan derivados, no introducidos arbitrariamente.

ajuste_liquidacion. PK E,S,id enlaza liquidación, documento, original de ajuste y periodo. naturaleza CARGO/ABONO da signo a monto positivo. motivo, fuente y actor justifican corrección. No reescribe liquidacion.total. Una reducción de deuda y una bonificación de costo son intenciones distintas aunque sus cifras coincidan.

gasto_operacion. PK E,S,id y FK a operación, proveedor, documento, gasto bonificado y periodo mantienen causa. naturaleza y clasificacion separan costo/bonificación y capitalizable/directo/otro. La referencia de proveedor permite detectar duplicados declarados. snapshot_fuente_externa conserva tarifa aceptada; su importe no se vuelve a consultar en FDW dentro de locks.

asignacion_costo. PK E,S,id y FK hacia posición y fuentes permiten seguir cuotas. Un CHECK XOR exige concepto, gasto o ajuste como única fuente. metodo, base_valor y lote_reparto_id explican distribución. periodo_id ancla efecto. La suma del reparto debe ser exactamente igual al importe, conservando residuo en centavos determinista.

asiento_valoracion. PK E,S,id enlaza posición, periodo, asignación y operación. clase, sentido e importe generan delta_inventario, delta_costo_venta y delta_perdida. Los deltas no son campos libres del formulario. Un costo tardío de compra puede atribuirse a la venta original de la posición ya vendida; una baja atribuye pérdida, no venta.

11.4.6 4.6 Dinero, historia e integración

pago. PK E,S,id conecta parte, origen de reembolso, documento y periodo. tipo es COBRO/PAGO; naturaleza NORMAL/REEMBOLSO, un eje distinto. REGISTRADO mantiene periodo y fecha_reconocimiento nulos hasta validar. importe, método, referencia y recibido_el describen caja. Disponible es derivado y puede quedar como anticipo sin fabricar ingreso adicional.

aplicacion_pago. PK E,S,id y FK hacia pago, liquidación, original de reverso y periodo modelan N:M con identidad. naturaleza APLICACION/REVERSO da signo a importe positivo. reverso_de_id único impide segundo contramovimiento del mismo original. FK no basta: el comando verifica contraparte, moneda, dirección, saldo y disponible bajo locks.

auditoria_evento. PK E,S,id conserva actor, comando, correlación, entidad y cambio JSON. El resultado se inserta junto al efecto, no después en otra transacción. Desde V13 el motivo de corrección puede recuperarse del resultado auditado del ticket nuevo. Los logs técnicos no deben copiar secretos por el hecho de existir auditoría interna.

evento_outbox. PK E,S,id guarda tipo, versión de payload, datos y propietario de despacho. creado_en no demuestra entrega. El evento nace con el hecho económico o físico y es inmutable. La tabla de entrega separa reintentos locales. La muerte en B puede actualizar A después, sin retener red dentro de la transacción de pérdida.

mensaje_inbox. PK consumidor,fuente_id,event_id deduplica consumo local. resultado y recibido_en conservan respuesta. No se publica como negocio replicado. Consumir muerte central no reconoce otra pérdida: devuelve resultado si el evento ya fue aplicado y permite responder a un acuse perdido.

clave_idempotencia. PK E,S,principal_id,comando,clave delimita intención. hash_peticion, estado y resultado distinguen replay de conflicto. expira_en es infinity en el comando actual. La persistencia es permanente para deduplicación financiera, no caché de sesión. Se consulta sólo por el principal correspondiente y después de autorización vigente.

infra.nodo_config. PK nodo y empresa_id definen identidad técnica local. No se replica: A y B deben seguir reconociéndose como sí mismos. La app no modifica este soporte para adjudicarse otra sede. Una restauración aislada debe reconstruir conexiones, no reutilizar suscripciones hacia el proyecto original.

infra.entrega_outbox. PK evento_id,destino conserva intentos, próximo intento, acuse, error y lease. Los nombres físicos son lease_token y lease_vence_en. No tiene FK declarada al evento en este catálogo: funciones validan su relación. La lease durable termina antes de red y un acuse antiguo no puede confirmar trabajo reclamado por otro.

infra.importacion_fuente. PK id y datos de empresa/sede/fuente/hash/corte/conteos documentan procedencia. CSV/OCR/FDW son vocabulario, no prueba de OCR implementado. Es soporte local; no atribuirle FK ausentes. Importar una propuesta no autoriza confirmar automáticamente importes.

11.4.7 4.7 Las cuatro tablas físicas adicionales

infra.tarifa_snapshot. PK id y FK E,actor_id→usuario y E,S→sucursal añaden dos referencias . ruta, precio numeric(14,2), hash_sha256, snapshot, capturado_en y aceptada_en representan captura privada. Precio fuente de cuatro decimales queda en JSON; el monto canónico a centavos se compara con gasto. El token sólo vale para actor/sede/empresa de captura.

infra.spring_session. PK primary_id y unicidad de session_id sostienen sesión JDBC. creation_time, last_access_time, max_inactive_interval, expiry_time y principal_name son técnicos. No

se replican ni sustituyen membresía vigente. El rol auth puede modificar sesiones; la app de negocio no. Restore invalida estado efímero.

infra.spring_session_attributes. PK session_primary_id,attribute_name y FK a spring_sessio n con CASCADE preservan atributos serializados. attribute_bytes contiene bytes privados que no se inspeccionan en el manual. CASCADE es adecuado para sesiones, no una autorización de borrar hechos económicos confirmados.

infra.flyway_schema_history. PK installed_rank y columnas version, description, type, script, checksum, installed_by/on, execution_time y success explican migración. No tiene FK empresa rial. Su éxito registra DDL/adopción, no todasRN aprobadas. La entrada baseline3 conserva la verdad de instalación existente.

11.5 5. El protocolo de un comando SQL

La firma vigente recibe ocho argumentos y retorna JSONB:

```
bovi.cmd_ejecutar(
    p_empresa uuid, p_sucursal uuid, p_actor uuid,
    p_clave text, p_hash text, p_correlation uuid,
    p_comando text, p_payload jsonb
) returns jsonb
```

No son ocho datos comerciales elegidos libremente por el formulario. Empresa y sede se resuelven en contexto autorizado; actor proviene de sesión; clave identifica intención; hash lo calcula el servidor sobre cuerpo canónico; correlation relaciona seguimiento; comando selecciona rama; payload aporta datos tipados de esa intención. El backend no acepta una página de proveedor fabricada por el navegador como procedencia auténtica.

La función es SECURITY DEFINER y se ejecuta con derechos del rol técnico dueño de comandos. Eso permite insertar hechos sin conceder DML directo a la app. También obliga a ser extremadamente explícito con scope y objetos cualificados. search_path=pg_catalog y referencias bovi/infra evitan que un objeto de otro esquema sustituya una función esperada. PUBLIC no tiene EXECUTE.

El control fn_scope comprueba empresa configurada, usuario activo, membresía vigente y rol co mpatible. ADMIN satisface roles operativos específicos. Si la intención modifica maestros, exige A. En hechos locales exige que la sede tenga como dueño el nodo actual. No utiliza current_use r como persona: current_user es la identidad técnica de ejecución, especialmente en SECURITY DEFINER.

Después se valida clave de16-80 caracteres y hash hexadecimal64. INSERT ON CONFLICT reclama el espacio de intención; SELECT FOR UPDATE bloquea la fila idempotente. Si el hash difiere se rechaza BV405. Si estado es COMPLETADA se devuelve resultado persistido. Sólo una intención nueva alcanza la lógica del agregado y sus comprobaciones de versión.

Ese orden resuelve un caso central: confirmar con versión0 incrementa el agregado a1. Si se perdió la respuesta, repetir misma clave y cuerpo con versión0 debe devolver la liquidación original, no412. En cambio, una nueva clave intentando modificar versión0 de un agregado que ya está en1 debe fallar. Replay y nueva escritura obsoleta son intenciones distintas.

Cada rama compone su resultado antes del commit. Al terminar se inserta auditoria_evento, ev ento_outbox y se marca idempotencia COMPLETADA con ese mismo resultado. Todo pertenece a la transacción local del comando. Si una FK o una validación falla, rollback elimina también reclamo nuevo, auditoría y outbox de esa intención. No se anuncia un éxito parcial ni se consulta después del commit para construir otra respuesta.

Fuente: db/migrations/V13__correccion_composicion_exacta.sql:2.

11.5.1 5.1 Leer las variables locales sin confundirlas con columnas

e,s,a son alias de empresa, sede y actor. j es el payload normalizado; n es la identidad del nodo. result guarda el JSON de respuesta y prev es una fila tipada de clave_idempotencia. op,original,pos,pay,liq,app y gasto son filas %ROWTYPE de operación, ticket original, posición, pago, liquidación, aplicación y gasto. Usar %ROWTYPE permite que el registro local corresponda al tipo de la tabla, pero no implica que se haya leído o bloqueado: eso depende del SELECT concreto.

r y c son record flexibles para resultados de consultas; lot y x son elementos JSON. idn comienza como UUID nuevo y puede reasignarse al recurso retornado. opid identifica operación, pid identifica pago/posición/periodo según la rama, lid liquidación, wid ticket, aid asignación y per periodo. La reutilización de pid exige leer su contexto: no se debe seguir una variable sólo por su nombre suponiendo que siempre significa pago.

amount es monto de la acción; totaln puede ser total comercial o importe remanente de reaplicación; cost representa costo o cuota temporal según rama. cnt cuenta posiciones; k e idx recorren distribución o conceptos; seqn forma folio o versión de cierre; contractn identifica la versión c ontractual. ref agrupa reparto; ids es arreglo de identidades; clas y nature seleccionan vocabularios de clase y naturaleza; f conserva fecha contextual; preserve decide si existe historia que obliga a mantener composición.

La directiva variable_conflict use_column evita una ambigüedad al coincidir nombres de variables y columnas. No sustituye escribir consultas cualificadas ni entender qué valor se compara. Un SELECT INTO no es un UPDATE y tampoco un lock salvo que incluya FOR UPDATE. Una variable numérica exacta no prueba redondeo correcto si después se materializa otro dato con una política distinta.

El bloque de DECLARE sirve como mapa de lectura. Conviene recorrerlo y después señalar dónde se asigna cada variable importante. Para confirmar compra, seguir per→op→contractn→totaln →lid→pos→cost. Para aplicar dinero, seguir per→pid→pay→lid→liq→amount. Para corregir ticket, seguir opid derivado del original→op→original→j→idn→result. Ese ejercicio revela dependencias reales y evita una explicación genérica de “la función valida todo”.

11.6 6. Aritmética decimal, unidades y materialización

Una cantidad exacta no empieza en el reporte: empieza cuando se recibe el dato. El contrato Dinero utiliza un objeto con importe como cadena y moneda MXN. Peso utiliza kg como cadena. Una tarifa puede tener cuatro decimales; un importe confirmado se materializa a dos. PostgreSQL trabaja con NUMERIC y Java transporta BigDecimal; el navegador no envía un total calculado para que la BD lo acepte como verdad.

Ejemplos de representación:

```
{"importe":"12000.01","moneda":"MXN"}
{"kg":"4900.000"}
```

La cadena conserva escala y evita pasar por una aproximación binaria flotante. No basta con llamar decimal a una variable si después una conversión usa double para sumar. La autoridad que confirma importes es SQL. Java y UI presentan los resultados canónicos y mantienen el contexto de intención; no implementan otra fórmula que pueda divergir del resultado persistid o.

numeric(18,2) expresa precisión total18 y escala2, no dieciocho enteros más dos decimales. numeric(14,3) conserva masa con tres decimales; numeric(14,4) admite tarifas con cuatro. Una restricción de precisión limita magnitud representable. Los CHECK evitan montos inválidos, NaN o infinitos según el campo. Las funciones comprueban también condiciones semánticas como importe de aplicación positivo y ya redondeado a centavos.

11.6.1 6.1 Neto, variación física y deducción contractual

Para un ticket BRUTO_TARA, sean B la masa bruta en kilogramos, T la tara y N el neto. La ecuación y sus condiciones son:

$$N = B - T, \qquad 0 \le T \le B, \qquad N > 0.$$

Bruto incluye el elemento que se descontará; tara es esa parte y neto representa masa liquidable registrada. Un ticket NETO_REPORTADO conserva un neto declarado sin inventar bruto y tara. La precisión y procedencia siguen siendo explícitas. No debe llenarse bruto con neto sólo para evitar NULL, porque se estaría fabricando una medición distinta de la real.

Para comparar dos tickets, sean W_i el peso inicial y W_f el final. La variación y porcentaje son:

$$\Delta W = W_i - W_f, \qquad v = 100\,\frac{W_i - W_f}{W_i}, \qquad W_i > 0.$$

W_i y W_f se expresan en kg; v se expresa en porcentaje. Si el peso final es mayor, la variación es negativa y se conserva ese signo. Una ganancia de peso no debe convertirse a cero por comodidad visual. Antes de calcular se exige igualdad del conjunto de animales; dos totales de lotes distintos no producen una merma comparable.

En GOLD-01, W_i=5000 kg y W_f=4900 kg. La diferencia es100 kg y el porcentaje2. Ese2 % describe evidencia física. No autoriza descontar nuevamente2 % del precio. La deducción contractual d existe en condicion_lote y se aplica solamente si se pactó. Mezclar ambas variables produciría doble descuento.

Cuando la unidad comercial es KG:

$$Q_{\text{kg}} = N\left(1 - \frac{d}{100}\right).$$

Q es cantidad liquidable; N es el neto del ticket seleccionado y d es porcentaje pactado. Si N=1000 kg y d=1.5, Q=985 kg. Si no se pactó deducción, d=0 y Q=N. Este ejemplo enseña la fórmula, no agrega un descuento inexistente a GOLD-01. El porcentaje de variación sigue siendo información independiente.

Para CABEZA, Q=n es el conteo de animales o posiciones seleccionadas en ese lote. No se usa el conteo total de la sede ni el número de filas después de una unión duplicada. La selección tiene unicidad por operación y las FK de tipo impiden contabilizar una selección de venta como compra.

Fuentes: db/migrations/V2__comandos.sql:70; docs/domain/states-and-calculations.md:47; db/migrations/V12__pesajes_reportes_canonicos.sql:430.

11.6.2 6.2 Base por lote y total comercial

Sean P_l la tarifa del lote l, Q_l su cantidad liquidable, C_l los cargos y B_l las bonificaciones pactadas. Se materializan base y total:

$$A_l = \text{round}_2(Q_l P_l), \qquad L_l = A_l + C_l - B_l, \qquad L = \sum_{l=1}^{m} L_l.$$

A_l es base monetaria a centavos; L_l total del lote y L total de operación. m es número de lotes con condiciones de la versión relevante. Se redondea al materializar el concepto, no después de sumar una tarifa ya truncada. La política HALF_UP a dos decimales responde al reconoci-

miento de cada importe persistido. Cargos y bonificaciones están expresados como importes y se materializan como conceptos distintos.

El SQL selecciona la versión máxima de contrato cuando la operación sigue BORRADOR; para una operación acordada o confirmada utiliza la versión aceptada. Esta elección evita simular siempre versión1 cuando existe reapertura. Una propuesta nueva no cambia una liquidación confirmada: la liquidación conserva numero_version y snapshot_condiciones.

Para una compra de5000 kg a48MXN/kg, la base es240000.00. Con gastos de adquisición posteriores de12000.00, el costo de posiciones llega a252000.00, pero la obligación comercial de compra no se modifica automáticamente por esos gastos. Esa separación es esencial: costo gerencial y deuda con la contraparte no son la misma cifra.

En una venta de4900 kg a60MXN/kg, la base es294000.00. Si hay un gasto directo de venta de8000.00, reduce margen, no necesariamente total adeudado por comprador. Los cargos que sí se pactaron con comprador deben estar en condiciones o ajustes de obligación, no camuflados como gastos propios.

11.6.3 6.3 Reparto exacto y residuo determinista

El reparto no utiliza pesos que no fueron medidos. Se admiten POR_CABEZA, POR_PESO_MEDIDO e IMPORTE_INDIVIDUAL. En los dos primeros, el gasto G se distribuye con base w_i para cada posición i. Por cabeza w_i=1. Por peso w_i es medición individual aprobada de la evidencia compatible.

$$a_i^* = G\,\frac{w_i}{\sum_{j=1}^{n} w_j}, \qquad r_i = \mathrm{round}_2(a_i^*), \qquad R = G - \sum_{i=1}^{n} r_i.$$

a_i estrella es cuota exacta antes de redondear; r_i cuota a centavos y R residuo. R no desaparece y tampoco se asigna a una posición arbitraria. El SQL convierte a centavos y corrige las primeras filas ordenadas por posicion_id ascendente. El orden estable hace reproducible el resultado al consultar o reintentar la misma intención.

Para G=1.00 y tres posiciones iguales, las cuotas exactas son un tercio. Después de redondear resultan0.33,0.33,0.33; queda0.01. La primera posición recibe el centavo y las cuotas finales son0.34,0.33,0.33. Ese es GOLD-03, un esperado fijo. No se obtiene llamando a la misma función que se está probando para fabricar el oráculo.

La corrección también puede ser negativa: si el redondeo provisional excedió G, el algoritmo resta centavos según el mismo orden. Para una bonificación, el signo nace de naturaleza validada; el usuario no introduce deltas monetarios libres. El importe acumulado de una posición no puede volverse negativo para ocultar un reparto excesivo.

En POR_PESO_MEDIDO se consultan pesos individuales del ticket aprobado ligado a condiciones de la liquidación de compra. Si una posición no tiene medición individual positiva, se rechaza BV018. Un ticket grupal de300 kg para dos animales no demuestra150 kg para cada uno. La ausencia de medición es información significativa y debe permanecer NULL.

En IMPORTE_INDIVIDUAL, se envían todas las posiciones una sola vez con cantidades explícitas positivas a centavos. Las identidades deben pertenecer al conjunto y la suma debe igualar el gasto. No se permite omitir una posición para absorber residuo sin explicación. Una distribución0.40/0.60 de1.00 es válida si corresponde a todo el conjunto;0.40/0.59 no cuadra aunque “casi” llegue al total.

Fuente: db/migrations/V2__comandos.sql:64; db/migrations/V2__comandos.sql:99.

11.6.4 6.4 Saldo, disponible y margen son tres resultados diferentes

Para una liquidación, sean L_0 total confirmado original, C ajustes CARGO, B ajustes ABONO y A aplicaciones netas. El total ajustado y saldo son:

$$L_{\text{ajustado}} = L_0 + \sum C - \sum B, \qquad S = L_{\text{ajustado}} - A.$$

Aplicaciones netas significa suma de APLICACION menos suma de REVERSO. L_0 permanece en la fila histórica. S no se introduce en un UPDATE manual para cuadrar. Si se modifica una aplicación, cambia la vista del saldo al añadir un contramovimiento, no al borrar evidencia.

Para un pago normal con importe P_0, aplicaciones netas A_p y reembolsos F:

$$D = P_0 - A_p - \sum F.$$

D es disponible. Puede ser positivo aunque una obligación ya tenga saldo cero: es un anticipo o saldo a favor. Un reembolso disminuye D y representa flujo real de caja. No cambia la venta reconocida ni el costo del inventario. El nuevo pago de naturaleza REEMBOLSO conserva referencia al original y toma dirección opuesta: COBRO original produce PAGO de devolución, y viceversa.

Para margen de venta, sean V la venta neta reconocida, K el costo de posiciones vendidas y G_d los gastos directos:

$$M = V - K - G_d.$$

M no usa cobros como ingreso. Un comprador que aún no pagó puede haber generado venta y margen reconocidos; dinero recibido como anticipo sin venta no genera margen. Separar devengo gerencial y caja evita que el reporte mejore o empeore sólo porque se aplicó un pago.

Los reportes calculan sobre todo el conjunto filtrado, no sobre la primera página visible. Si la cartera muestra25 filas de200, la UI no suma esas25 como si fueran toda la sede. La consulta y el contrato indican paginación, corte y fuente. El margen agrega el filtro completo y conserva faltantes, evitando etiquetar una cifra provisional como cierre definitivo.

11.6.5 6.5 Asientos de valoración y signos derivados

C es costo acumulado de una posición. Una adquisición genera inventario+C. Venta genera inventario−C y costo vendido+C. Baja sin venta genera inventario−C y pérdida+C. Un costo tardío δ se dirige al eje correspondiente al estado actual del ciclo. Una bonificación produce efecto contrario bajo límites que evitan un costo acumulado negativo.

Hecho	Delta de inventario	Otro efecto
Adquisición C	+C	Sin costo vendido ni pérdida.
Venta C	−C	Costo vendido+C.
Costo tardío capitalizado δ	+δ	Sin nueva deuda por sí solo.
Costo tardío ya vendido δ	0	Costo vendido+δ.
Baja C	−C	Pérdida+C, sin venta.
Costo tardío tras baja δ	0	Pérdida+δ.
Anulación antes de entrega C neto	+C neto	Costo vendido−C neto.

Las columnas delta_inventario, delta_costo_venta y delta_perdida son generadas. Clase y sentido determinan su fórmula a partir de importe. Un usuario no puede enviar tres números contradictorios para que un reporte aparente cuadrar. Los asientos preservan operación y posición para atribuir costo tardío a la venta original, incluso si su fuente administrativa fue la compra.

11.7 7. Recorridos reales de comandos

11.7.1 7.1 Maestros: crear sin duplicar identidad

crear_contraparte y crear_predio escriben en A después de fn_scope con modo maestro. Regist rar_animal crea identidad y recorre todo el arreglo de identificaciones, no sólo la primera. Cada código se valida y normaliza; la UK de sistema/código resuelve colisiones en el escritor central. Si una identificación posterior del mismo comando colisiona, rollback retira también animal e identificaciones anteriores de esa intención.

crear_usuario recibe un hash bcrypt generado por el backend, nunca acepta del cliente una contraseña ya presentada como hash para eludir políticas del servidor. actualizar_usuario conserva empresa y comprueba existencia. asignar_membresia enlaza usuario y rol con la sede au torizada. La cuenta ADMIN de una sede no puede introducir sucursal_id de otra en el cuerpo y aprovechar que el maestro vive en A.

Las altas maestras se replican hacia B. Si B todavía no observa una identidad, no debe inventar una copia para continuar compra. El servidor espera o informa disponibilidad según la opera ción. La replicación asíncrona tiene visibilidad eventual; la unicidad canónica no se consigue permitiendo que ambos nodos inventen filas y después las comparen por nombre.

11.7.2 7.2 Borrador, edición y acuerdo

crear_operacion obtiene folio mediante contador, inserta cabecera BORRADOR y contrato PRO-PUESTA versión1. Los lotes se numeran en orden del arreglo. La rama coloca selección de ani-males en compra o posiciones en venta, y condiciones por lote. Si se aporta neto para KG, se crea ticket propuesto con composición correspondiente, todavía sin aprobación implícita.

editar_operacion exige estado BORRADOR y version_lock actual bajo lock de operación. Una ACORDADA requiere reabrir, no PATCH directo. Si existe historia de pesajes o aceptación pre-via, preserve exige que el conjunto lote/elemento entrante coincida con el anterior. Se pueden cambiar condiciones de la propuesta vigente sin borrar condiciones aceptadas. No se elimina historia para facilitar el formulario.

Cuando no existe historia, la edición puede reconstruir selecciones y lotes del borrador. Cuando sí existe, conserva composición y ordinalidad. La política requiere_recepcion se cambia sólo si llega explícitamente. fecha_operacion también se conserva cuando PATCH omite ese cam-po: usar current_date como reemplazo automático habría alterado una fecha documental sin intención del usuario.

acordar_operacion bloquea operación, verifica BORRADOR/version y condiciones, acepta con-trato y cambia a ACORDADA. El trigger de V8 completa validación de acuerdo: no admite KG sin ticket aprobado y conjunto compatible. reabrir_operacion verifica ACORDADA/version y clona contrato/condiciones hacia la siguiente propuesta, incrementando version_lock y devolviendo BORRADOR. El contrato anterior queda ACEPTADA e inmutable.

11.7.3 7.3 Ticket propuesto, aprobación y corrección V13

registrar_pesaje bloquea operación y admite únicamente contexto editable BORRADOR/ACO RDADA. Calcula neto de bruto/tara cuando corresponde o conserva neto reportado. Inserta el conjunto indicado con pesos individuales reales o NULL. Aprobar exige composición y cambia PROPUESTA a APROBADO. Esa aprobación congela medición y asociaciones mediante triggers.

corregir_pesaje deriva opid del ticket original indicado por ruta/contexto. Después bloquea operación y original en ese orden. El original debe estar APROBADO. Si se envía operacion_id o numero_lote incompatible con el original, rechaza; no permite usar una ruta para modificar el estado de un ticket y crear el nuevo bajo otra operación.

El motivo es obligatorio, recortado y de1-200 caracteres. V13 calcula diferencia de conjuntos original−nuevo y nuevo−original. Si cualquiera contiene elementos, rechaza BV013 antes de

UPDATE del original. Un subconjunto de dos animales reducido a uno no pasa; un superconjunto que añade un tercero existente tampoco. Validar sólo FK habría permitido un tercero válido, pero ajeno al ticket.

Después de las guardas se cambia únicamente estado del original a CORREGIDO. Se crea ticket nuevo PROPUESTA con corrige_a_id, la misma composición y mediciones nuevas. En BORRADOR puede actualizarse la referencia de condición vigente al nuevo ticket KG; una condición aceptada no se reescribe por esa vía. El ticket corregido requiere su propia aprobación antes de utilizarlo.

El motivo forma parte del resultado del nuevo ticket y ese resultado se guarda en auditoria_evento.cambio e idempotencia. fn_pesaje_json lo recupera por entidad_id y comando corregir_pesaje. Un ticket antiguo sin motivo auditado devuelve motivo_correccion NULL; no se fabrica una justificación histórica. Replay ocurre antes de volver a exigir original APROBADO, pues el original ya está CORREGIDO después del primer éxito.

Fuentes: db/migrations/V13__correccion_composicion_exacta.sql:99; db/migrations/V12__pesajes_reportes_canonicos.sql:2.

11.7.4 7.4 Reserva y reloj de confirmación

reservar_venta bloquea operación y posiciones por id ascendente. Primero expira reservas activas cuyo vencimiento ya pasó. Comprueba estado DISPONIBLE, animal VIGENTE y ausencia de baja local. La nueva reserva tiene vencimiento explícito o valor por defecto de15 minutos desde reloj real. Un índice parcial impide dos ACTIVA para la misma posición.

liberar_reserva puede derivar operación a partir de reserva_id y procesa posiciones de la venta. Libera las ACTIVA relacionadas; no vende ni reconoce ingreso. Cancelar una operación libera reservas y cambia estado, sin crear liquidación. Un usuario que conserva pantalla antigua no puede interpretar una reserva liberada o vencida como compromiso vigente.

now permanece fijo al comienzo de transacción. Si una sesión empieza a las10:00 y espera un lock hasta10:16, una reserva que vencía10:15 podría parecer vigente al compararla con now. Por eso la confirmación usa clock_timestamp: vuelve a consultar reloj real durante el procedimiento. También hay una comprobación al final de consumo para detectar vencimiento ocurrido durante una confirmación larga. Si falla, toda la confirmación se revierte.

11.7.5 7.5 Confirmar compra y venta

La confirmación reconoce periodo explícito y bloquea su fila antes de los recursos financieros . Lee operación bajo lock, exige versión actual y estado BORRADOR/ACORDADA, comprueba lotes, cantidades y evidencia. Los tickets KG deben estar aprobados y tener el conjunto selec cionado. El contrato actual se acepta si todavía era propuesta, y se calcula total por función canónica.

Se inserta liquidación principal con dirección según tipo: compra POR_PAGAR, venta POR_COBRAR. Se copian nombre/RFC de contraparte y condiciones. Se materializan conceptos base, cargos y bonificaciones. Las filas y snapshots corresponden a la versión aceptada, de modo que una tarifa o nombre posterior no recalcula la obligación.

En compra, se bloquea identidad de cada animal en orden estable, exige VIGENTE y no habilita otro ciclo frente a una baja pendiente de revisión. Se crean posiciones con numero_ciclo siguiente. requiere_recepcion decide PENDIENTE_RECEPCION o DISPONIBLE. Los conceptos se distribuyen a posiciones con residuo exacto, y los asientos ADQUISICION reconocen costo inicial positivo.

En venta, las posiciones se bloquean por id y cada una exige DISPONIBLE y reserva ACTIVA vigente por clock_timestamp. Se toma su inventario acumulado como costo, se crea asiento VENTA, se cambia a VENDIDA_PENDIENTE_ENTREGA y se consume reserva. La entrega pos-

terior cierra físicamente el ciclo, sin crear otro asiento de venta. El resultado canónico es la liquidación completa y se persiste tal cual para replay.

11.8 8. Dinero aplicado, reversos y correcciones sin borrar originales

11.8.1 8.1 Registrar y validar un pago son actos distintos

registrar_pago crea una fila REGISTRADO de naturaleza NORMAL. La rama vigente deja periodo_id y fecha_reconocimiento NULL. Su importe y referencia describen dinero declarado; no descuenta una obligación ni reconoce venta. Exigir ya un cierre abierto para una simple captura de dinero habría confundido registro con validación. El estado permite comprobar esa separación en la consulta.

validar_pago exige CAJA y llama fn_periodo_payload con periodo y fecha elegidos. Cambia un REGISTRADO a VALIDADO y fija reconocimiento. Si no encuentra el pago en ese estado, rechaza. La validación no crea una aplicación implícita: el usuario puede tener un pago validado completamente disponible, por ejemplo un anticipo. La venta y su costo continúan independientes de esta caja.

El resultado de registrar, validar o reembolsar se construye desde v_disponible_pago, se normaliza a JSON decimal y conserva pago_id. Se guarda como respuesta del comando. No se debe presentar un disponible inventado por la UI sumando únicamente las aplicaciones que aparecen en una página.

11.8.2 8.2 Aplicar con saldo y disponible bajo locks

aplicar_pago bloquea primero periodo, luego pago y las obligaciones en orden estable de identidad. pay es la fila de dinero; liq la obligación; amount el monto de la aplicación. La rama admite un arreglo de aplicaciones y ordena por liquidacion_id. Esa jerarquía reduce el riesgo de que dos comandos bloqueen los mismos recursos en órdenes opuestos.

Antes de cada inserción comprueba pago VALIDADO y NORMAL. Pago y obligación deben pertenecer al mismo E/S, a la misma contraparte y moneda, y tener dirección compatible: un COBRO satisface POR_COBRAR; un PAGO satisface POR_PAGAR. La FK conserva scope estructural, pero no prueba esa compatibilidad semántica de dirección y contraparte.

La cantidad debe ser positiva, estar a centavos y no superar ni disponible del pago ni saldo de la obligación. Estas sumatorias se leen mientras se conservan los locks. Si dos conexiones compiten por pago600 y cada una pretende aplicar400, una puede aplicar400; la otra observa disponible200 y falla. Revisar disponible sólo antes del lock permitiría que ambas vieran600 y juntas aplicaran800.

El loop de aplicaciones pertenece a una sola transacción. Si la segunda obligación del arreglo es incompatible, no se conserva una primera aplicación parcial como éxito del comando. Se revierten todas las inserciones de esa intención. Esto facilita que un reintento de la misma clave tenga un significado definido.

11.8.3 8.3 Reversar no equivale a eliminar

revertir_aplicacion busca el original de naturaleza APLICACION y bloquea pago y liquidación asociados. Inserta una fila REVERSO con el mismo importe y reverso_de_id. La UK sobre referencia impide repetir reverso del mismo original. El CHECK exige que naturaleza REVERSO corresponda a un original enlazado.

Las vistas usan signo por naturaleza. Si existía aplicación1000, el reverso1000 lleva la suma neta a0 y recupera disponible. Las dos filas siguen explicando qué se hizo y qué se corrigió. DELETE

habría borrado esa explicación y perjudicado reconciliación entre saldo histórico, auditoría y c omprobantes.

No existe un permiso general de “cambiar importes históricos”. El comando de reverso utiliza actor, periodo y contexto actuales. Un reverso reconocido este mes puede afectar el saldo vivo de una obligación anterior sin modificar el corte ya guardado del periodo anterior. La cronología de reconocimientos continúa verificable.

11.8.4 8.4 Bonificación de obligación completamente pagada: GOLD-07

Considérese obligación1000 y pago1000 aplicado. Se solicita ABONO100. Si simplemente se insertara el ajuste, la obligación ajustada quedaría900 frente a aplicaciones netas1000. El modelo rechaza esa sobreaplicación. El comando guiado permite reaplicar_pagos:true para ejecutar la conciliación completa de manera atómica.

La rama ajustar_liquidacion obtiene periodo, bloquea los pagos involucrados por id y después liquidación. Comprueba que el abono no exceda total ajustado. Si excede saldo actualmente disponible para abonar, requiere la opción explícita de reverso y reaplicación. totaln se convierte en la obligación remanente después del abono, no en un nuevo total original.

Se recorren aplicaciones vigentes sin reverso por aplicada_en e id. Cada una se contrarresta. cost se usa aquí como cantidad que se reaplica: least(app.importe,totaln). Su nombre no significa costo ganadero en esta rama; leer la asignación revela el significado. El remanente disminuye después de cada reaplicación.

En el esperado fijo, reverso1000 recupera dinero y aplicación900 lo vuelve a vincular; ajuste ABONO100 deja obligación900 y saldo0. Disponible queda100. Reembolsar100 crea flujo contrario de caja y devuelve disponible0. Todos los pasos de ajuste/reconciliación se confirman juntos; el reembolso es una intención posterior de dinero real, no una eliminación de aplicació n.

Si reaplicar_pagos:false o ausente no autoriza la conciliación necesaria, la función devuelve BV020. Un fallo intermedio revierte reversos, nuevas aplicaciones y ajuste. El usuario no debe quedar frente a un estado temporal confirmado donde el pago desapareció o la obligación quedó negativa.

Fuente: db/migrations/V13__correccion_composicion_exacta.sql:214; db/tests/gold.py:96.

11.9 9. Gastos, costo tardío, baja y anulación

registrar_gasto exige periodo abierto para su reconocimiento contextual y crea un gasto BORRADOR. Si utiliza proveedor externo, recibe snapshot_id de una captura aceptada del mismo E/S/actor; compara el importe con el precio canónico local. No permite que el navegador envíe snapshot_fuente_externa arbitrario sin token. El JSON copiado al gasto conserva procedencia y aceptación.

repartir_gasto o registrar_costo_tardio vuelve a validar periodo, bloquea gasto y exige BORRADOR. Una BONIFICACION debe enlazar costo aprobado de misma operación y clasificación, sin superar saldo de bonificación permitido. Así se evita reducir tres veces un flete que sólo se reconoció una vez. El gasto original sigue consultable.

Cuando clasificación es CAPITALIZABLE, se obtienen posiciones de esa compra en orden y se bloquean. fn_reparto determina cuotas por método. Cada asignación conserva fuente y grupo. Si una cuota es cero no se inventa asiento monetario positivo; si haría negativo el costo acumulado, falla. La clase del asiento depende del estado de posición: inventario, venta reconocida o baja.

Un detalle importante es opid en costo tardío de venta. Aunque el gasto se registró sobre la compra, la posición ya vendida requiere atribuir delta_costo_venta a su venta original. La rama obtiene esa operación desde asiento VENTA. Sin esa reasignación, un reporte filtrado por venta omitiría un costo tardío real porque su origen administrativo era compra.

GOLD-02 comienza con10 animales a1000, adquisición10000 y gasto1000 distribuido:1100 por posición. Vender4 a1600 produce6400, costo4400 y gasto directo200; margen1800, inventario6 600. GOLD-06 añade500 a la compra,50 por posición: costo vendido4600, inventario6900 y mar gen1600. Bonificar200,20 por posición, reduce costo vendido a4520, inventario6780 y aumenta margen a1680. Las deudas comerciales originales no cambian automáticamente por esos gastos propios.

Una baja exige ADMIN, posición libre DISPONIBLE o PENDIENTE_RECEPCION, motivo permitido y documento existente del scope. Se rechaza si hay reserva vigente. Se toma costo capitalizado, se inserta asiento BAJA y se cambia posición a BAJA con motivo/cierre. GOLD-08 conserva costo1100 convertido en pérdida1100, inventario0 y, después de un costo tardío50, pérdida115 0. No hay venta ni cuenta por cobrar inventada.

Si motivo es MUERTE y el nodo es A, se actualiza identidad central dentro del contexto autoriz ado. Si es B, se crea ANIMAL_MUERTO junto a pérdida local y se entrega después. Una réplica que todavía no ve animal MUERTO no permite eludir la baja local mediante nueva adquisición: el comando y los estados conservan esa protección operativa.

anular_venta no es cancelar borrador. Exige venta CONFIRMADA, versión actual y ninguna entrega asociada. Bloquea periodo, operación, posiciones y liquidación. Las aplicaciones deben estar revertidas previamente: saldo debe coincidir con total ajustado. El comando restaura a inventario el costo neto de cada posición, crea contrasiento de venta y abona obligación para llevarla al efecto correspondiente, conservando liquidación original.

En ANUL-01, costo inicial vendido1100 más tardío100 se anula antes de entrega. El contrasiento restituye1200 a inventario y deja costo neto de venta0. Tomar únicamente1100 habría ignorado el ajuste tardío; borrar la venta habría perdido explicación de la restitución. La posición vuelve a DISPONIBLE y el ciclo no se transforma en otro ciclo de adquisición.

Fuente: db/migrations/V13__correccion_composicion_exacta.sql:235; db/migrations/V13__corr eccion_composicion_exacta.sql:271; db/migrations/V13__correccion_composicion_exacta.sql:3 31.

11.10 10. Periodos: elegir reconocimiento y conservar historia

Una fecha documental no determina automáticamente el mes de reconocimiento. El sistema distingue fecha_operacion o fecha_economica, fecha_reconocimiento y tiempo técnico registra do_en. Un documento fechado en otro mes puede conocerse hoy y reconocerse explícitamente en un periodo abierto. Cambiar un campo documental no debe insertar retroactivamente costo en un periodo que ya se cerró.

crear_periodo exige ADMIN y toma un advisory lock por empresa/sede para serializar la creación del calendario. El primer mes se declara explícitamente. Debe comenzar en primer día del mes y terminar en último; un siguiente periodo sólo es consecutivo y no puede coexistir con otro abierto. EXCLUDE agrega protección frente a rangos superpuestos. La aplicación no crea automáticamente un mes al descubrir que falta.

fn_periodo recibe e,s,f,pid: empresa, sede, fecha de reconocimiento e identificador de periodo. Ambos f y pid son obligatorios. SELECT FOR UPDATE toma la fila exacta de E/S/id; debe existir, estar ABIERTO y contener f entre desde/hasta. Devuelve pid para asignarlo al hecho. fn_perio do_payload sólo adapta las propiedades JSON a esos argumentos, no elige una fecha silenciosa para resolver un error.

cerrar_periodo bloquea esa misma fila. Si costo tardío la bloqueó primero, el cierre espera y después observa el costo confirmado; si cierre ganó, el costo espera y al retomar encuentra CERRADO, falla BV025 y no deja asignación, asiento, auditoría o intención nueva parcial. Una nueva intención puede elegir el siguiente mes abierto. No se modifica la fecha de un intento antiguo para hacerlo pasar como replay igual.

El inventario de corte suma asientos asociados a periodos cuyo hasta es anterior o igual al periodo objetivo. No filtra simplemente asiento.fecha_economica contra la fecha final, porque en datos históricos ese nombre puede haber transportado significados diferentes y un documento futuro no debe excluir reconocimiento actual. La relación periodo_id es la autoridad del calendario de cada hecho.

Costo vendido y pérdida se suman por periodo reconocido. Cartera se lee del saldo vivo mientras el lock y la política de único periodo abierto impiden que otro reconocimiento posterior cambie el mismo corte concurrentemente. Esa coherencia es local de sede; no promete un snapshot distribuido simultáneo de A y B.

El cierre inserta cierre_periodo_version con número siguiente, actor, motivo, corte, hash y totales. El cambio de periodo a CERRADO pertenece a la misma transacción. Reabrir sólo el último cerrado sin periodo posterior/abierto crea versión REAPERTURA y conserva las anteriores. No hace UPDATE de un cierre aceptado para volver a presentar otras cifras.

11.10.1 10.1 El reporte histórico desde V12

Al cerrar se materializa reporte_margen dentro de totales. Una consulta de ese periodo cerrado sin búsqueda ni filtro de operación puede devolver el snapshot y su numero_version_cierre. La etiqueta CERRADO se utiliza sólo cuando no contiene faltantes. Un cierre que aún tuvo revisión o gasto pendiente conserva PROVISIONAL, aunque su calendario esté cerrado.

Un caso QA fijó margen90 al cerrar. Se reconoció después costo20 en el siguiente periodo; el margen vivo bajó70, pero la consulta del cierre anterior siguió devolviendo el mismo snapshot90. No se cambia de etiqueta para ocultar diferencia: son dos preguntas legítimas, el corte anterior y el estado actual.

Los cierres anteriores a V12 que no contienen reporte_margen no se reconstruyen como supuesta historia aceptada. Devuelven PROVISIONAL y CIERRE_SIN_REPORTE_MARGEN_HISTORICO. Esta rama se revisó en código; no se encontró un caso principal A actualmente cerrado que permitiera atribuir una prueba física específica de ese legado. El límite se mantiene explícito.

Si se filtra por operación o q dentro de periodo cerrado, no se presenta el total global guardado como si fuera el desglose filtrado. La consulta explica REPORTE_FILTRADO_SIN_DESGLOSE_HISTORICO y provisionalidad. El contrato de cartera actual no admite periodo_id histórico: el backend lo rechaza, en vez de ignorarlo o inventar filas históricas no almacenadas.

Fuentes: db/migrations/V2__comandos.sql:55; db/migrations/V5__corte_por_periodo_reconocimiento.sql:1; db/migrations/V12__pesajes_reportes_canonicos.sql:12; db/migrations/V13__correccion_composicion_exacta.sql:300.

11.11 11. Privilegios, SECURITY DEFINER y frontera de confianza

Los roles PostgreSQL son cuentas técnicas, distintas de los roles humanos de aplicación. bovi_owner administra estructura y bovi_command posee funciones privilegiadas; son NOLOGIN. bovi_migrator puede asumir propiedad con privilegios administrativos separados. bovi_app_a y bovi_app_b conectan a negocio con SELECT permitido y EXECUTE de interfaces, sin INSERT/UPDATE/DELETE/DDL directo. bovi_auth lee identidad/membresía y tiene DML específicamente sobre sesiones. bovi_worker ejecuta funciones de entrega y referencia de archivos. Monitor y backup tienen propósitos distintos.

El rol técnico no inventa la persona autorizada. fn_scope comprueba p_actor contra usuario y membresía vigentes. Una sesión válida en el navegador no permite a un cuerpo elegir otra persona. Las cuentas SQL compartidas pertenecen al servidor confiable; no se distribuyen al navegador. El diseño no utiliza JWT directo de clientes sobre tablas ni declara RLS multiempresa completa.

SECURITY DEFINER permite que app ejecute una función que inserta hechos bajo derechos con trolados. Su seguridad depende de no exponer funciones generales de ejecución SQL, comprobar scope y dueño, cualificar objetos y fijar search_path seguro. Si una función simplemente aceptara tabla y sentencia arbitrarias, el NOLOGIN del dueño no impediría abuso mediante EXECUTE.

PUBLIC se revoca para funciones de negocio. App recibe las firmas necesarias. El SELECT de usuario se limita a columnas no sensibles; auth tiene lectura del hash para su función legítima. La función de consulta elimina secretos del JSON. Controles de API y restricciones SQL se complementan; conocer un UUID no reemplaza membresía ni propiedad.

Los triggers protegen hechos confirmados frente a UPDATE/DELETE, incluso en una simulación controlada con rol dueño de funciones. Esto demuestra algo distinto de app sin DML. La guardia de contrato aceptado preserva historia; la de pesaje permite únicamente transición APROBADO→CORREGIDO sin cambiar sus otros valores. La de composición impide editar animales de un ticket que ya no es PROPUESTA.

No se afirma que una cuenta superusuario carezca de capacidad administrativa para alterar el sistema. Las pruebas se hacen con roles definidos y contextos controlados. PGAudit y JIT son posibles conceptos de seguridad, pero este capítulo no declara que estén instalados ni certificados. La auditoría de negocio implementada es explícita en tablas y comandos.

Fuentes: db/bootstrap/01_roles.sql:1; db/migrations/V2__comandos.sql:3; db/migrations/V2__comandos.sql:29; db/migrations/V3__consultas_sesiones_grants.sql:151.

11.12 12. Propietarios fijos, replicación y fallos

A escribe maestros y centraliza autenticación. Los hechos locales se escriben en el nodo dueño de la sede. B no se convierte en escritor de A por indisponibilidad, ni A confirma una venta de B para continuar servicio. Pgpool trabaja raw con un backend fijo por puerto y sin promoción automática.

Las publicaciones bidireccionales replican tablas comerciales y filtran hechos por nodo_origen. Las suscripciones utilizan origin=none para evitar reenviar cambios que provienen de otro origen. Esa opción no resuelve conflictos si se permite a ambos nodos escribir el mismo agregado. La disciplina de propietario evita crear el conflicto; los constraints y conciliación detectan inconsistencias observables.

El apply de suscripción no representa una nueva intención comercial humana. Por eso no debe generar otra auditoría económica o volver a despachar cada fila recibida como orden. Outbox conserva propietario_despacho y el worker procesa hechos de su nodo. Inbox, claves idempotentes y estados de entrega son locales y no se publican como hechos comerciales.

Una partición puede retrasar visibilidad de maestros o hechos. Se informa fuente y antigüedad cuando se conoce; un timestamp agregado de sincronización no demuestra que toda identidad o tarifa esté actualizada. La lectura de réplica nunca transfiere propiedad de escritura. La conciliación revisa huérfanos y relaciones balanceadas después de convergencia.

Si A cae, nuevas solicitudes autenticadas de A y B fallan503 porque la autorización vigente es central. Una transacción B ya autorizada y en ejecución puede concluir en su propio nodo. Son situaciones diferentes: terminar un trabajo validado no equivale a admitir otro login o nueva petición sin A. La caída de B no cambia su dueño ni reinicia arbitrariamente el proceso para ocultarla.

Las pruebas registraron paradas físicas de contenedores propios A/B/proveedor y reanudación en finally. La partición de réplica se simuló deshabilitando stream apply en QA, no cortando una red física. Es una evidencia útil con alcance específico. Dos PostgreSQL en la misma laptop no son alta disponibilidad frente a pérdida del host.

11.13 13. Muerte B→A: outbox, inbox y lease durable

B reconoce baja, pérdida y evento ANIMAL_MUERTO en una transacción local. No necesita mantener una transacción financiera abierta esperando que A acepte red. El hecho económico ya queda íntegro; el evento durable expresa trabajo pendiente para actualizar identidad central.

reclamar_entregas recibe p_worker UUID y límite de1-100. Selecciona pendientes de despacho local, sin acuse, con lease vencida/ausente y próximo intento habilitado. Usa FOR UPDATE SKIP LOCKED para que otro trabajador pueda procesar diferentes pendientes. Guarda lease_token= p_worker y lease_vence_en treinta segundos después, e incrementa intentos.

La transacción de reclamo termina antes de conectar con A. El UUID identifica al trabajador de ese reclamo; no es actor comercial de una venta. A consume mediante consumir_muerte, que exige nodo A y empresa configurada, evento remoto convergente ANIMAL_MUERTO de B y payload idéntico al outbox observado. No acepta una orden arbitraria que declare otro animal muerto.

La PK del inbox IDENTIDAD_CENTRAL/B/eventid deduplica. Sólo el primer consumo actualiza animal de la sede correspondiente y registra auditoría central. No crea otro asiento de pérdida: la pérdida pertenece a B. Repetir el consumo devuelve estado APLICADO. La convergencia del evento y su validación evitan reconocer una identidad a partir de información remota que aún no se observó.

confirmar_entrega verifica evento, destino, lease_token y vigencia. Un acuse del trabajador antiguo no confirma una entrega reclamada después por otro. fallar_entrega guarda error saneado, próximo intento con demora lineal cinco segundos por intento limitada a cinco minutos y limpia lease. No guarda conninfo ni contraseña dentro del diagnóstico.

Si se pierde el acuse después de consumo A, B mantiene pendiente y entrega otra vez. El inbox devuelve resultado y luego se confirma entrega local. Si un worker abandona lease, otro puede reclamar al vencer. Las pruebas simularon abandono dejando vencer la lease y acuse perdido omitiendo la primera llamada; no se atribuye a esas pruebas haber matado un proceso Java.

despachar_outbox tiene una finalidad DEMO local para otros eventos, con inbox y acuse locales. No debe confundirse con garantizar entrega a una empresa externa real. archivo_referenciado permite al worker consultar si una huella documental sigue comprometida. La limpieza de bytes exige respuesta de ambos propietarios; si uno falla, no se elimina contenido basándose sólo en el otro.

Fuente: db/migrations/V3__consultas_sesiones_grants.sql:131.

11.14 14. FDW sin transacción financiera remota

El proveedor simulado conserva tarifa_transporte con precio numeric(14,4) y ticket_bascula con neto numeric(14,3). El rol remoto bovi_fdw_read tiene SELECT. En A, postgres_fdw proporciona ext_proveedor y el servidor prov_sim. B no conserva vínculo FDW de proveedor desde V9.

El backend solicita leer_tarifas_proveedor en A mediante autocommit, incluso si la sede interesada es B. La función comprueba actor y scope maestro, pagina con orden de ruta y construye procedencia, unidad, versión, fecha, precisión original y Dinero a centavos. Después llama capturar_tarifas_proveedor en el dueño de S, también fuera de locks financieros.

Capturar valida contexto E/S/actor, arreglo acotado, hash, fuente DEMO, moneda, política y coherencia entre cuatro decimales originales y dos canónicos. Guarda precio a centavos por el tipo físico14,2 y JSON de precisión original. Hash comprueba integridad del contenido transportado; no es una firma que convierta una entrada de navegador en fuente confiable. La API no permite suministrar esa página interna como body público.

aceptar_tarifa marca aceptación en captura local sólo para ese actor y scope. registrar_gasto copia snapshot aceptado y compara el monto local. Una fuente futura cambiada o caída no altera

gasto aceptado anterior. Si no se leyó una captura, no se inventa una tarifa para que la compra continúe.

La frontera real12000.0050 del proveedor se redondea a Dinero12000.01. El JSON conserv a12000.0050 y política HALF_UP_2; el catálogo de captura14,2 almacena12000.01. Por eso comparar gasto12000.01 con captura funciona. Una sospecha inicial de comparación contra cuatro decimales se descartó al verificar catálogo y ejecución, sin añadir una migración redund ante.

No se usa FDW para atomicidad monetaria o two phase commit de proveedor. Si la red se mantiene dentro de un lock de pago o periodo, una fuente lenta bloquearía negocio local y complicaría cierre. La lectura previa y token aceptado reducen esa dependencia, con procedencia explícita. El proveedor es DEMO, no SINIIGA, SENASICA ni empresa comercial real.

Fuente: db/migrations/V9__lector_a_captura_owner_fecha.sql:2; db/bootstrap/bootstrap.py:60.

11.15 15. Consultas canónicas: qué responde cada pregunta

bovi.consultar recibe e,s,a,recurso,params y retorna JSON. recurso no es una sentencia SQL del navegador: el CASE decide tablas y ramas permitidas. La función vuelve a validar scope. Las partes dinámicas provienen de una lista de recursos y filtros definidos, mientras valores se pasan con USING. No debe confundirse SQL dinámico interno controlado con permitir un WHERE arbitrario enviado por cliente.

tab selecciona tabla o vista; filtro reúne predicados permitidos; sql conserva sentencia formada. rows y detail guardan JSON; totaln es el conteo completo, size tamaño y page número de página. idn identifica un detalle; searchq es búsqueda recortada y pattern su versión escapada para ILIKE literal. statefilter y typefilter no aceptan cualquier palabra: el recurso define vocabulario admisible.

La paginación es base1. El desplazamiento se obtiene mediante:

$$\text{offset} = (\text{pagina} - 1)\,\text{tamano}.$$

Para página1 y tamaño25, offset0; para página2, offset25. ORDER BY estable evita que un mismo corte distribuya filas aleatoriamente entre páginas. El conteo total no es longitud de rows: una página de25 puede corresponder a200 resultados. Una solicitud de tamaño mayor que100 o página menor que1 se rechaza, no se normaliza silenciosamente.

En q, caracteres porcentual y guion bajo se escapan para tratarse literalmente. El sistema no convierte una búsqueda de usuario en comodín libre ni en SQL. Estado y tipo se comprueban según operaciones, posiciones o pagos. disponibles_para_venta no es una comparación de texto: el parámetro booleano verdadero filtra DISPONIBLE, animal VIGENTE, sin reserva vigente y sin baja local. False o ausencia incluye todas las posiciones del scope.

El detalle de operación une contratos, versión actual, lotes, selección y tickets. Los tickets provienen de fn_pesaje_json, por lo que la composición histórica no se reconstruye con animales actualmente presentes en un lote editado. El detalle de liquidación aporta conceptos y aplicaciones canónicas y la contraparte necesaria para aplicar dinero compatible. Documento retira clave interna y añade revisiones exactas.

simulador consulta fn_calcular_liquidacion sin reconocer compra o venta. No es un comando monetario independiente que el navegador pueda presentar como confirmación. Si falta peso KG, no se fabrica una cantidad cero ni una liquidación principal. Un resultado de cálculo es una previsualización; el comando vuelve a validar evidencia, periodo, versión y reserva antes de confirmar.

11.15.1 15.1 Variables del reporte y corte observable

fn_reporte recibe recurso y params y construye metadata con corte_en, fuente_consulta y anti guedad_replica_segundos. ownernode procede de sucursal; today utiliza su zona_horaria. state ment_timestamp representa corte de esa sentencia. En el propietario, antigüedad se expresa0; en una réplica donde no se dispone de medida individual fiable queda NULL, no un número inventado a partir de una estadística global.

Cartera selecciona sólo saldos positivos. direction admite POR_COBRAR/POR_PAGAR y overdue es booleano verdadero o falso. La propiedad vencida del JSON debe ser booleana; una cadena “true” no equivale al tipo esperado. Backend convierte el parámetro de query tipado antes de llegar a SQL. vence_el NULL produce dias_vencida NULL: no se declara mora de una obligación sin fecha.

La CTE filtered define el conjunto, paginated aplica límite/offset y el resultado conserva filas, pagina, tamano y total. etiqueta ACTUALIZADO expresa consulta actual. No hay una suma de primera página que parezca una cartera completa. Historial de corte se conserva en cierre; un parámetro nuevo que la consulta no implementa debe rechazarse en la interfaz, no aceptarse s ilenciosamente.

Margen utiliza CTE scoped, income, valuation, expense, relevant y flags. scoped delimita operaciones; income suma ventas y ajustes con el periodo propio de cada hecho; valuation suma costo y pérdida; expense suma directos aprobados. relevant evita evaluar faltantes de objetos ajenos al filtro. flags identifica gasto pendiente y revisión administrativa pendiente.

Es importante que una venta de un mes anterior pueda recibir costo reconocido hoy: el periodo del costo no se reemplaza por el periodo de la liquidación. A la vez, consulta de un cierre histórico sin filtros recupera snapshot; no suma datos tardíos de otro mes. Esta combinación distingue margen actual de corte aceptado y evita una cifra financiera que cambia de significado según la pantalla.

Fuente: db/migrations/V12__pesajes_reportes_canonicos.sql:12; db/migrations/V12__pesajes_r eportes_canonicos.sql:430.

11.16 16. Ejemplos seguros para estudiar sin modificar hechos

Para verificar versión del servidor y contar tablas ordinarias, una conexión de lectura autorizada puede utilizar consultas de catálogo. No se incluye una contraseña en la línea de comandos ni se imprime configuración de user mapping.

```
SELECT current_database(), current_setting('server_version');

SELECT n.nspname, count(*) AS tablas_ordinarias
FROM pg_class c
JOIN pg_namespace n ON n.oid=c.relnamespace
WHERE c.relkind='r' AND n.nspname IN ('bovi','infra')
GROUP BY n.nspname
ORDER BY n.nspname;

SELECT count(*) AS claves_externas
FROM pg_constraint k
JOIN pg_namespace n ON n.oid=k.connamespace
WHERE k.contype='f' AND n.nspname IN ('bovi','infra');
```

Estas consultas no leen filas empresariales. Su ejecución concreta en el capítulo obtuvo46/86 en ambos nodos. Si otra instalación devuelve cantidades distintas, se identifica base, nivel de migración y esquemas incluidos antes de concluir que existe error. No se borra una tabla para hacer coincidir una cifra documental.

Para estudiar una definición privilegiada, se inspeccionan propiedades de función, sin copiar c redenciales. Una consulta a pg_proc puede revisar prosecdef y configuración del search_path. La definición debe contrastarse con su última migración. Enumerar nombres por regex estático

no descubre todos los triggers creados en bloques DO dinámicos; por eso el atlas del manual advierte ese límite y el catálogo completa la observación.

Para ensayar un comando de negocio, la alternativa apropiada es un escenario QA propio con identidad y periodo conocidos. Se comienza BEGIN, se llama la firma con parámetros de servidor autenticado y se comprueban cifras fijas; después ROLLBACK. No se ejecuta ese ejemplo sobre main, ni se utiliza un hash inventado como si fuera la canonicalización de producción. El harness gold.py muestra una práctica concreta de rollback y savepoints para negativas.

Un SAVEPOINT permite probar una excepción sin perder toda la transacción de fixture. Después del error se hace ROLLBACK TO SAVEPOINT y se verifica que el hecho no apareció. Eso no significa que la aplicación deba atrapar cualquier error y confirmar el resto: el comando económico conserva atomicidad de su intención. La técnica pertenece a una prueba controlada.

11.17 17. Bootstrap, despliegue local y evidencia de rendimiento

bootstrap prepara roles, identidad local, maestros DEMO, publicaciones/suscripciones y proveedor simulado. Los datos financieros de la semilla posterior se construyen por comandos, no insertando totales deseados directamente en liquidacion o asiento_valoracion. La inicialización organizativa no es una venta demostrada ni un bootstrap comercial de producción.

Las guardas iniciales rechazan BOVI_ENV=production o perfil production antes de cualquier acceso o mutación de bootstrap/seed DEMO. Configuración preparada para una VM no ejecuta ese aprovisionamiento ni activa TLS real. Roles comerciales, empresa real, inventario inicial conciliado y secretos de producción requieren otra preparación autorizada.

La topología académica usa PostgreSQL18.6 y Pgpool4.7.3 raw, en loopback con puertos propios. A/B tienen límites de memoria de1GiB y proveedor512MiB; shared_buffers128MiB, wor k_mem4MiB, maintenance_work_mem64MiB y max_connections50 acotan consumo. Cada Pgpool tiene16 procesos con max_pool1. Conexiones de autenticación A, replicación, migración y administración consumen presupuesto adicional al negocio/worker.

Estos límites no prueban capacidad comercial. El benchmark propio utilizó QA separada,200 animales,300 operaciones,300 gastos,100 pagos y100 entregas creados mediante1901 comandos. Repetir1901 conservó identidades, resultados y conteos. Después se midieron800 lecturas canónicas por Pgpool con1 y4 clientes,100 warmups y4EXPLAIN ANALYZE BUFFERS.

La máquina era Intel i7-13620H con16 lógicos y unos15.7GiB reportados en el artefacto. Caché caliente, dataset sintético moderado y ausencia de HTTP/carga escritora delimitan la medición. Los planes del wrapper PL/pgSQL no muestran todos los planes internos. Se reportan percentiles reales, pero no se extrapola un SLO del piloto, un número de clientes contratados o escalabilidad frente a miles de escrituras simultáneas.

Fuentes: db/bootstrap/bootstrap.py:1; db/bootstrap/01_roles.sql:1; db/tests/benchmark-report.md:1; infra/compose/compose.academic.yml:1.

11.18 18. Glosario SQL orientado a esta implementación

Transacción local. Unidad que confirma o revierte todos los efectos del comando en un propieta rio. No incluye mágicamente otro PostgreSQL o un archivo remoto. Outbox y staging documental resuelven límites externos sin afirmar atomicidad distribuida inexistente.

FOR UPDATE. Bloquea filas seleccionadas frente a cambios incompatibles. Debe utilizarse antes de leer condiciones agregadas que luego se materializan. No comprueba versión del cliente por sí mismo: version_lock exige comparación explícita bajo el lock.

SKIP LOCKED. Permite que un trabajador omita pendientes que otro mantiene bloqueados. Es útil para colas, no para omitir silenciosamente una posición que debía formar parte de una venta.

Advisory transaction lock. Lock por clave de aplicación que dura la transacción. En crear_p eriodo serializa calendario por empresa/sede. No agrega una FK ni vuelve correctos datos que no cumplen predicados.

%ROWTYPE y record. El primero utiliza estructura de una tabla; el segundo adopta resultados variables. Un registro cargado puede estar obsoleto si se leyó sin lock. El tipo no equivale a autorización o vigencia.

JSONB. Representación consultable de contexto, snapshot y respuesta. No convierte payload de navegador en fuente confiable. Se valida forma, alcance y valores; un hash no demuestra autenticidad externa si quien lo envía también puede fabricarlo.

RETURN NEXT. Emite una fila de una función que retorna conjunto, como reparto. Permite que cada posición reciba base e importe explícitos. El consumidor todavía necesita garantizar que todas las cuotas se materializan en una misma transacción.

ON CONFLICT. Define respuesta al conflicto de unicidad. En idempotencia reclama o encuentra intención; en contador incrementa secuencia; en inbox evita segundo consumo. No es una instrucción universal de ignorar errores económicos.

Columna generada. Valor derivado por el motor de otras columnas, como delta de valoración. Protege contra tres signos contradictorios enviados por cliente, pero la clase y el sentido todavía requieren vocabulario y autorizaciones correctos.

Snapshot. Información conocida y materializada en un acto. Un snapshot de contraparte no sigue futuros renombres; un cierre no incorpora automáticamente costos de meses posteriores. Esa estabilidad explica por qué no basta consultar maestros actuales al imprimir historia.

P0001 y DETAIL. Las reglas de negocio utilizan SQLSTATE P0001 con JSON público que contiene código BV, regla_id y campos. BV412 o BV405 no son SQLSTATE inventados. Errores na tivos23503/23505/42501 conservan su mecanismo y se traducen controladamente en API, sin exponer SQL ni secretos.

Idempotencia. Misma intención autorizada y mismo contenido produce respuesta anterior; contenido distinto bajo la misma clave genera conflicto. No significa ejecutar dos veces y esperar casualmente que el estado quede igual.

Reverso. Nuevo hecho que contrarresta uno anterior y conserva referencia. No borra el original ni oculta el periodo en que se decidió corregir. Su unicidad impide anular repetidamente el mismo efecto.

11.19 19. Qué quedó comprobado y qué no debe prometerse

La evidencia local incluye157 llamadas de GOLD fijo,22 guardas de versiones,66 comandos QA de tickets/reportes/cierre, carreras reales con dos conexiones y barreras, roles/constraints/tr iggers, replicación, proveedor y fallos. Backend registró24 pruebas:15 unitarias,8HTTP reales por Caddy y1PostgreSQL18.6 limpio con migraciones1-13. UI registró16 casos integrados con límites propios. Estas cifras no son porcentajes de cobertura universal.

El restore V13 se realizó en proyecto aislado y preservó firmas, archivos, deduplicación, inbox y conexiones reconstruidas; invalidó sesiones y probó compra nueva1000, replay con ETag original, lookup y descarga con hash. Pausa8.038 s y recuperación28.616 s son medidas de ese ensayo en el mismo host. No prueban PITR, copia externa o recuperación frente a pérdida física de la laptop.

La lectura del catálogo hecha para este capítulo confirma estado estructural actual de A/B sin nuevas mutaciones. No repite fallos o carreras ni atribuye una ejecución reciente a logs anterio res. El propósito pedagógico es enseñar cómo verificar una afirmación: la tabla dice dónde vive

el hecho, la restricción qué combinación protege, el comando cuándo materializa, el test qué caso ejercitó y el artefacto quién lo ejecutó.

Una defensa sólida distingue garantía estructural, garantía transaccional, permiso técnico, autorización humana y resultado observado. “Existe FK” no demuestra mismo comprador; “hay lock” no demuestra If-Match; “replica” no demuestra alta disponibilidad; “pasó un GOLD” no demuestra todas las combinaciones; “se preparó production” no demuestra despliegue. Mantener esas diferencias hace el manual útil para estudiar el producto real y para decidir responsablemente su siguiente incremento.

11.20 20. Cuaderno de oráculos fijos para repasar

Un oráculo independiente dice qué resultado se esperaba antes de ejecutar la implementació n. No se recalcula la respuesta llamando a la misma función bajo prueba. Los siguientes valores proceden de los casos GOLD documentados y permiten comprobar si se está confundiendo obligación, costo, dinero o movimiento. Cada cifra debe explicarse también por sus filas fuente; coincidir accidentalmente en el total no prueba un historial correcto.

Caso	Esperado fijo	Qué distinción verifica
GOLD-01	Costo252000; margen34000; saldo244000; variación100kg/2 %	Peso y deducción contractual independientes.
GOLD-02	Costo4400; margen1800; inventario6600	Venta parcial conserva ciclos restantes.
GOLD-03	0.34+0.33+0.33=1.00	Residuo de centavos por identidad estable.
GOLD-04A	Obligación1000,pago600,aplicaciones400+200→saldo400, disponible0	Parciales y replay sin sobreaplicación.
GOLD-04B	Obligación1000,pago1200,aplica1000→saldo0,disponible200	Anticipo no es otra venta.
GOLD-05	Recompra genera ciclo2, nunca dos posiciones activas	Identidad animal separada de ciclo.
GOLD-06	Tardío4600/6900/1600; bonificación4520/6780/1680	Costo vendido/inventario/margen sin cambiar deuda.
GOLD-07	Reverso1000,reaplica900,abono100→saldo0,disponible100; reembolso→0	Corrección pagada atómica y caja separada.
GOLD-08	Inventario0,pérdida1100→1150	Baja no genera ingreso ni cartera.
GOLD-09	Ciclo anterior conserva costo y nuevo ciclo obtiene propio	Historia no se recicla.
GOLD-10	Costo entra antes del corte o fallaBV025sin efecto	Periodo como lock común de reconocimiento.
ANUL-01	Inventario restituido1200,costo neto de venta0	Anulación incorpora costo tardío previo.

En GOLD-04A, la primera aplicación400 deja saldo600 y disponible200. La segunda200 deja saldo400 y disponible0. Repetir la segunda con la misma clave devuelve el resultado anterior sin reducir nuevamente saldo. Una nueva intención por0.01 no puede pasar, porque no hay disponible. No sería suficiente comprobar únicamente que la obligación todavía tiene saldo400: deben cumplirse ambos límites.

En GOLD-04B sobra200 después de pagar totalmente la obligación. Ese importe se conserva como disponible del pago, no como margen ni ingreso de otra operación. Puede aplicarse a otra obligación compatible o reembolsarse con su propio hecho de caja y periodo. Un reporte que sumara dinero recibido como venta mostraría1200 en vez de1000 y mezclaría anticipo con rec onocimiento.

En GOLD-05 y09, entregar cierra el ciclo anterior y permite una nueva adquisición del mismo animal vigente. La historia del primer ciclo conserva costo y operación. Las condiciones actuales del segundo trato no recalculan el primero. La unicidad parcial impide dos ciclos activos; no impide conservar dos filas históricas cuando uno está cerrado.

En GOLD-10 importa ensayar ambos órdenes de la carrera, no dormir unos segundos y suponer simultaneidad. Dos conexiones y barrera observan qué lock bloquea al otro. Si costo ganó, el cierre refleja el importe; si cierre ganó, el reconocimiento posterior falla y rollback conserva ausencia de efectos parciales. El esperado es una alternativa coherente por orden, no una cifra fija única independiente de quién obtuvo el lock.

Fuentes: docs/domain/states-and-calculations.md:61; db/tests/gold.py:1; db/tests/closure_orde r.py:1; db/tests/weighing_reports.py:1.

Capítulo 12

Backend, comunicación y seguridad: cómo funciona el programa Java real

12.1 Alcance de este capítulo y forma de leerlo

Este capítulo explica el backend que existe en el repositorio de BoviCierre, a partir de una lectura completa de sus 24 archivos Java de producción, sus dos archivos de configuración de aplicación, el POM de Maven y el contrato OpenAPI 0.2.0. La lectura se realizó el 12 de octubre de 2026. Las pruebas que se citan al final son ejecuciones registradas el día anterior; escribir este capítulo no significa que se hayan vuelto a ejecutar servicios, comandos económicos o pruebas.

El lector puede seguir tres niveles. Primero, comprender la circulación de una solicitud: navegador, filtros, controlador, autorización, conexión propietaria, función SQL y respuesta. Segundo, estudiar cada clase y sus métodos. Tercero, reconstruir las secuencias completas de sesión, confirmación y archivo privado. Los nombres conservan la ortografía del código para que sea posible buscarlos con el editor. Las referencias tienen el formato ruta:línea; indican el comienzo del método o bloque pertinente, no una afirmación de que todo el comportamiento cabe en una sola línea.

Aquí, **campo** significa dato conservado dentro de un objeto Java; **variable local**, dato temporal de una llamada; **argumento**, valor que recibe un método; **salida**, su resultado o excepció n. Un objeto de Spring registrado como servicio o controlador suele compartirse entre solicitudes, mientras los argumentos y variables locales pertenecen a cada ejecución. Por ello importa distinguir las dependencias finales de los contadores concurrentes y de los objetos temporales.

Este texto también diferencia cuatro objetos que se confunden fácilmente: usuario, empresa, sede y nodo. El usuario es una identidad autenticada. La empresa es el ámbito empresarial configurado en el despliegue. La sede es la unidad autorizada donde ocurre una operación. El nodo A o B es el PostgreSQL que posee la capacidad de escribir un agregado. Solicitar una sede B no concede acceso a B: primero deben comprobarse usuario y membresía.

12.2 Qué está implementado y qué corresponde al diseño

La aplicación ejecutable usa Java 21, Spring Boot 4.1.1, Spring MVC, Spring Security, JDBC y Spring Session JDBC. Es un proceso Java con componentes distribuidos en paquetes. No hay un microservicio independiente por venta, pago o documento. Dentro del mismo proceso, los controladores reciben HTTP, AccessService consulta autorización, CommandService selecciona el propietario y PostgreSQL ejecuta las reglas económicas.

El documento de arquitectura plantea módulos como identidad, inventario, pagos y liquidaciones, además de carpetas domain, application y adapters. En los archivos inspeccionados no existen esas capas completas para cada módulo. Los paquetes reales son acceso, operaciones,

evidencia, evidencia/application, integraciones, reportes y shared. CoreController concentra varias familias de rutas y CommandService concentra la comunicación canónica JDBC. Presentar el diagrama planeado como una lista de clases ya programadas sería incorrecto.

En particular, **OwnerDatabaseRouter** y **BranchTransactionRunner** aparecen en el diseño, pero no como clases del backend actual. Su función se realiza de manera compacta en CommandService mediante mapas de JdbcTemplate y TransactionTemplate. Tampoco existen clases Java Money o PesoKg en producción. La representación monetaria que circula por los controladores es JSON con cadenas; los importes confirmados se calculan en PostgreSQL. La norma del proyecto pide NUMERIC y BigDecimal para aritmética exacta, pero este código Java no contiene una segunda calculadora financiera basada en BigDecimal.

Sí existe un puerto explícito: DocumentStorage. DocumentController depende de esa interfaz y PrivateDocuments implementa el almacenamiento local. Es una separación concreta, comprobable en el constructor y en los métodos invocados; no debe extrapolarse a puertos inexistentes para todas las demás operaciones.

La consecuencia para quien exponga el proyecto es sencilla: se puede explicar la arquitectura objetivo y después mostrar la implementación compacta que la materializa parcialmente. La autorización, elección del propietario, transacción local, idempotencia, almacenamiento privado y trabajador existen. Una separación exhaustiva de dominio, casos de uso y adaptadores por cada módulo es trabajo adicional.

12.3 Dependencias, configuración y entrada del proceso

12.3.1 Qué aporta el POM

El archivo `backend/pom.xml` fija spring-boot-starter-parent 4.1.1 y java.version 21. El parent administra un conjunto compatible de dependencias; no aparece una versión independiente escrita junto a cada starter. La versión del proyecto es 0.1.0-SNAPSHOT y el grupo es mx.bovicierre. El plugin spring-boot-maven-plugin prepara el empaquetado de la aplicación.

Dependencia	Función real en este backend	Alcance
spring-boot-starter-webmvc	Anotaciones HTTP, servidor y ResponseEntity	Producción
spring-boot-starter-validation	Validación del record Login y errores de argumentos	Producción
spring-boot-starter-security	Filtros, contexto de sesión, CSRF y BCrypt	Producción
spring-boot-starter-jdbc	JdbcTemplate, DataSourceTransactionManager y acceso JDBC	Producción
spring-boot-starter-session-jdbc	Persistencia de sesiones en PostgreSQL	Producción
postgresql	Driver JDBC y lectura estructurada de errores PostgreSQL	Producción
com.google.zxing:core 3.5.4	Construcción local de la matriz QR	Producción
starters de pruebas, security-test y testcontainers-postgresql	Pruebas Java, HTTP y PostgreSQL real	Test

El POM de runtime no añade Flyway. Las migraciones son una actividad separada. Tampoco incorpora una biblioteca de renderizado PDF: SettlementPdf construye el comprobante directa

mente. Testcontainers no se convierte en un servicio del producto; su dependencia tiene alcance test.

12.3.2 application.yml y el perfil de producción

`backend/src/main/resources/application.yml:1` establece apagado graceful, dirección predeterminada 127.0.0.1 y puerto 18080. BOVI_BIND_ADDRESS y BOVI_API_PORT permiten cambiar esos valores. En el Compose integrado el backend escucha internamente para que Caddy lo alcance; el acceso del usuario se realiza por el origen local 15180. El puerto interno de Java y el puerto publicado del proxy son cosas distintas.

La cookie se llama BOVI_SESSION, tiene HttpOnly y SameSite=lax, y Secure=false en la configuración local. HttpOnly impide que el JavaScript ordinario lea la cookie; no reemplaza la autorización del servidor. SameSite reduce algunos escenarios de envío entre sitios; no sustituye el token CSRF. El archivo application-production.yml:1 cambia Secure a true. Es una condición necesaria para el perfil HTTPS, pero ese pequeño archivo no configura por sí solo DNS, certificados, publicación segura o una plataforma productiva completa.

Spring Session usa un timeout de 30 minutos, la tabla infra.spring_session y initialize-schema: never. La aplicación no crea esa tabla al arrancar. Su existencia y permisos deben proceder de la preparación de la base. Las propiedades de apagado permiten esperar hasta 30 segundos por fase y por finalización de tareas programadas. Esto sirve al procedimiento de respaldo con parada ordenada; no hace que una copia arbitraria de archivos y base sea automáticamente coherente.

Los multipart tienen máximo por archivo de 20971520 bytes, exactamente 20 MiB, y máximo por solicitud de 22020096 bytes, 21 MiB. El margen de la solicitud cubre el envoltorio y campos adicionales. PrivateDocuments también comprueba el límite de contenido. La primera barrera puede producir 413; la comprobación del servicio produce un error de dominio cuando el archivo está vacío o excede su límite.

La configuración separa cinco conexiones: a, b, auth, worker-a y worker-b. Los valores predeterminados de negocio pasan por Pgpool local 19999 y 19998; auth apunta al PostgreSQL A técnico en 15432. Los trabajadores toman las URL de negocio y un usuario distinto, bovi_worker. Las contraseñas se reciben mediante propiedades de entorno; sus valores predeterminados están vacíos. Este capítulo no incluye valores funcionales.

BOVI_DOCUMENTS_DIR determina el directorio privado. Sin esa variable, la propiedad bovi.documents usa ./var/private-documents; el lanzamiento local documentado configura un directorio externo. bovi.outbox-enabled, alimentada por BOVI_OUTBOX_ENABLED, habilita o deshabilita el trabajador. El nombre de aplicación es bovicierre.

12.3.3 BoviCierreApplication: un método de arranque

Referencia: `backend/src/main/java/mx/bovicierre/BoviCierreApplication.java:8`. La clase tiene SpringBootApplication, que define el punto de configuración del proceso, y EnableScheduling, que permite las tareas programadas. No tiene campos propios permanentes.

main(String[] args) crea la variable application con una instancia de SpringApplication. Calcula directory a partir de BOVI_RUNTIME_DIR o del directorio privado de ejecución bajo user.home. Registra ApplicationPidFileWriter para escribir api.pid y finalmente llama application.run(args). El argumento args pasa a Spring; la variable directory no selecciona una sede ni una empresa, únicamente la ubicación del PID.

El llamador habitual es el lanzador Java del JAR. La salida no es una respuesta HTTP: es un contexto Spring en funcionamiento. El archivo PID permite que un procedimiento local identifique su proceso propio. No debe confundirse ese PID con el identificador de una transacción o con el de un contenedor.

12.3.4 DatabaseConfiguration: cinco pools y una transacción de sesión

Referencia: `backend/src/main/java/mx/bovicierre/shared/DatabaseConfiguration.java:15`. La clase Configuration no guarda un router mutable global. Crea beans con nombres explícitos que después se inyectan con Qualifier.

pool(Environment env, String name) es la fábrica privada. prefix se forma como bovi.datab ase más el nombre. config es un HikariConfig donde se cargan URL, usuario y contraseña obli gatorios. Los pools worker reciben máximo dos conexiones; los demás, cuatro. MinimumIdle es cero, el tiempo de adquisición es 3000 ms, el de validación 1000 ms y InitializationFailTimeou t es -1. Este último valor permite iniciar el pool sin exigir una conexión exitosa inmediata; las solicitudes posteriores todavía pueden fallar.

Los métodos **ownerA(env)** y **ownerB(env)** llaman pool con a y b. **workerA(env)** y **workerB(en v)** usan worker-a y worker-b. **authDataSource(env)** usa auth y está marcado SpringSessionDa taSource. **sessionTransactionManager(ds)** construye DataSourceTransactionManager sobre el bean authDataSource y lleva SpringSessionTransactionManager. Son seis métodos de publi- cación de beans más la fábrica privada; ninguno decide dinámicamente mover una transacción ya iniciada a otro servidor.

La entrada Environment representa configuración resuelta por Spring, no un objeto enviado por el navegador. La salida de las primeras cinco fábricas es un DataSource. La última devuelve el administrador dedicado a transacciones de sesión. Los llamadores son el contenedor Spring, C ommandService, AccessService, NodeHealth y OutboxWorker, según sus Qualifier.

Que existan cinco pools no significa cinco bases distintas: autorización y varios pools pueden llegar al mismo PostgreSQL A con roles distintos. La distinción separa permisos, capacidad de conexiones y tareas, aunque compartan servidor físico.

12.4 Comunicación HTTP y forma de los datos

12.4.1 Qué viaja desde el navegador

Los clientes usan rutas bajo /api/v1. La sede suele venir en /sedes/{sedeId}; los recursos usan UUID en la ruta. Los controladores reciben Authentication proporcionada por Spring. Un cuerpo que contenga actor_id no puede sustituir la identidad del contexto de seguridad. En CoreContro ller se rechazan explícitamente empresa_id, sucursal_id, actor_id y otros campos que el servidor debe determinar.

Una mutación ordinaria lleva un cuerpo JSON, X-XSRF-TOKEN y, para comandos de negocio, Ide mpotency-Key. Algunas mutaciones de operación también requieren If-Match. La cookie mantie- ne la sesión. Son controles con funciones diferentes: cookie para localizar la sesión, CSRF para proteger el envío, idempotencia para identificar un intento y ETag para comprobar la versión.

```
{
  "importe": "1200.00",
  "moneda": "MXN"
}
```

Este ejemplo demuestra la estructura Dinero. El valor monetario es una cadena, no 1200.00 como número JSON. El peso usa otra estructura:

```
{
  "kg": "300.000"
}
```

Un precio unitario puede ser la cadena “10.0000”. Los dos, tres o cuatro decimales dependen del tipo, no de un formato visual elegido por el navegador. Un peso individual desconocido puede omitirse en una propuesta o aparecer nulo en un resultado; el sistema no debe repartir el peso grupal y fingir que fue medido para cada animal.

12.4.2 Lecturas, comandos y documentos derivados

Una consulta recibe filtros y devuelve un JSON canónico o una representación derivada. Una mutación llama una función de comando. Exportar un CSV, dibujar un QR o generar el PDF de una liquidación consulta información existente; esas salidas no crean otra venta ni otra deuda.

Una excepción importante es consultar tarifas: el GET consulta el proveedor mediante A y guarda una captura técnica en el propietario de la sede. Por tanto, no es una lectura completamente libre de escritura local. La captura se mantiene separada de la transacción económica y de la aceptación comercial.

OpenAPI describe 70 operationId. Los nombres de parámetros animalId, operacionId o doc umentoId del contrato pueden aparecer como id en una anotación Java. Para comprender la equivalencia importa la posición y el tipo del segmento de ruta, no que ambos archivos utilicen el mismo nombre local.

Las respuestas de comando no se fabrican con una fórmula en el controlador. En el camino ordinario, PostgreSQL devuelve jsonb, JDBC lo convierte a texto y ResponseEntity lo publica como application/json. Ese transporte conserva las cadenas monetarias canónicas.

12.5 Autenticación y autorización explicadas clase por clase

12.5.1 AccessService: la identidad y las membresías vigentes

Referencia: `backend/src/main/java/mx/bovicierre/acceso/AccessService.java:14`. El único campo es auth, un JdbcTemplate construido sobre authDataSource. El constructor **AccessServi ce(source)** fija esa dependencia técnica. Este servicio no utiliza ownerB para decidir si alguien puede entrar.

login(String login) normaliza el texto con trim y minúsculas de Locale.ROOT. La consulta parametrizada busca id, empresa_id, nombre, hash_contrasena y estado. El filtro empresarial procede de infra.nodo_config, no del cuerpo de login. rows es la lista de coincidencias; se devuelve su primera fila únicamente cuando tiene tamaño uno. Cero o varias filas producen null. Este método localiza el registro; la comparación de contraseña ocurre en SessionController.

user(Authentication authentication) revisa ausencia, indicador de autenticación y tipo Anon ymousAuthenticationToken. Si el contexto no representa una sesión activa, lanza AUTH401. Su salida es un UUID convertido desde authentication.getName(). No toma el UUID de una cabecera libre. Lo llaman prácticamente todos los controladores protegidos y el filtro de autorización vigente.

current(UUID user) consulta id y nombre para un usuario ACTIVO, nuevamente limitado a la empresa configurada. Una cantidad distinta de una fila produce AUTH401 con el mensaje de sesión no autorizada. La cookie puede existir y, aun así, este método rechazar al usuario bloqueado. Lo llaman el filtro de seguridad, authorize y la consulta de sesión actual.

sites(UUID user) une membresia_usuario con sucursal, exige membresía vigente y ordena por código de sede y rol. Devuelve una lista de filas. Si el usuario tiene dos roles en una sede, inicialmente hay dos filas para esa sede. Cada fila contiene identidad de sede, empresa, código, nombre, nodo dueño y rol_codigo. La agrupación para la interfaz ocurre después.

authorize(UUID user, UUID site, boolean mutation) primero ejecuta current. Filtra las filas de sites para la sede solicitada; rows guarda ese subconjunto. Si está vacío, AUTH403. roles reúne códigos diferentes. Para una mutación exige al menos ADMIN, OPERADOR o CAJA, de modo que un AUDITOR solo no puede escribir. first aporta la empresa y el propietario. El resultado es el record Scope.

Scope conserva company, site, owner y roles, con accesores company(), site(), owner() y roles(). Java genera también constructor, igualdad, hashCode y toString propios de un record; no son comandos económicos escritos a mano. Un Scope es la decisión obtenida para esa llamada. No significa que los roles se copien permanentemente dentro de la cookie.

authorize solo aplica la primera clasificación de escritura. CommandService exige después el rol pertinente al comando y PostgreSQL valida su propio contexto. Esto evita atribuir a mutation=true una precisión que no tiene: permitir una escritura general no equivale a autorizar cualquier ajuste financiero.

12.5.2 AvailabilityFilter: comprobar A antes de procesar la API

Referencia: `backend/src/main/java/mx/bovicierre/acceso/AvailabilityFilter.java:19`. Es un OncePerRequestFilter registrado con la prioridad más alta. Su campo auth apunta al JdbcT emplate técnico central, creado por el constructor **AvailabilityFilter(ds)**.

shouldNotFilter(request) devuelve true cuando la URI no empieza con /api/ o cuando es exactamente /api/v1/salud. Es decir, omite el chequeo en esos casos. La salud necesita permanecer consultable incluso si A falla. El GET anónimo de CSRF sí queda dentro del chequeo de disponi bilidad.

doFilterInternal(request, response, chain) ejecuta select 1 sobre auth y después chain.doFi lter. Si aparece DataAccessException, utiliza ApiErrors.database y ApiErrors.write. No envía la solicitud de B a A como sustituto ni la de A a B: A es aquí dependencia de autorización y sesión.

Las variables request y response son objetos HTTP; chain representa el resto de filtros y despacho. La comprobación de conectividad no lee roles. Su papel es convertir la indisponibilidad central en una respuesta coherente antes de continuar. El bloque try también envuelve la cadena, por lo que puede recoger una DataAccessException que se propague desde ella.

12.5.3 SecurityConfiguration: seguridad de infraestructura

Referencia: `backend/src/main/java/mx/bovicierre/acceso/SecurityConfiguration.java:21`. Esta clase declara beans; no conserva campos propios. Conviene distinguir sus métodos porque no todos realizan la misma protección.

noGeneratedUsers() entrega un UserDetailsService cuya función de búsqueda siempre lanza UsernameNotFoundException. Evita que el producto se apoye en un usuario automático de demostración generado por configuración predeterminada. No contiene una contraseña fija.

passwordEncoder() crea BCryptPasswordEncoder con coste 12. **securityContextRepository ()** crea HttpSessionSecurityContextRepository, que guarda el contexto autenticado en sesión. **c srfRepository()** crea HttpSessionCsrfTokenRepository y fija la cabecera X-XSRF-TOKEN. Estos objetos se inyectan también en SessionController.

security(http, access, context, csrf) construye SecurityFilterChain. Declara públicas las rutas de CSRF, login y salud; las demás requieren autenticación. Configura CSRF con su repositorio y CsrfTokenRequestAttributeHandler, así como el repositorio del contexto de seguridad. Deshabilita request cache, formulario de login, HTTP Basic y logout automático porque las rutas JSON propias se ocupan de esas acciones.

Una ruta pública no queda exenta automáticamente de CSRF. Login es pública para que un usuario todavía anónimo pueda enviar credenciales, pero el POST sigue protegido. Esa distinción explica el GET de bootstrap: primero se obtiene el token y después se realiza la mutación.

El authenticationEntryPoint escribe AUTH401 en formato JSON. El accessDeniedHandler escribe AUTH403 por falta de permiso o CSRF inválido. No se redirige a una página HTML de contr aseña.

Dentro de security se crea además un filtro anónimo con su propio **doFilterInternal(request , response, chain)**. Después de AnonymousAuthenticationFilter, obtiene authentication desde SecurityContextHolder. Si no es anónima, verifica access.current(access.user(authentication)) . Agrega X-Correlation-ID y continúa la cadena. Captura ApiException y DataAccessException para que una revocación o un problema de base conserve la respuesta pública prevista.

Las autoridades de Spring de la sesión están vacías en el login actual; las reglas de negocio no dependen de anotaciones hasRole sobre una copia antigua de permisos. El control vigente

se consulta en AccessService. La conexión central y las comprobaciones SQL son parte de ese contrato.

12.5.4 SessionController: los cinco endpoints de sesión y salud

Referencia: `backend/src/main/java/mx/bovicierre/acceso/SessionController.java:26`. Sus campos son access, encoder, contexts, csrf, dummyHash y health. El constructor **SessionController(...)** recibe las dependencias y genera dummyHash a partir de un UUID aleatorio. Ese hash permite realizar una comparación BCrypt también cuando no se encontró usuario; reduce una diferencia evidente de camino, pero no debe describirse como una garantía formal de tiempo constante.

csrf(CsrfToken token) responde al GET /api/v1/sesion/csrf. Devuelve un mapa con token y headerName y establece Cache-Control: no-store. El valor de token lo suministra la infraestructura de CSRF. El cliente debe utilizar el nombre de cabecera recibido, no asumir otra variante.

login(Login input, request, response) verifica que la contraseña ocupe como máximo 72 bytes UTF-8. El record Login tiene NotBlank y Size para login y contrasena, pero una longitud de caracteres no es idéntica a una longitud de bytes. Una contraseña con caracteres multibyte puede alcanzar el límite de BCrypt con menos caracteres visibles.

user contiene el resultado de access.login. hash usa dummyHash si no hay usuario o elimina el prefijo de almacenamiento {bcrypt} si lo hay. Se compara la contraseña y se comprueba el estado ACTIVO. El mensaje de fallo no revela si el usuario existía o cuál de las condiciones falló.

Al tener éxito, login obtiene o crea la sesión, cambia su identificador, crea un contexto vacío y le agrega un UsernamePasswordAuthenticationToken autenticado cuyo nombre es el UUID del usuario. Las credenciales del token son null. Guarda el contexto y borra el token CSRF anterior con csrf.saveToken(null,...). Devuelve 204. El cliente necesita pedir un token nuevo después de esa rotación.

logout(request, response) obtiene la sesión sin crearla. Si existe, la invalida, limpia SecurityContextHolder y devuelve 204. El método no contiene una instrucción manual para borrar la cookie por nombre; la identidad deja de ser válida porque se invalidó la sesión del lado servidor y se limpió el contexto.

current(authentication) devuelve GET /api/v1/sesion/actual. id identifica al usuario; user contiene su nombre vigente. sites es un LinkedHashMap por UUID. Cada primera fila de sede se copia en entry, se elimina rol_codigo y se crean listas roles y capacidades. Las siguientes filas agregan roles a la sede ya existente. Toda sede recibe CONSULTAR; una fila que no sea AUDITOR puede incorporar ESCRIBIR.

Estas capacidades simplifican la interfaz, pero no representan permisos específicos para todos los comandos. CAJA puede mostrar ESCRIBIR y no estar autorizada para un ajuste reservado a ADMIN. La decisión final sigue en los servicios y el SQL.

health() publica GET /api/v1/salud delegando a NodeHealth.status. **Login** es un record de entrada con accesores login() y contrasena(): login no vacío y máximo 80 caracteres, contraseña no vacía y máximo 72 caracteres, además del control de bytes dentro del método.

12.5.5 UserController: crear usuarios sin enviar hashes desde el cliente

Referencia: `backend/src/main/java/mx/bovicierre/acceso/UserController.java:14`. Tiene access, commands y encoder; el constructor fija las tres dependencias.

El método privado **write(site, command, body, key, target, auth, request, status)** obtiene el usuario, autoriza mutación y exige ADMIN. Parsear el JSON ocurre después de esa comprobación. No admite contrasena fuera de crear_usuario ni permite hash_contrasena, empresa_id, sucursal_id o actor_id suministrados por el navegador. target, cuando existe, fija usuario_id desde la ruta.

fingerprint se calcula antes de reemplazar la contraseña por BCrypt. Esto importa porque BCrypt genera una sal aleatoria: dos codificaciones de la misma contraseña no producen el mismo texto. Si la intención idempotente usara el hash salado nuevo, un reintento legítimo podría parecer un contenido diferente.

password es el nodo retirado de payload. Debe ser textual, no blanco y ocupar hasta 72 bytes UTF-8. Luego se agrega hash_contrasena con prefijo {bcrypt}. La contraseña en claro no se manda como campo del comando SQL. En crear_usuario es obligatoria. La llamada usa la sobrecarga de CommandService que recibe fingerprint explícito. La huella de intención tampoco se publica en las respuestas del controlador ni en sus logs ordinarios.

create(...) atiende POST a usuarios, selecciona crear_usuario y responde 201. **update(...)** atiende PATCH /usuarios/{id}, selecciona actualizar_usuario y responde 200. **membership(...)** atiende POST /usuarios/{id}/membresias, selecciona asignar_membresia y responde 200. Sus parámetros proporcionan sede, cuerpo, clave, usuario objetivo y contexto HTTP; el método común aplica el resto.

Estos comandos son maestros y se ejecutan en A, incluso cuando la sede cuya membresía se administra es B. La creación de un usuario no implica por sí sola una membresía para todas las sedes. La asignación es un comando separado.

12.6 JsonSupport: frontera de datos, ETag y huella de intención

Referencia: `backend/src/main/java/mx/bovicierre/shared/JsonSupport.java:11`. Es una clase final de utilidades con constructor privado vacío. Su campo público estático JSON es un JsonMapper compartido. No conserva el cuerpo de la última solicitud.

object(String body) lee el árbol JSON. node debe existir y ser objeto; un array raíz, una cadena raíz o un texto inválido generan BV400. La salida es ObjectNode. Los controladores ordinarios que reciben String utilizan este método, mientras Login utiliza deserialización y validación del framework.

sorted(JsonNode value) es recursivo y privado. Para un objeto crea result y un TreeMap fields, ordena las claves y procesa cada hijo. Para un array crea otro array y procesa los hijos en su mismo orden. Para un escalar devuelve el valor. Ordenar las propiedades no significa ordenar los animales de una lista: cambiar el orden de un array sigue siendo un cambio de intención para esta huella.

hash(String command, JsonNode body) serializa el cuerpo ordenado, lo precede con el nombre de comando y un salto de línea, lo codifica en UTF-8 y calcula SHA-256 hexadecimal. La variable content es el arreglo de bytes firmado por esa huella; el nombre del comando evita que idéntico JSON de acciones distintas sea la misma intención.

$$H = \text{SHA256}\big(\text{UTF8}(comando \,\|\, \text{LF} \,\|\, JSON_{ordenado}(cuerpo))\big)$$

En la ecuación, LF representa el byte de salto de línea, 0 x0A, que el código inserta entre el comando y el JSON. La huella no es una firma digital ni prueba la identidad de quien envió el cuerpo. La autenticación, el ámbito y la clave completan el control. Un cuerpo inválido puede tener una huella calculable y después ser rechazado: generar hash no equivale a validar el dominio.

version(String etag) lanza 428 si If-Match está ausente. Exige un entero no negativo entre comillas, sin ceros iniciales salvo cero, y con hasta 18 dígitos. Extrae el texto interior y lo convierte a long. No acepta un ETag débil con W/, una lista de versiones ni un sufijo de compresión. El ETag sirve como precondición de versión, no como clave de sesión.

animalIdentifications(ObjectNode payload) toma identificaciones o construye una forma compatible a partir de sistema y codigo. Exige array no vacío. Cada identification debe ser objeto, sistema y codigo textuales, sistema en SINIIGA, SINIDA, INTERNO u OTRO, y código de 1 a 40

caracteres ASCII permitidos. Debe contener al menos una letra o dígito; espacios o guiones solos no bastan.

decimals(JsonNode node) recorre objetos y arrays. Rechaza entradas llamadas total, saldo, disponible, margen y costo_vigente porque el servidor las deriva. Para importe, monto, cargos_pactados y bonificaciones_pactadas admite un objeto Dinero con importe y moneda MXN o una cadena decimal. Los objetos se recorren nuevamente y se valida su importe interior.

Familia	Forma textual comprobada	Responsabilidad restante
Importe, monto, cargos y bonificaciones	Dos decimales; signo opcional y hasta 16 dígitos enteros	SQL decide signo válido y efecto económico
Precio unitario	Cuatro decimales; sin signo negativo y hasta 10 dígitos enteros	SQL comprueba condiciones de precio
Kg y pesos nombrados, valor	Tres decimales; sin signo negativo y hasta 11 dígitos enteros	SQL valida tara, neto y magnitud
Cantidad, deducción_pct y base_valor	Cadena con hasta seis decimales según el patrón general	SQL valida unidad y rango

Este validador no es un validador integral de todo OpenAPI. Reconoce una lista de nombres de campo y formas decimales; no convierte cada petición en un DTO Java específico. Para explicar la seguridad correctamente, deben separarse forma JSON, campos protegidos, autorización y restricciones canónicas SQL.

12.7 CommandService: enrutamiento, transacción local y observabilidad

Referencia: `backend/src/main/java/mx/bovicierre/shared/CommandService.java:16`. Esta es la pieza central de comunicación económica. Todos los controladores de comandos la llaman, incluyendo usuarios, documentos y aceptación de tarifas.

12.7.1 Campos y constructor

access conserva AccessService. **jdbc** es el mapa A/B de JdbcTemplate; **transactions**, el mapa A/B de TransactionTemplate. El constructor **CommandService(access, a, b)** crea un administrador DataSourceTransactionManager independiente por fuente. Las claves A y B fijan la relación entre plantilla JDBC y transacción del mismo propietario. No hay un DataSource por defecto que concentre accidentalmente todas las escrituras en A.

MASTER contiene crear_contraparte, registrar_animal, crear_predio, crear_usuario, actualizar_usuario y asignar_membresia. Es una clasificación por nombre de comando controlado por el servidor. **LOG** escribe mensajes estructurados. **processStarted** conserva el instante de creación del servicio.

Los LongAdder **successes**, **commandErrors** y **commandNanos** acumulan resultados y duración de comandos. **queries**, **queryErrors**, **slowQueries** y **queryNanos** acumulan datos de consultas medidas. LongAdder permite incrementos concurrentes; estos números no son asientos financieros ni están protegidos por la transacción SQL. Reiniciar Java reinicia los contadores.

12.7.2 Las dos sobrecargas command

command(user, site, command, payload, key, correlation) calcula JsonSupport.hash y llama la sobrecarga completa. **command(user, site, command, payload, key, correlation, hash)**

acepta una huella previamente construida. UserController la necesita para la contraseña y DocumentController para su intención documental.

La ejecución completa comienza con started, owner inicialmente NO_RESUELTO, outcome inicialmente FALLO y errorCode nulo. scope resulta de authorize con mutation=true. owner se vuelve A para un maestro o el propietario de scope para un agregado local.

required selecciona los roles que pueden realizar la acción:

Clasificación escrita en Java	Roles admitidos	Ejemplos por nombre
Administración	ADMIN	Nombres que contienen periodo; anular_venta, registrar_baja, registrar_revision, ajustar_liquidacion y administración de usuarios
Caja	ADMIN o CAJA	registrar_pago, validar_pago, aplicar_pago, revertir_aplicacion y reembolsar_pago
Operación general	ADMIN u OPERADOR	Otros comandos, como borradores, pesajes, reservas, documentos y aceptación de tarifa

Esta tabla reproduce la clasificación del código, no una inferencia sobre cualquier combinación de payload. Por ejemplo, el controlador de movimientos usa el nombre genérico registrar_movimiento; no debe afirmarse que Java aplica automáticamente toda regla de un alias registrar_baja por el mero texto del formulario. Las reglas específicas del comando y sus datos se comprueban también en el SQL.

Después se valida Idempotency-Key: no nula, no blanca y longitud de 16 a 80. Se ejecutan decimals y, para registrar_animal, animalIdentifications. node repite la elección A o scope.owner. Si no existe una plantilla para ese nodo, devuelve BV501/503. No intenta enviar al nodo contrario.

Cada attempt abre una transacción local mediante transactions.get(node).execute. Dentro del callback usa exactamente jdbc.get(node) y una sentencia parametrizada:

```
select bovi.cmd_ejecutar(?,?,?,?,?,?,?,?::jsonb)::text
```

Los ocho argumentos son empresa autorizada, sede autorizada, usuario, clave, huella, correlación, comando y payload serializado. Los signos de interrogación son parámetros JDBC; el cuerpo no se concatena como SQL. Las funciones y tablas internas pertenecen a las migraciones, no a una instrucción arbitraria recibida del navegador.

response es el JSON devuelto. TransactionTemplate cierra la transacción antes de que execute entregue el resultado al llamador. Si todo concluye, outcome pasa a COMPLETADA. La API no ejecuta después una consulta financiera para reconstruir el resultado de un commit conocido; recibe el resultado persistible producido por el comando canónico.

12.7.3 Qué se reintenta y qué no

Si DataAccessException tiene como causa más específica una SQLException 40001 o 40P01, y attempt es menor que dos, se espera un tiempo acotado y se repite la misma intención. El programa permite como máximo tres ejecuciones de transacción, con dos esperas. La espera aleatoria base va de 20 a 80 ms y se multiplica por attempt+1: primero 20-80 ms; después 40-160 ms.

Si el hilo se interrumpe, se restablece su indicador y se lanza 503. Otros errores no activan este bucle. Una desconexión de respuesta con resultado incierto no se transforma en un reintento ciego ilimitado. El cliente dispone de la consulta autenticada de resultado.

```
autorizar y elegir propietario
para intento = 0, 1, 2:
    ejecutar una transacción en ese propietario
    si éxito: devolver JSON canónico
    si SQLSTATE es 40001 o 40P01 y aún hay intento:
        esperar un intervalo acotado
        repetir con la misma clave, huella y correlación
    en otro caso: propagar error
registrar siempre duración y resultado público
```

El catch de ApiException copia su código y clasifica RECHAZADA para estados menores de 500 o FALLO para los demás. El catch de RuntimeException obtiene un mapeo para registrar un código seguro, pero vuelve a lanzar la excepción original. El advice HTTP será quien produzca la respuesta correspondiente.

El finally suma elapsed en nanosegundos, incrementa un contador y crea event con evento, comando, correlation_id, nodo_dueno, duracion_ms y resultado. Puede incluir codigo. No agrega payload, clave, hash de petición o contraseña. Este registro ayuda a localizar una ejecución; la auditoría económica permanece en PostgreSQL.

12.7.4 query, providerTariffs, timedQuery y diagnostics

query(user, site, resource, params) autoriza lectura. usuarios exige ADMIN. Un lookup de comando maestro también usa A. Contrapartes, animales, animal, predios y usuarios se consultan en A; los demás recursos van al propietario de la sede. Ejecuta bovi.consultar con empresa, sede, usuario, recurso y params JSON mediante timedQuery. No abre la TransactionTemplate de un comando para una consulta ordinaria.

providerTariffs(user, site, params) autoriza lectura y hace dos llamadas separadas. Primero timedQuery en A invoca leer_tarifas_proveedor. quotes contiene la página de fuente y contexto interno. Luego otra llamada, en el dueño de la sede, invoca capturar_tarifas_proveedor con ese resultado interno. Las conexiones trabajan en autocommit y la primera llamada termina antes de iniciar la segunda. El cliente no entrega una página supuestamente leída del proveedor.

timedQuery(node, resource, query) recibe un Supplier. started mide el comienzo; outcome y errorCode describen el resultado. Un fallo incrementa queryErrors y se propaga. El finally incrementa queries y queryNanos. Si dura al menos un segundo, incrementa slowQueries y escribe CONSULTA_LENTA con recurso, nodo, duración y resultado. No imprime la sentencia ni los parámetros.

diagnostics() devuelve mapas con instante de proceso, advertencia de reinicio de contadores, comandos completados y errores, duración total, consultas totales, errores, lentas, umbral de 1000 ms y duración total. Las conversiones de duración son unidades temporales, no operaciones monetarias.

Las denegaciones que ocurren antes de timedQuery no forman parte de sus contadores de consulta. Tampoco se cuenta cada select de AccessService como consulta económica medida. Esta distinción evita llamar al diagnóstico un medidor exhaustivo de toda actividad JDBC.

12.8 CoreController: un adaptador HTTP compartido

Referencia: `backend/src/main/java/mx/bovicierre/operaciones/CoreController.java:16`. Los campos access y commands se fijan en el constructor **CoreController(access, commands)**. Esta clase concentra las familias de recursos; no contiene el cálculo de costo, saldo o margen.

12.8.1 Los métodos auxiliares

relative(request) toma uri y elimina el prefijo hasta la sede. Por ejemplo, de una ruta completa extrae operaciones/{id}/confirmar. Se utiliza para determinar qué recurso o identificador co-

rresponde a la ruta registrada. Está apoyado en la estructura de esas rutas, no es un router de PostgreSQL.

result(json, status) prepara ResponseEntity con application/json. Lee node para saber si hay version_lock y, en ese caso, agrega un ETag con ese número entre comillas. Devuelve el texto JSON recibido. No agrega una versión inventada a objetos que no la contienen.

write(site, command, body, key, match, idKey, id, version, status, authentication, request) concentra el camino de escritura. payload se construye como objeto JSON; cuerpo ausente o blanco se convierte en objeto vacío. Rechaza campos de ámbito, origen y version_lock enviados en el cuerpo. Si hay id de ruta, lo inserta usando idKey.

Para corregir_pesaje comprueba que corrige_a_id opcional sea textual, interpretable como UUID y coincidente con la ruta. reason debe ser motivo textual no blanco de hasta 200 unidades de longitud Java. Se fija después corrige_a_id al original de la ruta. El SQL V13 comprueba el conjunto de animales; el controlador no pretende reconstruir esa composición desde el lote actual.

Cuando version es true, write obtiene version_lock de JsonSupport.version(match). La clave match representa If-Match, no un entero libre del cuerpo. La llamada a commands.command proporciona usuario del contexto y correlación de request. La variable json recibe el resultado y response lo representa. Para registrar_gasto se obtiene gasto_id o id y se agrega Location de su recurso, con el mismo cuerpo confirmado.

integer(value) convierte un texto a int; si falla, BV400 de paginación. **validateIdFilter(name, value)** valida únicamente periodo_id y operacion_id. Además de UUID.fromString, exige que la representación canónica coincida sin distinguir mayúsculas. Esto rechaza formas abreviadas que un parser permisivo podría aceptar.

12.8.2 Lecturas públicas de esta clase

compare(sedeId, inicial_id, final_id, auth) construye params de dos UUID y consulta com paracion_pesajes. El cálculo de variación o la respuesta NO_COMPARABLE se realiza en SQL; Java no divide porcentajes.

commandResult(sedeId, comando, clave, auth) limita comando a letras minúsculas y guion bajo, de 3 a 60 caracteres, y la clave a 16-80. Consulta el recurso comando con esos filtros. La base limita por principal y sede, y CommandService dirige los maestros a A. Una ausencia devuelve un error de recurso; no prueba que el intento original jamás haya confirmado.

list(sedeId, query, auth, request) convierte la ruta relativa a resource reemplazando barras por guiones bajos. Rechaza periodo_id para reportes_cartera porque esa cartera es actual; la historia del cierre se consulta por su versión. params recibe UUID verificados y booleanos reales para disponibles_para_venta y vencida. Otros filtros siguen siendo textos. page y size tienen valores predeterminados 1 y 25, y límites página positiva y tamaño 1-100.

one(sedeId, id, query, auth, request) identifica el recurso específico por terminación o prefijo: periodo_versiones, operacion_pesajes, operacion_expediente, simulador, animal, operacion, liquidacion, pago, reparto_gasto o gasto. Revisa los filtros UUID, agrega el id de ruta y consulta. La variable relative aquí es un texto local distinto del método que lo produjo.

12.8.3 Escrituras y selección de acciones

create(...) obtiene suffix, el último segmento, y busca un comando en un mapa fijo. Contrapartes, predios y animales son maestros; operaciones, pagos, gastos, periodos y movimientos utilizan su selector correspondiente. Pasa status 201 a write.

edit(...) atiende PATCH de operación. Llama write con editar_operacion, operacion_id, el UUID de ruta, comprobación de versión y respuesta 200.

action(...) obtiene suffix y selecciona el comando con switch. Para reabrir distingue una operación de un periodo examinando la ruta. Calcula idKey según la primera familia: operación,

pesaje, reserva, liquidación, gasto, pago, aplicación o periodo. También calcula status: 201 para nuevas entidades o movimientos de ciertas acciones; 200 para otras.

Las acciones confirmar, cancelar, anular y acordar requieren versión, al igual que reabrir_operacion. Reabrir_periodo no utiliza en este controlador la misma bandera. La protección de periodo y su estado se realiza en el comando canónico; no debe atribuirse al método una comprobación If-Match que no ejecuta.

Familia de acción	Selector SQL elegido	Respuesta de este adaptador
acordar / reabrir operación / confirmar	acordar_operacion / reabrir_operacion / confirmar_operacion	200; versión requerida
cancelar / anular	cancelar_operacion / anular_venta	200; versión requerida
pesajes / aprobar / correcciones	registrar_pesaje / aprobar_pesaje / corregir_pesaje	201 / 200 / 201
reservas / liberar	reservar_venta / liberar_reserva	201 / 200
ajustes / reparto	ajustar_liquidacion / repartir_gasto	201 / 201
validar / aplicaciones / reembolsos	validar_pago / aplicar_pago / reembolsar_pago	200 / 201 / 201
reverso	revertir_aplicacion	201
cierre o cerrar / reabrir periodo	cerrar_periodo / reabrir_periodo	201 / 200
revisiones de operación	registrar_revision	200

El controlador conserva alias como cerrar junto a cierre y una ruta de revisiones de operación. El inventario de OpenAPI más adelante enumera las rutas contratadas; las anotaciones pueden incluir compatibilidades adicionales.

12.9 ApiException y ApiErrors: del fallo interno al error público

12.9.1 La excepción transportable

Referencia: `backend/src/main/java/mx/bovicierre/shared/ApiException.java:3`. ApiException extiende RuntimeException y tiene status, code, rule y fields. status y code son finales. rule puede quedar nula y fields inicia como lista vacía. **ApiException(status, code, message)** guarda el estado HTTP y código, y entrega message al constructor de RuntimeException.

No incluye una conexión ni una sentencia SQL. Su mensaje debe ser apto para el usuario. El detalle de campos es una lista de mapas con nombres públicos; el advice puede publicar esas piezas.

12.9.2 Métodos de ApiErrors

Referencia: `backend/src/main/java/mx/bovicierre/shared/ApiErrors.java:15`. La clase RestControllerAdvice no tiene campos persistentes. **correlation(request)** reutiliza un UUID del atributo correlation_id si existe; de lo contrario crea id, lo guarda y lo devuelve. El filtro y los controladores usan el mismo atributo para mantener una correlación por solicitud.

body(ex, id) construye un LinkedHashMap con codigo, mensaje, correlation_id y campos; agrega regla_id cuando no es nula. **write(response, ex, id)** se usa desde filtros, fuera del camino habitual del controlador. Fija estado, codificación UTF-8, application/problem+json y X-Correlation-ID antes de escribir el JSON.

api(ex, request) es el handler de ApiException: devuelve ResponseEntity con estado, media type de problema y body. **oversized(ex, request)** traduce MaxUploadSizeExceededExceptio n a 413/BV023. **invalid(ex, request)** convierte cabeceras faltantes, incompatibilidad de tipos, mensajes HTTP ilegibles y validación del argumento en 400/BV400.

Hay dos métodos llamados **database**. La sobrecarga estática **database(Throwable ex)** examina el fallo. La sobrecarga handler **database(RuntimeException ex, request)** atiende DataAcce ssException y TransactionException, llama a la anterior y después a api. **internal()** devuelve 500/BV500 con un mensaje genérico.

12.9.3 Cómo se valida el DETAIL de PostgreSQL

database(Throwable) avanza por cause hasta hallar SQLException. Si no existe, una imposibilidad de conexión JDBC reconocida produce 503; los demás fallos genéricos producen internal. Cuando SQLSTATE es P0001 y hay ServerErrorMessage, interpreta DETAIL como JSON.

error es el árbol del detalle. code debe cumplir BV seguido de una categoría 0, 4 o 5 y dos dígitos; rule debe cumplir RN y tres dígitos. fields debe ser un array de hasta 50 elementos. Cada elemento necesita campo y problema no blancos, de hasta 160 caracteres y sin controles. publicFields copia únicamente esas dos propiedades. Un detalle mal formado no se repite en la respuesta: se transforma en error interno.

El mensaje que escribió PostgreSQL no se publica sin control. El mapper elige mensajes españoles conocidos para clave repetida, versión, recurso, disponibilidad, reserva, sobreaplicación y periodo; usa mensajes generales para los demás. Esto conserva trazabilidad mediante códigos sin filtrar nombres internos, fragmentos SQL o datos privados de una excepción.

Error reconocido	Resultado HTTP	Lectura práctica
BV412, BV428, BV404	412, 428, 404	Versión obsoleta, precondición ausente o recurso no disponible
Otros BV4xx del detalle	409	Conflicto; BV403 es reserva vencida, no permiso HTTP403
BV0xx del detalle	422	Regla de dominio incumplida
BV5xx del detalle o conexión reconocida	503	Dependencia requerida no disponible
23505 con restricción uk_	409/BV401	Conflicto de unicidad conocido
23001 con fk_	409/BV401	Referencias impiden modificar
23503 con fk_ o CHECK reconocido	422/BV015	Referencia de ámbito o condición declarativa
23502, 22P02, 22001, 22003, 22007	422/BV015	Dato requerido, tipo, longitud, rango o fecha inválidos
42501	403/AUTH403	Permiso rechazado
40001 o 40P01 tras los reintentos	409/BV409	Conflicto concurrente, conservar la misma clave

Los nombres de restricción se revisan para las categorías que lo necesitan; no toda excepción se presenta como indisponibilidad. Una SQL no reconocida termina como 500, lo cual permite distinguir un defecto de programación de un dueño caído.

Un ejemplo público, sin valores reales de sesión, es:

```
{
  "codigo": "BV412",
  "mensaje": "La versión cambió; recargue el recurso.",
  "correlation_id": "00000000-0000-4000-8000-000000000001",
  "campos": [],
```

```
  "regla_id": "RN-017"
}
```

La correlación del ejemplo es ilustrativa. Para soporte se conserva el UUID de la respuesta real; no se necesita enviar al operador una contraseña o el texto de una sentencia para localizar el intento.

12.10 Archivos privados: puerto, adaptador, controlador y limpieza

12.10.1 DocumentStorage: un contrato pequeño y real

Referencia: `backend/src/main/java/mx/bovicierre/evidencia/application/DocumentStorage.java:7`. La interfaz expone **stage(MultipartFile file)** y **path(String sha256)**. stage devuelve un Upload cuyo ciclo de vida debe cerrarse; path devuelve una ruta local válida para contenido autorizado previamente por el llamador.

El record **Stored** contiene sha256, size, mime y name, con accesores sha256(), size(), mime() y name(). La interfaz anidada **Upload** extiende AutoCloseable y expone stored() y close(). El controlador puede usar try-with-resources sin conocer FileLock o Hikari. Los metadatos de negocio siguen perteneciendo al comando SQL.

12.10.2 PrivateDocuments: almacenamiento por huella

Referencia: `backend/src/main/java/mx/bovicierre/evidencia/PrivateDocuments.java:27`. **root** es el directorio absoluto normalizado; **LOCKS** es un ConcurrentHashMap estático de bloqueos locales por directorio y huella. El constructor **PrivateDocuments(root)** normaliza la ruta configurada. No acepta una ruta física como parámetro del usuario HTTP.

store(file) es un método de conveniencia: llama stage y devuelve stored dentro de un try que inmediatamente cierra el Upload. DocumentController utiliza directamente stage para conservar el bloqueo durante el comando SQL. No debe atribuirse a store el mismo alcance de bloqueo alrededor de una transacción que sucede después de devolverlo.

La clase interna **Upload** conserva stored y lock. Su constructor privado une el resultado de escritura y el HeldLock. **stored()** devuelve los metadatos del contenido; **close()** delega la libe ración. El record privado **HeldLock** contiene local, channel y file: respectivamente ReentrantL ock, FileChannel y FileLock. Su **close()** libera primero el bloqueo de archivo, cierra el canal y desbloquea el ReentrantLock en finally; si aparece IOException, registra un aviso genérico. Sus accesores generados no agregan reglas nuevas.

directories() crea root y .locks y aplica permisos POSIX rwx—— a ambos. **lock(hash, wait)** obtiene el ReentrantLock con computeIfAbsent. En modo wait bloquea; en modo sin espera usa tryLock y puede devolver null. Abre el archivo .locks/{hash}.lock con permisos rw——- y obtiene un FileLock entre procesos. local impide colisiones dentro de la JVM; el bloqueo de archivo coordina procesos que comparten directorio. Si falla, cierra channel y libera local.

stage(file) comprueba archivo no vacío y máximo de 20 MiB. bytes contiene el contenido completo en memoria. mime proviene de type y hash de SHA-256. Crea directorios y obtiene held esperando el bloqueo. temporary tiene una ruta de huella, UUID aleatorio y sufijo .part.

El método crea y escribe temporary, fija sus permisos y la mueve atómicamente a root/{hash}, con reemplazo permitido. name toma el nombre original o documento; reemplaza caracteres no permitidos y lo limita a 160. Devuelve Upload con Stored y el bloqueo aún retenido. El finally elimina la parte temporal si persiste. Un error de E/S produce BV503; no se realiza un movimiento no atómico alternativo.

type(bytes) examina firmas iniciales: %PDF-, los ocho bytes PNG o los tres primeros JPEG. Devuelve application/pdf, image/png o image/jpeg. Otro contenido produce 422/BV023. Es detección de firma, no un análisis estructural integral ni un escáner antimalware.

path(sha256) exige exactamente 64 caracteres hexadecimales minúsculos, resuelve bajo root y exige archivo regular sin seguir enlace simbólico final. Lee bytes y recalcula hash. Una diferencia produce 409 de integridad; indisponibilidad de lectura, 503. Devuelve Path únicamente después de esa comprobación. Quien invoca path debe haber autorizado el documento; conocer su huella no sustituye ese paso.

12.10.3 cleanup y CleanupMain: seguridad por candidato

cleanup(execute, referenced) devuelve el record **Cleanup**, cuyos campos son modo, candidatos, eliminados y omitidos y cuyos accesores tienen esos nombres. Las listas de resultado se copian antes de devolverlas.

paths recorre el directorio. Solo considera archivos regulares, sin seguir enlaces, que parezcan una huella o un .part asociado y sean anteriores a 24 horas. hash extrae los primeros 64 ca racteres. Con un bloqueo sin espera, omite contenido ocupado. Después vuelve a comprobar existencia, antigüedad y referenced.test(hash) bajo bloqueo. Solo entonces agrega un candidato y, si execute es true, lo elimina.

La limpieza predeterminada diagnostica. No elimina un archivo reciente ni uno que ambos chequeos no hayan permitido considerar huérfano. Tampoco es una transacción global de sistema de archivos: si falla una comprobación después de borrar candidatos previos, aquellos borrados ya realizados no se revierten.

Referencia del lanzador: `backend/src/main/java/mx/bovicierre/evidencia/CleanupMain.java:9`. El constructor privado impide uso ordinario como objeto. **main(args)** acepta cero argumentos o únicamente –execute. env contiene el entorno; secrets selecciona directorio externo por B OVI_SECRETS_DIR, alternativa BOVI_SECRET_DIR o ubicación predeterminada. password se lee del archivo worker_password sin imprimirse. directory determina el almacenamiento.

Abre conexiones a y b con rol bovi_worker y prepara qa y qb para archivo_referenciado(?), cada una con timeout de cinco segundos. El Predicate usado por cleanup ejecuta ambos chequeos y devuelve inA o inB. Cualquier fallo impide eliminar ese candidato y corta el procedimiento; si ni siquiera se pueden abrir ambas conexiones, no inicia el escaneo. result se imprime como JSON de diagnóstico. Este main no inicia Spring, HTTP ni los trabajadores programados.

12.10.4 DocumentController y sus cuatro operaciones

Referencia: `backend/src/main/java/mx/bovicierre/evidencia/DocumentController.java:16`. Tiene files de tipo DocumentStorage, access y commands. El constructor **DocumentControlle r(files, access, commands)** permite inyectar el adaptador local mediante el puerto.

upload(…) recibe MultipartFile archivo, tipo_documental, operación opcional, documento previo opcional, clave y contexto. user y scope se obtienen del servidor. Rechaza tipo documental blanco o mayor de 60 caracteres. El try de files.stage conserva upload hasta que termine el comando.

stored aporta huella, tamaño, MIME y nombre saneado. payload incorpora esos datos y storag e_key: una huella de almacenamiento que combina empresa, sede, usuario, clave y huella del contenido. Esta clave técnica no es la ruta del archivo físico. Si hay operación, se agrega su ID. Si hay documento previo, previous se consulta con autorización, se agrega la referencia y se calcula version como previa más uno.

El comando registrar_documento usa una huella explícita de intención que incluye el nombre original recibido. No solo compara los bytes: cambiar metadatos o nombre con la misma clave puede entrar en conflicto. Devuelve 201 con el JSON canónico. Al salir del bloque se libera el bloqueo físico.

Publicar el contenido ocurre antes del SQL. Si el SQL rechaza o la respuesta queda incierta, puede quedar un objeto sin referencia. No se borra precipitadamente: podría existir un commit cuyo

resultado no llegó. La antigüedad, el chequeo de referencias A/B y el mismo bloqueo sustentan la limpieza posterior.

metadata(sedeId, id, auth) consulta documento y devuelve JSON. **download(...)** consulta esos metadatos con ámbito autorizado, obtiene metadata, verifica files.path de sha256 y entrega F ileSystemResource. Usa application/octet-stream, nombre de descarga basado en el UUID y X-Content-Type-Options: nosniff. El navegador no recibe una ruta libre a abrir en disco.

review(...) parsea el cuerpo. Si falta documentos, construye una lista con el documento de la ruta y resultado. Si el cliente ya proporcionó la lista, no la reemplaza. Delega registrar_revisio n, cuya autorización ADMIN y validación de vínculos/versiones corresponden al servicio y SQL. Devuelve 200. Es importante no confundir esta construcción condicional con la comprobación estricta corrige_a_id del controlador de pesajes.

12.11 Integraciones: tarifas, outbox y estado de operación

12.11.1 ProviderController

Referencia: `backend/src/main/java/mx/bovicierre/integraciones/ProviderController.java:15`. El constructor recibe y conserva access y commands. **tariffs(sedeId, query, auth)** convierte pagina y tamano, por defecto 1 y 25, limita tamaño a 100 y ruta a 160 caracteres. params contiene números de paginación y, si existe, ruta. Devuelve application/json con no-store mediante providerTariffs.

La consulta se lee exclusivamente en A. Después se guarda la captura en el dueño local. B no necesita consultar directamente el servidor externo para aceptar una captura ya recibida. El importe posterior se toma de esa captura local, no de una tarifa reenviada libremente por el navegador.

accept(sedeId, body, key, auth, request) exige un objeto con exactamente snapshot_id textual y comprobable como UUID. No acepta precio, hash o contexto empresarial adicionales del cliente. Invoca aceptar_tarifa con clave y correlación y devuelve 201. La aceptación y el gasto son comandos locales distintos; el gasto canónico verifica que la captura corresponda a actor y ámbito.

12.11.2 OutboxWorker: transacciones separadas, sin atomicidad distribuida

Referencia: `backend/src/main/java/mx/bovicierre/integraciones/OutboxWorker.java:16`. Sus campos jdbc y transactions tienen mapas A/B propios, construidos con workerA y workerB; enabled decide si actúa. worker es un UUID nuevo para esta instancia del proceso. unavailable es un HashSet que evita repetir el mismo aviso en cada ciclo; LOG publica mensajes técnicos g enéricos.

El constructor **OutboxWorker(a, b, enabled)** crea las dos plantillas y transacciones con roles de trabajador. El único método de trabajo **dispatch()** está programado con espera inicial de 10000 ms y demora fija de 5000 ms entre ejecuciones. No es una solicitud HTTP.

Primero, si enabled es false, sale. Para node A y B, llama despachar_outbox(50) en una transacción local. Un fallo de un nodo se registra una vez y conserva trabajo pendiente; una recuperación puede registrarse cuando unavailable pierde esa marca.

Después reclama en B hasta 20 entregas con reclamar_entregas(worker). claimed es JSON; cada event contiene id, empresa, sede y payload. La transacción de reclamo termina antes de contactar A. Para cada evento, una transacción A ejecuta consumir_muerte y una transacción B posterior confirma entrega con id y worker.

Si consume o confirma falla, otra transacción B llama fallar_entrega y registra IDENTIDAD como pendiente. Si tampoco puede completar ese manejo, el catch exterior conserva la reserva temporal hasta vencer y avisa que el propietario reintentará.

```
B: reclamar entregas y confirmar la transacción local
por cada evento reclamado:
    A: consumir identidad de forma idempotente y confirmar
    B: confirmar acuse para este trabajador
si falla:
    B: registrar fallo/backoff, cuando sea posible
    conservar evento pendiente o esperar vencimiento de lease
```

La lease y el backoff persistentes se definen en SQL; Java utiliza sus funciones, no un temporizador como prueba de posesión del evento. Si A confirmó y el acuse B se perdió, el consumo se puede repetir. La deduplicación de inbox evita repetir el efecto central. El diseño admite reintentos y no debe anunciar entrega externa exactamente una vez.

12.11.3 NodeHealth y DiagnosticsController

Referencia: `backend/src/main/java/mx/bovicierre/shared/NodeHealth.java:11`. El constructor **NodeHealth(a, b, auth, workerA, workerB)** crea nodes para probar A, B y AUTORIZACI ON_A, y pools para observar esas tres fuentes más WORKER_A y WORKER_B.

status() ejecuta select 1 en cada nodes y construye statuses con DISPONIBLE o NO_DISPONI BLE. Devuelve estado ACTIVO del proceso y los estados separados. El endpoint responde 200; estado ACTIVO no significa que las tres dependencias estén disponibles.

pools() examina cada fuente Hikari. bean es HikariPoolMXBean: result recibe activos, inactivos, esperando y máximo. Si el pool todavía no dispone del bean, informa NO_DISPONIBLE. No publica URL ni contraseña.

Referencia: `backend/src/main/java/mx/bovicierre/integraciones/DiagnosticsController.java:15`. Conserva access, commands y health por su constructor. **snapshot(sedeId, auth)** exige ADMIN vigente. sync intenta consultar sincronizacion; si hay DataAccessException, conserva un estado público de indisponibilidad en esa sección. result combina diagnostics, pools, sincronizacion y capturado_en. Devuelve no-store.

Un operador no obtiene estos detalles por conocer la ruta. La respuesta tampoco es un certificado de retraso cero de cada flujo de réplica: informa el corte que proporciona la consulta canónica y la disponibilidad instantánea de conexiones.

12.12 CSV, QR y PDF: producir representaciones sin reconocer otra operación

12.12.1 CsvController: tres formas de reporte

Referencia: `backend/src/main/java/mx/bovicierre/reportes/CsvController.java:15`. Tiene access y commands, fijados por **CsvController(access, commands)**. **export(sedeId, tipo, auth)** admite cartera, margen e inventario; otro tipo produce 404. user se obtiene de la sesión y rows acumula los nodos que serán exportados.

Para page desde uno, consulta reportes_ más tipo con tamaño 100. response es el JSON SQL. En margen agrega el objeto completo como una fila y termina: el margen canónico es un agregado, no una página que Java deba sumar. Para cartera, items sale de filas; para inventario utiliza elementos y conserva una compatibilidad con items. Agrega esas filas y termina al alcanzar total o cuando la página está vacía. Si todavía faltan filas y ya alcanzó 10000, rechaza el reporte grande.

La salida es text/csv UTF-8 con Content-Disposition y nombre por tipo. La implementación acumula las filas en memoria; no es un streaming de millones de registros ni toma filtros libres del endpoint JSON para la descarga.

cell(value) examina trimmed, el texto sin espacios iniciales. Si comienza por un signo de fórmula =, +, - o @, o por tabulación/salto de línea de control al inicio original, antepone una comilla

simple. Después duplica comillas dobles y encierra toda celda entre comillas. Esta protección evita que un nombre de contraparte se interprete ordinariamente como fórmula al abrir la hoja.

csv(rows) devuelve cadena vacía si no hay filas. keys toma las propiedades de la primera fila y define las columnas; csv es el StringBuilder. Por cada row construye cells: escalares se vuelven texto y objetos o arrays se serializan como JSON. Usa CRLF entre registros. Un Dinero permanece como objeto JSON escapado en la celda; no se convierte en double ni se recalcula.

12.12.2 AnimalQrController: matriz local para una ficha autorizada

Referencia: `backend/src/main/java/mx/bovicierre/evidencia/AnimalQrController.java:15`. El constructor **AnimalQrController(access, commands)** conserva las dependencias. **qr(sedeId, id, auth)** consulta animal con ámbito autorizado antes de crear el gráfico. url contiene /?sede={UUID}#animales/{UUID}; no incluye cookie ni token. Devuelve SVG con no-store.

svg(url) llama QRCodeWriter, con formato QR_CODE y dimensiones automáticas. matrix contiene los módulos. Los índices y y x recorren la matriz; cada módulo negro agrega a path un cuadrado SVG mediante M, h y v. El resultado tiene viewBox, fondo blanco, un path negro y etiqueta accesible. Ninguna API externa recibe los identificadores.

El QR contiene una referencia de navegación, no una credencial. Al abrir la ficha se aplican los controles de sesión. La generación local tampoco demuestra haber probado una cámara de teléfono físico; eso corresponde a evidencia de dispositivo aparte.

12.12.3 SettlementController y SettlementPdf

Referencia: `backend/src/main/java/mx/bovicierre/evidencia/SettlementController.java:13`. El constructor conserva access y commands. **receipt(sedeId, id, auth)** consulta liquidacion, convierte su JSON a liquidation y llama SettlementPdf.render. La salida son bytes application/pdf y un nombre liquidacion-{id}.pdf. No hay llamada a confirmar_operacion ni a registrar_pago .

Referencia del generador: `backend/src/main/java/mx/bovicierre/evidencia/SettlementPdf.java:9`. Su campo ENCODING es windows-1252 y su constructor privado evita instanciar la utilidad. **money(value)** concatena importe y moneda si recibió un objeto o toma su texto si recibió una forma compatible. **bytes(text)** aplica ENCODING. Ninguno calcula un saldo.

render(liquidation) arma lines: encabezado de comprobante interno no fiscal, IDs, contraparte, dirección, total, cargos, abonos, aplicaciones, saldo y vencimiento. concept recorre conceptos y agrega cantidad, unidad, precio e importe como textos canónicos. Se agrega una aclaración de que el documento deriva de hechos confirmados.

objects reúne objetos PDF en bytes. Primero se crean catálogo, colección de páginas y fuente Helvetica con WinAnsiEncoding. pages resulta de dividir las líneas en grupos de 40; kids enumera sus referencias. Cada page genera un objeto de página y otro de contenido; contentId señala este último. MediaBox usa 612 × 792 puntos, tamaño carta, no A4.

content escribe comandos de texto: tamaño 10, avance 14 y punto inicial. Se escapan barra inversa y paréntesis y se eliminan controles que podrían romper el texto. stream contiene los bytes del contenido; object agrega su longitud y delimitadores. pdf empieza con PDF-1.4, offsets guarda la posición de cada objeto y xref la posición de la tabla de referencias. El trailer señala el catálogo y startxref permite encontrar la tabla.

Esta estructura explica por qué el archivo puede abrirse sin una biblioteca PDF adicional. No incrusta una fuente Unicode completa, no genera firma digital y no es un documento fiscal. El PDF de este comprobante del producto tampoco es el compilador del manual que el lector está consultando.

12.13 Tres secuencias completas para reconstruir la comunicación

12.13.1 Secuencia 1: iniciar sesión y obtener una sede autorizada

1. El navegador solicita GET /api/v1/sesion/csrf. AvailabilityFilter prueba A técnico en `AvailabilityFilter.java:23` y la configuración permite acceso anónimo en `SecurityConfiguration.java:27`.
2. SessionController.csrf, `SessionController.java:37`, devuelve token y headerName, con no-store. El navegador conserva la cookie de sesión y utiliza la cabecera indicada. Un token ilustrativo no sirve en otro contexto real.
3. POST /api/v1/sesion/login envía login y contrasena en JSON y el token CSRF. La validación del record y el control de 72 bytes están en `SessionController.java:38`.
4. AccessService.login, `AccessService.java:17`, busca el usuario en la empresa configurada. SessionController compara BCrypt, exige ACTIVO y usa dummyHash si falta una identidad.
5. `SessionController.java:46` crea o utiliza la sesión y cambia su identificador. Guarda el contexto autenticado, invalida el CSRF anterior y devuelve 204.
6. El navegador vuelve a solicitar CSRF y consulta /sesion/actual. `SessionController.java:59` agrupa sedes, roles y capacidades. El usuario puede elegir una sede de esa lista.
7. En solicitudes posteriores el filtro de `SecurityConfiguration.java:38` vuelve a consultar current. Bloquear al usuario afecta una nueva petición aunque el navegador conserve su cookie.

```
GET CSRF -> cookie de sesión + token inicial
POST login + token -> usuario ACTIVO y contraseña comprobados
rotación de sesión -> 204 y token antiguo invalidado
GET CSRF nuevo -> token de la sesión autenticada
GET sesión actual -> sedes y roles vigentes
```

Las referencias abreviadas de esta sección tienen el prefijo `backend/src/main/java/mx/bovicierre/acceso/`. El flujo no usa JWT, HTTP Basic ni una contraseña fija introducida por el ejemplo.

12.13.2 Secuencia 2: confirmar una operación y recuperar una respuesta perdida

1. GET /api/v1/sedes/{sedeId}/operaciones/{id} usa CoreController.one, `operaciones/CoreController.java:96`. El JSON que contiene version_lock produce ETag por result, línea24.
2. El operador prepara un intento con la versión original, Idempotency-Key nueva y cuerpo de reconocimiento. El precio y lotes pertenecen al borrador; el navegador no envía un total autoritativo.

```
{
  "periodo_id": "00000000-0000-4000-8000-000000000002",
  "fecha_reconocimiento": "2026-10-12"
}
```

3. POST /operaciones/{id}/confirmar lleva If-Match, por ejemplo “3”, y X-XSRF-TOKEN. action, `CoreController.java:113`, elige confirmar_operacion; write, línea30, fija operacion_id y version_lock del servidor.
4. `CommandService.java:32` autoriza nuevamente, escoge el dueño y abre su TransactionTemplate. No abre otra transacción en el nodo opuesto.
5. PostgreSQL V13 reclama la intención bajo el ámbito y principal. La fila idempotente y el retorno previo aparecen en `db/migrations/V13__correccion_composicion_exacta.sql:12`. Si la intención está completada, devuelve el resultado original antes de la nueva comprobación de versión.
6. Para una ejecución nueva, el SQL comprueba periodo de reconocimiento y versión bajo bloqueo, condiciones, pesajes y recursos. La entrada a confirmación se observa en V13:129. Los cálculos y escrituras son responsabilidad del capítulo de base de datos.

7. El SQL produce hechos económicos, auditoría, outbox y resultado del intento. La transacción local confirma; Java devuelve ese JSON. Una excepción revierte las escrituras de esa ejecución.
8. Si el navegador perdió la respuesta, conserva clave, cuerpo y ETag originales. Consulta GET /comandos/confirmar_operacion/{clave}, `CoreController.java:64`. Un resultado completado permite recuperar el objeto sin otro POST.
9. Si el resultado aún no es visible, una ausencia no demuestra rollback: la transacción puede seguir en curso o la consulta puede fallar. Cuando corresponda reintentar, se conserva exactamente la intención. Cambiar fecha, versión o cuerpo con la misma clave provoca conflicto.

$$\text{Misma clave y misma intención} \Rightarrow \text{resultado persistido anterior}$$

$$\text{Misma clave y distinta intención} \Rightarrow 409$$

Este retorno no implica volver a crear liquidación, deuda o asientos. Los bloqueos y la clave persistida coordinan intentos simultáneos; la cabecera del navegador, por sí sola, no garantiza idempotencia.

12.13.3 Secuencia 3: publicar una versión documental y descargarla

1. El cliente envía multipart a /api/v1/sedes/{sedeId}/documentos con archivo y tipo_documental; opcionalmente operación y documento previo. Conserva su clave de intento y token CSRF.
2. DocumentController.upload, `evidencia/DocumentController.java:22`, obtiene usuario y Scope y valida el tipo documental. No utiliza una ruta de disco enviada por el cliente.
3. PrivateDocuments.stage, `evidencia/PrivateDocuments.java:64`, valida tamaño y firma, calcula SHA-256, adquiere bloqueo local y de archivo, escribe .part y publica el contenido mediante movimiento atómico.
4. Upload devuelve Stored y mantiene el bloqueo. DocumentController construye metadata y storage_key. Si es una nueva versión, consulta el previo y obtiene su número para incrementarlo.
5. commands.command ejecuta registrar_documento con su intención y transacción local. Un commit deja metadata y resultado idempotente; el cierre del bloque libera el bloqueo. Un rechazo puede dejar contenido físico huérfano, sin inventar metadata aprobada.
6. Un retry igual puede recuperar el metadata anterior. La limpieza no intenta resolver esa incertidumbre borrando inmediatamente el archivo.
7. GET /documentos/{id}/contenido consulta primero metadata autorizado. PrivateDocuments.path, línea116, verifica ruta, archivo regular y hash. DocumentController.download, línea42, publica el recurso como adjunto, sin interpretación de tipo en el navegador.
8. Una revisión administrativa referencia la versión documental correspondiente; crear otra versión no convierte retroactivamente la primera revisión en revisión de todos los contenidos futuros.
9. Pasadas 24 horas, el procedimiento de limpieza puede diagnosticar objetos sin referencia. CleanupMain consulta A y B bajo rol técnico. Solo el modo explícito de ejecución elimina candidatos comprobados y no bloqueados.

Aquí no hay una transacción distribuida que englobe simultáneamente PostgreSQL y el disco. La protección procede del protocolo de staging, huella, bloqueo durante metadata, idempotencia y limpieza conservadora.

12.14 Inventario del contrato HTTP y sus llamadores Java

Todas las rutas siguientes llevan el prefijo /api/v1. Para compactar la lectura, **S** representa /sedes/{sedeId}, **O** representa operaciones/{operacionId}, **P** pagos/{pagoId}, **D** documentos/{documentoId}, **L** liquidaciones/{liquidacionId}, **G** gastos/{gastoId} y **T** periodos/{periodoId}. La tabla enumera los 70 operationId, no 70 clases.

Método y ruta relativa	operationId del contrato	Método Java principal
POST /sesion/login	login	SessionController.login
POST /sesion/logout	logout	SessionController.logout
GET /sesion/actual	sesionActual	SessionController.current
GET /sesion/csrf	obtenerCsrf	SessionController.csrf
GET /salud	consultarSaludProceso	SessionController.health
GET S/contrapartes	listarContrapartes	CoreController.list
POST S/contrapartes	crearContraparte	CoreController.create
POST S/animales	registrarIdentidadAnimal	CoreController.create
GET S/animales	listarAnimales	CoreController.list
GET S/animales/{animalId}	fichaAnimal	CoreController.one
GET S/animales/{animalId}/qr	qrAnimal	AnimalQrController.qr
GET S/operaciones	bandejaOperaciones	CoreController.list
POST S/operaciones	crearBorrador	CoreController.create
GET S/O	obtenerOperacion	CoreController.one
PATCH S/O	editarBorrador	CoreController.edit
POST S/O/confirmar	confirmarOperacion	CoreController.action
POST S/O/pesajes	proponerPesaje	CoreController.action
POST S/pesajes/{pesajeId}/aprobar	aprobarPesaje	CoreController.action
POST S/pesajes/{pesajeId}/correcciones	corregirPesaje	CoreController.action
POST S/O/reservas	reservarPosiciones	CoreController.action
POST S/reservas/{reservaId}/liberar	liberarReserva	CoreController.action
GET S/inventario/posiciones	listarPosiciones	CoreController.list
POST S/inventario/movimientos	registrarMovimiento	CoreController.create
GET S/L	detalleLiquidacion	CoreController.one
POST S/L/ajustes	ajustarLiquidacion	CoreController.action
GET S/L/comprobante	comprobanteInterno	SettlementController.receipt
POST S/gastos	registrarGasto	CoreController.create
GET S/gastos	listarGastos	CoreController.list
GET S/G/reparto	previsualizarReparto	CoreController.one
POST S/G/reparto	aprobarReparto	CoreController.action
POST S/pagos	registrarPago	CoreController.create
GET S/pagos	listarPagos	CoreController.list
POST S/P/aplicaciones	aplicarPago	CoreController.action
POST S/aplicaciones/{aplicacionId}/reverso	revertirAplicacion	CoreController.action
POST S/P/reembolsos	registrarReembolso	CoreController.action
GET S/reportes/cartera	reporteCartera	CoreController.list
GET S/reportes/margen	reporteMargen	CoreController.list
POST S/T/cierre	cerrarPeriodo	CoreController.action
POST S/documentos	subirDocumento	DocumentController.upload
GET S/documentos	listarDocumentos	CoreController.list

Método y ruta relativa	operationId del contrato	Método Java principal
GET S/D/contenido	descargarDocumento	DocumentController.download
GET S/integraciones/proveedor/tarifas	consultarTarifasProveedor	ProviderController.tariffs
GET S/liquidaciones	listarLiquidaciones	CoreController.list
POST S/P/validar	validarPago	CoreController.action
GET S/comandos/{comando}/{clave}	consultarResultadoComando	CoreController.commandResult
GET S/usuarios	listarUsuarios	CoreController.list
POST S/usuarios	crearUsuario	UserController.create
GET S/periodos	listarPeriodos	CoreController.list
POST S/periodos	crearPeriodo	CoreController.create
GET S/predios	listarPredios	CoreController.list
POST S/predios	crearPredio	CoreController.create
PATCH S/usuarios/{usuarioId}	editarUsuario	UserController.update
POST S/usuarios/{usuarioId}/membresias	asignarMembresia	UserController.membership
POST S/T/reabrir	reabrirPeriodo	CoreController.action
GET S/T/versiones	consultarVersionesCierre	CoreController.one
POST S/O/acordar	acordarOperacion	CoreController.action
POST S/O/reabrir	reabrirOperacion	CoreController.action
POST S/O/cancelar	cancelarOperacion	CoreController.action
POST S/O/anular	anularOperacion	CoreController.action
GET S/O/simulador	simularOperacion	CoreController.one
GET S/O/expediente	consultarExpediente	CoreController.one
GET S/P	consultarPago	CoreController.one
GET S/G	consultarGasto	CoreController.one
GET S/D	consultarDocumento	DocumentController.metadata
POST S/D/revisiones	revisarDocumento	DocumentController.review
GET S/pesajes/comparacion	compararPesajes	CoreController.compare
GET S/reportes/{tipo}/csv	exportarCsv	CsvController.export
GET S/sincronizacion	consultarSincronizacion	CoreController.list
POST S/integraciones/proveedor/tarifas/aceptaciones	aceptarTarifa	ProviderController.accept
GET S/integraciones/diagnostico	consultarDiagnostico	DiagnosticsController.snapshot

El catálogo muestra reutilización de métodos. Para conocer un contrato completo hay que consultar su esquema, cabeceras y errores OpenAPI; compartir CoreController.action no hace iguales un reverso, una confirmación y una reserva.

12.15 Cobertura, evidencia y límites de este capítulo

Se explicaron las 24 unidades de fuente de producción, sus constructores explícitos, todos sus métodos declarados y los campos relevantes. También se describieron los records anidados y sus accesores generados, el filtro anónimo de seguridad, argumentos HTTP y variables que deciden ámbito, transacción, hash, metadatos, representación y reintentos. Los imports y métodos heredados del framework no se presentan como lógica nueva de BoviCierre.

La evidencia `backend/verification-result-v13.json` registra 24 pruebas aprobadas: 15 unitarias, ocho HTTP por Caddy15180 y un PostgreSQL 18.6 fresco con V1-V13. Incluye contenido de

CSV, composición de pesajes, motivos, idempotencia, sesiones, archivos y controles de autoriz ación. Este conjunto no prueba exhaustivamente las 70 operaciones y todas las combinaciones de entrada.

Hay límites concretos que deben conservarse al explicar el código. Los permisos detallados se verifican en servicios y SQL, no en la capacidad genérica ESCRIBIR. La sesión y autorización dependen de A. No hay failover automático de propietario, 2PC ni transacción conjunta entre disco y base. Los uploads y exports usan memoria y tamaño acotado; LOCKS no tiene una política explícita de expulsión en esta clase. El almacenamiento requiere permisos POSIX y movimiento atómico del sistema de archivos utilizado.

La firma inicial de un archivo no es antimalware. El PDF no incrusta Unicode completo ni firma fiscalmente un documento. El QR no acredita pruebas de cámara móvil. La aplicación inspeccionada no implementa en estas clases un mecanismo de rate limiting de login; BCrypt y mensajes uniformes no sustituyen ese control. Las métricas no son historial persistente, p95 de HTTP ni certificación de capacidad. Estos límites identifican el alcance ejecutable; no autorizan nuevas modificaciones del producto por escribir el manual.

Finalmente, una clase compacta puede cumplir una responsabilidad sin estar separada en diez archivos. Eso explica por qué el router real es una decisión en CommandService y por qué hay un puerto documental concreto. Para evaluar mantenibilidad futura conviene comparar responsabilidades observadas con la arquitectura planeada, conservando primero las reglas SQL, el contrato y la evidencia.

12.16 Fuentes primarias y localización en el repositorio

Prefijo **J**: `backend/src/main/java/mx/bovicierre/`. Las referencias J de esta lista deben expandirse con ese prefijo. Cada archivo Java fue leído completo, no únicamente la línea citada.

Fuente	Punto de entrada para estudiar	Contenido utilizado
`J/BoviCierreApplication.java:8`	main	Inicio, scheduling y PID
`J/shared/DatabaseConfiguration.java:15`	pool y beans	Cinco conexiones y transacción de sesión
`J/shared/CommandService.java:16`	command y query	Router compacto, transacciones, reintentos y diagnóstico
`J/shared/JsonSupport.java:11`	utilidades	JSON, decimales, hash, ETag e identificaciones
`J/shared/ApiException.java:3`	constructor	Excepción pública
`J/shared/ApiErrors.java:15`	handlers	Traducción de SQLSTATE y UTF-8
`J/shared/NodeHealth.java:11`	status y pools	Disponibilidad y Hikari
`J/acceso/AccessService.java:14`	authorize	Usuario, empresa, sedes y roles
`J/acceso/AvailabilityFilter.java:19`	doFilterInternal	Disponibilidad central
`J/acceso/SecurityConfiguration.java:21`	security	Contexto, CSRF y filtros
`J/acceso/SessionController.java:26`	login/current	Sesión y salud
`J/acceso/UserController.java:14`	write	Usuarios, BCrypt y membresías

Fuente	Punto de entrada para estudiar	Contenido utilizado
J/operaciones/CoreController.java:16	write/action	Adaptador de recursos económicos
J/evidencia/application/DocumentStorage.java:7	interfaces	Puerto y ciclos de contenido
J/evidencia/PrivateDocuments.java:27	stage/cleanup/path	Adaptador local y locks
J/evidencia/DocumentController.java:16	upload	Metadata, contenido y revisión
J/evidencia/CleanupMain.java:9	main	Diagnóstico/limpieza técnica
J/evidencia/AnimalQrController.java:15	qr/svg	QR local
J/evidencia/SettlementController.java:13	receipt	Comprobante autorizado
J/evidencia/SettlementPdf.java:9	render	Estructura PDF
J/integraciones/ProviderController.java:15	tariffs/accept	Fuente y captura aceptada
J/integraciones/OutboxWorker.java:16	dispatch	Reclamo, consumo y acuse
J/integraciones/DiagnosticsController.java:15	snapshot	Diagnóstico ADMIN
J/reportes/CsvController.java:15	export/csv/cell	Representaciones y protección de celdas
backend/pom.xml:1	Dependencias	Boot, Java, JDBC, seguridad, sesión y ZXing
backend/src/main/resources/application.yml:1	Propiedades	Límites, conexiones, sesión y apagado
backend/src/main/resources/application-production.yml:1	Perfil	Cookie Secure
docs/api/openapi.yaml:1	Contrato completo	70 operationId; tipos, cabeceras y estados
docs/development/backend-context.md:1	Contexto vigente	V13/V12, evidencia y despliegue integrado
docs/architecture/overview.md:1	Diseño	Módulos y clases planeados, diferenciados del código
db/migrations/V13__correccion_composicion_exacta.sql:12	Frontera SQL	Resultado idempotente y confirmación canónica
backend/verification-result-v13.json:1	Registro histórico	24 pruebas ejecutadas el 11-oct-2026

Para verificar afirmaciones de cálculo y restricciones físicas, continuar con el capítulo de PostgreSQL y las migraciones canónicas. Este capítulo muestra cómo el programa Java llega a esas funciones, cómo protege y presenta la comunicación, y dónde termina su responsabilidad.

Capítulo 13

Frontend de BoviCierre: manual didáctico del software implementado

13.1 1. Qué hace este capítulo y cómo leerlo

Este capítulo explica el frontend real de BoviCierre, una aplicación para registrar y cerrar compras y ventas de ganado en pie, conservar su evidencia y conocer las obligaciones por cobrar o pagar. Se basa en la lectura del código vigente el 12 de octubre de 2026. Los recorridos y resultados citados corresponden a la evidencia local del 11 de octubre; preparar este capítulo no ejecutó nuevamente la aplicación ni alteró registros. Todos los ejemplos son DEMO, sintéticos y didácticos. Sus identificadores se deben sustituir por los devueltos por una API autorizada. No contienen contraseñas, cookies ni credenciales.

El lector puede empezar por el recorrido operativo y regresar al catálogo de componentes cuando encuentre un nombre desconocido. También puede estudiar primero las capas de datos, porque explican por qué ciertos botones permanecen deshabilitados después de un problema de red. La interfaz no consiste sólo en una colección de formularios: conserva la relación entre una intención humana, una versión de un expediente y el resultado que confirmó el servidor. Comprender esa relación permite diagnosticar un error sin repetir accidentalmente una operación económica.

La separación principal es la siguiente. React organiza controles, presenta resultados y conserva el estado de una interacción. La API autentica, autoriza y conduce comandos hacia las reglas del dominio. PostgreSQL calcula importes y valida las transiciones económicas. Por tanto, una liquidación visible no se obtiene sumando celdas en el navegador. Es una representación de un resultado canónico recibido. Aunque el código use `Number` para páginas, números de lote o contadores, esos usos no constituyen aritmética monetaria. Importes y pesos viajan como cadenas decimales.

Hay quince pantallas conceptuales en la documentación de producto, pero el menú implementado contiene diecinueve destinos. Algunas pantallas conceptuales abarcan una lista y un detalle; otras acciones adicionales, como documentos, predios o periodos, aparecen como destinos propios. Esa diferencia no significa que existan diecinueve aplicaciones ni diecinueve diseños independientes. Todos los destinos reutilizan la misma estructura, campos, tablas, avisos y reglas de recuperación.

Fuentes principales: `frontend/src/App.tsx:7`, `frontend/src/screens.tsx:8`, `frontend/src/ui.tsx:4`, `frontend/src/api.ts:13`, `docs/ui/flows-and-screens.md:18`, `docs/development/frontend-context.md:1`.

13.2 2. Mapa de archivos, dependencias y ejecución

La aplicación tiene una entrada pequeña, un contenedor de navegación, una colección de pantallas y cuatro módulos compartidos. Esta organización permite seguir un dato desde el campo donde se captura hasta la solicitud HTTP que lo envía. No hay que buscar una segunda biblioteca de componentes ni un estado global basado en Redux: las dependencias de ejecución declaradas son React y React DOM. El resto corresponde a compilación, tipos, generación del contrato y pruebas.

Archivo o grupo	Responsabilidad	Punto de entrada
`main.tsx` y `index.html`	Montar React y cargar estilos	`frontend/src/main.tsx:5`
`App.tsx`	Acceso, sede, navegación, catálogos y despacho de vistas	`frontend/src/App.tsx:19`
`screens.tsx`	Operaciones, liquidaciones, dinero y controles específicos	`frontend/src/screens.tsx:11`
`ui.tsx`	Recursos, avisos, campos, comandos, tablas y listas	`frontend/src/ui.tsx:4`
`api.ts`	HTTP, CSRF, errores e intentos persistidos	`frontend/src/api.ts:1`
`domain.ts` y `workflow.ts`	Representación decimal, tipos y composición de lotes	`frontend/src/domain.ts:1`
`styles.css`	Identidad, estructura y adaptación de tamaños	`frontend/src/styles.css:1`
`generated/api-schema.ts`	Tipos producidos desde OpenAPI	`frontend/package.json:16`
`tests` y `tests_e2e`	Verificación unitaria y recorridos reales	`frontend/playwright.config.ts:2`

Las versiones declaradas son React 19.2.4, TypeScript 5.9.3, Vite 8.0.3 y Playwright 1.58.2. El archivo de paquetes fija las versiones y el archivo de bloqueo permite una instalación reproducible mediante `npmci`. Los entornos documentados son Node 20.20.2 o 24.21.0; el rango técnico de `engines` es más amplio y no debe confundirse con una prueba realizada en cada versión que admite ese rango. Saber qué está declarado y qué se ejecutó evita convertir compatibilidad teórica en evidencia práctica.

La entrada integrada es `http://localhost:15180`, servida por Caddy y los contenedores locales. Vite ofrece una alternativa de desarrollo en `http://127.0.0.1:15173`, con proxy de `/api` hacia `http://127.0.0.1:18080`. El navegador sigue utilizando rutas relativas del mismo origen. No descubre nodos PostgreSQL ni selecciona al propietario de una sede. La elección de base de datos pertenece al backend. Tampoco deben iniciarse dos procesos de API sobre el mismo puerto para intentar resolver una falla del frontend.

Script	Qué hace	Qué no demuestra por sí solo
`dev`	Inicia Vite en loopback y puerto 15173	Persistencia correcta o recorrido aprobado
`generate:api`	Produce tipos desde OpenAPI	Que el servidor entregue todos los DTO correctamente
`typecheck`	Comprueba TypeScript con `tsc-b`	Resultado funcional en un navegador
`test`	Ejecuta pruebas unitarias con `tsx`	Comandos reales de PostgreSQL
`build`	Comprueba tipos y genera archivos estáticos	Rendimiento GPU ni aceptación humana

Script	Qué hace	Qué no demuestra por sí solo
`preview`	Sirve la compilación local	Despliegue comercial
`test:e2e`	Ejecuta Playwright según su configuración	Cobertura universal o prueba en teléfono físico

Estos comandos se explican para que el lector los entienda; no son instrucciones para ejecutarlos indiscriminadamente sobre un entorno compartido. Los recorridos E2E crean datos DEMO y necesitan una ventana coordinada. La prueba física del proveedor además requiere que otro responsable detenga y reanude únicamente ese servicio. El runner no controla Docker por su cuenta.

Fuentes: `frontend/package.json:1`, `frontend/vite.config.ts:3`, `frontend/tsconfig.json:1`, `frontend/README.md:15`, `frontend/playwright.config.ts:2`.

13.3 3. HTML inicial y montaje de React

`index.html` declara idioma español, UTF-8, viewport adaptable, color de tema y el elemento `root`. Ese elemento es el punto donde React administra el árbol de interfaz. El archivo HTML no contiene una lista precargada de ganado ni una sesión de ejemplo. Su tarea es ofrecer una base mínima y cargar el módulo de entrada. El título de la pestaña identifica el producto, mientras que los encabezados internos cambian con la pantalla activa.

`main.tsx` importa React, `createRoot`, `App` y la hoja de estilos. La llamada `createRoot(document.getElementById('root')!)` establece una única raíz. El operador de afirmación no nula comunica a TypeScript que el HTML esperado contiene ese elemento; no constituye una recuperación si alguien elimina el contenedor. Después se renderiza `App` dentro de `React.StrictMode`. Este modo ayuda a detectar comportamientos problemáticos durante el desarrollo, pero puede ejecutar ciclos adicionales de efectos. Por eso una consulta anónima inicial puede repetirse y debe distinguirse de un comando económico duplicado.

Los eventos de envío requieren una acción del usuario y las claves de idempotencia se conservan en el módulo de comandos. El montaje de React no confirma una compra. Esta distinción es útil al interpretar la consola: ver una consulta repetida en desarrollo no permite concluir que se hayan creado dos obligaciones. Hay que identificar método, ruta y clave del comando antes de diagnosticar duplicación.

El CSS se carga desde la entrada, de modo que componentes de acceso, catálogo y dinero reciben los mismos tokens. No existe una hoja distinta para cada sede. La sede modifica el ámbito de datos, no la identidad gráfica. Si la aplicación no muestra nada, la revisión comienza por el HTML, la carga del módulo y errores JavaScript; si muestra una pantalla pero falla una solicitud, el diagnóstico se mueve a `request`, sus parámetros y el servidor.

Fuentes: `frontend/index.html:2`, `frontend/src/main.tsx:1`, `frontend/src/main.tsx:5`.

13.4 4. Tipos del dominio y representación exacta

`domain.ts` exporta cinco tipos básicos. `Money` reutiliza el esquema `Dinero` generado desde OpenAPI. `Sede` contiene identidad, empresa, código, nombre, roles y algunos datos opcionales de capacidad o disponibilidad. `Session` conserva las sedes autorizadas y admite tanto la identidad superior como una estructura de usuario usada como compatibilidad. `Row` representa objetos de consulta con campos desconocidos e identificador opcional. `Page` contiene `elementos`, página, tamaño y total.

Usar `Row` permite que una tabla genérica reciba columnas distintas, pero también significa que TypeScript no garantiza automáticamente cada campo leído. Por eso el código verifica tipos al mostrar dinero, peso o listas, y ofrece textos de ausencia. Los esquemas generados se emplean

con mayor precisión en entradas específicas, como `PredioNuevo` y `PesajeNuevo`. Es importante reconocer esta combinación: el frontend tiene tipos de contrato, aunque no todas sus lecturas sean objetos completamente especializados.

13.4.1 4.1 Función decimal

Firma: `decimal(value:unknown):string`. Su responsabilidad es representar una cadena numérica recibida, sin convertirla a coma flotante. Si el valor no es una cadena decimal reconocida, devuelve `Nodisponible`. Cuando es válido, separa parte entera y fracción, añade comas de agrupación a la parte entera y conserva exactamente la fracción. Por ejemplo, `9007199254740993.01` se muestra como `9,007,199,254,740,993.01`, pese a superar el entero que JavaScript representa con precisión como `number`.

La función acepta un signo negativo para presentación, porque un valor calculado puede necesitarlo. No normaliza la escala ni decide si un importe está permitido en una captura. También conserva `0.000` como `0.000`: tres decimales de peso no se reducen silenciosamente a dos. La llaman `money`, `text` y la comparación de tickets. Su error visible es un texto de ausencia, no una excepción que derribe la pantalla.

13.4.2 4.2 Función money

Firma: `money(value:unknown):string`. Comprueba que exista un objeto con `importe`, pasa ese importe a `decimal` y añade la moneda. Cuando falta moneda utiliza la presentación MXN; cuando falta el objeto esperado muestra `Nodisponible`. Se emplea en resumen de liquidación, disponible del pago, tarifas, reportes e inventario. No calcula el saldo y no valida que una moneda arbitraria sea comercialmente compatible. Esas guardas siguen perteneciendo al servidor.

La diferencia entre formato y cálculo debe quedar clara. Presentar `1000.00` con una coma facilita la lectura, pero no cambia su valor ni introduce un nuevo hecho. El texto mostrado nunca se vuelve a analizar con `parseFloat` para confirmar una operación. Si la API omite saldo, la función no inventa `0.00`; la ausencia permanece visible y permite investigar el DTO.

13.4.3 4.3 Función canonicalDecimal

Firma: `canonicalDecimal(value:string,scale:number):string`. Valida una captura positiva mediante una expresión regular, elimina ceros iniciales redundantes y completa ceros hasta la escala solicitada. Para escala dos permite hasta dieciséis dígitos enteros; para tres, once; para cuatro, diez; las otras escalas usadas por porcentajes tienen otra cota. La salida de `048.5` con escala cuatro es `48.5000`. La salida de `0` con escala dos es `0.00`.

Rechaza exponentes, comas de miles, negativos, infinitos, texto vacío y exceso de precisión . Rechazar `1.001` al pedir dos decimales evita que la interfaz decida un redondeo económico . Su responsabilidad es validar y normalizar representación. Si hay que redondear un cálculo confirmado, lo hace SQL conforme a la política canónica. `canonicalDecimal` lanza una excepción con mensaje de captura; `CommandForm` la captura, enfoca el resumen y deja al usuario corregir antes de enviar.

13.4.4 4.4 Función text

Firma: `text(value:unknown):string`. Es el adaptador de presentación más utilizado. Traduce ausencia a `Nodisponible`, booleanos a `Sí` o `No`, dinero mediante `money`, peso mediante `decimal` y la unidad kg, arreglos mediante la unión de sus representaciones e identificaciones mediante sistema y código declarado. Un objeto genérico que no reconoce se representa como `Verdetall e`. Finalmente sustituye guiones bajos de estados por espacios.

Esta función explica por qué el código SQL puede usar `PENDIENTE_RECEPCION` y el usuario leer `P ENDIENTERECEPCION`. También explica que un objeto de JSON no se vuelque íntegramente en una

celda. La representación compacta facilita trabajar con datos, aunque no sustituye una vista de detalle cuando el objeto posee información que la tabla no expone.

13.4.5 4.5 Función today

Firma: `today(date=newDate()):string`. Obtiene año, mes y día con `Intl.DateTimeFormat` en `America/Mexico_City` y construye una fecha ISO. No utiliza simplemente la fecha UTC de `toISOString`, porque una tarde mexicana puede corresponder al día siguiente en UTC. La usan fechas predeterminadas del trato, pagos, gastos y reconocimiento. Su parámetro opcional permite comprobar instantes fijos en pruebas.

Una fecha predeterminada es una ayuda de captura, no una decisión contable automática. El usuario sigue seleccionando periodo y fecha, y el servidor comprueba que la combinación sea admisible. El ejemplo unitario `2026-10-12T01:00:00Z` produce `2026-10-11` en México, mostrando el problema que se evitó.

Fuentes: `frontend/src/domain.ts:2`, `frontend/src/domain.ts:7`, `frontend/src/domain.ts:12`, `frontend/src/domain.ts:17`, `frontend/src/domain.ts:22`, `frontend/src/domain.ts:32`, `frontend/tests/domain.test.ts:8`.

13.5 5. Dos auxiliares que preservan la identidad comercial

`replaceLot(lots:Row[],replacement:Row):Row[]` reemplaza el lote con el mismo `numero_lote`, conserva los demás y ordena el resultado por ese número. No modifica el arreglo original. La ordenación usa números de lote, no precios. `LotEditor` lo llama para construir el PATCH completo de condiciones. Es relevante porque el servidor puede relacionar la ordinalidad del arreglo con la identidad del lote: editar el primero no debe enviarlo al final ni intercambiar animales con el segundo.

La prueba unitaria conserva la referencia del lote intacto y comprueba que el precio original del primer objeto no cambie. Además, el recorrido real editó dos lotes antes y después de un ticket aprobado. Esto no convierte al auxiliar en un motor contractual: su alcance es construir una composición ordenada. La autorización para editar, el estado de borrador y la conservación de versiones se validan en el servidor.

`operationAnimals(operation:Row,positions:Row[]=[]):string[]` obtiene los animales de cada lote. En una compra puede leer `lote.animales`. En una venta tiene que resolver `lote.posiciones` contra las posiciones consultadas y obtener sus `animal_id`. Si una posición no tiene identidad resuelta, lanza un error que exige consultar toda la composición. Nunca utiliza un arreglo vacío de animales como sustituto silencioso de una venta conocida.

El error se captura antes de enviar un ticket. Esta conducta protege un dato fundamental: el pesaje corresponde a un conjunto concreto de animales. Tener trescientos kilogramos sin saber qué identidades integraron el grupo no equivale a un ticket válido. `OperationEditor` usa el auxiliar al presentar pesos opcionales y al construir la propuesta; `useSalePositions` lo emplea para verificar que terminó de resolver las posiciones requeridas.

Fuentes: `frontend/src/workflow.ts:3`, `frontend/src/workflow.ts:8`, `frontend/src/screens.tsx:14`, `frontend/src/screens.tsx:20`, `frontend/tests/workflow.test.ts:6`.

13.6 6. La frontera HTTP: solicitar, reconocer errores y recuperar comandos

El módulo `api.ts` concentra reglas que sería peligroso copiar de manera distinta en cada pantalla. Sus variables de módulo son `csrf`, `csrfHeader` y `principal`. No son tokens almacenados en disco por el frontend. `csrf` conserva el valor actual en memoria; `csrfHeader` comienza con un nombre conocido y se ajusta a la cabecera autorizada que entrega el servidor. `principal`

identifica a quién pertenece el espacio de intentos persistidos. El navegador maneja la cookie de sesión; el código no la extrae para guardarla en `localStorage`.

13.6.1 6.1 ApiError y clasificación del resultado

`ApiError` extiende `Error`. Su constructor recibe `status` y detalles opcionales: mensaje, regla, referencia de correlación y lista de campos problemáticos. Para 401 explica que terminó la sesión; para 403 y 404 utiliza un mismo mensaje de falta de acceso, evitando diferencias innecesarias de presentación. Para 412 avisa que otra persona cambió el registro y para 428 que falta su versión. Para 503 ofrece conservar la captura. El estado cero representa un resultado que no se pudo conocer por red o por interpretación de la respuesta.

Una excepción no describe automáticamente el estado económico del servidor. Un POST pudo confirmar y perder su respuesta. Por eso `keepAttempt(error:Error):boolean` clasifica si debe conservarse la intención. Retiene errores que no sean `ApiError`, estado cero, 401, 408, 409, 425, 429 y cualquier 5 xx. Descarta el intento pendiente en rechazos conocidos como 400, 403, 404, 412, 413, 422 o 428, permitiendo corregir una captura resuelta. El 409 tiene tratamiento conservador y ofrece una acción explícita para resolver y volver a capturar.

Situación observada	Tratamiento del intento	Siguiente conducta
Respuesta perdida o 5xx	Mantener clave, cuerpo y versión	Consultar resultado antes de reenviar
Sesión expirada	Mantener almacenamiento por principal	Reautenticar al mismo usuario
Comando en curso 425	Mantener intención	Esperar y volver a consultar
Conflicto 409	Mantener hasta resolución explícita	Revisar conflicto, no cambiar clave a escondidas
Regla 422 o captura 400	Resolver el intento rechazado	Corregir campos y formular una intención nueva
Versión 412 o 428	Resolver el envío inválido	Obtener versión vigente antes de continuar

13.6.2 6.2 setPrincipal, clearSession y refreshCsrf

`setPrincipal(id:string|undefined)` actualiza la identidad que forma el prefijo del almacena miento. La llaman `App` al recuperar una sesión o completar login y las pruebas de aislamiento . `clearSession()` borra CSRF e identidad en memoria y emite `session-cleared` cuando existe ventana. No recorre ni elimina las entradas anteriores de `sessionStorage`. Por ello el mismo principal puede recuperar posteriormente un intento, mientras que otro principal no obtiene la misma clave de almacenamiento.

`refreshCsrf()` solicita `/sesion/csrf` mediante `request`, lee token y nombre de cabecera y admite únicamente `X-CSRF-TOKEN` o `X-XSRF-TOKEN`. Un nombre distinto provoca error. Después actualiza las variables en memoria. El acceso lo llama antes del POST de login y otra vez después, para utilizar el contexto que corresponde a la sesión recién establecida. Una prueba unitaria comprueba la cabecera recibida, sin presentar ese ensayo como una prueba de cookie real.

Estas funciones separan tres conceptos que suelen confundirse: identidad de usuario, cookie de sesión y defensa CSRF. La identidad organiza el almacenamiento local; la cookie autentica ante el servidor; el token CSRF acompaña escrituras del mismo origen. Ninguno de esos mecanismos sustituye la autorización de una sede o un recurso en el backend.

13.6.3 6.3 request

Firma: `request<T>(path:string,options:RequestInit={}):Promise<{data:T;etag:string|null}>`. Construye `Headers` a partir de las opciones. Si el cuerpo es una cadena y falta tipo de contenido, declara JSON. Si la petición tiene método explícito distinto de GET y existe CSRF,

añade la cabecera correspondiente. Ejecuta `fetch('/api/v1'+path,...)` con `credentials:'same-origin'`. Las rutas de negocio llegan normalmente como `/sedes/<sede>/...`.

Un fallo de `fetch` se convierte en `ApiError(0)`. Una respuesta no exitosa intenta leer sus detalles JSON; si es 401, emite `session-expired` antes de lanzar el error. Una respuesta 204 devuelve datos indefinidos. Las demás respuestas exitosas se interpretan como JSON; si esa interpretación falla, vuelve a tratar el resultado como desconocido. Finalmente conserva la cabecera `ETag`, que no es una cantidad económica, sino una versión para concurrencia.

Los recursos cancelables pasan `signal` en `options`. La función no conserva una caché global de respuestas ni reintenta automáticamente. Tampoco incorpora un selector de base de datos. Si una vista muestra datos obsoletos, hay que investigar su ciclo de montaje, revisión y petición; no existe un almacén universal oculto que garantice frescura. Las descargas y QR utilizan enlaces o imágenes del mismo origen, pues `request` espera JSON y no es el adaptador de todos los tipos binarios.

13.6.4 6.4 Attempt, createAttempt y executeAttempt

`Attempt` contiene `path`, `method`, `body`, `key` y una versión `etag` opcional. `createAttempt(path,body,method='POST',etag?)` serializa el cuerpo inmediatamente y crea una UUID mediante `crypto.randomUUID()`. Serializar en ese momento evita que una mutación posterior del objeto de captura cambie el contenido de un reintento. La clave se crea una vez por intención; no una vez por cada clic posterior.

`executeAttempt<T>(attempt,checkResult=false)` permite consultar previamente el comando. Si `checkResult` está activo y el resultado está completado, devuelve el resultado persistido sin nuevo POST. Si no está completado y la consulta permite continuar, utiliza `request` con el método y cuerpo originales, `Idempotency-Key` y `If-Match` cuando el intento posee ETag. Un envío de edición queda ligado, por tanto, tanto al contenido como a la versión que el usuario revisó.

La ecuación conceptual es una identidad de intención, no una operación numérica:

$$I = (\text{principal}, \text{sede}, \text{ruta}, \text{método}, \text{clave}, \text{cuerpo}, \text{versión})$$

Cambiar cualquiera de sus elementos relevantes después de un resultado desconocido no constituye necesariamente un reintento seguro. La interfaz protege la captura para que primero se resuelva el comando anterior. Este principio resulta especialmente importante para pagos, ajustes, gastos y archivos, donde un registro duplicado podría representar una decisión humana que nunca ocurrió.

13.6.5 6.5 storageKey, readPending, rememberPending y forgetPending

`storageKey(path)` es interno. Devuelve el prefijo `bovi:pending:` seguido del principal y la ruta, o no devuelve clave si falta identidad. La ruta incluye la sede. `readPending(path)` lee y analiza la entrada correspondiente; si el almacenamiento o JSON falla, devuelve ausencia. `rememberPending(attempt)` guarda el intento serializado cuando existe principal. `forgetPending(path)` elimina solamente esa entrada. Los errores de almacenamiento se absorben, porque el formulario montado aún puede conservar su intento en memoria.

Este almacenamiento es por pestaña. No es una cola offline ni una sincronización de borradores entre dispositivos. Tampoco cifra el contenido comercial que una intención ordinaria guarda. La aplicación evita persistir contraseña o bytes de un archivo en sus formularios especiales, pero un cuerpo ordinario puede contener concepto, identificadores e importe. El acceso al mismo origen y la seguridad del entorno del navegador siguen siendo relevantes.

Después de recargar, un formulario ordinario puede encontrar su intención y ofrecer recuperación. Un campo que alguien estaba escribiendo pero que nunca envió no adquiere esa persistencia automáticamente. Es diferente decir «el borrador guardado existe en SQL», «el intento enviado

está pendiente» y «un texto todavía vive sólo en un `useState`». El capítulo usa esas expresiones por separado.

13.6.6 6.6 commandName y commandResult

`commandName(attempt)` es interno y traduce ruta y método al nombre canónico de comando. Reconoce creación de contraparte, predio, animal, operación, gasto, pago, documento, periodo y usuario; reconoce PATCH de operación o usuario; y distingue acciones como confirmar, acordar, reabrir, aprobar, corregir, repartir, aplicar, revertir, reembolsar o asignar membresía. La aceptación de tarifa posee un caso específico por su ruta más profunda. Reabrir una operación se diferencia de reabrir un periodo.

`commandResult<T>(attempt)` obtiene ese nombre y la sede desde la ruta, y consulta `/sedes/<sede>/comandos/<comando>/<clave>`. `COMPLETADA` devuelve `resultado`. `EN_CURSO` lanza 425 y evita otro envío. Un estado desconocido produce resultado incierto. Un 404 del lookup significa que no encontró ese comando para continuar con el mismo intento; otros errores se propagan. Si la ruta no tiene nombre reconocido, devuelve `completed:false`, una razón para mantener sincronizada esa traducción al añadir comandos.

El lookup no revela resultados de cualquier usuario. El servidor autentica y verifica el ámbito. El frontend organiza las claves para ayudar al usuario correcto, pero la garantía final no puede depender de que alguien no altere `sessionStorage`. Las pruebas de reintento verifican que después de una respuesta entregada como 502 el navegador recupera el resultado con un único POST real.

Fuentes: `frontend/src/api.ts:1`, `frontend/src/api.ts:6`, `frontend/src/api.ts:11`, `frontend/src/api.ts:13`, `frontend/src/api.ts:23`, `frontend/src/api.ts:27`, `frontend/src/api.ts:33`, `frontend/src/api.ts:42`, `frontend/tests/domain.test.ts:19`, `frontend/tests/workflow.test.ts:25`.

13.7 7. La biblioteca pequeña de interfaz compartida

13.7.1 7.1 useResource y la consulta local

Firma: `useResource<T>(path:string,revision=0)`. Es un hook de consulta, usado por listas y detalles. Conserva un estado con datos opcionales, ETag opcional, `loading` y error; además conserva un contador local de recarga. Un efecto depende de ruta, revisión y contador. Cada ejecución marca carga, crea `AbortController`, solicita datos y actualiza estado sólo mientras permanece activa. Al desmontarse o cambiar dependencias, desactiva el efecto y aborta la solicitud.

El retorno añade `reload`, una función que incrementa el contador. Pulsar «Actualizar consulta» no ejecuta un cálculo en memoria; produce otra consulta. El parámetro `revision` permite que una escritura confirmada invalide consultas asociadas. Es habitual que `onDone` incremente una revisión de pantalla y que las listas vuelvan a cargar sus datos canónicos.

Este hook no mantiene resultados antiguos durante la carga: comienza una nueva consulta con `{loading:true}`. Tampoco es una caché compartida entre pantallas. El aislamiento depende de rutas con sede, de la clave de montaje del contenedor y de la cancelación. Al cambiar de B a A, un resultado tardío de B no debe volver a poblar la vista. El recorrido real retuvo precisamente una consulta de B y comprobó su aborto al cambiar de sede.

13.7.2 7.2 Notice y State

`Notice({error})` produce un aviso con `role="alert"` cuando existe error. Expone el mensaje y, si es `ApiError` con regla, añade regla y referencia de correlación. No transforma una negativa en éxito. Si no existe error, no renderiza contenido. Lo llaman formularios, recursos y pantallas que realizan alguna actualización directa.

State({loading,error,empty,reload,children}) organiza estados comunes. Cargando muestra un texto con role="status". Error muestra Notice y «Volver a consultar». Vacío muestra una explicación inicial. Si ninguno aplica, devuelve los hijos. El orden importa: una carga no se presenta como lista vacía y un error no se confunde con ausencia de registros. Algunas vistas proporcionan mensajes vacíos más específicos dentro de sus hijos.

reload sólo pertenece a consultas. El reintento de un comando tiene otro botón y otra lógica. Mezclar ambos sería peligroso: repetir un GET para conocer datos actuales no equivale a repetir un POST económico. La biblioteca hace visible esa diferencia mediante funciones y controles distintos.

13.7.3 7.3 FieldErrors, FormScope, Field y FormField

FieldErrors es un contexto con mensajes por nombre. FormScope aporta un prefijo de identificadores por formulario. Field recibe etiqueta, nombre, tipo, obligatoriedad, valor controlado, onChange, opciones, ayuda, error y deshabilitación. Construye un identificador usando el contexto; conecta etiqueta y control; y asocia ayuda o error mediante aria-describedby. Añade aria-invalid cuando corresponde.

Si recibe opciones genera un select con una opción inicial «Selecciona…». En caso contrario genera input. Contraseñas de acceso permiten gestores con current-password; login usa username. Los campos de importe, precio y neto utilizan inputMode="decimal", que facilita el teclado apropiado pero no valida por sí mismo una regla monetaria. La obligatoriedad nativa y la normalización decimal son defensas complementarias.

FormField, exportado desde App.tsx, envuelve Field con un useState inicial. Firma resumida: FormField({name,label,initial,options,required,type,help}). Permite escribir campos de captura sin repetir el par valor/setter. El valor inicial se toma al montar; por eso varias pantallas utilizan key para remontar un formulario cuando cambia de destino o versión. Modificar la prop initial de un componente ya montado no equivale a sincronizar su estado automáticamente.

13.7.4 7.4 CommandForm y sus funciones internas

Firma: CommandForm({path,build,method='POST',etag,children,label,onDone}). build convierte FormData en el cuerpo que exige la acción. onDone recibe el resultado del servidor y puede navegar o aumentar una revisión. El componente posee un identificador useId, intento pendiente, banderas busy y success, error y una referencia al resumen de mensajes. Inicializa el intento mediante readPending(path).

La función interna submit cancela el envío nativo. Si ya hay un intento, ejecuta exactamente ese intento con lookup previo. Si no existe, lee el formulario, llama a build, crea y guarda el intento y lo ejecuta. Un error de construcción, como decimal inválido, se muestra sin enviar un cuerpo defectuoso. La función interna execute administra carga, error y éxito; elimina el pendiente después de un resultado confirmado; llama a onDone; y consulta keepAttempt al fallar. Finalmente vuelve a habilitar la acción de espera, pero puede mantener protegidos los campos por una intención no resuelta.

El fieldset queda deshabilitado mientras se espera o existe un pendiente. La acción principal cambia entre el rótulo original, «Esperando respuesta…» y «Reintentar el mismo comando». Para 409 aparece «Resolver conflicto y volver a capturar», que elimina expresamente el pendiente. Esa acción exige revisión humana del conflicto; no genera automáticamente otro POST. Los errores de campos producen enlaces hacia los controles identificados con el prefijo del formulario, y el resumen se enfoca después del envío fallido.

Un error de prueba ilustró una consecuencia de este diseño: buscar un formulario por el texto «Confirmar venta» deja de encontrarlo después de un resultado incierto, porque el botón cambia de nombre. La prueba final utiliza un campo estable del formulario. Entender la máquina de estados evita corregir el producto para satisfacer un selector de prueba equivocado.

13.7.5 7.5 DataTable y List

`DataTable({rows,columns,onSelect})` presenta una tabla dentro de una región enfocable y d esplazable. Cada columna tiene clave, etiqueta y una marca opcional de dinero. Encabezados utilizan `scope="col"`. Las celdas pasan por `text`; el estado aparece como badge. Las columnas monetarias tienen alineación y cifras tabulares. Si existe `onSelect`, añade «Abrir» con texto de contexto accesible, de modo que dos botones visualmente iguales no sean indistinguibles para quien navega por controles.

`List({path,columns,revision=0,onSelect,filters=[]})` combina consulta paginada, búsqueda y filtros. Conserva página, texto escrito, búsqueda aplicada y valores de filtros. Envía página desde uno y tamaño veinte. El texto escrito no produce necesariamente una solicitud por tecla: enviar «Buscar» copia la consulta recortada y reinicia página. Cambiar un filtro también regresa a la primera página. «Anterior» se deshabilita en página uno y «Siguiente» cuando ya se alcanza el total.

La búsqueda y los filtros consultan el servidor. La tabla no descarga toda la sede para filtrarla en JavaScript. Un detalle puede seleccionarse mediante `onSelect`, pero una lista sin ese callback permanece de consulta. El número de resultados pertenece al servidor; no se deduce contando sólo las filas visibles. Algunas pantallas usan consultas especiales fuera de `List`, como cartera, pero conservan el mismo criterio de paginación y mensajes.

Fuentes: `frontend/src/ui.tsx:4`, `frontend/src/ui.tsx:10`, `frontend/src/ui.tsx:11`, `frontend /src/ui.tsx:17`, `frontend/src/ui.tsx:19`, `frontend/src/ui.tsx:23`, `frontend/src/ui.tsx:30`, `frontend/src/ui.tsx:33`, `frontend/src/App.tsx:15`.

13.8 8. App: acceso, sede y navegación

`App()` es el componente exportado por defecto. Sus estados principales son sesión, carga inicial, error global, sede, pantalla y fila activa. La fila activa aporta un identificador para abrir detalles. Al montar, registra listeners de expiración de sesión y cambio de hash, intenta recuperar `/sesio n/actual`, refresca CSRF y establece principal. Un 401 anónimo inicial se maneja como ausencia de sesión; otros errores pueden aparecer en acceso. La limpieza del efecto elimina los listeners.

Un segundo efecto observa la sesión e intenta seleccionar el parámetro `sede` de la URL sólo si coincide con una sede autorizada y disponible. Conocer una UUID no concede acceso. El selector inicial muestra las sedes de la sesión, sus roles y disponibilidad declarada. Si no hay sedes, explica que se debe pedir acceso. La interfaz no utiliza una sede de ejemplo escondida para que el recorrido parezca conectado.

`readRoute()` interpreta el hash como pantalla e identificador. Usa operaciones como destino predeterminado y decodifica el identificador. `chooseSede(site)` escribe la sede en el query string mediante `replaceState` y actualiza el estado. `go(next,row?)` cambia pantalla y fila activa, conserva la sede, construye el hash, añade historial con `pushState` e intenta enfocar el título. No usa una biblioteca de router: el enrutamiento local está contenido en estas funciones y el listener de hash.

`Brand()` representa el nombre y símbolo del producto; su SVG decorativo está oculto al árbol accesible. `description(screen)` selecciona una explicación breve de la pantalla o utiliza una descripción general. Son funciones de presentación sin solicitudes. El nombre de la sede y los roles del encabezado provienen de la sesión, no del diseño gráfico.

`Login({onLogin,initialError})` conserva login, contraseña, error y estado de espera. Enviar cancela la navegación del formulario, obtiene CSRF anónimo, envía login, elimina la contraseña del estado después de éxito, refresca CSRF y recupera la sesión. `onLogin` comunica el resultado al contenedor. Una negativa permanece como mensaje; un envío pendiente cambia la etiqueta del botón. El capítulo no reproduce el contenido de contraseña ni ofrece una credencial DEMO fija.

`Workspace({sede,screen,active,go})` construye `base='/sedes/'+sede.id`, conserva una revisión local y define `bump` para aumentarla después de comandos. Calcula `operator` a partir de ADMIN u OPERADOR. Despacha cada destino hacia listas, formularios o componentes especial izados. Su clave de montaje combina sede, pantalla e identificador activo. Cambiar ámbito desmonta el estado anterior, evitando conservar una selección de animal o un formulario de otra o peración.

`str(f,name)` obtiene texto de `FormData`, usando vacío cuando falta. `cash(f,name='importe')` construye `{importe:canonicalDecimal(...,2),moneda:'MXN'}`. No suma dinero. `PartyField({base,initial,name,label,required})` combina búsqueda explícita y select de contrapartes: conserva query y search, solicita hasta cien resultados y puede buscar por texto o identidad. Se reutiliza para comprador/vendedor, responsable de predio y proveedor de gasto. Si la consulta falla muestra aviso. La búsqueda permite encontrar un registro que no estaba entre las primeras cien opciones; no es un paginador completo del catálogo.

13.8.1 8.1 Las quince pantallas conceptuales y sus destinos

Pantalla conceptual	Destino o composición real	Fuente
1. Acceso y selección	`Login` y selector de sede de `App`	`frontend/src/App.tsx:26`
2. Bandeja	`operaciones`, lista con tipo y estado	`frontend/src/App.tsx:37`
3. Contrapartes	Directorio y alta; predios como destino adicional	`frontend/src/App.tsx:38`
4. Ficha animal	Lista, `AnimalDetail` y `AnimalCycles`	`frontend/src/App.tsx:40`
5. Inventario	Posiciones, costo, estado y CSV	`frontend/src/App.tsx:41`
6. Compra y venta	Alta de borrador y `OperationEditor`	`frontend/src/App.tsx:35`
7. Tickets	Lista de operaciones y `TicketHistory`	`frontend/src/App.tsx:52`
8. Liquidación	Lista y `Settlement`	`frontend/src/App.tsx:44`
9. Dinero	`Payments`, validación y aplicación	`frontend/src/App.tsx:45`
10. Expediente	Lista de operaciones y `CaseFile`	`frontend/src/App.tsx:36`
11. Cartera	`Report` con búsqueda y vencimiento	`frontend/src/App.tsx:46`
12. Margen	`Report` con agregados canónicos	`frontend/src/App.tsx:46`
13. Usuarios	`Users` y controles ADMIN	`frontend/src/App.tsx:54`
14. Sincronización	`ControlView` de consulta	`frontend/src/App.tsx:55`
15. Simulador	Lista y `Simulator`	`frontend/src/App.tsx:56`

Los destinos adicionales son predios, movimientos, gastos, periodos y documentos; parte de su función se integra en las pantallas conceptuales. El menú agrupa Operaciones, Inventario, Dinero y Control. En escritorio aparece una navegación lateral; en móvil, un selector nativo agrupado. Un destino desconocido produce un mensaje de función sin endpoint autorizado, en lugar de inventar datos. Ese fallback no demuestra que una función pendiente esté conectada.

13.8.2 8.2 Catálogos y ficha animal

Las contrapartes tienen nombre, tipo de persona, RFC y teléfono; el alta usa `CommandForm` y luego aumenta revisión. Los predios agregan nombre obligatorio, municipio, entidad, UPP declarada y

responsable opcional. El constructor recorta el nombre y rechaza un texto vacío. Su aviso aclara que la declaración comercial no acredita propiedad ni autorización sanitaria. Los animales se registran con sexo y una identificación de sistema INTERNO, SINIIGA, SINIDA u OTRO.

`AnimalDetail({base,id})` solicita la ficha, muestra situación y convergencia, presenta identificaciones y solicita el QR mediante una imagen autorizada del mismo origen. La imagen no concede una sesión. Su texto alternativo explica que abre una ficha con acceso autorizado. `AnimalCycles({base,id})` conserva página y consulta posiciones con `q` igual al animal, tamaño veinte y sede en la ruta. Muestra número de ciclo, estado y costo vigente. La clave adicional `base+id` reinicia su paginación al cambiar animal o sede.

Una recompra no debe sobrescribir el costo de un ciclo cerrado anterior. La tabla expone precisamente esa historia: un animal permanente puede tener un ciclo vendido y entregado y otro disponible. La disponibilidad se consulta en inventario; no se obtiene suponiendo que toda identidad registrada está lista para vender. Las altas de catálogo no confirman por sí mismas operaciones económicas.

Fuentes: `frontend/src/App.tsx:13`, `frontend/src/App.tsx:14`, `frontend/src/App.tsx:16`, `frontend/src/App.tsx:18`, `frontend/src/App.tsx:19`, `frontend/src/App.tsx:23`, `frontend/src/App.tsx:31`, `frontend/src/App.tsx:33`, `frontend/src/App.tsx:59`.

13.9 9. Operaciones, lotes, pesajes y acuerdos

`PeriodFields({base})` consulta hasta cien periodos y presenta sólo los que llegan como ABIERTO; ofrece fecha de reconocimiento con `today()`. No abre un periodo automáticamente. `recognition(f)` exporta un constructor con `periodo_id` y `fecha_reconocimiento`; lo reutilizan confirmación, validación de pago, aplicación, gastos, ajustes, reversos, reembolsos y bajas. Cada comando conserva su contexto económico explícito.

`inputDecimal(value)` es interno y adapta valores iniciales de condiciones: lee `importe` si recibe dinero, devuelve vacío para `undefined` y convierte otros valores a texto. No calcula ni redondea. Lo llama `LotEditor` para campos de cargos y bonificaciones. Este auxiliar de representación no sustituye la validación posterior de `canonicalDecimal` o del contrato.

`OperationEditor({base,sede,id,go})` conserva revisión, paso y último ticket propuesto. Consulta la operación y su ETag, resuelve posiciones de venta y comprueba permiso de operación. Los pasos son condiciones, pesaje y revisión. Muestra folio, estado operativo, estado físico, contraparte y versión. Un bloque plegable presenta contratos anteriores. Si la operación está confirmada, comunica que cierre económico, recepción, entrega y pago son hechos separados; permite consultar liquidación sin ofrecer reescribir el cierre.

El constructor de ticket selecciona un lote, conserva el neto como `Peso{kg}` de escala tres, convierte fecha/hora a ISO y añade folio y documento opcional. Los animales salen de `operationAnimals`, no de un conteo. Los pesos individuales se incluyen únicamente cuando el usuario los escribió. El total del lote no se divide entre animales para inventar mediciones individuales. Después del POST, el componente conserva el ticket y puede ofrecer su aprobación explícita.

`useSalePositions(base,operation)` es un hook interno distinto de `useResource`. Su identidad de consulta es la composición de posiciones serializada para una venta. Mantiene datos, carga y error; depende de base e identidad, por lo que no descarta identidades resueltas sólo porque una recarga transitoria deje momentáneamente ausente el detalle. Su función interna `load` consulta inventario por páginas de cien hasta resolver el conjunto requerido o alcanzar el total. Después usa `operationAnimals` para comprobar que no faltó alguna posición. El efecto también cancela su solicitud al cambiar ámbito o desmontarse.

`LotForm(props)` conserva el número de lote seleccionado. Si el estado es ACORDADA muestra condiciones aceptadas y un comando de reapertura con ETag. Si es editable, permite elegir lotes, añadir el siguiente número, montar `LotEditor` por clave y aceptar condiciones mediante `/acordar`. `LotEditor` conserva búsqueda y selección, consulta animales para compra o posiciones para

venta y deshabilita posiciones reservadas o no disponibles. Guardar envía todas las condiciones a través de `replaceLot`, conservando el resto de lotes.

Los campos incluyen unidad KG o CABEZA, precio con cuatro decimales, pesaje base, deducción con seis decimales y cargos/bonificaciones con dos. La preferencia de recepción se obtiene del acuerdo actual y se conserva al editar. Guardar un borrador KG sin ticket puede ser útil para preparar el expediente, pero no autoriza aceptar ni confirmar un acuerdo incompleto. El servidor exige evidencia aprobada en las transiciones correspondientes. Reabrir conserva versiones aceptadas y produce una propuesta nueva; no edita la historia anterior.

La reserva de venta envía posiciones guardadas y vencimiento ISO. Confirmar utiliza reconocimiento y ETag. La interfaz no manda el total calculado. Una reserva vencida o ajena provoca rechazo y exige revisión de disponibilidad. Un botón deshabilitado en la lista es una ayuda; el servidor debe seguir rechazando dos confirmaciones competidoras aunque ambos usuarios hayan visto una posición disponible antes de reservar.

13.9.1 9.1 Historial, corrección y comparación

`TicketHistory({base,id,sede})` conserva revisión y ticket seleccionado. Acepta una respuesta paginada o un arreglo de tickets y construye una columna de composición con animal e indicación de peso individual. Cuando no hubo medición muestra «Sin medición individual». Presenta versión corregida, motivo y fecha. Sólo ADMIN u OPERADOR obtiene acciones de selección para corregir; otros perfiles siguen consultando.

`TicketCorrection({base,ticket,onDone})` tiene tres ramas. Una PROPUESTA puede aprobarse después de revisión. Un ticket distinto de APROBADO conserva historia y no ofrece corrección editable. Un APROBADO sin composición original completa muestra un aviso y exige recargar; no toma animales del lote actual como reemplazo. Para una corrección válida envía neto nuevo, motivo, `corrige_a_id`, número de lote y exactamente los animales originales. La aprobación posterior enlaza la propuesta y conserva el original como CORREGIDO.

`TicketComparison({base,tickets})` conserva primer y último identificador y ofrece dos selectores. Cuando ambos existen monta `ComparisonResult({base,first,last})`. Este componente consulta el endpoint canónico y muestra NO_COMPARABLE cuando los grupos difieren. Si recibe COMPARABLE presenta variación kg y porcentaje devueltos como cadenas. No calcula la diferencia ni la convierte automáticamente en descuento. Una tercera rama avisa si el servidor no entregó comparación disponible.

Fuentes: `frontend/src/screens.tsx:8`, `frontend/src/screens.tsx:9`, `frontend/src/screens.tsx:10`, `frontend/src/screens.tsx:11`, `frontend/src/screens.tsx:16`, `frontend/src/screens.tsx:24`, `frontend/src/screens.tsx:29`, `frontend/src/screens.tsx:64`, `frontend/src/screens.tsx:68`, `frontend/src/screens.tsx:109`.

13.10 10. Liquidaciones, dinero y correcciones económicas

`Settlement({base,sede,id,go})` consulta una liquidación y conserva revisión. ADMIN o CAJA puede pasar a registrar cobro o pago. El resumen muestra saldo, vencimiento y dirección; el detalle separa operación económica, entrega física y contraparte. Conceptos, total original, cargos, abonos, aplicaciones netas y saldo provienen del servidor. El comprobante se abre como recurso autorizado y se identifica como interno, no CFDI.

El formulario de ajuste sólo se presenta para ADMIN. Envía naturaleza, monto canónico, motivo, fuente, reconocimiento y `reaplicar_pagos`. Esa bandera solicita conciliación atómica cuando un abono reduce una obligación pagada. No significa que React ejecute primero reversos y después nuevas aplicaciones en varias llamadas. Una sola intención llega al servidor, que conserva originales y registra los hechos necesarios dentro de su transacción.

`FinancialHistory({base,data,caja,onDone})` muestra ajustes y aplicaciones. Conserva una aplicación seleccionada. Cuando el perfil puede operar caja y la fila no es ya un REVERSO, ofrece

`/aplicaciones/<id>/reverso` con motivo y reconocimiento. El original sigue visible; el callback recarga datos. Esta acción independiente sirve para otros casos y no reemplaza la conciliación atómica solicitada por el ajuste guiado.

`Payments({base,sede,context})` conserva revisión, pago seleccionado, error de recarga, destino, búsqueda aplicada y página de obligaciones. El contexto de una liquidación puede preseleccionar dirección, contraparte y UUID buscada. La función interna `refreshPayment(id)` recupera el pago completo mediante `request`, actualiza la selección y aumenta revisión; `bump` la llama después de validaciones o aplicaciones. Estas consultas directas no añaden un `AbortControlle r` propio como `useResource`; el ámbito de la pantalla sigue estando separado por el montaje del contenedor.

La lista de pagos filtra estado REGISTRADO o VALIDADO y tipo COBRO o PAGO. Registrar dinero envía importe, contraparte, método, referencia y fecha. No exige en ese formulario el mismo reconocimiento que validar. Un pago REGISTRADO ofrece validación con periodo y fecha; uno VALIDADO permite aplicar. La lista de obligaciones consulta veinte por página, incorpora búsqueda y muestra candidatos con saldo positivo, misma contraparte y dirección compatibl e. Ese filtrado ayuda a seleccionar, pero el servidor vuelve a validar empresa, sede, moneda, dirección y disponibilidad.

Una aplicación envía un arreglo con liquidación e importe, además de reconocimiento. El destino sólo aparece si todavía está dentro del conjunto compatible visible. No se descuenta saldo localmente al pulsar el botón: después de éxito se recupera el pago y las obligaciones. `RefundForm( {base,payment,onDone})` permite documentar salida del disponible con importe, método, fecha, referencia y reconocimiento. Conserva el pago original y utiliza otra intención y otra clave.

Tres magnitudes deben leerse por separado: obligación pendiente, disponible de un pago y resultado económico de una venta. Un anticipo puede existir sin liquidación saldada; una entrega física puede estar pendiente con saldo cero; un margen puede incorporar gastos que no aumentan la deuda del comprador. Las fórmulas siguientes describen categorías canónicas, no operaciones ejecutadas por la interfaz:

$$\text{Saldo de obligación} = \text{Total original} + \text{Cargos} - \text{Abonos} - \text{Aplicaciones netas}$$

$$\text{Disponible del pago} = \text{Importe validado} - \text{Aplicaciones netas} - \text{Reembolsos}$$

La UI recibe cada resultado de la consulta autorizada. Si una cifra falta, muestra ausencia. No deriva otra cifra para hacer que el desglose parezca completo.

Fuentes: `frontend/src/screens.tsx:33`, `frontend/src/screens.tsx:36`, `frontend/src/screens .tsx:38`, `frontend/src/screens.tsx:97`, `frontend/src/screens.tsx:98`.

13.11 11. Gastos, tarifas aceptadas y periodos

`ExpenseCapture({base,onDone})` conserva naturaleza COSTO o BONIFICACION y snapshot aceptado. Renderiza primero `ProviderTariffs` y después el formulario. Sin snapshot, el usuario captura importe mediante `cash`; con snapshot, se presenta la tarifa aceptada y el cuerpo utiliza el `precio` recibido y su `snapshot_id`. La UI no permite escribir una procedencia externa arbitraria para hacerla pasar por una aceptación válida.

El gasto incluye operación, clasificación CAPITALIZABLE, DIRECTO_VENTA u OTRO, concepto, fecha económica declarada y reconocimiento explícito. Proveedor, referencia de origen y documento son opcionales. Si se elige BONIFICACION, aparece el gasto original que se bonifica. Cambiar la naturaleza cambia campos visibles, pero no elimina el hecho original. La composición del cuerpo y la autorización se vuelven a comprobar en SQL. Un gasto directo de venta puede afectar margen sin incrementar la obligación con la contraparte.

`ProviderTariffs({base,onAccepted})` es interno. Conserva snapshot seleccionado y puede inicializarlo desde un intento de aceptación pendiente. Consulta cien tarifas, encuentra la fila seleccionada y adapta errores de red o 503 a un mensaje contextual del proveedor. No afirma que la sede o ambos nodos estén caídos. El select expone ruta, precio y versión; una ficha muestra fuente, unidad y vigencia. La aceptación envía sólo `snapshot_id`. Al confirmar, comunica al padre la fila consultada correspondiente. Cambiar selección elimina el snapshot aceptado del padre hasta aceptar la nueva versión.

La aceptación y el gasto son comandos distintos. Consultar una tarifa tampoco equivale a aceptarla. El backend lee la fuente mediante el lector permitido y captura el snapshot en el propietario de la sede; esa topología no se controla desde un select. La prueba de UI identifica la fuente como DEMO simulada. El lector no debe concluir que existe un convenio con un proveedor comercial ni una consulta sanitaria oficial.

`ExpenseDetail({base,sede,id})` conserva revisión y método. Solicita una previsualización de reparto. Para POR_CABEZA o POR_PESO_MEDIDO muestra bases e importes canónicos. Para IMPORTE_INDIVIDUAL utiliza la consulta por cabeza para conocer destinos, pero oculta sus importes calculados y pide una captura individual por posición. No ofrece esa previsualización como aprobación de los valores manuales. El servidor exige composición y suma exacta del gasto.

El POST de reparto incluye método y reconocimiento; en modo individual agrega `asignaciones` con posición y dinero. Un constructor rechaza intentar repartir individualmente sin destinos c onsultados. Cambiar método remonta el formulario mediante clave, evitando reutilizar campos de otro método. `onDone` aumenta revisión. La UI no ajusta la última posición con una diferencia calculada localmente para «cuadrar» centavos; esa política corresponde al cálculo canónico.

`Periods({base,sede})` conserva revisión y fila seleccionada. Todos consultan la lista; ADMIN puede seleccionar para administrar. `PeriodVersions({base,id})` consulta versiones inmutables y admite respuesta paginada o arreglo. Muestra número, tipo, motivo y corte. El formulario elige `/cierre` cuando el periodo está ABIERTO y `/reabrir` en el otro caso, siempre con motivo. Después de éxito elimina selección y recarga.

El alta solicita primer y último día de un mes. El texto explica continuidad y un único periodo abierto, pero la UI no reemplaza la validación de orden, límites y conciliación del servidor. La prueba real cerró y reabrió el periodo corriente y conservó versiones. El siguiente mes se comprobó en QA aislado; no se adelantó por iniciativa de la prueba el calendario de la demostración principal.

Fuentes: `frontend/src/screens.tsx:83`, `frontend/src/screens.tsx:87`, `frontend/src/screens.tsx:88`, `frontend/src/screens.tsx:92`, `frontend/src/screens.tsx:96`.

13.12 12. Archivos privados y usuarios sensibles

`Documents({base,sede})` conserva revisión y documento seleccionado. Presenta lista con tipo, formato y versión; ADMIN u OPERADOR obtiene carga. Elegir una fila monta `DocumentDetail({base,sede,id})` por identificador. Ese detalle consulta metadatos, muestra si el contenido es sintético, ofrece descarga autorizada y muestra revisiones. ADMIN puede registrar una revisión con operación, criterio, resultado y documento exacto. El callback vuelve a consultar el detalle.

`DocumentUpload({base,onDone})` necesita un tratamiento especial porque `File` y bytes no deben guardarse como un JSON ordinario. Conserva intento, carga, error, éxito y una referencia `upload` al `FormData` en memoria. Su función interna `submit(form)` consulta primero el resultado si ya existe comando pendiente. Si encuentra COMPLETADA, termina sin subir otra vez. Si necesita enviar, verifica que haya archivo, tamaño máximo veinte MiB y MIME PDF, JPEG o PNG.

Calcula SHA-256 con `crypto.subtle.digest` sobre los bytes y forma una huella con nombre, tamaño, hash, tipo documental y operación vinculada. La huella se guarda como cuerpo del `Attempt`; el archivo real permanece en memoria. Después de recargar, seleccionar un archivo diferente o cambiar tipo/vínculo no coincide con la intención antigua y produce error. La nue-

va selección debe reproducir contenido y metadatos del mismo envío. Cuando la consulta de resultado confirma la subida, no se necesita recapturar bytes.

El POST real usa `FormData`, no la huella JSON. `request` no impone `Content-Type:application/json` porque el cuerpo no es cadena; el navegador construye el límite multipart. Se envía la misma clave de idempotencia. Después de éxito borra pendiente y referencia de memoria. En un rechazo conocido permite corregir; ante resultado incierto conserva lo necesario para recuperar. El atributo `accept` ayuda a elegir archivo, pero las verificaciones del cliente y del servidor siguen siendo necesarias.

`Users({base,sede})` conserva revisión y usuario seleccionado. Una lista muestra nombre, login, roles y estado. ADMIN obtiene alta sensible, PATCH de nombre/estado y membresía por sede. El PATCH permite ACTIVO o BLOQUEADO. La membresía envía un código ADMIN, OPERADOR, CAJA o AUDITOR y un booleano `vigente`; retirar un rol es un comando explícito, no borrar visualmente una etiqueta. La autorización vigente se comprueba también en el backend.

`SensitiveUserCreate({base,onDone})` conserva intento, bandera de espera, error, éxito y una referencia de texto `body`. Su `submit` consulta el resultado pendiente antes de intentar crear de nuevo. Construye JSON con login, nombre y contraseña sólo en memoria; exige contraseña no vacía; obtiene SHA-256 del contenido completo y persiste exclusivamente la huella con la clave. Si se recarga y el comando no terminó, el mismo contenido debe recapturarse para coincidir con la huella. La contraseña nunca se copia a `sessionStorage`.

El POST utiliza el JSON original en memoria. Después de éxito elimina referencia y pendiente, informa al contenedor y solicita reset del formulario. El callback incorpora nombre y estado ACTIVO para presentar inmediatamente el usuario recién creado, mientras que la lista puede consultarse otra vez. Crear un usuario no concede automáticamente todos los roles de todas las sedes. Por ello la prueba de AUDITOR realiza una membresía separada antes de iniciar su nueva sesión.

La protección mediante huella no convierte este mecanismo en un almacén de contraseñas ni en un servicio de autenticación offline. La sesión y el hash de contraseña del servidor siguen perteneciendo a otra capa. La prueba unitaria y la prueba UI comprueban ausencia de contraseña persistida; no se presenta ese resultado como una auditoría completa de penetración.

Fuentes: `frontend/src/screens.tsx:46`, `frontend/src/screens.tsx:47`, `frontend/src/screens.tsx:49`, `frontend/src/screens.tsx:51`, `frontend/src/screens.tsx:99`, `frontend/src/screens.tsx:100`, `frontend/src/screens.tsx:102`.

13.13 13. Expediente, movimientos, reportes y simulador

`CaseFile({base,id})` solicita `/operaciones/<id>/expediente`. Admite eventos o hechos en la estructura consultada y extrae documentos vinculados. Representa fechas, tipo o acción y descripción o estado en una línea temporal. Cuando faltan eventos lo declara; cuando faltan documentos no inventa una evidencia. Sus enlaces descargan la versión autorizada del mismo origen. El botón de actualizar utiliza `reload`, de modo que la historia permanece sustentada por consultas, no por una narración construida de clics locales.

`MovementCapture({base,sede})` conserva tipo RECEPCION inicialmente. El usuario captura posiciones separadas por espacios o comas; el constructor produce un arreglo no vacío de textos y convierte fecha/hora local a ISO. ENTREGA exige identificador de la venta confirmada. BAJA aparece sólo para ADMIN y agrega motivo PERDIDA o MUERTE, documento y reconocimiento. RECEPCION y ENTREGA no incluyen reconocimiento económico nuevo en ese cuerpo: son hechos físicos distintos del cierre ya confirmado.

`ControlView({base,kind})` consulta usuarios o sincronización, pero la navegación actual emplea `Users` para administración y `ControlView` para sincronización. Su rama de usuarios es una representación de consulta reutilizable, no un segundo CRUD completo. La rama de sincronización muestra estado, última actualización y avisos cuando llegan. No presenta automáticamente todos los campos internos del diagnóstico ni nombres de hosts. Un valor ausente se mantiene

como ausencia y una etiqueta ASINCRONA no se reemplaza por «todo sincronizado» por conveniencia.

`Simulator({base,id})` consulta el cálculo de una operación, presenta total y conceptos y etiqueta «No registra hechos». Si hay faltantes recibidos los muestra. Un borrador KG incompleto puede producir error y no un total artificial de cero. Consultar el simulador no genera inventario ni obligación, no acepta condiciones y no confirma. Su resultado sirve para revisar una propuesta, no para saltarse la acción económica explícita.

`Report({base,kind,go})` conserva página, texto escrito, búsqueda aplicada y filtro de vencimiento. Para cartera construye query con tamaño veinte y booleano textual `true` o `false` en `vencida`; el backend normaliza la entrada. Muestra `filas`, total y controles de página. Elegir una fila navega hacia la liquidación. Para margen no agrega paginación local, porque recibe un agregado canónico de todo el ámbito consultado.

Ambos reportes muestran corte, etiqueta de consolidación, fuente y antigüedad cuando la API los informa. Una antigüedad nula se explica expresamente; no se transforma en cero. Las métricas de margen son venta neta, costo vendido, gastos directos, margen y pérdidas. El componente llama a `money` para mostrarlas, pero no suma o resta esos campos. El corte identifica la consulta, no promete una consistencia global instantánea entre todos los streams de replicación.

Los enlaces CSV de cartera, margen e inventario apuntan a `/api/v1/sedes/<sede>/reportes/<tipo>/csv`. Dicen «Exportar CSV de toda la sede». No llevan la búsqueda o filtros de la tabla. Un lector que buscó una liquidación y luego descarga debe esperar toda la sede autorizada, no únicamente la fila visible. La descarga usa cookie y autorización del servidor. No se llama a un servicio externo para producir el archivo ni se convierte dinero a números binarios para escribirlo en el cliente.

Fuentes: `frontend/src/screens.tsx:41`, `frontend/src/screens.tsx:75`, `frontend/src/screens.tsx:79`, `frontend/src/screens.tsx:80`, `frontend/src/screens.tsx:82`, `frontend/src/App.tsx:41`.

13.14 14. Recorrido completo con cuerpos de solicitud didácticos

Este recorrido combina las acciones de formularios con la forma de sus mensajes. Los UUID siguientes son marcadores sintácticamente válidos para estudiar el contrato; no son instrucciones para crear registros sobre una sede desconocida. En una ejecución real se toman identificadores de las respuestas y se obtiene la versión mediante GET. Se supone una sede autorizada, una contraparte DEMO, dos animales, un periodo abierto y roles suficientes para operación y caja. La ilustración conserva el caso de dos cabezas a500.0000 MXN: el servidor devolvió1000.00 en el recorrido evidenciado.

13.14.1 14.1 Preparar identidad y crear borrador

Primero se inicia sesión sin copiar sus secretos al capítulo, se selecciona sede y se crea o consulta una contraparte. El cuerpo comercial de alta puede ser:

```
{"nombre":"DEMO Contraparte didáctica","tipo_persona":"FISICA"}
```

Cada identidad animal necesita su propia alta; un ejemplo de entrada es:

```
{"sexo":"M","identificaciones":[{"sistema":"INTERNO","codigo":"DEMO-MANUAL-001"}]}
```

Después, el formulario de Nueva operación envía a `/sedes/<S>/operaciones`:

```
{
  "tipo":"COMPRA",
  "parte_id":"10000000-0000-4000-8000-000000000001",
  "fecha_operacion":"2026-10-11",
  "requiere_recepcion":true
}
```

Crear el borrador devuelve su identificador, pero no significa que haya posiciones ni deuda. Abrir ese identificador produce GET del detalle y su ETag. La edición usa la versión recibida, no presupone que un POST de creación haya entregado la cabecera. Los tests de fixtures también consultan GET entre transiciones por esta razón.

13.14.2 14.2 Guardar condiciones, aceptar y confirmar

El PATCH a `/sedes/<S>/operaciones/<OC>` conserva un arreglo ordenado de lotes. Para dos animales, su forma simplificada es:

```
{
  "lotes":[{
    "numero_lote":1,
    "animales":[
      "20000000-0000-4000-8000-000000000001",
      "20000000-0000-4000-8000-000000000002"
    ],
    "unidad_precio":"CABEZA",
    "precio_unitario":"500.0000"
  }],
  "requiere_recepcion":true
}
```

Lo acompañan CSRF, `Idempotency-Key` del intento e `If-Match` con el ETag obtenido. El servidor guarda la propuesta. Luego «Aceptar condiciones» envía `{}` a `/acordar` con versión vigente. Si el acuerdo está aceptado y se desea modificarlo, se utiliza `/reabrir`, se guarda otra propuesta y se acepta otra versión; no se edita directamente la versión anterior.

Confirmar compra envía únicamente su reconocimiento económico:

```
{
  "periodo_id":"30000000-0000-4000-8000-000000000001",
  "fecha_reconocimiento":"2026-10-11"
}
```

El total esperado de esta ilustración es1000.00, pero ese campo no está en el cuerpo de con firmación. El servidor devuelve una liquidación cuyo `total` y `saldo` usan dinero canónico. La navegación puede tomar `data.id` como identificador de la liquidación; no debe conservarlo erróneamente como UUID de operación. El inventario resultante será pendiente de recepción porque así se pactó.

13.14.3 14.3 Recibir físicamente y preparar venta

En Inventario se buscan los animales y se copian los identificadores de posición devueltos. Recibir utiliza otro comando:

```
{
  "tipo":"RECEPCION",
  "posiciones":[
    "40000000-0000-4000-8000-000000000001",
    "40000000-0000-4000-8000-000000000002"
  ],
  "ocurrido_en":"2026-10-11T16:00:00.000Z"
}
```

No incluye pago ni un segundo total de compra. Después de confirmarlo se consulta la disponi bilidad. Para vender se crea otro borrador con tipo VENTA y contraparte autorizada. Su PATCH selecciona posiciones, no identidades sin ciclo:

```
{
  "lotes":[{
    "numero_lote":1,
    "posiciones":[
      "40000000-0000-4000-8000-000000000001",
      "40000000-0000-4000-8000-000000000002"
    ],
    "unidad_precio":"CABEZA",
    "precio_unitario":"500.0000"
  }],
```

```
  "requiere_recepcion":false
}
```

Se aceptan condiciones y se reserva el conjunto antes de confirmar. El vencimiento siempre debe ser futuro para la intención actual:

```
{
  "posiciones":[
    "40000000-0000-4000-8000-000000000001",
    "40000000-0000-4000-8000-000000000002"
  ],
  "vence_en":"2026-10-11T20:00:00.000Z"
}
```

La fecha es didáctica, no válida para ejecutar el ejemplo en otro día. Si venció, no se inventa una extensión en el navegador. El usuario revisa disponibilidad y captura una nueva reserva conforme a la respuesta. Confirmar venta vuelve a usar periodo y fecha explícitos, versión vigente y clave propia. El servidor decide inventario y obligación en la misma transacción.

13.14.4 14.4 Variante por kg

Cuando el precio es KG, guardar puede preparar la composición antes de tener ticket, pero aceptar exige base aprobada. El ticket enviado conserva identidades animales, aunque la venta se haya seleccionado por posiciones:

```
{
  "numero_lote":1,
  "modo":"NETO_REPORTADO",
  "neto":{"kg":"300.000"},
  "ticket_folio":"DEMO-MANUAL-PESO",
  "pesado_en":"2026-10-11T17:00:00.000Z",
  "animales":[
    {"animal_id":"20000000-0000-4000-8000-000000000001","peso_individual":{"kg":"100.000"}},
    {"animal_id":"20000000-0000-4000-8000-000000000002","peso_individual":{"kg":"200.000"}}
  ]
}
```

Los pesos individuales sólo se incluyen si existen mediciones. Una propuesta se aprueba mediante su endpoint y se vincula a la condición. Con precio10.0000 y neto300.000, el servidor devolvió 3000.00 en la regresión real. La igualdad didáctica describe ese ejemplo, no una implementación JavaScript:

$$300.000\ \text{kg} \times 10.0000\ \text{MXN}/\text{kg} = 3000.00\ \text{MXN}$$

13.14.5 14.5 Registrar, validar y aplicar dinero

Desde liquidación se abre Cobros y pagos con contexto de dirección y contraparte. Registrar utiliza:

```
{
  "parte_id":"10000000-0000-4000-8000-000000000001",
  "tipo":"COBRO",
  "importe":{"importe":"1000.00","moneda":"MXN"},
  "metodo":"EFECTIVO",
  "referencia":"DEMO Manual",
  "recibido_el":"2026-10-11"
}
```

La creación de pago no salda automáticamente la liquidación. Se valida con reconocimiento y se elige una obligación compatible. Aplicar usa un cuerpo diferente:

```
{
  "periodo_id":"30000000-0000-4000-8000-000000000001",
  "fecha_reconocimiento":"2026-10-11",
  "aplicaciones":[{
    "liquidacion_id":"50000000-0000-4000-8000-000000000001",
    "importe":{"importe":"1000.00","moneda":"MXN"}
  }]
}
```

Después se recuperan saldo y disponible. Un saldo cero recibido confirma la aplicación económica, no la entrega física. Si el comprador aún no recibió ganado, ENTREGA se registra por separado con posiciones, venta confirmada y fecha. Una bonificación posterior de100.00 puede solicitar `reaplicar_pagos:true` en un ABONO ADMIN; el servidor conserva originales, deja aplicaciones netas900.00 y disponible100.00 para reembolso. La interfaz no ejecuta esa conciliación sumando renglones ni enviando tres comandos ocultos.

Fuentes: `frontend/src/App.tsx:35`, `frontend/src/screens.tsx:31`, `frontend/src/screens.tsx:14`, `frontend/src/screens.tsx:39`, `frontend/src/screens.tsx:82`, `frontend/tests_e2e/closure.spec.ts:33`, `frontend/tests_e2e/closure.spec.ts:41`, `frontend/tests_e2e/closure.spec.ts:88`.

13.15 15. Estados, borrador y resultado incierto: cómo pensar una falla

Un borrador existe en SQL sólo después de que el servidor confirma su alta o edición. Pasar de un paso visual a otro no guarda automáticamente los campos. «Guardar condiciones del lote» es una acción explícita; el callback actualiza la revisión y consulta el detalle. Dejar una página con texto no enviado puede perder ese texto. La aplicación no ofrece captura offline ni autosave universal, y esa ausencia debe explicarse al usuario antes de asumir que una propuesta quedó persistida.

Para un comando ordinario, el instante de envío congela cuerpo, ruta, método y versión en `Attempt`. Mientras no se conozca el resultado, el formulario bloquea sus campos. Recargar puede recuperar esa intención por principal y ruta, pero no restaura necesariamente toda la presentación previa. Por ejemplo, puede ser necesario volver al detalle correcto; el contenido original que se reenvía proviene del intento, no de escribir valores aproximados en controles nuevos.

Un 401 provoca un evento que lleva de regreso al acceso. La identidad y CSRF de memoria se eliminan; el almacenamiento queda separado por el usuario anterior. Al entrar como otro usuario no aparece su intento. Al entrar como el mismo usuario puede recuperarse y consultar el resultado. Este mecanismo no permite continuar con una cookie expirada ni sustituir la autorización del lookup. Cada respuesta económica recuperada debe proceder del servidor autenticado.

Un 412 significa que la versión revisada ya no es la actual. El usuario obtiene el registro vigente y vuelve a valorar sus cambios. Un 428 significa que no acompañó versión cuando se necesitaba. No se arreglan enviando una versión inventada. La interfaz utiliza ETag del GET y, en el editor, dispone de una representación entre comillas de `version_lock` recibido como fallback. Esa versión sigue viniendo del servidor. El proxy integrado debe conservar el ETag fuerte; transformarlo con compresión de API puede romper la comparación.

Un 422 conocido habilita corregir la captura porque el intento anterior fue rechazado. Un 5 xx o una respuesta que no puede interpretarse no tiene esa certeza. Lo correcto es consultar primero. Tampoco se afirma que cada 500 o504 haya sido provocado realmente en E2E: su clasificación está cubierta por unitarias. La prueba502 alteró sólo la entrega de una respuesta201 real, lo que reproduce precisamente el riesgo de desconocer una confirmación ya realizada.

En cargas documentales y creación sensible la recuperación es más cuidadosa. Se guarda clave y huella, no el contenido protegido. Si el lookup confirma, termina sin pedir otra subida o contraseña. Si no confirma y la memoria desapareció al recargar, se recaptura exactamente el contenido original. Cambiar el archivo o contraseña mientras la intención está pendiente produciría otra decisión, por lo que la comparación de huella rechaza el cambio.

La UX de conflicto mantiene una referencia de regla para soporte y un paso de resolución visible. No anuncia «todo está bien» después de un timeout ni muestra un saldo cero para ocultar una consulta ausente. Para diagnosticar, conviene separar: error de captura antes de enviar, rechazo conocido del servidor, versión antigua y resultado desconocido. Esa clasificación permite actuar

con propósito y evita tratar todos los avisos como una invitación a pulsar Guardar repetidamen te.

Fuentes: frontend/src/ui.tsx:26, frontend/src/api.ts:11, frontend/src/api.ts:19, fronten d/src/screens.tsx:53, frontend/src/screens.tsx:102, frontend/tests/workflow.test.ts:21, frontend/tests_e2e/closure.spec.ts:90.

13.16 16. CSS, accesibilidad y adaptación de tamaños

La hoja de estilos define fondo #F6F7F4, texto #17211B, acción #1F5A43, secundario #5A685F, borde #D8E0D8, superficie suave #E9EFE9 y error #9F2922. La fuente comienza por Inter como preferencia y continúa con familias del sistema; no hay una descarga de fuente declarada en esa hoja. Los tokens reducen decisiones aisladas y permiten que acceso, inventario y dinero conserven la misma identidad. El foco visible usa tres píxeles de color cálido y separación del control.

El layout de escritorio usa app-shell flexible, lateral fijo y un workspace con min-width:0. Este último detalle permite que el contenido flexible encoja sin imponer su ancho intrínseco al documento. main posee ancho completo y máximo1600 px. Las tarjetas panel utilizan borde, radio y espaciado consistente. split organiza dos columnas mediante minmax, y form-grid distribuye campos. facts y metric-grid utilizan columnas adaptables para información y métricas.

Los botones tienen objetivo mínimo44 px y los controles de campo45 px. Las cifras monetarias usan font-variant-numeric:tabular-nums, alineación derecha y una representación completa. Un identificador largo puede envolver mediante overflow-wrap:anywhere; no se utiliza como motivo para abreviar una UUID y hacer ambiguo el registro seleccionado. Los estados combinan texto y badge; el color por sí solo no contiene toda la información.

Las tablas desplazan dentro de table-region, que tiene overflow-x:auto y ancho máximo disp onible. En móvil la tabla conserva un ancho mínimo570 px: el usuario recorre la región, no toda la página. La regla .text-button{position:relative} y el anclaje del texto sr-only evitan que un texto invisible posicionado fuera de su control extienda el documento. Una región accesible con tabulación permite llegar a la tabla mediante teclado y comprender qué conjunto contiene.

A1100 px el lateral y espaciados se reducen y el layout dividido pasa a una columna. A767 px se apilan bloques, resumen económico y formulario. Las reglas posteriores ocultan la navegación lateral y muestran mobile-navigation, un select nativo con grupos. No deben interpretarse reglas anteriores del archivo aisladas de la cascada: al final existe una corrección que reemplaza el menú horizontal móvil por el selector compacto. Esa secuencia explica el estilo aplicado realmente.

El acceso móvil reduce la banda gráfica a la marca y coloca el formulario antes de una región promocional extensa. La tipografía de inputs pasa a16 px. skip aparece al obtener foco y salta hacia contenido. Los campos poseen etiqueta, errores asociados y ayudas. El resumen de error se puede enfocar; no se mueve el foco arbitrariamente al cambiar cada carácter. La preferencia prefers-reduced-motion elimina animaciones, incluido el indicador giratorio.

La evidencia de390,768 y1440 px comprueba ausencia de desplazamiento horizontal global en los recorridos registrados. No es equivalente a probar un teclado virtual de teléfono, cada combinación de texto largo, un lector de pantalla o conformidad WCAG global. La suite de teclado sí completa un flujo principal real sin ratón ni locator.focus; el uso de un focus interno de la aplicación para anunciar navegación o error es parte del comportamiento del producto y no una trampa del runner.

Fuentes: frontend/src/styles.css:1, frontend/src/styles.css:3, frontend/src/styles.css: 5, frontend/src/styles.css:6, frontend/src/styles.css:8, frontend/src/styles.css:11, fro ntend/src/ui.tsx:21, frontend/src/App.tsx:29.

13.17 17. Pruebas: qué verifican y qué límites conservan

Hay trece pruebas unitarias entre `domain.test.ts` y `workflow.test.ts`. Verifican formato exacto de un importe mayor que el entero seguro de JavaScript, captura decimal y rechazo de expon entes/negativos/exceso de escala, conservación de cuerpo y ETag al mutar el objeto original, estructura accesible de campos, cabecera CSRF, fecha mexicana, recuperación de lookup sin POST adicional, estado EN_CURSO, identificadores únicos entre formularios, orden de lotes, composición animal de venta y aislamiento del intento al expirar sesión.

Varias unitarias sustituyen `fetch` o almacenamiento por objetos controlados. Esa técnica permite observar exactamente qué se envía y restaurar el objeto original al terminar. No es evidencia de PostgreSQL real. El render a markup comprueba atributos, pero no certifica navegación con lector de pantalla. El capítulo mantiene esas distinciones porque una prueba pequeña es valiosa precisamente cuando se conoce el comportamiento acotado que demuestra.

La configuración Playwright usa un trabajador, sin paralelismo ni retries, timeout de caso240 s y expectativas15 s. Abre Chrome instalado con ventana visible y perfil separado. Puede recibir `BO VI_UI_TEST_URL`; el valor de desarrollo predeterminado es15173 y la tanda final usó Caddy1518 0. No utiliza trazas de navegación para versionar cookies o contraseñas. Los secretos necesarios permanecen en el directorio privado local y no se reproducen aquí.

`runtime.ts` exporta un `test` extendido con fixture automática `runtimeLog`. Escucha errores de consola, errores JavaScript y respuestas HTTP de error; conserva título, estado, duración y URL base en JSON temporal. Los contextos secundarios necesitan observación propia: el caso de reservas registra por separado su segundo operador. Por ello un total de errores del contexto principal no debe presentarse como un inventario exhaustivo de cualquier ventana imaginable.

Grupo de prueba	Comportamiento concreto	Fuente
Flujo principal	Compra, recepción, venta, cobro, abono atómico, reembolso, archivo y QR	`frontend/tests_e2e/closure.spec.ts:27`
Respuesta perdida	POST real confirmado, entrega abortada y lookup sin segundo POST	`frontend/tests_e2e/closure.spec.ts:48`
Auditor	Alta/membresía real, controles ocultos y escritura directa403	`frontend/tests_e2e/closure.spec.ts:57`
Gastos	CompraKG, negativa422, pesos medidos, tres repartos y tarifa aceptada	`frontend/tests_e2e/closure.spec.ts:63`
Acuerdos	Reapertura, edición y dos versiones aceptadas conservadas	`frontend/tests_e2e/closure.spec.ts:73`
Periodos	Cierre/reapertura corriente e historia de versiones	`frontend/tests_e2e/closure.spec.ts:76`
Regresión KG	Dos lotes preservados y venta 300.000kg/3000.00	`frontend/tests_e2e/closure.spec.ts:81`
HTTP502	Entrega simulada502 después de POST201; clave conservada	`frontend/tests_e2e/closure.spec.ts:90`
Cambio de sede	GET retenido B abortado al pasar a A	`frontend/tests_e2e/closure.spec.ts:94`
Predios	Validación, alta, búsqueda, páginas20+1 y refresh	`frontend/tests_e2e/additional.spec.ts:21`
Ciclos	Entrega física y recompra con dos ciclos históricos	`frontend/tests_e2e/additional.spec.ts:27`

Grupo de prueba	Comportamiento concreto	Fuente
Corrección	Original100kg, versión110kg, motivo y grupo incompatible	`frontend/tests_e2e/additional.spec.ts:32`
Reportes	Corte/fuente y descarga CSV real de toda la sede	`frontend/tests_e2e/additional.spec.ts:37`
Reservas	Dos usuarios, conflicto409 y vencimiento real409	`frontend/tests_e2e/additional.spec.ts:42`
Teclado390	Compra→recepción→venta→pago sin ratón ni locator.focus	`frontend/tests_e2e/keyboard.spec.ts:21`
Proveedor físico	GET503, captura preservada y recuperación200	`frontend/tests_e2e/provider.spec.ts:6`

Los helpers de `closure.spec.ts` buscan contrapartes y animales mediante controles reales, seleccionan periodos y esperan una respuesta por método/ruta. Los de `additional.spec.ts` pueden crear precondiciones DEMO mediante API explícita; su función `operation` obtiene GET y ETag entre transiciones. Por eso no se atribuye a clicks la creación de cada precondición. El helper `responsive` cambia viewport y comprueba el ancho del documento, además de capturar.

En `keyboard.spec.ts`, `tabTo` llega al destino usando Tab y sólo lee si obtuvo foco; `enter`, `input`, `select` y `datetime` utilizan eventos de teclado. El datetime se captura por segmentos del control nativo de Chrome. Sólo identidad y contraparte son fixtures API; el flujo financiero utiliza formularios. La evidencia registra534 eventos,364Tab, ancho390 y scrollWidth375. Estos números describen esa corrida, no una cantidad universal de pasos para cualquier conjunto de datos.

La prueba física del proveedor requiere `BOVI_PROVIDER_OUTAGE=true`. Si no existe esa autorización coordinada, se omite con razón explícita. En la tanda final se ejecutó aparte, sin omisiones. Después del503 crea un archivo `ready`; espera que el responsable reanude, compruebe salud y escriba literalmente `resumed`; entonces pulsa volver a consultar. No acepta precio ni crea gasto durante la caída. El login sí registra sesión, por lo que el resultado declara cero escrituras comerciales, no ausencia total de POST.

El cierre registrado fue15/15 integrados y1/1 proveedor físico, cero retries/skips. El reportero de la tanda15 mostró1,3 min redondeados; las duraciones individuales sumaron73.520 ms, sin confundir esa suma con tiempo global. El proveedor tuvo47.431,536 ms de tanda y46.406 ms de caso. Se registraron24 entradas esperadas de consola en contextos principales y cero errores JavaScript;401 anónimo,400 de motivo excesivo,409,403,422,502 inducido,503 físico y entrega abortada se clasificaron. No se anuncia consola vacía.

La GPU registrada fue Intel integrada mediante ANGLE/OpenGL, sin SwiftShader. Eso demuestra el dispositivo de esas capturas, no una medición FPS. La restauración de datos, las pruebas de nodos y el benchmark SQL tienen sus propias evidencias del coordinador; no se atribuyen al runner frontend. Tampoco hay entrevistas, prueba con clientes reales, aprobación humana o despliegue público derivados de estos resultados.

Fuentes: `frontend/tests/domain.test.ts:8`, `frontend/tests/workflow.test.ts:6`, `frontend/playwright.config.ts:2`, `frontend/tests_e2e/runtime.ts:3`, `frontend/tests_e2e/keyboard.spec.ts:8`, `frontend/tests_e2e/provider.spec.ts:7`, `docs/testing/evidence/ui/bovicierre-ui-final-run.json:1`, `docs/development/frontend-context.md:25`.

13.18 18. Guía de estudio y lectura del código sin alterar datos

Para estudiar el sistema conviene seguir primero una lectura sin ejecución. Abra `main.tsx`, identifique `App`, siga `Workspace` hacia una pantalla y anote su ruta. Después busque el `build` del formulario y el helper que normaliza sus campos. Compare el cuerpo con OpenAPI y observe qué campos no envía: total y saldo de confirmación no se calculan en GUI. Continúe hacia `CommandForm`, `createAttempt` y `request`, y termine identificando el `onDone` que consulta o navega después del resultado.

Una segunda lectura puede seguir el camino de un error. Pregunte qué ocurre antes del envío, qué ocurre ante422 y qué cambia ante502. Observe cuándo se borra el intento, dónde queda el ETag y cuál es la identidad que forma la clave de almacenamiento. En archivos o usuarios, examine la diferencia entre huella persistida y contenido de memoria. Este ejercicio revela más del diseño que memorizar colores o endpoints aislados.

Una tercera lectura puede estudiar estado y alcance. Localice `key` en `Workspace`, `AnimalCycles`, `LotEditor`, correcciones y destinos de pago. Identifique efectos que abortan consultas y callbacks directos que recargan recursos. No suponga que todo prop inicial actualiza un `useState` ya montado. La clave de React, las dependencias del efecto y la ruta con sede trabajan juntas, pero cada mecanismo resuelve una parte distinta del problema.

Por último, relacione funciones con evidencia. Una función de formato se estudia con unitarias; una recuperación de POST confirmado requiere la prueba de red correspondiente; una afirmación responsive requiere sus métricas y capturas; una incidencia física requiere coordinación de servicios. La documentación puede explicar cómo repetir un caso, pero no sustituye su ejecución. Durante una sesión compartida no se lanzan pruebas que escriben mientras otro responsable realiza backup o altera funciones SQL.

El frontend implementado permite trabajar con evidencia y estados concretos. Su claridad depende de conservar las distinciones que estructuran el dominio: identidad frente a ciclo, borrador frente a cierre, consulta frente a aceptación de tarifa, dinero registrado frente a aplicación, reverso frente a borrado y resultado incierto frente a rechazo conocido. Cada una tiene componentes, mensajes y comandos visibles en el código descrito. La revisión humana del producto sigue siendo un paso separado del hecho de que esas funciones existan y sus casos locales hayan pasado.

Capítulo 14

Atlas de scripts Python y sus variables

El atlas registra firmas y nombres de variables desde AST de Python, con línea inicial y final. Un nombre local sólo existe durante su función; una constante de módulo puede compartirse dentro del script. No confundir estos nombres con variables de entorno. La lista incluye asignaciones directas y targets de bucles; argumentos, bindings de excepciones y closures deben leerse con el fragmento de código para conocer su alcance exacto.

14.1 scripts/doctor.py

Diagnóstico local sin modificar servicios ni mostrar configuración privada.

Variables de módulo: `COMMANDS`.

14.1.1 inspect_command

Firma: `inspect_command(command)`. Fuente `scripts/doctor.py:21-29`.

Consulta una versión con timeout y registra ausencia o exit code sin alterar servicios.

Variables locales asignadas: `lines`, `result`.

14.1.2 listening

Firma: `listening(port)`. Fuente `scripts/doctor.py:32-35`.

Comprueba si hay un listener TCP en127.0.0.1 para el puerto indicado.

14.1.3 resources

Firma: `resources()`. Fuente `scripts/doctor.py:38-41`.

Obtiene sistema, shell, CPU y memoria desde información del host.

Variables locales asignadas: `memory`, `total`.

14.1.4 main

Firma: `main()`. Fuente `scripts/doctor.py:44-57`.

Entrada CLI que interpreta las opciones y coordina las funciones del archivo; su firma no concede permisos comerciales.

Variables locales asignadas: `result`.

14.2 scripts/academic.py

Arranque académico integrado; rechaza un backend manual concurrente.

Variables de módulo: `ROOT`.

14.2.1 busy

Firma: `busy(port)`. Fuente `scripts/academic.py:11-14`.

Detecta el puerto de API manual ocupado antes del arranque integrado.

14.2.2 main

Firma: `main()`. Fuente `scripts/academic.py:17-28`.

Entrada CLI que interpreta las opciones y coordina las funciones del archivo; su firma no concede permisos comerciales.

Variables locales asignadas: `docs`, `env`, `private`.

14.3 scripts/dev.py

Ejecuta API y Vite en primer plano; Ctrl+C detiene únicamente sus hijos.

Variables de módulo: `ROOT`, `COMMANDS`.

14.3.1 stop

Firma: `stop(children)`. Fuente `scripts/dev.py:16-25`.

Termina únicamente los grupos de procesos hijos creados por make dev; espera y limita la escalada.

Variables locales asignadas: `child`.

14.3.2 main

Firma: `main()`. Fuente `scripts/dev.py:28-50`.

Entrada CLI que interpreta las opciones y coordina las funciones del archivo; su firma no concede permisos comerciales.

Variables locales asignadas: `child`, `children`, `command`, `directory`.

14.4 scripts/recovery.py

Respaldo privado y ensayo aislado BoviCierre; nunca elimina volúmenes.

Variables de módulo: `ROOT`, `PRIVATE`, `SECRETS`, `DOCS`, `SOURCE`.

14.4.1 run

Firma: `run(args,**kwargs)`. Fuente `scripts/recovery.py:24-29`.

Ejecuta una utilidad y conserva su salida; en recuperación filtra el error para no divulgar cadenas de conexión.

Variables locales asignadas: `result`.

14.4.2 sql

Firma: `sql(project,node,query)`. Fuente `scripts/recovery.py:32-37`.

Ejecuta SQL sobre un nodo identificado; en recuperación verifica la etiqueta del contenedor antes de utilizarlo.

Variables locales asignadas: `label`, `name`.

14.4.3 manifest

Firma: `manifest(directory)`. Fuente `scripts/recovery.py:40-49`.

Recorre archivos privados, rechaza enlaces simbólicos y obtiene SHA-256 por ruta relativa.

Variables locales asignadas: `directory`, `path`, `records`, `str`.

14.4.4 signature

Firma: `signature(project,node)`. Fuente `scripts/recovery.py:52-54`.

Obtiene conteos y huellas de filas de negocio replicables para comprobar convergencia.

Variables locales asignadas: `tables`.

14.4.5 local_signature

Firma: `local_signature(project,node)`. Fuente `scripts/recovery.py:57-58`.

Comprueba tablas locales de deduplicación, inbox, entrega, captura y Flyway que no se comparan como réplicas.

14.4.6 private_dir

Firma: `private_dir(path)`. Fuente `scripts/recovery.py:61-63`.

Crea un directorio nuevo con permisos privados; la existencia previa es un error.

14.4.7 backend_mode

Firma: `backend_mode(pid)`. Fuente `scripts/recovery.py:66-84`.

Distingue la API Docker propia de la Java nativa comprobando proceso, cwd, puerto y runtime.

Variables locales asignadas: `container`, `environment`, `native`, `pid`, `pid_file`, `process`, `running`, `state`.

14.4.8 backup

Firma: `backup(destination,pid)`. Fuente `scripts/recovery.py:87-153`.

Pausa la API propia, espera coherencia, copia documentos/secretos y dumps, escribe manifiesto y reanuda en finally.

Variables locales asignadas: `_`, `a`, `active`, `api_container`, `artifacts`, `b`, `container`, `data`, `env`, `local_signatures`, `log`, `log_path`, `node`, `path`, `pending`, `process`, `report`, `restart`, `roles`, `source_documents`, `source_secrets`, `started`.

14.4.9 restore

Firma: `restore(destination)`. Fuente `scripts/recovery.py:157-211`.

Crea un destino aislado, restaura hashes/datos/roles y reconstruye FDW y suscripciones sólo hacia el destino.

Variables locales asignadas: `command`, `compose`, `config`, `container`, `digest`, `env`, `expected`, `name`, `node`, `port`, `project`, `records`, `report`, `roles`, `service`, `started`, `work`.

14.4.10 verify_http

Firma: `verify_http(destination)`. Fuente `scripts/recovery.py:214-271`.

API restaurada en el puerto exclusivo18180; jamás usa el puerto fuente18080.

Variables locales asignadas: `animal`, `base`, `confirmed`, `csrf`, `data`, `detail`, `document`, `documents`, `etag`, `headers`, `inventory`, `key`, `lookup`, `opener`, `operation`, `party`, `party_body`, `path`, `period`, `periods`, `raw`, `recognition`, `report`, `session`, `site`, `today`.

14.4.11 main

Firma: `main()`. Fuente `scripts/recovery.py:274-301`.

Entrada CLI que interpreta las opciones y coordina las funciones del archivo; su firma no concede permisos comerciales.

Variables locales asignadas: `args`, `parser`, `path`.

14.4.12 call

Firma: `call(method,path,payload=None,key=None,match=None,binary=False)`. Fuente `scripts/recovery.py:221-232`.

Helper HTTP local que conserva cookies/CSRF y comprueba el resultado o el contrato según el script.

Variables locales asignadas: `csrf`, `data`, `headers`, `raw`.

14.5 scripts/sync-task-tables.py

Genera las tablas/listas del README desde el backlog canónico, sin cambiar estados.

Variables de módulo: `ROOT`, `backlog`, `sections`, `readme`, `text`, `start`, `end`, `text`.

14.6 tools/esquema/comparar_catalogo.py

Contrasta columnas, tipos, nullabilidad, PK, UK declaradas y FK lógicas contra PostgreSQL propio.

Variables de módulo: `SQL`.

14.6.1 qualified

Firma: `qualified(name)`. Fuente `tools/esquema/comparar_catalogo.py:23-24`.

Resuelve el nombre del objeto con su esquema para compararlo sin ambigüedad.

14.6.2 normalize_type

Firma: `normalize_type(value)`. Fuente `tools/esquema/comparar_catalogo.py:27-29`.

Normaliza representaciones del catálogo para contrastarlas con el modelo lógico.

Variables locales asignadas: `value`.

14.6.3 compare

Firma: `compare(tables,catalog)`. Fuente `tools/esquema/comparar_catalogo.py:32-61`.

Compara columnas/tipos/nullabilidad y claves con el catálogo; no demuestra todas las RN.

Variables locales asignadas: `_`, `actual`, `actual_columns`, `actual_fks`, `actual_keys`, `column`, `columns`, `errors`, `expected`, `expected_fks`, `foreign_keys`, `key`, `keys`, `name`, `primary`, `table`, `unique`, `unique_keys`.

14.6.4 main

Firma: `main()`. Fuente `tools/esquema/comparar_catalogo.py:64-86`.

Entrada CLI que interpreta las opciones y coordina las funciones del archivo; su firma no concede permisos comerciales.

Variables locales asignadas: `args`, `catalog`, `errors`, `labels`, `parser`, `raw`, `table`.

14.7 tools/dev/check_api_contract.py

Valida tráfico DEMO real contra OpenAPI; no guarda payloads ni cookies.

Variables de módulo: `ROOT`.

14.7.1 main

Firma: `main()`. Fuente `tools/dev/check_api_contract.py:17-72`.

Entrada CLI que interpreta las opciones y coordina las funciones del archivo; su firma no concede permisos comerciales.

Variables locales asignadas: `args`, `content`, `contract`, `csrf`, `errors`, `headers`, `media`, `op`, `opener`, `parser`, `payload`, `raw`, `request`, `resolver`, `response`, `results`, `schema`, `secret`, `site`, `spec`, `suffix`, `url`, `validator`.

14.7.2 call

Firma: `call(method,path,template=None,body=None,expected=200)`. Fuente `tools/dev/check_api_contract.py:30-57`.

Helper HTTP local que conserva cookies/CSRF y comprueba el resultado o el contrato según el script.

Variables locales asignadas: `content`, `errors`, `headers`, `media`, `op`, `payload`, `raw`, `request`, `response`, `schema`, `spec`, `validator`.

14.8 db/bootstrap/bootstrap.py

Bootstrap aislado, repetible; nunca opera contenedores fuera de bovicierre-academic.

Variables de módulo: `ROOT`, `SECRET`, `PROJECT`, `DATABASE`, `E`, `A`, `B`, `U`, `masters`, `local`, `restore_converged`, `seed`, `provider`, `fdw`.

14.8.1 lit

Firma: `lit(v)`. Fuente `db/bootstrap/bootstrap.py:13-13`.

Escapa literal SQL usado en bootstrap controlado; no es el mecanismo HTTP de filtros públicos.

14.8.2 sql

Firma: `sql(node,query)`. Fuente `db/bootstrap/bootstrap.py:14-21`.

Ejecuta SQL sobre un nodo identificado; en recuperación verifica la etiqueta del contenedor antes de utilizarlo.

Variables locales asignadas: `error`, `file`, `p`.

14.8.3 secret

Firma: `secret(n)`. Fuente `db/bootstrap/bootstrap.py:22-22`.

Lee el archivo privado requerido por bootstrap; su valor se excluye de este manual.

14.9 db/seed/demo.py

Semilla pequeña repetible por comandos y claves permanentes; no borra ni inserta hechos fina ncieros.

Variables de módulo: `ROOT`, `SECRETS`, `fixture`, `dataset`, `day`, `label`, `e`, `s`, `actor`.

14.9.1 query

Firma: `query(sql,args=())`. Fuente `db/seed/demo.py:21-22`.

Ejecuta una consulta parametrizada de la semilla y recupera su resultado para comprobar el estado persistido.

Variables locales asignadas: `row`.

14.9.2 money

Firma: `money(value)`. Fuente `db/seed/demo.py:23-23`.

Construye el DTO importe/moneda con cadena decimal MXN, sin aritmética flotante.

14.9.3 command

Firma: `command(name,step,body)`. Fuente `db/seed/demo.py:24-26`.

Construye clave y hash del payload y llama cmd_ejecutar; no inserta hechos financieros directa mente.

Variables locales asignadas: `digest`.

14.10 db/tests/benchmark.py

Comandos DEMO idempotentes y lecturas canónicas QA; jamás modifica main.

Variables de módulo: `ROOT`, `FIXTURE`, `DB`, `DATASET`, `E`, `S`, `U`, `STATE`, `CALLS`, `FINANCIAL`.

14.10.1 connect

Firma: `connect(pool=False,readonly=False)`. Fuente `db/tests/benchmark.py:32-43`.

Abre una conexión psycopg A con timeouts; puede pasar por Pgpool y limitarse a lectura para medir.

Variables locales asignadas: `options`, `secret`.

14.10.2 guard

Firma: `guard(connection)`. Fuente `db/tests/benchmark.py:46-60`.

Verifica base QA, servidor18, empresaDEMO, nodoA y las guardas V13 antes del benchmark.

Variables locales asignadas: `company`, `definition`.

14.10.3 command

Firma: `command(connection,name,step,body,financial)`. Fuente `db/tests/benchmark.py:63-73`.

Construye clave y hash del payload y llama cmd_ejecutar; no inserta hechos financieros directa mente.

Variables locales asignadas: `CALLS`, `digest`, `key`, `name`, `payload`.

14.10.4 money

Firma: `money(amount)`. Fuente `db/tests/benchmark.py:76-77`.

Construye el DTO importe/moneda con cadena decimal MXN, sin aritmética flotante.

14.10.5 counts

Firma: `counts(connection)`. Fuente `db/tests/benchmark.py:80-84`.

Obtiene conteos de tablas relevantes para comprobar que el replay no duplicó hechos.

14.10.6 prepare

Firma: `prepare()`. Fuente `db/tests/benchmark.py:87-190`.

Crea el dataset moderado QA por comandos canónicos y contrasta esperados fijos; conserva su manifiesto.

Variables locales asignadas: `actual`, `animal`, `before`, `expected`, `expense`, `financial`, `index`, `key`, `party`, `payment`, `payments`, `period`, `position`, `purchase`, `purchases`, `result`, `sale`, `sales`, `schema`, `settlement`, `settlements`, `started`, `state`, `tag`.

14.10.7 utc

Firma: `utc()`. Fuente `db/tests/benchmark.py:193-194`.

Devuelve el instante actual UTC en formato ISO8601 para fechar evidencia.

14.10.8 revision

Firma: `revision()`. Fuente `db/tests/benchmark.py:197-201`.

Registra commit y estado dirty del checkout en el momento de la ejecución.

14.10.9 machine

Firma: `machine()`. Fuente `db/tests/benchmark.py:204-212`.

Lee CPU, memoria y carga del host desde procfs; no estima consumo exclusivo de PostgreSQL.

Variables locales asignadas: `cpu`, `mem`.

14.10.10 percentiles

Firma: `percentiles(samples)`. Fuente `db/tests/benchmark.py:215-218`.

Obtiene mediana p50 y p95 por rango ceil(0.95*n), además de mínimos/máximos y tamaño de muestra.

14.10.11 params

Firma: `params(resource,index,state)`. Fuente `db/tests/benchmark.py:221-227`.

Selecciona filtros, páginas e IDs conocidos para rotar consultas sobre el dataset autorizado.

14.10.12 measure

Firma: `measure()`. Fuente `db/tests/benchmark.py:230-285`.

Calienta y mide consultas canónicas con una/cuatro conexiones, conserva muestras y planes con límites declarados.

Variables locales asignadas: `barrier`, `begin`, `body`, `elapsed`, `index`, `name`, `output`, `plan`, `query`, `report`, `resources`, `samples`, `started`, `state`, `value`, `workers`.

14.10.13 main

Firma: `main()`. Fuente `db/tests/benchmark.py:288-315`.

Entrada CLI que interpreta las opciones y coordina las funciones del archivo; su firma no concede permisos comerciales.

Variables locales asignadas: `args`, `mock`, `mode`, `parser`, `report`, `summary`.

14.10.14 git

Firma: `git(*args)`. Fuente `db/tests/benchmark.py:198-200`.

Consulta commit/estado mediante subprocess sin modificar Git.

14.10.15 execute

Firma: `execute(connection,name,index)`. Fuente `db/tests/benchmark.py:243-249`.

Cronometra una consulta canónica del benchmark y valida su resultado antes de conservar la muestra.

Variables locales asignadas: `begin`, `body`, `elapsed`, `value`.

14.10.16 worker

Firma: `worker(worker_id)`. Fuente `db/tests/benchmark.py:262-272`.

Mide con una conexión persistente por participante y una barrera por recurso; respeta el máximo de cuatro.

Variables locales asignadas: `begin`, `index`, `name`, `output`, `samples`.

Capítulo 15

Atlas de funciones SQL y declaraciones locales

Este atlas conserva la última definición textual de cada función encontrada en las migraciones ordenadas V1-V13. Es una referencia estática al código; las migraciones que alteran objetos con bloques dinámicos deben interpretarse con su explicación y la evidencia de catálogo. La última definición de `cmd_ejecutar` está enV13, la consulta principal y el reporte canónico enV12. Las declaraciones muestran tipos y valores iniciales públicos del código, nunca una configuración privada ejecutada.

15.1 bovi.archivo_referenciado

Fuente de la definición: `db/migrations/V3__consultas_sesiones_grants.sql:150`.

Argumentos: `p_sha256text`.

Retorno y propiedades: `booleanLANGUAGEsqlSTABLESECURITYDEFINERSETsearch_path=pg_catalog`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn_` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.2 bovi.capturar_tarifas_proveedor

Fuente de la definición: `db/migrations/V9__lector_a_captura_owner_fecha.sql:17`.

Argumentos: `euuid,suuid,auuid,pagina_fuentejsonb`.

Retorno y propiedades: `jsonbLANGUAGEplpgsqlSECURITYDEFINERSETsearch_path=pg_catalog`.

Variables locales declaradas:

- `itemjsonb`.
- `snapjsonb`.
- `siduuid`.
- `resultjsonb:='[]'`.
- `source_pricenumeric`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn_` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.3 bovi.cmd_ejecutar

Fuente de la definición: db/migrations/V13__correccion_composicion_exacta.sql:2.

Argumentos: p_empresauuid,p_sucursaluuid,p_actoruuid,p_clavetext,p_hashtext,p_correlationuuid,p_comandotext,p_payloadjsonb.

Retorno y propiedades: jsonbLANGUAGEplpgsqlSECURITYDEFINERSETsearch_path=pg_catalog.

Variables locales declaradas:

- euuid:=p_empresa.
- suuid:=p_sucursal.
- auuid:=p_actor.
- jjsonb:=p_payload.
- ntext:=bovi.fn_nodo().
- resultjsonb.
- prevbovi.clave_idempotencia%ROWTYPE.
- opbovi.operacion%ROWTYPE.
- originalbovi.pesaje%ROWTYPE.
- posbovi.posicion_inventario%ROWTYPE.
- paybovi.pago%ROWTYPE.
- liqbovi.liquidacion%ROWTYPE.
- appbovi.aplicacion_pago%ROWTYPE.
- gastobovi.gasto_operacion%ROWTYPE.
- rrecord.
- crecord.
- lotjsonb.
- xjsonb.
- idnuuid:=uuidv7().
- opiduuid:=coalesce((j->>'operacion_id')::uuid,(j->>'id')::uuid).
- piduuid.
- liduuid.
- widuuid.
- aiduuid.
- peruuid.
- amountnumeric.
- totalnnumeric.
- costnumeric.
- cntinteger.
- kinteger.
- idxinteger:=0.
- seqninteger.
- contractnsmallint.
- preserveboolean:=false.
- clastext.
- naturetext.
- fdate:=coalesce((j->>'fecha_economica')::date,(j->>'fecha_operacion')::date,current_date).
- refuuid.
- idsuuid[].
- statetext.

Ramas de despacho presentes en esta definición: aceptar_tarifa, crear_usuario, actualizar_usuario, asignar_membresia, crear_contraparte, crear_predio, registrar_animal, COMPRA, acordar_operacion, aprobar_pesaje, COMPRA, registrar_pago, validar_pago, aplicar_pago, revertir_aplicacion, reembolsar_pago, COBRO, ajustar_liquidacion, registrar_gasto, crear_periodo, registrar_documento, registrar_revision, cancelar_operacion, anular_venta.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn _` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.4 bovi.confirmar_entrega

Fuente de la definición: `db/migrations/V3__consultas_sesiones_grants.sql:140`.

Argumentos: `eventiduuid,workeruuid`.

Retorno y propiedades: `voidLANGUAGEplpgsqlSECURITYDEFINERSETsearch_path=pg_catalog`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn _` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.5 bovi.consultar

Fuente de la definición: `db/migrations/V12__pesajes_reportes_canonicos.sql:430`.

Argumentos: `euuid,suuid,auuid,recursotext,paramsjsonb`.

Retorno y propiedades: `jsonbLANGUAGEplpgsqlSECURITYDEFINERSETsearch_path=pg_catalog`.

Variables locales declaradas:

- `tabtext`.
- `filtrotext:=''`.
- `sqltext`.
- `rowsjsonb`.
- `detailjsonb`.
- `totalninteger`.
- `sizeinteger:=coalesce((params->>'tamano')::integer,25)`.
- `pageinteger:=coalesce((params->>'pagina')::integer,1)`.
- `idnuuid:=(params->>'id')::uuid`.
- `searchqtext:=nullif(btrim(params->>'q'),'')`.
- `patterntext`.
- `statefiltertext:=nullif(params->>'estado','')`.
- `typefiltertext:=nullif(params->>'tipo','')`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn _` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.6 bovi.consultar_tarifas_proveedor

Fuente de la definición: `db/migrations/V9__lector_a_captura_owner_fecha.sql:35`.

Argumentos: `euuid,suuid,auuid,paramsjsonb`.

Retorno y propiedades: `jsonbLANGUAGEplpgsqlSECURITYDEFINERSETsearch_path=pg_catalog`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn _` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.7 bovi.consumir_muerte

Fuente de la definición: `db/migrations/V3__consultas_sesiones_grants.sql:121`.

Argumentos: `euuid,suuid,eventiduuid,payloadjsonb`.

Retorno y propiedades: `jsonbLANGUAGEplpgsqlSECURITYDEFINERSETsearch_path=pg_catalog`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn_` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.8 bovi.despachar_outbox

Fuente de la definición: `db/migrations/V3__consultas_sesiones_grants.sql:113`.

Argumentos: `p_limiteintegerDEFAULT50`.

Retorno y propiedades: `jsonbLANGUAGEplpgsqlSECURITYDEFINERSETsearch_path=pg_catalog`.

Variables locales declaradas:

- `rrecord`.
- `cntinteger:=0`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn_` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.9 bovi.fallar_entrega

Fuente de la definición: `db/migrations/V3__consultas_sesiones_grants.sql:145`.

Argumentos: `eventiduuid,workeruuid`.

Retorno y propiedades: `voidLANGUAGEplpgsqlSECURITYDEFINERSETsearch_path=pg_catalog`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn_` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.10 bovi.fn_calcular_liquidacion

Fuente de la definición: `db/migrations/V2__comandos.sql:70`.

Argumentos: `euuid,suuid,opuuid`.

Retorno y propiedades: `TABLE(numero_lotesmallint,cantidadnumeric,unidadtext,precionumeric,basenumeric,cargosnumeric,bonificacionesnumeric,totalnumeric,pesaje_iduuid)LANGUAGEsqlSTABLESETsearch_path=pg_catalog`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn_` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.11 bovi.fn_codigo

Fuente de la definición: `db/migrations/V2__comandos.sql:78`.

Argumentos: `ctext`.

Retorno y propiedades: `textLANGUAGEplpgsqlIMMUTABLESETsearch_path=pg_catalog`.

Variables locales declaradas:

- `normtext:=upper(c)`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn_` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.12 bovi.fn_json_decimal

Fuente de la definición: `db/migrations/V3__consultas_sesiones_grants.sql:24`.

Argumentos: `jjsonb`.

Retorno y propiedades: `jsonbLANGUAGEplpgsqlIMMUTABLESETsearch_path=pg_catalog`.

Variables locales declaradas:

- `ktext`.
- `vjsonb`.
- `resultjsonb:='{}'`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn_` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.13 bovi.fn_liquidacion_json

Fuente de la definición: `db/migrations/V3__consultas_sesiones_grants.sql:35`.

Argumentos: `euuid,suuid,liduuid`.

Retorno y propiedades: `jsonbLANGUAGEsqlSTABLESETsearch_path=pg_catalog`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn_` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.14 bovi.fn_nodo

Fuente de la definición: `db/migrations/V2__comandos.sql:2`.

Argumentos: ‘’.

Retorno y propiedades: `textLANGUAGEsqlSTABLESETsearch_path=pg_catalog`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn_` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.15 bovi.fn_normalizar_payload

Fuente de la definición: `db/migrations/V2__comandos.sql:81`.

Argumentos: `cmdtext,jjsonb`.

Retorno y propiedades: `jsonbLANGUAGEplpgsqlIMMUTABLESETsearch_path=pg_catalog`.

Variables locales declaradas:

- `outjjsonb:=j`.
- `lotjsonb`.
- `lotsjsonb:='[]'`.
- `vtext`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn_` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.16 bovi.fn_periodo

Fuente de la definición: `db/migrations/V2__comandos.sql:55`.

Argumentos: `euuid,suuid,fdate,piduuid`.

Retorno y propiedades: `uuidLANGUAGEplpgsqlSETsearch_path=pg_catalog`.

Variables locales declaradas:

- `rbovi.periodo_cierre%ROWTYPE`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn_` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.17 bovi.fn_periodo_payload

Fuente de la definición: `db/migrations/V2__comandos.sql:63`.

Argumentos: `euuid,suuid,jjsonb`.

Retorno y propiedades: `uuidLANGUAGEsqlSETsearch_path=pg_catalog`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn_` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.18 bovi.fn_pesaje_json

Fuente de la definición: `db/migrations/V12__pesajes_reportes_canonicos.sql:2`.

Argumentos: `euuid,suuid,piduuid`.

Retorno y propiedades: `jsonbLANGUAGEsqlSTABLESETsearch_path=pg_catalog`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn_` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.19 bovi.fn_repartir

Fuente de la definición: `db/migrations/V2__comandos.sql:64`.

Argumentos: `mnumeric,ninteger`.

Retorno y propiedades: `TABLE(numerointeger,importenumeric)LANGUAGEsqlIMMUTABLESETsearch_path=pg_catalog`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn_` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.20 bovi.fn_reparto

Fuente de la definición: db/migrations/V2__comandos.sql:99.

Argumentos: euuid,suuid,giduuid,metodotext,asignacionesjsonbDEFAULTNULL.

Retorno y propiedades: TABLE(posicion_iduuid,base_valornumeric,importenumeric)LANGUAGEplpgsqlSTABLESETsearch_path=pg_catalog.

Variables locales declaradas:

- gbovi.gasto_operacion%ROWTYPE.
- idsuuid[].
- weightsnumeric[].
- valuesnnumeric[].
- sourceweightnumeric.
- entryjsonb.
- amountnumeric.
- totalweightnumeric.
- roundedtotalnumeric.
- residualinteger.
- idxinteger.
- signninteger.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo fn_ agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.21 bovi.fn_reporte

Fuente de la definición: db/migrations/V12__pesajes_reportes_canonicos.sql:12.

Argumentos: euuid,suuid,recursotext,paramsjsonb.

Retorno y propiedades: jsonbLANGUAGEplpgsqlSTABLESETsearch_path=pg_catalog.

Variables locales declaradas:

- metadatajsonb.
- resultjsonb.
- patterntext.
- qtext:=nullif(btrim(params->>'q'),'').
- sizeinteger:=coalesce((params->>'tamano')::integer,25).
- pageinteger:=coalesce((params->>'pagina')::integer,1).
- periodiduuid:=(params->>'periodo_id')::uuid.
- operationiduuid:=(params->>'operacion_id')::uuid.
- periodstatetext.
- faltantesjsonb:='[]'.
- ownernodetext.
- closurerecord.
- todaydate.
- directiontext:=params->>'direccion'.
- overdueboolean.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo fn_ agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.22 bovi.fn_scope

Fuente de la definición: db/migrations/V2__comandos.sql:3.

Argumentos: euuid,suuid,auuid,roltextDEFAULTNULL,maestrobooleanDEFAULTfalse.

Retorno y propiedades: voidLANGUAGEplpgsqlSETsearch_path=pg_catalog.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo fn _ agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.23 bovi.fn_validar_conciliacion_propietario

Fuente de la definición: db/migrations/V3__consultas_sesiones_grants.sql:38.

Argumentos: euuid,suuid,auuid.

Retorno y propiedades: jsonbLANGUAGEplpgsqlSECURITYDEFINERSETsearch_path=pg_catalog.

Variables locales declaradas:

- fkrecord.
- localcolstext.
- joincolstext.
- nonnulltext.
- missingbigint.
- totalmissingbigint:=0.
- errorsjsonb:='[]'.
- settlementsbigint.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo fn _ agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.24 bovi.leer_tarifas_proveedor

Fuente de la definición: db/migrations/V9__lector_a_captura_owner_fecha.sql:2.

Argumentos: euuid,suuid,auuid,paramsjsonb.

Retorno y propiedades: jsonbLANGUAGEplpgsqlSECURITYDEFINERSETsearch_path=pg_catalog.

Variables locales declaradas:

- rrecord.
- siduuid.
- snapjsonb.
- resultjsonb:='[]'.
- countninteger:=0.
- totalninteger.
- pageinteger:=coalesce((params->>'pagina')::integer,1).
- sizeinteger:=coalesce((params->>'tamano')::integer,25).

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo fn _ agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.25 bovi.reclamar_entregas

Fuente de la definición: db/migrations/V3__consultas_sesiones_grants.sql:131.

Argumentos: p_workeruuid,p_limiteintegerDEFAULT50.

Retorno y propiedades: jsonbLANGUAGEplpgsqlSECURITYDEFINERSETsearch_path=pg_catalog.

Variables locales declaradas:

- rrecord.
- claimedjsonb:='[]'.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo fn _ agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.26 bovi.trg_acuerdo_completo

Fuente de la definición: db/migrations/V8__borrador_kg_y_expiracion.sql:5.

Argumentos: ".

Retorno y propiedades: triggerLANGUAGEplpgsqlSETsearch_path=pg_catalog.

Variables locales declaradas:

- versmallint.
- rrecord.
- wbovi.pesaje%ROWTYPE.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo fn _ agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.27 bovi.trg_aprobado

Fuente de la definición: db/migrations/V2__comandos.sql:29.

Argumentos: ".

Retorno y propiedades: triggerLANGUAGEplpgsqlSETsearch_path=pg_catalog.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo fn _ agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.28 bovi.trg_borrador

Fuente de la definición: db/migrations/V2__comandos.sql:22.

Argumentos: ".

Retorno y propiedades: triggerLANGUAGEplpgsqlSETsearch_path=pg_catalog.

Variables locales declaradas:

- jjsonb:=CASEWHENTG_OP='DELETE'THENto_jsonb(OLD)ELSEto_jsonb(NEW)END.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo fn _ agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.29 bovi.trg_inmutable

Fuente de la definición: `db/migrations/V2__comandos.sql:20`.

Argumentos: “.

Retorno y propiedades: `triggerLANGUAGEplpgsqlSETsearch_path=pg_catalog`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn _` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.30 bovi.trg_pesaje_composicion

Fuente de la definición: `db/migrations/V2__comandos.sql:35`.

Argumentos: “.

Retorno y propiedades: `triggerLANGUAGEplpgsqlSETsearch_path=pg_catalog`.

Variables locales declaradas:

- `jjsonb:=CASEWHENTG_OP='DELETE'THENto_jsonb(OLD)ELSEto_jsonb(NEW)END`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn _` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.31 bovi.trg_propietario

Fuente de la definición: `db/migrations/V2__comandos.sql:9`.

Argumentos: “.

Retorno y propiedades: `triggerLANGUAGEplpgsqlSETsearch_path=pg_catalog`.

Variables locales declaradas:

- `jjsonb:=to_jsonb(NEW)`.
- `oldjjsonb:=to_jsonb(OLD)`.
- `ntext:=bovi.fn_nodo()`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn _` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

15.32 bovi.trg_snapshot_tarifa

Fuente de la definición: `db/migrations/V4__capturas_proveedor_dto.sql:11`.

Argumentos: “.

Retorno y propiedades: `triggerLANGUAGEplpgsqlSETsearch_path=pg_catalog`.

Para entender sus efectos, consulta la explicación SQL del cuerpo principal y las RN. El prefijo `fn _` agrupa auxiliares; una función SECURITY DEFINER requiere los grants explícitos y el contexto validado. Una llamada no autoriza por sí sola al usuario humano.

Capítulo 16

Referencia completa del contrato HTTP

Esta referencia se extrae de OpenAPI0.2.0 en la revisión92 d3 d41. Describe el acuerdo versionado; sus respuestas declaradas no equivalen a una ejecución nueva de cada combinación . Las clases y casos de prueba explicados en el cuerpo identifican la implementación y las evidencias reales. Los nombres de parámetros y esquemas conservan su ortografía para buscar directamente en el código.

16.1 POST /sesion/login

Inicia sesión (cookie HttpOnly)

Identificador: `login`.

Parámetro y ubicación	Tipo	Requerido y propósito
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: object. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
204	Sesión creada (Set-Cookie)	Sin cuerpo descrito
401	Sin sesión	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error

16.2 POST /sesion/logout

Logout

Identificador: `logout`.

Parámetro y ubicación	Tipo	Requerido y propósito
`X-XSRF-TOKEN` (header)	string	Sí.

HTTP	Significado declarado	Cuerpo y media type
204	Sesión cerrada	Sin cuerpo descrito
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error

16.3 GET /sesion/actual

Usuario actual, sedes y capacidades autorizadas

Identificador: `sesionActual`.

HTTP	Significado declarado	Cuerpo y media type
200	OK	application/json: SesionActual
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error

16.4 GET /sedes/{sedeId}/contrapartes

Listar contrapartes

Identificador: `listarContrapartes`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`pagina` (query)	integer	No.
`tamano` (query)	integer	No.
`q` (query)	string	No.

HTTP	Significado declarado	Cuerpo y media type
200	OK	application/json: objeto
403	Sin permiso para la sede o la acción	application/problem+json: Error
401	Sin sesión	application/problem+json: Error

HTTP	Significado declarado	Cuerpo y media type
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.5 POST /sedes/{sedeId}/contrapartes

Crear contraparte

Identificador: `crearContraparte`. Escritura maestra; se ejecuta en el nodo A.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: ContraparteNueva. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Creada	application/json: Contraparte
409	Conflicto de concurrencia, version_lock o idempotencia (misma clave, payload distinto)	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.6 POST /sedes/{sedeId}/animales

Registrar identidad animal

Identificador: `registrarIdentidadAnimal`. Ejecutado en el maestro A. Si la sede es B, la respuesta indica PENDIENTE_CONVERGENCIA hasta que la identidad sea visible en B.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: object. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Registrado	application/json: Animal
409	Conflicto de concurrencia, version_lock o idempotencia (misma clave, payload distinto)	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.7 GET /sedes/{sedeId}/animales

listarAnimales

Identificador: `listarAnimales`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`pagina` (query)	integer	No.
`tamano` (query)	integer	No.
`q` (query)	string	No. Búsqueda literal acotada; % y _ no actúan como comodines.

HTTP	Significado declarado	Cuerpo y media type
200	OK	application/json: objeto
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.8 GET /sedes/{sedeId}/animales/{animalId}

Ficha animal

Identificador: `fichaAnimal`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`animalId` (path)	string (uuid)	Sí.

HTTP	Significado declarado	Cuerpo y media type
200	Ficha con ciclos históricos	application/json: FichaAnimal
404	No existe o no es accesible (política uniforme)	application/problem+json: Error
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.9 GET /sedes/{sedeId}/operaciones

Bandeja operaciones

Identificador: `bandejaOperaciones`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`pagina` (query)	integer	No.
`tamano` (query)	integer	No.
`estado` (query)	EstadoOperacion	No.
`tipo` (query)	string	No.
`q` (query)	string	No. Búsqueda literal acotada; % y _ no actúan como comodines.

HTTP	Significado declarado	Cuerpo y media type
200	OK	application/json: objeto
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.10 POST /sedes/{sedeId}/operaciones

Crear borrador

Identificador: `crearBorrador`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: OperacionNueva. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Borrador creado	application/json: Operacion
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.11 GET /sedes/{sedeId}/operaciones/{operacionId}

Obtener operacion

Identificador: `obtenerOperacion`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`operacionId` (path)	string (uuid)	Sí.

HTTP	Significado declarado	Cuerpo y media type
200	OK	application/json: Operacion
404	No existe o no es accesible (política uniforme)	application/problem+json: Error
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error

HTTP	Significado declarado	Cuerpo y media type
400	Forma o parámetro inválido	application/problem+json: Error

16.12 PATCH /sedes/{sedeId}/operaciones/{operacionId}

Editar borrador

Identificador: `editarBorrador`. Edita lotes, selecciones y condiciones de la versión en curso. Sólo BORRADOR/ACORDADA.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`operacionId` (path)	string (uuid)	Sí.
`If-Match` (header)	string	Sí. version_lock recibido en ETag
`X-XSRF-TOKEN` (header)	string	Sí.
`Idempotency-Key` (header)	string	Sí.

Cuerpo application/json: EdicionBorrador. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
200	Actualizado	application/json: Operacion
409	Conflicto de concurrencia, version_lock o idempotencia (misma clave, payload distinto)	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
412	If-Match no corresponde a la versión revisada	application/problem+json: Error
428	Falta If-Match obligatorio	application/problem+json: Error
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.13 POST /sedes/{sedeId}/operaciones/{operacionId}/confirmar

Confirmar operacion

Identificador: `confirmarOperacion`. ConfirmarCompra o ConfirmarVenta según el tipo (RN-021, RN-022). Atómico.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`operacionId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`If-Match` (header)	string	Sí. version_lock recibido en ETag
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: Reconocimiento. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
200	Confirmada (o resultado previo si es reintento con la misma clave y payload)	application/json: Liquidacion
409	Conflicto de concurrencia, version_lock o idempotencia (misma clave, payload distinto)	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
412	If-Match no corresponde a la versión revisada	application/problem+json: Error
428	Falta If-Match obligatorio	application/problem+json: Error
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.14 POST /sedes/{sedeId}/operaciones/{operacionId}/pesajes

Proponer pesaje

Identificador: `proponerPesaje`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`operacionId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: PesajeNuevo. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Propuesto	application/json: Pesaje
422	Viola una regla de negocio	application/problem+json: Error
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.15 POST /sedes/{sedeId}/pesajes/{pesajeId}/aprobar

Aprobar pesaje

Identificador: `aprobarPesaje`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`pesajeId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

HTTP	Significado declarado	Cuerpo y media type
200	Aprobado y composición congelada	application/json: Pesaje
422	Viola una regla de negocio	application/problem+json: Error
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.16 POST /sedes/{sedeId}/pesajes/{pesajeId}/correcciones

Corregir pesaje

Identificador: `corregirPesaje`. Crea un nuevo pesaje que corrige al original (no lo edita).

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`pesajeId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: PesajeCorreccion. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Corrección registrada	application/json: Pesaje
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.17 POST /sedes/{sedeId}/operaciones/{operacionId}/reservas

Reservar posiciones

Identificador: `reservarPosiciones`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`operacionId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: object. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Reservas activas	application/json: colección de Reserva
409	Posición no disponible (otra reserva activa o vendida)	application/problem+json: Error
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error

HTTP	Significado declarado	Cuerpo y media type
422	Viola una regla de negocio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.18 POST /sedes/{sedeId}/reservas/{reservaId}/liberar

Liberar reserva

Identificador: `liberarReserva`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`reservaId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

HTTP	Significado declarado	Cuerpo y media type
200	Liberada	application/json: Reserva
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.19 GET /sedes/{sedeId}/inventario/posiciones

Listar posiciones

Identificador: `listarPosiciones`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`pagina` (query)	integer	No.
`tamano` (query)	integer	No.
`estado` (query)	EstadoPosicion	No.
`disponibles_para_venta` (query)	boolean	No. true: sólo posiciones disponibles, sin reserva activa vigente ni identidad bloqueada; false: no aplicar ese filtro.
`q` (query)	string	No. Filtro de búsqueda acotado.

HTTP	Significado declarado	Cuerpo y media type
200	OK	application/json: objeto
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.20 POST /sedes/{sedeId}/inventario/movimientos

Registrar movimiento

Identificador: `registrarMovimiento`. Recepción, entrega o baja. No genera venta, costo ni obligación (RN-020).

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: object. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Registrado	Sin cuerpo descrito
422	Viola una regla de negocio	application/problem+json: Error
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.21 GET /sedes/{sedeId}/liquidaciones/{liquidacionId}

Detalle liquidacion

Identificador: `detalleLiquidacion`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`liquidacionId` (path)	string (uuid)	Sí.

HTTP	Significado declarado	Cuerpo y media type
200	Snapshot, ajustes, aplicaciones y saldo	application/json: Liquidacion
404	No existe o no es accesible (política uniforme)	application/problem+json: Error
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.22 POST /sedes/{sedeId}/liquidaciones/{liquidacionId}/ajustes

Ajustar liquidacion

Identificador: `ajustarLiquidacion`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`liquidacionId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: object. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Ajuste registrado	application/json: Liquidacion
422	Viola una regla de negocio	application/problem+json: Error
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.23 GET /sedes/{sedeId}/liquidaciones/{liquidacionId}/comprobante

Comprobante interno

Identificador: `comprobanteInterno`. PDF interno (no es CFDI). No genera ningún hecho económico.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`liquidacionId` (path)	string (uuid)	Sí.

HTTP	Significado declarado	Cuerpo y media type
200	PDF	application/pdf: string (binary)
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.24 POST /sedes/{sedeId}/gastos

Registrar gasto

Identificador: `registrarGasto`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: GastoNuevo. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Gasto registrado	application/json: Gasto
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.25 GET /sedes/{sedeId}/gastos

listarGastos

Identificador: `listarGastos`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`pagina` (query)	integer	No.
`tamano` (query)	integer	No.
`q` (query)	string	No. Búsqueda literal acotada; % y _ no actúan como comodines.

HTTP	Significado declarado	Cuerpo y media type
200	OK	application/json: objeto
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.26 GET /sedes/{sedeId}/gastos/{gastoId}/reparto

Previsualizar reparto

Identificador: `previsualizarReparto`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`gastoId` (path)	string (uuid)	Sí.
`metodo` (query)	string	Sí.
`pagina` (query)	integer	No.
`tamano` (query)	integer	No.

HTTP	Significado declarado	Cuerpo y media type
200	Asignaciones propuestas (sin persistir)	application/json: objeto
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error

HTTP	Significado declarado	Cuerpo y media type
400	Forma o parámetro inválido	application/problem+json: Error

16.27 POST /sedes/{sedeId}/gastos/{gastoId}/reparto

Aprobar reparto

Identificador: `aprobarReparto`. Aprueba el gasto y crea asignaciones y asientos según destino económico (incluye costos tardíos).

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`gastoId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: DistribucionInput. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Aprobado	Sin cuerpo descrito
422	Viola una regla de negocio	application/problem+json: Error
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.28 POST /sedes/{sedeId}/pagos

Registrar pago

Identificador: `registrarPago`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: PagoNuevo. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Registrado	application/json: Pago

HTTP	Significado declarado	Cuerpo y media type
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.29 GET /sedes/{sedeId}/pagos

listarPagos

Identificador: `listarPagos`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`pagina` (query)	integer	No.
`tamano` (query)	integer	No.
`q` (query)	string	No. Búsqueda literal acotada; % y _ no actúan como comodines.
`estado` (query)	string	No.
`tipo` (query)	string	No.

HTTP	Significado declarado	Cuerpo y media type
200	OK	application/json: objeto
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.30 POST /sedes/{sedeId}/pagos/{pagoId}/aplicaciones

Aplicar pago

Identificador: `aplicarPago`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`pagoId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: object. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Aplicado	application/json: Pago
422	Excede disponible o saldo (RN-029)	application/problem+json: Error
409	Conflicto de concurrencia, version_lock o idempotencia (misma clave, payload distinto)	application/problem+json: Error
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.31 POST /sedes/{sedeId}/aplicaciones/{aplicacionId}/reverso

Revertir aplicacion

Identificador: `revertirAplicacion`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`aplicacionId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: object. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Reverso completo registrado	Sin cuerpo descrito
409	Conflicto de concurrencia, version_lock o idempotencia (misma clave, payload distinto)	application/problem+json: Error
401	Sin sesión	application/problem+json: Error

HTTP	Significado declarado	Cuerpo y media type
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.32 POST /sedes/{sedeId}/pagos/{pagoId}/reembolsos

Registrar reembolso

Identificador: `registrarReembolso`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`pagoId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: object. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Reembolso registrado	application/json: Pago
422	Viola una regla de negocio	application/problem+json: Error
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.33 GET /sedes/{sedeId}/reportes/cartera

Reporte cartera

Identificador: `reporteCartera`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`pagina` (query)	integer	No.
`tamano` (query)	integer	No.

Parámetro y ubicación	Tipo	Requerido y propósito
`direccion` (query)	string	No.
`vencida` (query)	boolean	No.
`q` (query)	string	No. Filtro de búsqueda acotado.

HTTP	Significado declarado	Cuerpo y media type
200	Cartera con corte	application/json: ReporteCartera
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.34 GET /sedes/{sedeId}/reportes/margen

Reporte margen

Identificador: `reporteMargen`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`operacion_id` (query)	string (uuid)	No.
`periodo_id` (query)	string (uuid)	No.
`q` (query)	string	No. Filtro de búsqueda acotado.

HTTP	Significado declarado	Cuerpo y media type
200	Margen etiquetado PROVISIONAL/ACTUALIZADO/CERRADO	application/json: ReporteMargen
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.35 POST /sedes/{sedeId}/periodos/{periodoId}/cierre

Cerrar periodo

Identificador: `cerrarPeriodo`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`periodoId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: object. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Versión de cierre creada	Sin cuerpo descrito
409	Conflicto de concurrencia, version_lock o idempotencia (misma clave, payload distinto)	application/problem+json: Error
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.36 POST /sedes/{sedeId}/documentos

Subir documento

Identificador: `subirDocumento`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo multipart/form-data: object. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Documento (metadatos y sha256)	application/json: Documento
422	Viola una regla de negocio	application/problem+json: Error
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error

HTTP	Significado declarado	Cuerpo y media type
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
413	Archivo excede 20 MiB	application/problem+json: Error
415	Contenido real de archivo no admitido	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.37 GET /sedes/{sedeId}/documentos

Consultar versiones de documentos privadas

Identificador: `listarDocumentos`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`q` (query)	string	No. Búsqueda literal acotada; % y _ no actúan como comodines.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: objeto
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.38 GET /sedes/{sedeId}/documentos/{documentoId}/contenido

Descargar documento

Identificador: `descargarDocumento`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`documentoId` (path)	string (uuid)	Sí.

HTTP	Significado declarado	Cuerpo y media type
200	Contenido autorizado	application/octet-stream: string (binary)
404	No existe o no es accesible (política uniforme)	application/problem+json: Error
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.39 GET /sedes/{sedeId}/integraciones/proveedor/tarifas

Consultar tarifas proveedor

Identificador: `consultarTarifasProveedor`. Consulta y captura técnica local de proveedor simulado por el dueño de la sede; sin locks financieros. No-store. Una captura no constituye todavía un gasto.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`ruta` (query)	string	No.
`pagina` (query)	integer	No.
`tamano` (query)	integer	No.

HTTP	Significado declarado	Cuerpo y media type
200	Tarifas	application/json: objeto
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.40 GET /sesion/csrf

obtenerCsrf

Identificador: `obtenerCsrf`.

HTTP	Significado declarado	Cuerpo y media type
200	OK	application/json: Csrf
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error

16.41 GET /sedes/{sedeId}/liquidaciones

listarLiquidaciones

Identificador: `listarLiquidaciones`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`pagina` (query)	integer	No.
`tamano` (query)	integer	No.
`q` (query)	string	No. Búsqueda literal acotada; % y _ no actúan como comodines.

HTTP	Significado declarado	Cuerpo y media type
200	OK	application/json: objeto
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.42 POST /sedes/{sedeId}/pagos/{pagoId}/validar

Valida dinero registrado y su periodo

Identificador: `validarPago`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`pagoId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: Reconocimiento. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
200	OK	application/json: Pago
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.43 GET /sedes/{sedeId}/comandos/{comando}/{clave}

Recuperar un resultado propio comprometido

Identificador: `consultarResultadoComando`. Un 404 no prueba rollback de una petición anterior. Reintentar conserva clave, contenido y ETag.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`comando` (path)	string	Sí.
`clave` (path)	string	Sí.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado comprometido	application/json: object
404	Sin resultado visible en este instante	Sin cuerpo descrito
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.44 GET /sedes/{sedeId}/usuarios

Consultar usuarios

Identificador: `listarUsuarios`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`pagina` (query)	integer	No.

Parámetro y ubicación	Tipo	Requerido y propósito
`tamano` (query)	integer	No.
`q` (query)	string	No. Búsqueda literal acotada; % y _ no actúan como comodines.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: objeto
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.45 POST /sedes/{sedeId}/usuarios

Crear usuario por ADMIN

Identificador: `crearUsuario`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: UsuarioNuevo. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Resultado persistido del servidor	application/json: ResultadoComando
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
409	Conflicto de concurrencia, version_lock o idempotencia (misma clave, payload distinto)	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error

HTTP	Significado declarado	Cuerpo y media type
400	Forma o parámetro inválido	application/problem+json: Error

16.46 GET /sedes/{sedeId}/periodos

Consultar periodos

Identificador: `listarPeriodos`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`pagina` (query)	integer	No.
`tamano` (query)	integer	No.
`q` (query)	string	No. Búsqueda literal acotada; % y _ no actúan como comodines.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: objeto
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.47 POST /sedes/{sedeId}/periodos

Abrir mes calendario consecutivo

Identificador: `crearPeriodo`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: object. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Resultado persistido del servidor	application/json: ResultadoComando
401	Sin sesión	application/problem+json: Error

HTTP	Significado declarado	Cuerpo y media type
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
409	Conflicto de concurrencia, version_lock o idempotencia (misma clave, payload distinto)	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.48 GET /sedes/{sedeId}/predios

Consultar predios

Identificador: `listarPredios`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`pagina` (query)	integer	No.
`tamano` (query)	integer	No.
`q` (query)	string	No. Búsqueda literal acotada; % y _ no actúan como comodines.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: objeto
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.49 POST /sedes/{sedeId}/predios

Crear predio declarado

Identificador: `crearPredio`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: PredioNuevo. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Resultado persistido del servidor	application/json: ResultadoComando
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
409	Conflicto de concurrencia, version_lock o idempotencia (misma clave, payload distinto)	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.50 PATCH /sedes/{sedeId}/usuarios/{usuarioId}

Cambiar nombre o vigencia del acceso

Identificador: `editarUsuario`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`usuarioId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: object. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: ResultadoComando
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error

HTTP	Significado declarado	Cuerpo y media type
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
409	Conflicto de concurrencia, version_lock o idempotencia (misma clave, payload distinto)	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.51 POST /sedes/{sedeId}/usuarios/{usuarioId}/membresias

Asignar o retirar rol en la sede autorizada

Identificador: `asignarMembresia`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`usuarioId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: object. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: ResultadoComando
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
409	Conflicto de concurrencia, version_lock o idempotencia (misma clave, payload distinto)	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.52 POST /sedes/{sedeId}/periodos/{periodoId}/reabrir

Reabrir último periodo sin periodos posteriores

Identificador: `reabrirPeriodo`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`periodoId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: object. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: ResultadoComando
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
409	Conflicto de concurrencia, version_lock o idempotencia (misma clave, payload distinto)	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.53 GET /sedes/{sedeId}/periodos/{periodoId}/versiones

Consultar versiones históricas conservadas

Identificador: `consultarVersionesCierre`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`periodoId` (path)	string (uuid)	Sí.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: object
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error

HTTP	Significado declarado	Cuerpo y media type
400	Forma o parámetro inválido	application/problem+json: Error

16.54 POST /sedes/{sedeId}/operaciones/{operacionId}/acordar

Acordar operación conservando versiones

Identificador: `acordarOperacion`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`operacionId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.
`If-Match` (header)	string	Sí. version_lock recibido en ETag

Cuerpo application/json: object. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: ResultadoComando
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
409	Conflicto de concurrencia, version_lock o idempotencia (misma clave, payload distinto)	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
412	If-Match no corresponde a la versión revisada	application/problem+json: Error
428	Falta If-Match obligatorio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.55 POST /sedes/{sedeId}/operaciones/{operacionId}/reabrir

Reabrir operación conservando versiones

Identificador: `reabrirOperacion`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`operacionId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.
`If-Match` (header)	string	Sí. version_lock recibido en ETag

Cuerpo application/json: object. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: ResultadoComando
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
409	Conflicto de concurrencia, version_lock o idempotencia (misma clave, payload distinto)	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
412	If-Match no corresponde a la versión revisada	application/problem+json: Error
428	Falta If-Match obligatorio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.56 POST /sedes/{sedeId}/operaciones/{operacionId}/cancelar

Cancelar operación conservando versiones

Identificador: `cancelarOperacion`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`operacionId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.
`If-Match` (header)	string	Sí. version_lock recibido en ETag

Cuerpo application/json: object. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: ResultadoComando
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
409	Conflicto de concurrencia, version_lock o idempotencia (misma clave, payload distinto)	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
412	If-Match no corresponde a la versión revisada	application/problem+json: Error
428	Falta If-Match obligatorio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.57 POST /sedes/{sedeId}/operaciones/{operacionId}/anular

Anular operación conservando versiones

Identificador: `anularOperacion`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`operacionId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.
`If-Match` (header)	string	Sí. version_lock recibido en ETag

Cuerpo application/json: Reconocimiento. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: ResultadoComando
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error

HTTP	Significado declarado	Cuerpo y media type
409	Conflicto de concurrencia, version_lock o idempotencia (misma clave, payload distinto)	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
412	If-Match no corresponde a la versión revisada	application/problem+json: Error
428	Falta If-Match obligatorio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.58 GET /sedes/{sedeId}/operaciones/{operacionId}/simulador

Previsualizar sin reconocer hechos

Identificador: `simularOperacion`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`operacionId` (path)	string (uuid)	Sí.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: object
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.59 GET /sedes/{sedeId}/operaciones/{operacionId}/expediente

Consultar hechos y documentos de la operación

Identificador: `consultarExpediente`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`operacionId` (path)	string (uuid)	Sí.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: object
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.60 GET /sedes/{sedeId}/pagos/{pagoId}

Consultar dinero disponible canónico

Identificador: `consultarPago`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`pagoId` (path)	string (uuid)	Sí.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: Pago
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.61 GET /sedes/{sedeId}/gastos/{gastoId}

Consultar gasto persistido y procedencia

Identificador: `consultarGasto`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`gastoId` (path)	string (uuid)	Sí.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: Gasto
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.62 GET /sedes/{sedeId}/documentos/{documentoId}

Consultar metadatos y revisión exacta

Identificador: `consultarDocumento`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`documentoId` (path)	string (uuid)	Sí.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: Documento
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.63 POST /sedes/{sedeId}/documentos/{documentoId}/revisiones

Registrar revisión administrativa de esta versión

Identificador: `revisarDocumento`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`documentoId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: object. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: ResultadoComando
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
409	Conflicto de concurrencia, version_lock o idempotencia (misma clave, payload distinto)	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.64 GET /sedes/{sedeId}/pesajes/comparacion

Comparar exclusivamente conjuntos compatibles

Identificador: `compararPesajes`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`inicial_id` (query)	string (uuid)	Sí.
`final_id` (query)	string (uuid)	Sí.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: object
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.65 GET /sedes/{sedeId}/animales/{animalId}/qr

Exportar recurso autenticado

Identificador: `qrAnimal`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`animalId` (path)	string (uuid)	Sí.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	image/svg+xml: string
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.66 GET /sedes/{sedeId}/reportes/{tipo}/csv

Exportar recurso autenticado

Identificador: `exportarCsv`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`tipo` (path)	string	Sí.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	text/csv: string
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.67 GET /sedes/{sedeId}/sincronizacion

Consultar incidencias del propietario

Identificador: `consultarSincronizacion`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.

HTTP	Significado declarado	Cuerpo y media type
200	Resultado persistido del servidor	application/json: object
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error
400	Forma o parámetro inválido	application/problem+json: Error

16.68 POST /sedes/{sedeId}/integraciones/proveedor/tarifas/aceptaciones

Aceptar captura local sin consultar de nuevo el proveedor

Identificador: `aceptarTarifa`.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.
`Idempotency-Key` (header)	string	Sí.
`X-XSRF-TOKEN` (header)	string	Sí.

Cuerpo application/json: object. Obligatorio.

HTTP	Significado declarado	Cuerpo y media type
201	Captura aceptada para registrar gasto en el mismo actor/sede	application/json: ResultadoComando
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
409	Conflicto de concurrencia, version_lock o idempotencia (misma clave, payload distinto)	application/problem+json: Error
422	Viola una regla de negocio	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error

16.69 GET /sedes/{sedeId}/integraciones/diagnostico

Diagnóstico técnico acotado de ADMIN

Identificador: `consultarDiagnostico`. Cache-Control:no-store; sin URLs, credenciales ni SQL. Duraciones acumuladas de proceso, sin histogramas ni certificaciónSLO.

Parámetro y ubicación	Tipo	Requerido y propósito
`sedeId` (path)	string (uuid)	Sí.

HTTP	Significado declarado	Cuerpo y media type
200	Contadores del proceso y pools reales, sólo ADMIN; reinician al reiniciar API.	application/json: DiagnosticoProceso
401	Sin sesión	application/problem+json: Error
403	Sin permiso para la sede o la acción	application/problem+json: Error
503	Dependencia no disponible; una respuesta perdida puede requerir comprobar resultado con la misma clave.	application/problem+json: Error

16.70 GET /salud

Salud del proceso y conectividad de componentes

Identificador: `consultarSaludProceso`.

HTTP	Significado declarado	Cuerpo y media type
200	El proceso sigue activo aunque un nodo falle; no sustituye disponibilidad auten ticada.	application/json: object

Capítulo 17

Diccionario completo de DTO y tipos API

Un DTO es el objeto que cruza HTTP. Un `$ref` reutiliza la definición de un tipo; no es una FK. `required` indica presencia obligatoria, `nullable` o un tipo que incluye null indica ausencia admitida. Un número transportado como string preserva su precisión y escala; el formato visual no cambia el valor canónico.

17.1 Decimal2

Definición contractual reutilizable de tipo Decimal2.

Tipo: string.

Patrón: `^-?\d{1,16}\.\d{2}$`.

17.2 Decimal3

Definición contractual reutilizable de tipo Decimal3.

Tipo: string.

Patrón: `^\d{1,11}\.\d{3}$`.

17.3 Decimal4

Definición contractual reutilizable de tipo Decimal4.

Tipo: string.

Patrón: `^\d{1,10}\.\d{4}$`.

17.4 Dinero

Definición contractual reutilizable de tipo Dinero.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`importe`	Decimal2; requerido	Ver definición Decimal2
`moneda`	string; requerido	enum=[‘MXN’]

17.5 Peso

Definición contractual reutilizable de tipo Peso.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`kg`	Decimal3; requerido	Ver definición Decimal3

17.6 Error

Definición contractual reutilizable de tipo Error.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`codigo`	string; requerido	
`mensaje`	string; requerido	
`regla_id`	string; opcional	
`correlation_id`	string (uuid); requerido	
`campos`	colección de object; opcional	

17.7 Pagina

Definición contractual reutilizable de tipo Pagina.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`pagina`	integer; requerido	
`tamano`	integer; requerido	
`total`	integer; requerido	

17.8 SesionActual

Definición contractual reutilizable de tipo SesionActual.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`usuario_id`	string (uuid); requerido	
`nombre`	string; requerido	
`sedes`	colección de object; requerido	

17.9 ContraparteNueva

Definición contractual reutilizable de tipo ContraparteNueva.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`tipo_persona`	string; requerido	enum=[‘FISICA’, ‘MORAL’]
`nombre`	string; requerido	maxLength=200
`rfc`	string; opcional	maxLength=13
`telefono`	string; opcional	maxLength=20

17.10 Contraparte

Definición contractual reutilizable de tipo Contraparte.

Tipo: objeto.

allOf: ContraparteNueva, object.

17.11 Animal

Definición contractual reutilizable de tipo Animal.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`id`	string (uuid); opcional	
`sede_operadora_id`	string (uuid); opcional	
`situacion`	string; opcional	enum=[‘VIGENTE’, ‘IDENTIDAD_EN_REVISION’, ‘MUERTO’]
`estado_convergencia`	string; opcional	enum=[‘VISIBLE’, ‘PENDIENTE_CONVERGENCIA’]
`identificaciones`	colección de object; opcional	

17.12 FichaAnimal

Definición contractual reutilizable de tipo FichaAnimal.

Tipo: objeto.

allOf: Animal, object.

17.13 EstadoOperacion

Definición contractual reutilizable de tipo EstadoOperacion.

Tipo: string.

Valores permitidos: `BORRADOR`, `ACORDADA`, `CONFIRMADA`, `CANCELADA`, `ANULADA`.

17.14 EstadoPosicion

Definición contractual reutilizable de tipo EstadoPosicion.

Tipo: string.

Valores permitidos: `PENDIENTE_RECEPCION`, `DISPONIBLE`, `VENDIDA_PENDIENTE_ENTREGA`, `CERRADA_VENTA`, `BAJA`.

17.15 OperacionResumen

Definición contractual reutilizable de tipo OperacionResumen.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`id`	string (uuid); opcional	
`tipo`	string; opcional	enum=[‘COMPRA’, ‘VENTA’]
`folio`	string; opcional	
`contraparte`	string; opcional	
`estado`	EstadoOperacion; opcional	Ver definición EstadoOperacion
`estado_entrega`	string; opcional	enum=[‘PENDIENTE’, ‘PARCIAL’, ‘COMPLETA’]
`pendientes`	colección de string; opcional	

17.16 Operacion

Definición contractual reutilizable de tipo Operacion.

Tipo: objeto.

allOf: OperacionResumen, object.

17.17 Version

Definición contractual reutilizable de tipo Version.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`numero_version`	integer; opcional	
`estado`	string; opcional	enum=[‘PROPUESTA’, ‘ACEPTADA’, ‘SUPERADA’]
`condiciones`	colección de object; opcional	

17.18 EdicionBorrador

Definición contractual reutilizable de tipo EdicionBorrador.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`lotes`	colección de objeto; requerido	minItems=1
`acordar`	boolean; opcional	
`requiere_recepcion`	boolean; opcional	

17.19 PesajeNuevo

Definición contractual reutilizable de tipo PesajeNuevo.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`numero_lote`	integer; requerido	
`modo`	string; requerido	enum=[‘BRUTO_TARA’, ‘NETO_REPORTADO’]
`bruto`	Peso; opcional	Ver definición Peso
`tara`	Peso; opcional	Ver definición Peso
`neto`	Peso; requerido	Ver definición Peso
`ticket_folio`	string; opcional	
`pesado_en`	string (date-time); requerido	
`documento_id`	string (uuid); opcional	
`animales`	colección de object; requerido	

17.20 PesajeCorreccion

Definición contractual reutilizable de tipo PesajeCorreccion.

Tipo: objeto.

allOf: PesajeNuevo, object.

17.21 Pesaje

Definición contractual reutilizable de tipo Pesaje.

Tipo: objeto.

allOf: PesajeNuevo, object.

17.22 Reserva

Definición contractual reutilizable de tipo Reserva.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`id`	string (uuid); opcional	
`posicion_id`	string (uuid); opcional	
`estado`	string; opcional	enum=[‘ACTIVA’, ‘CONSUMIDA’, ‘LIBERADA’, ‘EXPIRADA’]
`vence_en`	string (date-time); opcional	

17.23 Posicion

Definición contractual reutilizable de tipo Posicion.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`id`	string (uuid); opcional	
`animal_id`	string (uuid); opcional	
`numero_ciclo`	integer; opcional	

Campo	Tipo y presencia	Restricción o significado
`estado`	EstadoPosicion; opcional	Ver definición EstadoPosicion
`reservada`	boolean; opcional	Derivado de una reserva ACTIVA
`costo_vigente`	Dinero; opcional	Ver definición Dinero

17.24 Liquidacion

Definición contractual reutilizable de tipo Liquidacion.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`id`	string (uuid); opcional	
`operacion_id`	string (uuid); opcional	
`direccion`	string; opcional	enum=[‘POR_PAGAR’, ‘POR_COBRAR’]
`total`	Dinero; opcional	Ver definición Dinero
`ajustes_cargo`	Dinero; opcional	Ver definición Dinero
`ajustes_abono`	Dinero; opcional	Ver definición Dinero
`aplicaciones_netas`	Dinero; opcional	Ver definición Dinero
`saldo`	Dinero; opcional	Ver definición Dinero
`vence_el`	string (date); opcional	
`conceptos`	colección de object; opcional	

17.25 GastoNuevo

Definición contractual reutilizable de tipo GastoNuevo.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`operacion_id`	string (uuid); requerido	
`naturaleza`	string; requerido	enum=[‘COSTO’, ‘BONIFICACION’]
`clasificacion`	string; requerido	enum=[‘CAPITALIZABLE’, ‘DIRECTO_VENTA’, ‘OTRO’]
`importe`	Dinero; requerido	Ver definición Dinero
`concepto`	string; requerido	maxLength=160
`proveedor_parte_id`	string / null (uuid); opcional	
`referencia_origen`	string / null; opcional	maxLength=60
`bonifica_gasto_id`	string / null (uuid); opcional	
`fecha_economica`	string (date); requerido	
`periodo_id`	string (uuid); requerido	
`fecha_reconocimiento`	string (date); requerido	
`snapshot_id`	string (uuid); opcional	

17.26 Asignacion

Definición contractual reutilizable de tipo Asignacion.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`posicion_id`	string (uuid); opcional	
`base_valor`	string; opcional	
`importe`	Dinero; opcional	Ver definición Dinero
`destino`	string; opcional	enum=[‘INVENTARIO’, ‘COSTO_VENTA’, ‘PERDIDA’]

17.27 PagoNuevo

Definición contractual reutilizable de tipo PagoNuevo.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`parte_id`	string (uuid); requerido	
`tipo`	string; requerido	enum=[‘COBRO’, ‘PAGO’]
`importe`	Dinero; requerido	Ver definición Dinero
`metodo`	string; requerido	enum=[‘TRANSFERENCIA’, ‘EFECTIVO’, ‘CHEQUE’, ‘OTRO’]
`referencia`	string / null; opcional	maxLength=60
`recibido_el`	string (date); requerido	

17.28 Pago

Definición contractual reutilizable de tipo Pago.

Tipo: objeto.

allOf: PagoNuevo, object.

17.29 Documento

Definición contractual reutilizable de tipo Documento.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`id`	string (uuid); opcional	
`tipo_documental`	string; opcional	
`sha256`	string; opcional	pattern=[1]{64}$
`tamano_bytes`	integer; opcional	
`mime`	string; opcional	
`version`	integer; opcional	

17.30 ReporteCartera

Definición contractual reutilizable de tipo ReporteCartera.

Tipo: object.

[1] 0-9a-f

Campo	Tipo y presencia	Restricción o significado
`corte_en`	string (date-time); requerido	
`etiqueta`	string; requerido	enum=[‘ACTUALIZADO’]
`fuente_consulta`	string; requerido	enum=[‘PROPIETARIO’, ‘REPLICA’]
`antiguedad_replica_segundos`	integer / null; requerido	0 para lectura propietaria; null si la medida de réplica es desconocida.
`pagina`	integer; requerido	minimum=1
`tamano`	integer; requerido	minimum=1; maximum=100
`total`	integer; requerido	minimum=0
`filas`	colección de object; requerido	

17.31 ReporteMargen

Definición contractual reutilizable de tipo ReporteMargen.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`corte_en`	string (date-time); requerido	
`etiqueta`	string; requerido	enum=[‘PROVISIONAL’, ‘ACTUALIZADO’, ‘CERRADO’]
`fuente_consulta`	string; requerido	enum=[‘PROPIETARIO’, ‘REPLICA’]
`antiguedad_replica_segundos`	integer / null; requerido	0 para lectura propietaria; null si la medida de réplica es desconocida.
`venta_neta`	Dinero; requerido	Ver definición Dinero
`costo_vendido`	Dinero; requerido	Ver definición Dinero
`gastos_directos`	Dinero; requerido	Ver definición Dinero
`margen`	Dinero; requerido	Ver definición Dinero
`perdidas`	Dinero; requerido	Ver definición Dinero
`faltantes`	colección de string; requerido	

17.32 TarifaExterna

Definición contractual reutilizable de tipo TarifaExterna.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`fuente`	string; opcional	
`ruta`	string; opcional	
`precio`	Decimal4; opcional	Ver definición Decimal4
`unidad`	string; opcional	
`vigente_desde`	string (date); opcional	
`version`	integer; opcional	

17.33 Csrf

Definición contractual reutilizable de tipo Csrf.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`token`	string; requerido	
`headerName`	string; requerido	

17.34 Reconocimiento

Definición contractual reutilizable de tipo Reconocimiento.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`periodo_id`	string (uuid); requerido	
`fecha_reconocimiento`	string (date); requerido	

17.35 LoteCompraInput

Definición contractual reutilizable de tipo LoteCompraInput.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`numero_lote`	integer; requerido	minimum=1
`descripcion`	string; opcional	maxLength=120
`animales`	colección de string (uuid); requerido	minItems=1
`unidad_precio`	string; requerido	enum=[‘KG’, ‘CABEZA’]
`precio_unitario`	Decimal4; requerido	Ver definición Decimal4
`pesaje_base_id`	string (uuid); opcional	
`deduccion_pct`	string; opcional	pattern=^1,2.1,6$
`cargos_pactados`	Decimal2; opcional	Ver definición Decimal2
`bonificaciones_pactadas`	Decimal2; opcional	Ver definición Decimal2

17.36 LoteVentaInput

Definición contractual reutilizable de tipo LoteVentaInput.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`numero_lote`	integer; requerido	minimum=1
`descripcion`	string; opcional	maxLength=120
`unidad_precio`	string; requerido	enum=[‘KG’, ‘CABEZA’]
`precio_unitario`	Decimal4; requerido	Ver definición Decimal4
`pesaje_base_id`	string (uuid); opcional	
`deduccion_pct`	string; opcional	pattern=^1,2.1,6$
`cargos_pactados`	Decimal2; opcional	Ver definición Decimal2
`bonificaciones_pactadas`	Decimal2; opcional	Ver definición Decimal2
`posiciones`	colección de string (uuid); requerido	minItems=1

17.37 DistribucionInput

Definición contractual reutilizable de tipo DistribucionInput.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`metodo`	string; requerido	enum=[‘POR_CABEZA’, ‘POR_PESO_MEDIDO’, ‘IMPORTE_INDIVIDUAL’]
`periodo_id`	string (uuid); requerido	
`fecha_reconocimiento`	string (date); requerido	
`asignaciones`	colección de object; opcional	

17.38 Gasto

Definición contractual reutilizable de tipo Gasto.

Tipo: objeto.

allOf: GastoNuevo, object.

17.39 ResultadoComando

Definición contractual reutilizable de tipo ResultadoComando.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`id`	string (uuid); opcional	
`version_lock`	integer; opcional	minimum=0

17.40 OperacionNueva

Definición contractual reutilizable de tipo OperacionNueva.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`tipo`	string; requerido	enum=[‘COMPRA’, ‘VENTA’]
`parte_id`	string (uuid); requerido	
`fecha_operacion`	string (date); requerido	
`requiere_recepcion`	boolean; opcional	En compra: conserva la posición pendiente de recepción física cuando es true. default=False
`lotes`	colección de objeto; opcional	minItems=1

17.41 Usuario

Definición contractual reutilizable de tipo Usuario.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`id`	string (uuid); requerido	
`nombre`	string; requerido	
`login`	string; requerido	
`estado`	string; requerido	enum=[‘ACTIVO’, ‘BLOQUEADO’]
`roles`	colección de string; opcional	

17.42 UsuarioNuevo

Definición contractual reutilizable de tipo UsuarioNuevo.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`login`	string; requerido	minLength=1; maxLength=100
`nombre`	string; requerido	minLength=1; maxLength=200
`contrasena`	string; requerido	Máximo 72 bytes UTF-8 para bcrypt; no se devuelve. minLength=1; maxLength=72

17.43 Periodo

Definición contractual reutilizable de tipo Periodo.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`id`	string (uuid); requerido	
`fecha_inicio`	string (date); opcional	
`fecha_fin`	string (date); opcional	
`estado`	string; requerido	enum=[‘ABIERTO’, ‘CERRADO’]
`desde`	string (date); requerido	
`hasta`	string (date); requerido	

17.44 Predio

Definición contractual reutilizable de tipo Predio.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`id`	string (uuid); requerido	
`nombre`	string; requerido	
`municipio`	string; opcional	
`entidad_federativa`	string; opcional	
`parte_responsable_id`	string (uuid); opcional	

17.45 PredioNuevo

Definición contractual reutilizable de tipo PredioNuevo.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`nombre`	string; requerido	
`municipio`	string; opcional	
`entidad_federativa`	string; opcional	
`parte_responsable_id`	string (uuid); opcional	
`clave_upp_declarada`	string; opcional	

17.46 CierreVersion

Definición contractual reutilizable de tipo CierreVersion.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`numero_version`	integer; requerido	
`periodo_id`	string (uuid); requerido	
`totales`	object; opcional	
`corte_en`	string (date-time); opcional	
`tipo`	string; opcional	enum=[‘CIERRE’, ‘REAPERTURA’]

17.47 CalculoLote

Definición contractual reutilizable de tipo CalculoLote.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`numero_lote`	integer; opcional	
`cantidad`	string; opcional	
`unidad`	string; opcional	enum=[‘KG’, ‘CABEZA’]
`precio`	Decimal4; opcional	Ver definición Decimal4
`base`	Dinero; opcional	Ver definición Dinero
`total`	Dinero; opcional	Ver definición Dinero

17.48 TarifaCapturada

Definición contractual reutilizable de tipo TarifaCapturada.

Tipo: objeto.

allOf: TarifaExterna, object.

17.49 DiagnosticoProceso

Definición contractual reutilizable de tipo DiagnosticoProceso.

Tipo: object.

Campo	Tipo y presencia	Restricción o significado
`proceso`	object; requerido	
`comandos`	object; requerido	
`consultas`	object; requerido	
`pools`	object; requerido	
`sincronizacion`	object; requerido	Estado SQL observado; antigüedad desconocida se mantiene null. Si falla dependencia: estadoNO_DISPONIBLE y código público.
`capturado_en`	string (date-time); requerido	

Capítulo 18

Definiciones canónicas completas

Los siguientes anexos preservan reglas, campos, casos de uso y decisiones de arquitectura. Complementan la explicación, permiten buscar RN/ENT/UC/ADR y mantienen los límites del alcance. Los ADR anteriores deben interpretarse con sus notas de sustitución por decisiones p osteriores. La autoridad del esquema físico es la última definición aplicada por las migraciones; no se ejecuta SQL desde estos párrafos.

18.1 Reglas de negocio RN-001..RN-045

Fuente: `docs/domain/business-rules.md`.

Fuente canónica de las reglas. Origen: **MP** = megaprompt §5 (propuesta del producto); **DEC-xx** = decisión registrada en `docs/research/evidence-register.md`. Línea base de diseño: 202 6-10-10. Al 2026-10-11 existen implementación SQL/Java y pruebas locales por incremento; el resultado y alcance de cada ejecución se registran en evidencia (referencia del repositorio: `../testing/evidence.md`). La presencia de una RN o un TEST en esta tabla no acredita por sí sola su cumplimiento completo.

Convención de errores

Errores de dominio levantados por PL/pgSQL con `RAISEEXCEPTION...USINGERRCODE='P0001',DETAIL=json` (objeto validado con `codigo`, `regla_id` y campos públicos) y mapeados por la API: `BV0xx` → 422 (regla de dominio), `BV4xx` → 409 (conflicto/concurrencia/idempotencia), `BV5xx` → 503 (dueño no disponible). Errores nativos relevantes: `23505` unique_violation → 409, `23503` foreig n_key_violation → 422, `23514` check_violation → 422, `42501` insufficient_privilege → 403 (sólo pruebas de rol), `40001/40P01` → reintento acotado.

Actores: **ADM** administrador · **OPE** operador · **CAJ** caja · **AUD** auditor · **SIS** sistema (comando/ trigger).

- **ID:** RN-001. **Regla:** Aislamiento por empresa: un usuario sólo accede a empresas y sedes autorizadas; referencias cruzadas entre empresas se rechazan también en BD. **Razón:** Evitar fuga y mezcla de datos. **Actor:** Todos. **Precondición:** Sesión válida y membresía . **Transición / efecto:** Ninguna: guarda. **Datos:** membresia_usuario, todas las FK compu estas. **Error:** 403 / `23503`. **Implementación prevista:** Spring Security + FK compuestas (E,...). **Prueba:** TEST-14, TEST-15. **Origen:** MP.
- **ID:** RN-002. **Regla:** Cada animal conserva identidad estable; identificaciones oficiales y UUID interno son campos distintos. **Razón:** Recompra e historial. **Actor:** OPE. **Precondición:** -. **Transición / efecto:** Alta de `animal` independiente de sus códigos. **Datos:** animal, animal_identificacion. **Error:** -. **Implementación prevista:** Tablas separadas. **Prueba:** TEST-05. **Origen:** MP.

- **ID:** RN-003. **Regla:** Identidad maestra e identificadores se registran en A; unicidad por empresa verificada allí. **Razón:** Evitar dos UUID para el mismo arete. **Actor:** OPE en A. **Precondición:** Nodo = A. **Transición / efecto:** RegistrarIdentidadAnimal inserta en A. **Datos:** animal_identificacion UK(E,sistema,codigo_normalizado). **Error:** `23505`→409 `BV401`. **Implementación prevista:** Comando en A + UNIQUE. **Prueba:** TEST-13. **Origen:** MP.
- **ID:** RN-004. **Regla:** Sede operadora del animal inmutable en el MVP; propietario, custodio, ubicación y nodo son conceptos distintos. **Razón:** Escritor único por agregado. **Actor:** SIS. **Precondición:** -. **Transición / efecto:** UPDATE de `sede_operadora_id` rechazado. **Datos:** animal. **Error:** `BV010`. **Implementación prevista:** Trigger de columna inmutable. **Prueba:** TEST-17. **Origen:** MP.
- **ID:** RN-005. **Regla:** Animal y posición son distintos; recompra permitida tras cerrar el ciclo previo; no doble venta del ciclo vigente. **Razón:** Ciclos de costo independientes. **Actor:** OPE. **Precondición:** Ciclo previo CERRADA_VENTA. **Transición / efecto:** Nueva posición num ero_ciclo+1. **Datos:** posicion_inventario. **Error:** `BV011`. **Implementación prevista:** Confi rmarCompra. **Prueba:** TEST-05. **Origen:** MP.
- **ID:** RN-006. **Regla:** Sólo una posición activa por animal en la empresa; toda posición deriva de una adquisición registrada. **Razón:** Inventario sin saldos manuales. **Actor:** SIS. **Precondición:** -. **Transición / efecto:** -. **Datos:** posicion_inventario UNIQUE parcial (E ,animal_id) WHERE estado activo. **Error:** `23505`→409. **Implementación prevista:** Índice único parcial; sin INSERT directo para app. **Prueba:** TEST-05. **Origen:** MP.
- **ID:** RN-007. **Regla:** Un lote agrupa animales/posiciones con historia; una compra alimenta varias ventas y una venta usa posiciones de varias compras. **Razón:** Realidad comercial. **Actor:** OPE. **Precondición:** -. **Transición / efecto:** -. **Datos:** lote_operacion, seleccion_*. **Error:** -. **Implementación prevista:** Modelo N:M. **Prueba:** TEST-02. **Origen:** MP.
- **ID:** RN-008. **Regla:** La selección de un animal/posición es única entre lotes de una misma operación; composición confirmada histórica. **Razón:** Evitar duplicar un animal en un trato. **Actor:** OPE. **Precondición:** Operación BORRADOR para editar. **Transición / efecto:** -. **Datos:** seleccion_compra UK(E,S,op,animal), seleccion_venta UK(E,S,op,posicion). **Error:** `23505`→409. **Implementación prevista:** UNIQUE + trigger inmutable tras confirmar. **Prueba:** TEST-17. **Origen:** MP.
- **ID:** RN-009. **Regla:** Toda operación tiene empresa, sede, tipo, contraparte, moneda, versión y estado; no admite animales de otra sede escritora. **Razón:** Propiedad de escritura. **Actor:** OPE. **Precondición:** -. **Transición / efecto:** -. **Datos:** operacion NOT NULL; FK (E,S,animal) contra UK(E,sede_operadora_id,id). **Error:** `23503`. **Implementación prevista:** FK compuesta. **Prueba:** TEST-14. **Origen:** MP.
- **ID:** RN-010. **Regla:** Compra y venta por cuenta propia son los únicos modos; un tercero puede ser comprador y vendedor. **Razón:** Alcance MVP. **Actor:** OPE. **Precondición:** -. **Transición / efecto:** -. **Datos:** operacion.tipo CHECK IN (‘COMPRA’,‘VENTA’); parte sin rol fijo. **Error:** `23514`. **Implementación prevista:** CHECK + sin ISA disjunta. **Prueba:** TEST-30. **Origen:** MP.
- **ID:** RN-011. **Regla:** Condiciones de precio, pesaje, ajustes y redondeo se versionan; una versión nueva no recalcula confirmadas. **Razón:** Snapshot histórico. **Actor:** OPE/ADM. **Precondición:** -. **Transición / efecto:** Nueva contrato_version numero+1. **Datos:** contrato_ version, condicion_lote. **Error:** `BV012` si se edita aceptada. **Implementación prevista:** Trigger inmutable en versión aceptada. **Prueba:** TEST-17. **Origen:** MP.
- **ID:** RN-012. **Regla:** Pesos en kg con precisión declarada; tara ≤ bruto y neto = bruto − tara. **Razón:** Integridad de medición. **Actor:** OPE. **Precondición:** -. **Transición / efecto:** -. **Datos:** pesaje CHECK. **Error:** `23514`. **Implementación prevista:** CHECK de fila + no NaN/∞. **Prueba:** TEST-18. **Origen:** MP.
- **ID:** RN-013. **Regla:** Un pesaje de lote conserva la lista exacta de animales; no se inventan pesos individuales. **Razón:** Evidencia real. **Actor:** OPE. **Precondición:** -. **Transición / efecto:** AprobarPesaje congela composición. **Datos:** pesaje_animal.peso_individual_kg

NULL si no medido. **Error:** BV013. **Implementación prevista:** Comando + trigger inmutable. **Prueba:** TEST-18. **Origen:** MP.

- **ID:** RN-014. **Regla:** Sólo se comparan pesajes del mismo conjunto; si difiere la composición → "no comparable". **Razón:** Evitar merma falsa. **Actor:** SIS. **Precondición:** -. **Transición / efecto:** Comparación devuelve estado NO_COMPARABLE. **Datos:** pesaje_animal. **Error:** -. **Implementación prevista:** Función de comparación por igualdad de conjuntos. **Prueba:** TEST-19. **Origen:** MP.
- **ID:** RN-015. **Regla:** Variación de peso y deducción contractual son independientes; no descontar dos veces. **Razón:** Precio justo. **Actor:** SIS. **Precondición:** -. **Transición / efecto:** Liquidación usa sólo base y deducciones pactadas. **Datos:** condicion_lote. **Error:** -. **Implementación prevista:** Función canónica de liquidación. **Prueba:** TEST-01. **Origen:** MP.
- **ID:** RN-016. **Regla:** OCR/CSV generan propuestas; un responsable confirma antes de usarlas. **Razón:** Control humano. **Actor:** OPE. **Precondición:** -. **Transición / efecto:** Estado PROPUESTA → CONFIRMADA. **Datos:** pesaje.estado, infra.importacion_fuente. **Error:** BV014. **Implementación prevista:** Estado separado. **Prueba:** TEST-18. **Origen:** MP.
- **ID:** RN-017. **Regla:** Un borrador puede estar incompleto; confirmar exige partidas, contraparte, condiciones válidas, pesos cuando procedan y autorización. **Razón:** Calidad del cierre. **Actor:** OPE. **Precondición:** BORRADOR/ACORDADA. **Transición / efecto:** → CONFIRMADA. **Datos:** operacion, condicion_lote, pesaje. **Error:** BV015. **Implementación prevista:** Validaciones en ConfirmarCompra/Venta. **Prueba:** TEST-01. **Origen:** MP.
- **ID:** RN-018. **Regla:** Reserva atómica en el nodo propietario, bloqueando posiciones en orden estable; una reserva activa por posición. **Razón:** Evitar doble venta. **Actor:** OPE. **Precondición:** Posición DISPONIBLE, operación VENTA no confirmada. **Transición / efecto:** Crea reserva ACTIVA. **Datos:** reserva_venta UNIQUE parcial (E,S,posicion_id) WHERE estado='ACTIVA'. **Error:** 23505/BV402→409. **Implementación prevista:** SELECT…FORUPDATEORDERBYid + índice parcial. **Prueba:** TEST-10. **Origen:** MP, DEC-11.
- **ID:** RN-019. **Regla:** Reservas con vencimiento explícito; confirmar tras vencer revalida disponibilidad. **Razón:** No retener inventario indefinidamente. **Actor:** SIS. **Precondición:** -. **Transición / efecto:** ExpirarReserva: ACTIVA→EXPIRADA. **Datos:** reserva_venta.vence_en. **Error:** BV403. **Implementación prevista:** Comando (no índice con now()). **Prueba:** TEST-20. **Origen:** MP.
- **ID:** RN-020. **Regla:** Venta, entrega física y cobro son ejes separados. **Razón:** Un solo evento económico. **Actor:** SIS. **Precondición:** -. **Transición / efecto:** -. **Datos:** operacion.estado, movimiento_inventario, aplicacion_pago. **Error:** -. **Implementación prevista:** Estados independientes. **Prueba:** TEST-32. **Origen:** MP.
- **ID:** RN-021. **Regla:** ConfirmarCompra crea posiciones, valoración inicial, liquidación principal confirmada y obligación en una transacción local. **Razón:** Atomicidad. **Actor:** OPE. **Precondición:** RN-017. **Transición / efecto:** Operación → CONFIRMADA; posiciones PENDIENTE_RECEPCION/DISPONIBLE. **Datos:** posicion_inventario, liquidacion, concepto_liquidacion, asignacion_costo, asiento_valoracion, auditoria_evento, evento_outbox. **Error:** BV015, BV404. **Implementación prevista:** Función cmd_confirmar_compra. **Prueba:** TEST-01, TEST-02. **Origen:** MP, DEC-02.
- **ID:** RN-022. **Regla:** ConfirmarVenta compromete posiciones, consume reservas, registra costo de venta, liquidación y obligación; la entrega posterior no duplica. **Razón:** Atomicidad y reconocimiento único. **Actor:** OPE. **Precondición:** Reservas ACTIVAS vigentes. **Transición / efecto:** Posiciones → VENDIDA_PENDIENTE_ENTREGA. **Datos:** idem + reserva_venta. **Error:** BV403, BV402. **Implementación prevista:** Función cmd_confirmar_venta. **Prueba:** TEST-10, TEST-32. **Origen:** MP.
- **ID:** RN-023. **Regla:** Liquidación confirmada inmutable; ajustes/devoluciones/reversos conservan referencia, motivo, actor y fecha. **Razón:** Auditoría. **Actor:** SIS. **Precondición:** -. **Transición / efecto:** UPDATE/DELETE rechazado. **Datos:** liquidacion, concepto_liquida

cion. **Error:** `BV016`. **Implementación prevista:** Trigger + privilegios. **Prueba:** TEST-17. **Origen:** MP.

- **ID:** RN-024. **Regla:** Tarifa nueva, tercero renombrado o documento posterior no alteran el snapshot de una liquidación. **Razón:** Historia. **Actor:** SIS. **Precondición:** -. **Transición / efecto:** -. **Datos:** liquidacion.snapshot_*. **Error:** -. **Implementación prevista:** Columnas snapshot sin FK en cascada de UPDATE. **Prueba:** TEST-17. **Origen:** MP.
- **ID:** RN-025. **Regla:** Cada gasto: costo de adquisición asignable, gasto directo de venta u otro; no se suma dos veces. **Razón:** Margen correcto. **Actor:** OPE. **Precondición:** -. **Transición / efecto:** -. **Datos:** gasto_operacion.clasificacion. **Error:** `BV017`. **Implementación prevista:** CHECK + validación de duplicado por referencia de origen. **Prueba:** TEST-01. **Origen:** MP, DEC-08.
- **ID:** RN-026. **Regla:** Asignaciones de un gasto suman exactamente el total; método y residuo deterministas. **Razón:** Cuadre al centavo. **Actor:** SIS. **Precondición:** Gasto aprobado. **Transición / efecto:** DistribuirCosto. **Datos:** asignacion_costo. **Error:** `BV018`. **Implementación prevista:** Función de reparto con residuo por orden de id. **Prueba:** TEST-03. **Origen:** MP.
- **ID:** RN-027. **Regla:** Costo tardío modifica el margen mediante ajuste trazable, sin editar liquidación ni periodo cerrado. **Razón:** Historia. **Actor:** OPE. **Precondición:** Periodo destino ABIERTO. **Transición / efecto:** RegistrarCostoTardio crea asignaciones y asientos. **Datos:** asignacion_costo, asiento_valoracion. **Error:** `BV019`. **Implementación prevista:** Comando. **Prueba:** TEST-06. **Origen:** MP.
- **ID:** RN-028. **Regla:** El pago existe separado de su aplicación; N:M pago-obligación. **Razón:** Anticipos y pagos parciales. **Actor:** CAJ. **Precondición:** -. **Transición / efecto:** -. **Datos:** pago, aplicacion_pago. **Error:** -. **Implementación prevista:** Tabla asociativa. **Prueba:** TEST-04. **Origen:** MP.
- **ID:** RN-029. **Regla:** Aplicaciones no exceden disponible del pago ni saldo de la obligació n. **Razón:** Sin sobreaplicación. **Actor:** CAJ. **Precondición:** Locks de pago y liquidación . **Transición / efecto:** AplicarPago. **Datos:** aplicacion_pago. **Error:** `BV020`. **Implementación prevista:** Comando con FOR UPDATE en orden estable. **Prueba:** TEST-04, TEST-11. **Origen:** MP.
- **ID:** RN-030. **Regla:** Pago y aplicación: misma empresa, sede, contraparte, moneda y dirección compatible. **Razón:** Coherencia. **Actor:** CAJ. **Precondición:** -. **Transición / efecto:** -. **Datos:** pago.tipo vs operacion.tipo. **Error:** `BV021`. **Implementación prevista:** FK compuestas + validación. **Prueba:** TEST-11. **Origen:** MP, DEC-06.
- **ID:** RN-031. **Regla:** Importe no aplicado queda como anticipo/saldo a favor; no es venta adicional. **Razón:** Separar caja y margen. **Actor:** SIS. **Precondición:** -. **Transición / efecto:** -. **Datos:** saldo derivado del pago. **Error:** -. **Implementación prevista:** Vista de saldos. **Prueba:** TEST-04. **Origen:** MP.
- **ID:** RN-032. **Regla:** Misma clave + mismo payload devuelve el resultado previo; misma clave + payload distinto = conflicto. **Razón:** Idempotencia. **Actor:** SIS. **Precondición:** -. **Transición / efecto:** Reclamo de clave. **Datos:** clave_idempotencia. **Error:** `BV405`→409. **Implementación prevista:** Tabla + hash SHA-256 del payload canónico. **Prueba:** TEST-12. **Origen:** MP.
- **ID:** RN-033. **Regla:** Reversar conserva original y crea contramovimiento; se verifica efecto en aplicaciones relacionadas. **Razón:** Historia. **Actor:** CAJ/ADM. **Precondición:** Original no revertido. **Transición / efecto:** RevertirAplicacion. **Datos:** aplicacion_pago.reverso_de_id UNIQUE. **Error:** `BV022`. **Implementación prevista:** Comando. **Prueba:** TEST-07. **Origen:** MP, DEC-09.
- **ID:** RN-034. **Regla:** Importes con NUMERIC y BigDecimal; nunca float/double. **Razón:** Exactitud. **Actor:** -. **Precondición:** -. **Transición / efecto:** -. **Datos:** todos los importes. **Error:** -. **Implementación prevista:** Tipos; prueba de lint. **Prueba:** TEST-03. **Origen:** MP.

- **ID:** RN-035. **Regla:** Margen = ventas reconocidas − costo de posiciones vendidas − gastos directos; cobros no afectan margen. **Razón:** Separar devengo y caja. **Actor:** SIS. **Precondición:** -. **Transición / efecto:** -. **Datos:** vistas de reporte. **Error:** -. **Implementación prevista:** Vista `v_margen_operacion`. **Prueba:** TEST-01. **Origen:** MP.
- **ID:** RN-036. **Regla:** Si faltan costos o revisiones, margen PROVISIONAL; nunca “utilidad neta”. **Razón:** Honestidad del reporte. **Actor:** SIS. **Precondición:** -. **Transición / efecto:** Etiqueta. **Datos:** vista de margen. **Error:** -. **Implementación prevista:** Columna derivada. **Prueba:** TEST-01. **Origen:** MP.
- **ID:** RN-037. **Regla:** Evidencias con versión, hash, tamaño, tipo y control de acceso; hash detecta cambios, no prueba autoría. **Razón:** Integridad de archivos. **Actor:** OPE. **Precondición:** -. **Transición / efecto:** -. **Datos:** documento. **Error:** `BV023`. **Implementación prevista:** SHA-256 + almacenamiento privado. **Prueba:** TEST-34. **Origen:** MP.
- **ID:** RN-038. **Regla:** Cancelación, pérdida, muerte, devolución y venta son motivos distintos con evidencia y autorización; no borrar animales. **Razón:** Trazabilidad. **Actor:** ADM. **Precondición:** -. **Transición / efecto:** RegistrarBajaInventario → BAJA. **Datos:** posicion_inventario.motivo_baja, animal.situacion. **Error:** `BV024`. **Implementación prevista:** Comando; sin DELETE. **Prueba:** TEST-08, TEST-33. **Origen:** MP, DEC-03/04.
- **ID:** RN-039. **Regla:** Periodo cerrado impide ediciones retroactivas; reapertura con permiso y bitácora. **Razón:** Cierres confiables. **Actor:** ADM. **Precondición:** -. **Transición / efecto:** Cerrar/ReabrirPeriodo. **Datos:** periodo_cierre, cierre_periodo_version. **Error:** `BV025`. **Implementación prevista:** Lock de fila de periodo. **Prueba:** TEST-09. **Origen:** MP.
- **ID:** RN-040. **Regla:** Una réplica retrasada no autoriza operaciones críticas; decide el escritor propietario. **Razón:** Consistencia. **Actor:** SIS. **Precondición:** -. **Transición / efecto:** -. **Datos:** -. **Error:** `BV501`→503. **Implementación prevista:** OwnerDatabaseRouter. **Prueba:** TEST-22. **Origen:** MP.
- **ID:** RN-041. **Regla:** Si el dueño no está disponible la respuesta puede quedar pendiente de comprobar; no hay cambio automático de dueño. **Razón:** Evitar split-brain. **Actor:** SIS. **Precondición:** -. **Transición / efecto:** -. **Datos:** -. **Error:** `BV501`→503. **Implementación prevista:** Router sin failover. **Prueba:** TEST-22. **Origen:** MP.
- **ID:** RN-042. **Regla:** Dato del proveedor FDW no es histórico por sí mismo: se guarda snapshot aceptado localmente. **Razón:** Reproducibilidad. **Actor:** OPE. **Precondición:** -. **Transición / efecto:** AceptarCotizacion crea snapshot. **Datos:** gasto_operacion/documento snapshot. **Error:** -. **Implementación prevista:** Copia local con hash. **Prueba:** TEST-25. **Origen:** MP.
- **ID:** RN-043. **Regla:** Confirmar una operación y su auditoría es atómico; evento de integración en outbox en la misma transacción. **Razón:** Consistencia. **Actor:** SIS. **Precondición:** -. **Transición / efecto:** -. **Datos:** auditoria_evento, evento_outbox. **Error:** -. **Implementación prevista:** Insert en el comando. **Prueba:** TEST-24. **Origen:** MP.
- **ID:** RN-044. **Regla:** Entrega externa admite reintentos; outbox no garantiza exactamente una vez. **Razón:** Honestidad técnica. **Actor:** SIS. **Precondición:** -. **Transición / efecto:** -. **Datos:** infra.entrega_outbox, mensaje_inbox. **Error:** -. **Implementación prevista:** Inbox deduplica. **Prueba:** TEST-24. **Origen:** MP.
- **ID:** RN-045. **Regla:** Datos de demostración identificados como sintéticos; sin documentos oficiales aparentando autenticidad. **Razón:** Ética y legal. **Actor:** Todos. **Precondición:** -. **Transición / efecto:** -. **Datos:** seed con prefijo DEMO. **Error:** -. **Implementación prevista:** Seed marcado. **Prueba:** revisión manual. **Origen:** MP.

18.2 Glosario

Fuente: `docs/domain/glossary.md`.

Nombre canónico en código entre `backticks`. Datos de ejemplo **sintéticos**.

- **Término:** Empresa cliente (`empresa`). **Definición:** Organización que contrata BoviCierre; todo dato pertenece a una. **Ejemplo:** "Acopio Demo SA" (sintética). **No confundir con:** La contraparte con la que compra/vende.
- **Término:** Sede (`sucursal`). **Definición:** Centro operativo de la empresa; cada operación pertenece a una. **Ejemplo:** Sede A "Centro", Sede B "Norte". **No confundir con:** Predio/establecimiento; nodo de BD.
- **Término:** Nodo escritor. **Definición:** Instancia PostgreSQL que acepta escrituras de un agregado: A (maestros + sede A) o B (sede B). **Ejemplo:** Una venta de la sede B se escribe sólo en B. **No confundir con:** Sede (una sede *tiene* un nodo dueño); Pgpool.
- **Término:** Contraparte (`parte`). **Definición:** Persona u organización con la que se compra o vende; puede ser ambas cosas en momentos distintos. **Ejemplo:** "Productor Sintético 01". **No confundir con:** Empresa cliente; usuario.
- **Término:** Establecimiento (`predio`). **Definición:** Lugar físico declarado de origen, destino o custodia. **Ejemplo:** "Rancho Demo El Llano". **No confundir con:** Sede; UPP.
- **Término:** UPP. **Definición:** Unidad de Producción Pecuaria: clave de registro oficial de un establecimiento productivo; se guarda como texto declarado con evidencia. **Ejemplo:** `UPP-DEMO-0001` (ficticia). **No confundir con:** Validación de autenticidad (BoviCierre no la certifica).
- **Término:** PSG. **Definición:** Prestador de Servicios Ganaderos: registro oficial para quien compra/acopia/comercializa; se guarda como texto declarado. **Ejemplo:** `PSG-DEMO-01`. **No confundir con:** Permiso de movilización.
- **Término:** Animal (`animal`). **Definición:** Identidad permanente de un bovino dentro de la empresa; UUID técnico. **Ejemplo:** `animal7b1…`. **No confundir con:** Posición de inventario.
- **Término:** Identificación oficial (`animal_identificacion`). **Definición:** Código de arete/registro (SINIIGA/SINIDA [F04]) asociado a un animal, con sistema, vigencia y evidencia. **Ejemplo:** `SINIIGADEMO-000123`. **No confundir con:** UUID; QR interno (no sustituye arete).
- **Término:** Lote (`lote_operacion`). **Definición:** Grupo comercial de animales/posiciones dentro de una operación, numerado. **Ejemplo:** Lote 1 de la compra C-0001. **No confundir con:** Pesaje; corral físico.
- **Término:** Membresía de lote. **Definición:** Pertenencia de un animal (compra) o posición (venta) a un lote: `seleccion_compra` / `seleccion_venta`. **Ejemplo:** Animal X en lote 2. **No confundir con:** Composición de un pesaje.
- **Término:** Posición / ciclo de inventario (`posicion_inventario`). **Definición:** Tenencia de un animal entre una adquisición y su salida; tiene costo propio. **Ejemplo:** Ciclo 2 del animal X tras recompra. **No confundir con:** Animal.
- **Término:** Custodia. **Definición:** Quién tiene físicamente al animal; se registra en movimientos. **Ejemplo:** Transportista durante el viaje. **No confundir con:** Propiedad comercial; nodo dueño.
- **Término:** Operación (`operacion`). **Definición:** Compra o venta por cuenta propia con una contraparte; tipo inmutable. **Ejemplo:** Compra C-0001. **No confundir con:** Liquidación; pago.
- **Término:** Acuerdo / versión (`contrato_version`). **Definición:** Conjunto numerado de condiciones aceptadas para una operación. **Ejemplo:** v2 aceptada. **No confundir con:** Condición por lote.
- **Término:** Condición por lote (`condicion_lote`). **Definición:** Precio, unidad (kg/cabeza), pesaje base y deducciones pactadas para un lote en una versión. **Ejemplo:** 48.0000 MXN/kg, base pesaje de origen. **No confundir con:** Tarifa de catálogo.

- **Término:** Pesaje (`pesaje`). **Definición:** Medición de báscula de un conjunto exacto de animales. **Ejemplo:** Ticket 4471, 5,000.000 kg. **No confundir con:** Peso estimado.
- **Término:** Bruto / tara / neto. **Definición:** Neto = bruto − tara; si el ticket sólo trae neto: modo NETO_REPORTADO. **Ejemplo:** 12,000 − 7,000 = 5,000 kg. **No confundir con:** -.
- **Término:** Variación física. **Definición:** Diferencia observada entre dos pesajes compatibles del mismo conjunto. **Ejemplo:** 100 kg (2 %). **No confundir con:** Deducción contractual.
- **Término:** Ajuste contractual. **Definición:** Deducción o cargo pactado expresamente en la condición. **Ejemplo:** 1 % de deducción pactada. **No confundir con:** Merma “habitual” supuesta.
- **Término:** Liquidación comercial (`liquidacion`). **Definición:** Snapshot inmutable del cálculo de una operación confirmada; es la obligación con la contraparte (DEC-05). **Ejemplo:** 240,000.00 por pagar. **No confundir con:** CFDI; PDF.
- **Término:** Obligación. **Definición:** Saldo por cobrar (venta) o pagar (compra) de una liquidación + ajustes − aplicaciones. **Ejemplo:** 244,000 por cobrar. **No confundir con:** Pago.
- **Término:** Pago (`pago`). **Definición:** Movimiento de dinero registrado (COBRO o PAGO); existe aunque no esté aplicado. **Ejemplo:** Transferencia de 50,000. **No confundir con:** A plicación.
- **Término:** Aplicación (`aplicacion_pago`). **Definición:** Asignación de un importe de un pago a una liquidación. **Ejemplo:** 50,000 a la venta V-0001. **No confundir con:** Pago.
- **Término:** Reverso. **Definición:** Contramovimiento que anula un registro conservando el original (aplicación, ajuste, asiento). **Ejemplo:** Reverso de aplicación 400. **No confundir con:** Corrección (nueva versión) / reembolso (dinero real).
- **Término:** Corrección. **Definición:** Nuevo registro que sustituye a otro por error de captura, enlazado al original. **Ejemplo:** Pesaje corregido. **No confundir con:** Reverso.
- **Término:** Reembolso / reintegro. **Definición:** Devolución real de dinero que consume disponibilidad del pago origen. **Ejemplo:** Devolver 100 al comprador. **No confundir con:** Reverso de aplicación.
- **Término:** Costo de adquisición. **Definición:** Precio de compra + gastos capitalizables asignados a cada posición. **Ejemplo:** 1,100 por posición (GOLD-02). **No confundir con:** Costo de venta.
- **Término:** Costo de venta. **Definición:** Costo de las posiciones cuya venta se reconoció. **Ejemplo:** 4,400. **No confundir con:** Gasto directo de venta.
- **Término:** Gasto directo de venta. **Definición:** Gasto atribuible a una venta que no se capitaliza en inventario. **Ejemplo:** Flete de entrega 200. **No confundir con:** Costo de adquisi ción.
- **Término:** Margen provisional. **Definición:** Margen con costos o revisiones faltantes; siempre etiquetado. **Ejemplo:** “Provisional: falta flete”. **No confundir con:** Margen cerrado.
- **Término:** Margen cerrado. **Definición:** Margen incluido en un cierre de periodo publicado (versión inmutable). **Ejemplo:** Cierre oct v1. **No confundir con:** Margen acumulado actu alizado.

Sinónimos aceptados

sede = sucursal · contraparte = tercero = parte · establecimiento = predio · acuerdo = contrat o_version · ciclo = posición.

Denominaciones oficiales

La NOM-001-SAG/GAN-2015 usa **SINIDA**; en la operación del sistema aparece **SINIIGA** [F04-F05]. Se guarda el sistema declarado en `animal_identificacion.sistema`. Un establecimiento registrado, el propietario comercial y la sede que escribe en PostgreSQL son conceptos distintos.

Unidades y precisión

kg NUMERIC(14,3) · tarifas NUMERIC(14,4) · importes NUMERIC(18,2) · tasas NUMERIC(9,6) · moneda MXN · cantidades enteras positivas · redondeo HALF_UP a 2 decimales en importes.

18.3 Diccionario de datos

Fuente: `docs/database/data-dictionary.md`.

Convenciones comunes (no se repiten por tabla):

- `empresa_iduuidNOTNULL`, `sucursal_iduuidNOTNULL` en tablas locales; `iduuidNOTNULLDEFAULTuuidv7()`.
- `nodo_origenchar(1)NOTNULLCHECK(nodo_origenIN('A','B'))`, lo deriva el comando; trigger `trg_guarda_propietario` rechaza INSERT/UPDATE cuando `nodo_origen ≠ infra.nodo_config.nodo` (no corre en apply porque `session_replication_role=replica`).
- Importes `numeric(18,2)`, kg `numeric(14,3)`, tarifas `numeric(14,4)`, tasas `numeric(9,6)`; todos con `CHECK(xNOTIN('NaN','Infinity','-Infinity'))` como defensa explícita (comportamiento de NaN/∞ con typmod a verificar en TASK-A-009 con prueba negativa).
- Instantes `timestamptz`; fechas civiles `date`; presentación en America/Mexico_City.
- Hechos comprometidos: `ONUPDATERESTRICTONDELETERESTRICT`; la app no tiene privilegio DELETE en tablas de hechos.
- Sensibilidad: **P** personal (LFPDPPP [F10]), **C** comercial confidencial, **T** técnica, **S** secreta.
- Replicación: **A→B** maestros originados en A; **L↔** local del dueño (A→B y B→A según nodo_origen); **local** no se publica.

ENT-01 empresa

Propósito: cliente del SaaS; raíz del aislamiento (RN-001). Dueño: A · Replicación A→B · Sensibilidad C. - **Columna:** id. **Tipo:** uuid. **Nulo:** no. **Default:** uuidv7(). **Dominio / nota:** PK.

- **Columna:** razon_social. **Tipo:** varchar(200). **Nulo:** no. **Default:** . **Dominio / nota:** .
- **Columna:** rfc. **Tipo:** varchar(13). **Nulo:** sí. **Default:** . **Dominio / nota:** P/C.
- **Columna:** estado. **Tipo:** varchar(12). **Nulo:** no. **Default:** ‘ACTIVA’. **Dominio / nota:** ACTIVA, SUSPENDIDA.
- **Columna:** creado_en. **Tipo:** timestamptz. **Nulo:** no. **Default:** now(). **Dominio / nota:** .

Válido: `('AcopioDemoSA',NULL)`. Rechazado: estado ‘BORRADA’.

ENT-02 sucursal

Propósito: sede y su nodo dueño (RN-004, RN-040). PK(E,id); UK(E,codigo). Dueño A · A→B · C. - **Columna:** codigo. **Tipo:** varchar(10). **Nulo:** no. **Default:** . **Nota:** `^[A-Z0-9-]+$`.

- **Columna:** nombre. **Tipo:** varchar(120). **Nulo:** no. **Default:** . **Nota:** .
- **Columna:** nodo_dueno. **Tipo:** char(1). **Nulo:** no. **Default:** . **Nota:** A, B; inmutable (trigger).
- **Columna:** zona_horaria. **Tipo:** varchar(40). **Nulo:** no. **Default:** ‘America/Mexico_City’. **Nota:** .

Índices: PK, UK. Válido: (E,'CEN','A'). Rechazado: dos sedes con código 'CEN' en la misma empresa.

ENT-03 parte

Propósito: contraparte que compra y/o vende (RN-010). PK(E,id); UK parcial(E,rfc). A · A→B · **P**. - **Columna:** tipo_persona. **Tipo:** varchar(6). **Nulo:** no. **Default:** . **Nota:** FISICA, MORAL.

- **Columna:** nombre. **Tipo:** varchar(200). **Nulo:** no. **Default:** . **Nota:** .
- **Columna:** rfc. **Tipo:** varchar(13). **Nulo:** sí. **Default:** . **Nota:** normalizado mayúsculas.
- **Columna:** telefono. **Tipo:** varchar(20). **Nulo:** sí. **Default:** . **Nota:** P.
- **Columna:** es_sintetico. **Tipo:** boolean. **Nulo:** no. **Default:** false. **Nota:** RN-045.

Índices: `(E,lower(nombre))` búsqueda. Rechazado: mismo RFC dos veces en la empresa.

ENT-04 predio

Propósito: origen/destino/custodia declarados. PK(E,id); FK(E,parte_responsable_id). A · A→B · C/P. Columnas: nombre varchar(160) NOT NULL; clave_upp_declarada varchar(40) NULL (texto declarado, no certificado); municipio varchar(80); entidad_federativa varchar(40); parte_responsable_id uuid NULL. Rechazado: parte_responsable_id de otra empresa (FK compuesta falla, TEST-14).

ENT-05 usuario

Propósito: usuario del producto (no rol PostgreSQL). PK(E,id); UK(E,login_normalizado). A · A→B · **S** (hash). Columnas: login_normalizado varchar(80) NOT NULL (`lower(trim())`); nombre varchar(120); hash_contrasena varchar(255) NOT NULL (formato `{bcrypt}`/`{argon2}` de Spring Security DelegatingPasswordEncoder); estado varchar(10) DEFAULT 'ACTIVO' (ACTIVO, BLOQUEADO); creado_en. Nunca se loguea ni se indexa en el grafo el hash.

ENT-06 rol_aplicacion

PK(codigo varchar(12)); descripcion. Valores: ADMIN, OPERADOR, CAJA, AUDITOR. A→B · T.

ENT-07 membresia_usuario

PK(E,S,usuario_id,rol_codigo); vigente boolean DEFAULT true. Autoriza por sede (RN-001). A · A→B · T. Índice: `(E,usuario_id)WHEREvigente`. Rechazado: rol inexistente.

ENT-08 animal

Propósito: identidad permanente (RN-002, RN-004). PK(E,id); UK(E,sede_operadora_id,id). A · A→B · C. - **Columna:** sede_operadora_id. **Tipo:** uuid. **Nulo:** no. **Default:** . **Nota:** FK (E,·) → sucursal; inmutable (trigger).

- **Columna:** sexo. **Tipo:** char(1). **Nulo:** no. **Default:** . **Nota:** M, H.
- **Columna:** fecha_nacimiento_declarada. **Tipo:** date. **Nulo:** sí. **Default:** . **Nota:** declarada.
- **Columna:** raza_declarada. **Tipo:** varchar(60). **Nulo:** sí. **Default:** . **Nota:** .
- **Columna:** situacion. **Tipo:** varchar(24). **Nulo:** no. **Default:** 'VIGENTE'. **Nota:** VIGENTE, IDENTIDAD_EN_REVISION, MUERTO (DEC-04).
- **Columna:** creado_en. **Tipo:** timestamptz. **Nulo:** no. **Default:** now(). **Nota:** .

Rechazado: `UPDATEanimalSETsede_operadora_id=…` (BV010).

ENT-36 animal_identificacion

Propósito: códigos oficiales/internos con historia (RN-003). PK(E,id); UK(E,sistema,codigo_normalizado). A · A→B · C. - **Columna:** animal_id. **Tipo:** uuid. **Nulo:** no. **Nota:** FK (E,·) → animal.

- **Columna:** sistema. **Tipo:** varchar(10). **Nulo:** no. **Nota:** SINIIGA, SINIDA, INTERNO, OTRO.
- **Columna:** codigo_declarado. **Tipo:** varchar(40). **Nulo:** no. **Nota:** tal como se capturó.
- **Columna:** codigo_normalizado. **Tipo:** varchar(40). **Nulo:** no. **Nota:** `regexp_replace(upper(codigo),'[-]','','g');INTERNOadmiteASCIIalfanumérico,espacio/guion;rechazavacíoocaracteresfueradepolítica`.
- **Columna:** estado. **Tipo:** varchar(12). **Nulo:** no. **Nota:** VIGENTE, INACTIVA, EN_REVISION.
- **Columna:** vigente_desde / vigente_hasta. **Tipo:** date. **Nulo:** no / sí. **Nota:** CHECK hasta ≥ desde.
- **Columna:** evidencia_sucursal_id, evidencia_documento_id. **Tipo:** uuid. **Nulo:** sí. **Nota:** FK documento (ambas o ninguna).

Índice parcial: UNIQUE(E, animal_id, sistema) WHERE estado='VIGENTE'. Rechazado: mismo código normalizado en otro animal (TEST-13).

ENT-09 operacion

Propósito: compra o venta por cuenta propia (RN-009, RN-010). PK(E,S,id); UK(E,S,tipo,folio); UK(E,S,id,tipo). Dueño: nodo de la sede · L↔ · C. - **Columna:** tipo. **Tipo:** varchar(6). **Nulo:** no. **Default:** . **Nota:** COMPRA, VENTA; inmutable.

- **Columna:** folio. **Tipo:** varchar(20). **Nulo:** no. **Default:** . **Nota:** `C-2026-000123`; de folio_contador; no fiscal.
- **Columna:** parte_id. **Tipo:** uuid. **Nulo:** no. **Default:** . **Nota:** → parte.
- **Columna:** predio_origen_id / predio_destino_id. **Tipo:** uuid. **Nulo:** sí. **Default:** . **Nota:** → predio.
- **Columna:** moneda. **Tipo:** char(3). **Nulo:** no. **Default:** 'MXN'. **Nota:** CHECK = 'MXN' en MVP.
- **Columna:** estado. **Tipo:** varchar(12). **Nulo:** no. **Default:** 'BORRADOR'. **Nota:** BORRADOR, ACORDADA, CONFIRMADA, CANCELADA, ANULADA.
- **Columna:** estado_entrega. **Tipo:** varchar(10). **Nulo:** no. **Default:** 'PENDIENTE'. **Nota:** PENDIENTE, PARCIAL, COMPLETA (mantenido por comando de movimientos).
- **Columna:** numero_version_aceptada. **Tipo:** smallint. **Nulo:** sí. **Default:** . **Nota:** NOT NULL si CONFIRMADA (CHECK).
- **Columna:** fecha_operacion. **Tipo:** date. **Nulo:** no. **Default:** . **Nota:** .
- **Columna:** version_lock. **Tipo:** integer. **Nulo:** no. **Default:** 0. **Nota:** concurrencia optimista de borrador.
- **Columna:** creado_por / creado_en / confirmada_en. **Tipo:** uuid / timestamptz / timestamptz. **Nulo:** no/no/sí. **Default:** . **Nota:** .

Índices: `(E,S,estado,fecha_operacion)` bandeja; `(E,parte_id)` historial de contraparte. Rechazado: tipo 'CONSIGNA'.

ENT-10 lote_operacion

PK(E,S,operacion_id,numero_lote smallint CHECK > 0); descripcion varchar(120). Débil. L↔. Editable sólo si la operación está en BORRADOR (trigger).

ENT-11 seleccion_compra

PK(E,S,operacion_id,numero_lote,animal_id); UK(E,S,operacion_id,animal_id); `tipo_operacion varchar(6)NOTNULLDEFAULT'COMPRA'CHECK(='COMPRA')`. FKs a lote, a operacion(E,S,id,tipo) y a animal(E,sede_operadora_id,id). L↔. Rechazado: animal de la sede A en compra de sede B (FK falla, RN-009); mismo animal en dos lotes de la misma compra (UK).

ENT-12 seleccion_venta

Igual que ENT-11 con `posicion_id` y `tipo_operacion='VENTA'`; UK(E,S,operacion_id,posicion_id). L↔.

ENT-13 posicion_inventario

Propósito: ciclo de inventario (RN-005, RN-006). PK(E,S,id). L↔ · C. - **Columna:** animal_id. **Tipo:** uuid. **Nulo:** no. **Nota:** → animal(E,sede_operadora_id,id).

- **Columna:** numero_ciclo. **Tipo:** smallint. **Nulo:** no. **Nota:** > 0; max+1 por animal.
- **Columna:** compra_operacion_id, compra_numero_lote. **Tipo:** uuid, smallint. **Nulo:** no. **Nota:** → seleccion_compra (con animal_id).
- **Columna:** estado. **Tipo:** varchar(26). **Nulo:** no. **Nota:** PENDIENTE_RECEPCION, DISPONIBLE, VENDIDA_PENDIENTE_ENTREGA, CERRADA_VENTA, BAJA.
- **Columna:** motivo_baja. **Tipo:** varchar(10). **Nulo:** sí. **Nota:** PERDIDA, MUERTE; NOT NULL ⇔ estado = BAJA.
- **Columna:** abierta_en / cerrada_en. **Tipo:** timestamptz. **Nulo:** no / sí. **Nota:** cerrada_en NOT NULL ⇔ estado terminal.

Índices: UNIQUE parcial `(E,animal_id)WHEREestadoIN(activos)`; `(E,S,estado)` inventario disponible. Sin INSERT/UPDATE directos de la app (sólo comandos). Rechazado: segunda posición activa del mismo animal (TEST-05).

ENT-14 movimiento_inventario

Hecho físico inmutable (RN-020). Columnas: posicion_id NOT NULL; operacion_id NULL; tipo varchar(12) (RECEPCION, ENTREGA, BAJA, REVERSO); ocurrido_en timestamptz; predio_id, custodio_parte_id, documento_id NULL; reverso_de_id UNIQUE NULL; registrado_por. Índice `(E,S,posicion_id,ocurrido_en)`. L↔.

ENT-15 reserva_venta

Columnas: operacion_id, numero_lote, posicion_id (→ seleccion_venta); estado varchar(10) (ACTIVA, CONSUMIDA, LIBERADA, EXPIRADA); creada_en; vence_en timestamptz NOT NULL CHECK > creada_en; cerrada_en NULL. Índice UNIQUE parcial `(E,S,posicion_id)WHEREestado='ACTIVA'` (RN-018); `(E,S,vence_en)WHEREestado='ACTIVA'` para el expirador. L↔.

ENT-16 contrato_version

PK(E,S,operacion_id,numero_version smallint > 0). estado (PROPUESTA, ACEPTADA, SUPERADA); politica_redondeo varchar(12) DEFAULT 'HALF_UP_2'; requiere_recepcion boolean DEFAULT false; documento_id NULL; aceptada_por/aceptada_en NULL. Inmutable una vez ACEPTADA (RN-011). Pueden conservarse varias versiones ACEPTADAS históricas: `operacion.numero_version_aceptada` identifica la que referencia la operación. La restricción parcial propuesta inicialmente para permitir sólo una ACEPTADA no existe en el DDL vigente de `V1__esquema.sql`; la PK distingue cada versión sin modificar las aceptadas anteriores.

ENT-17 condicion_lote

- **Columna:** unidad_precio. **Tipo:** varchar(6). **Nulo:** no. **Default:** . **Nota:** KG, CABEZA.
- **Columna:** precio_unitario. **Tipo:** numeric(14,4). **Nulo:** no. **Default:** . **Nota:** > 0.
- **Columna:** base_pesaje. **Tipo:** varchar(10). **Nulo:** no. **Default:** ‘NINGUNA’. **Nota:** NINGUNA (por cabeza), PESAJE.
- **Columna:** pesaje_base_id. **Tipo:** uuid. **Nulo:** sí. **Default:** . **Nota:** NOT NULL ⇔ base_pesaje = ‘PESAJE’.
- **Columna:** deduccion_pct. **Tipo:** numeric(9,6). **Nulo:** no. **Default:** 0. **Nota:** 0 ≤ x < 100; sólo pactada (RN-015).
- **Columna:** cargos_pactados / bonificaciones_pactadas. **Tipo:** numeric(18,2). **Nulo:** no. **Default:** 0. **Nota:** ≥ 0.

Rechazado: unidad KG sin pesaje base.

ENT-18 pesaje

- **Columna:** operacion_id, numero_lote. **Tipo:** uuid, smallint. **Nulo:** no. **Nota:** → lote_operacion.
- **Columna:** modo. **Tipo:** varchar(14). **Nulo:** no. **Nota:** BRUTO_TARA, NETO_REPORTADO.
- **Columna:** bruto_kg, tara_kg. **Tipo:** numeric(14,3). **Nulo:** sí. **Nota:** NOT NULL ⇔ BRUTO_TARA; tara ≤ bruto.
- **Columna:** neto_kg. **Tipo:** numeric(14,3). **Nulo:** no. **Nota:** > 0; = bruto − tara si BRUTO_TARA.
- **Columna:** ticket_folio. **Tipo:** varchar(40). **Nulo:** sí. **Nota:** .
- **Columna:** bascula. **Tipo:** varchar(80). **Nulo:** sí. **Nota:** descripción declarada.
- **Columna:** pesado_en. **Tipo:** timestamptz. **Nulo:** no. **Nota:** .
- **Columna:** estado. **Tipo:** varchar(10). **Nulo:** no. **Nota:** PROPUESTA, APROBADO, CORREGIDO.
- **Columna:** fuente. **Tipo:** varchar(8). **Nulo:** no. **Nota:** MANUAL, CSV, OCR, FDW (RN-016).
- **Columna:** corrige_a_id. **Tipo:** uuid. **Nulo:** sí. **Nota:** UNIQUE → pesaje.

Rechazado: bruto 5,000, tara 6,000 (TEST); NETO_REPORTADO con bruto informado.

ENT-19 pesaje_animal

PK(E,S,pesaje_id,animal_id); peso_individual_kg numeric(14,3) NULL (> 0 si existe; sólo medido, RN-013). Inmutable tras APROBADO.

ENT-20 gasto_operacion

- **Columna:** operacion_id. **Tipo:** uuid. **Nulo:** no. **Nota:** .
- **Columna:** naturaleza. **Tipo:** varchar(12). **Nulo:** no. **Nota:** COSTO, BONIFICACION.
- **Columna:** clasificacion. **Tipo:** varchar(14). **Nulo:** no. **Nota:** CAPITALIZABLE, DIRECTO_VENTA, OTRO (RN-025).
- **Columna:** importe. **Tipo:** numeric(18,2). **Nulo:** no. **Nota:** > 0 (signo derivado de naturaleza).
- **Columna:** concepto. **Tipo:** varchar(160). **Nulo:** no. **Nota:** .
- **Columna:** proveedor_parte_id. **Tipo:** uuid. **Nulo:** sí. **Nota:** .

- **Columna:** referencia_origen. **Tipo:** varchar(60). **Nulo:** sí. **Nota:** UK parcial con proveedor: detecta duplicados.
- **Columna:** bonifica_gasto_id. **Tipo:** uuid. **Nulo:** sí. **Nota:** NOT NULL ⇔ BONIFICACION.
- **Columna:** estado. **Tipo:** varchar(10). **Nulo:** no. **Nota:** BORRADOR, APROBADO.
- **Columna:** fecha_economica. **Tipo:** date. **Nulo:** no. **Nota:** .
- **Columna:** snapshot_fuente_externa. **Tipo:** jsonb. **Nulo:** sí. **Nota:** tarifa FDW aceptada: fuente, ref, fecha, unidad, hash (RN-042).

ENT-21 asignacion_costo

Columnas: posicion_id; **exactamente una** de (fuente_liquidacion_id + fuente_numero_concepto) / fuente_gasto_id / fuente_ajuste_id (`CHECK(num_nonnulls(fuente_liquidacion_id,fuente_gasto_id,fuente_ajuste_id)=1)`); metodo (IMPORTE_INDIVIDUAL, POR_CABEZA, POR_PESO_MEDIDO); base_valor numeric(14,3); importe numeric(18,2) firmado (≠ 0); lote_reparto_id uuid (agrupa un reparto); periodo_id. Índice `(E,S,lote_reparto_id)` para verificar Σ = fuente.

ENT-22 liquidacion

- **Columna:** operacion_id. **Tipo:** uuid. **Nulo:** no. **Nota:** UK(E,S,operacion_id) principal.
- **Columna:** numero_version. **Tipo:** smallint. **Nulo:** no. **Nota:** versión aceptada.
- **Columna:** direccion. **Tipo:** `varchar(10)`. **Nulo:** no. **Nota:** POR_PAGAR (compra), POR_COBRAR (venta).
- **Columna:** total. **Tipo:** numeric(18,2). **Nulo:** no. **Nota:** ≥ 0; = Σ conceptos (validado en comando).
- **Columna:** snapshot_contraparte_nombre / _rfc. **Tipo:** varchar. **Nulo:** no / sí. **Nota:** RN-024.
- **Columna:** snapshot_condiciones. **Tipo:** jsonb. **Nulo:** no. **Nota:** condiciones aceptadas tal cual.
- **Columna:** vence_el. **Tipo:** date. **Nulo:** sí. **Nota:** cartera vencida.
- **Columna:** periodo_id. **Tipo:** uuid. **Nulo:** no. **Nota:** periodo de reconocimiento.
- **Columna:** confirmada_por / confirmada_en. **Tipo:** . **Nulo:** no. **Nota:** .

Inmutable (trigger BV016). Índice `(E,S,direccion,vence_el)` cartera.

ENT-23 concepto_liquidacion

PK(E,S,liquidacion_id,numero_concepto smallint). tipo (BASE, CARGO, BONIFICACION, DEDUCCION); descripcion; numero_lote NULL; cantidad numeric(14,3); unidad (KG, CABEZA, MXN); precio_unitario numeric(14,4) NULL; importe numeric(18,2) firmado (negativo para BONIFICACION/DEDUCCION, CHECK por tipo); pesaje_id NULL. Inmutable.

ENT-24 ajuste_liquidacion

naturaleza (CARGO, ABONO); monto numeric(18,2) > 0; motivo varchar(200) NOT NULL; fuente (PRECIO_GANADO, BONIFICACION_COMERCIAL, ANULACION, OTRO); documento_id NULL; revierte_ajuste_id UNIQUE NULL; periodo_id NOT NULL; fecha_reconocimiento date NOT NULL dentro del periodo abierto; registrado_por/en.

ENT-25 pago

- **Columna:** parte_id. **Tipo:** uuid. **Nulo:** no. **Nota:** .
- **Columna:** tipo. **Tipo:** varchar(10). **Nulo:** no. **Nota:** COBRO (entra), PAGO (sale); naturaleza separada NORMAL/REEMBOLSO.

- **Columna:** moneda. **Tipo:** char(3). **Nulo:** no. **Nota:** MXN.
- **Columna:** importe. **Tipo:** numeric(18,2). **Nulo:** no. **Nota:** > 0.
- **Columna:** metodo. **Tipo:** varchar(14). **Nulo:** no. **Nota:** TRANSFERENCIA, EFECTIVO, CHEQUE, OTRO.
- **Columna:** referencia. **Tipo:** varchar(60). **Nulo:** sí. **Nota:** .
- **Columna:** recibido_el. **Tipo:** date. **Nulo:** no. **Nota:** .
- **Columna:** estado. **Tipo:** varchar(10). **Nulo:** no. **Nota:** REGISTRADO, VALIDADO, REVERTIDO.
- **Columna:** naturaleza. **Tipo:** varchar(10). **Nulo:** no. **Nota:** NORMAL / REEMBOLSO; no sustituye dirección COBRO/PAGO.
- **Columna:** periodo_id. **Tipo:** uuid. **Nulo:** sí. **Nota:** NULL en registro; obligatorio al validar y para reembolso.
- **Columna:** fecha_reconocimiento. **Tipo:** date. **Nulo:** sí. **Nota:** NULL en registro; dentro del periodo al validar.
- **Columna:** pago_origen_id. **Tipo:** uuid. **Nulo:** sí. **Nota:** NOT NULL ⇔ naturaleza REEMBOLSO.

Rechazado: importe 0; REEMBOLSO sin origen.

ENT-26 aplicacion_pago

pago_id, liquidacion_id NOT NULL; importe numeric(18,2) > 0; naturaleza (APLICACION, REVERSO); reverso_de_id UNIQUE NULL; aplicada_en; registrado_por; periodo_id uuid y fecha_reconocimiento date NOT NULL. El registro de pago no cambia cartera; validación/aplicación/reverso/reembolso identifican reconocimiento explícito. Índices `(E,S,liquidacion_id)`, `(E,S,pago_id)`.

ENT-27 documento

- **Columna:** tipo_documental. **Tipo:** varchar(24). **Nulo:** no. **Nota:** TICKET_BASCULA, CONTRATO, GUIA_MOVILIZACION, COMPROBANTE_PAGO, IDENTIFICACION, OTRO.
- **Columna:** clave_almacenamiento. **Tipo:** varchar(80). **Nulo:** no. **Nota:** aleatoria; UK; nunca ruta del cliente.
- **Columna:** sha256. **Tipo:** char(64). **Nulo:** no. **Nota:** `^[0-9a-f]{64}$`.
- **Columna:** tamano_bytes. **Tipo:** bigint. **Nulo:** no. **Nota:** 1..20 MiB.
- **Columna:** mime. **Tipo:** varchar(60). **Nulo:** no. **Nota:** application/pdf, image/jpeg, image/png.
- **Columna:** version. **Tipo:** smallint. **Nulo:** no. **Nota:** 1 + versión previa.
- **Columna:** documento_previo_id. **Tipo:** uuid. **Nulo:** sí. **Nota:** UNIQUE.
- **Columna:** emisor, folio, fecha_emision, vigente_hasta, origen_texto, destino_texto, finalidad. **Tipo:** varios. **Nulo:** sí. **Nota:** según tipo (§4.2).
- **Columna:** es_sintetico. **Tipo:** boolean. **Nulo:** no. **Nota:** RN-045.

Sensibilidad C/P. Bytes fuera de la BD (DocumentStorage).

ENT-28 operacion_documento

PK(E,S,operacion_id,documento_id); rol varchar(16).

ENT-29 auditoria_evento

ocurrido_en; actor_id NULL; comando varchar(60); correlation_id uuid; entidad varchar(40); entidad_id uuid; cambio jsonb (sólo campos pertinentes, sin secretos). Insertado por el comando en el nodo origen; replicado como dato; nunca generado por apply. Índice `(E,S,entidad,entidad_id)`.

ENT-30 evento_outbox

tipo_evento varchar(60); version_payload smallint; payload jsonb; propietario_despacho char(1); creado_en. Sólo `propietario_despacho=nodolocal` se despacha.

ENT-31 mensaje_inbox (local)

PK(consumidor varchar(40), fuente_id varchar(40), event_id uuid); empresa_id, sucursal_id NULL; recibido_en; resultado varchar(10).

ENT-32 clave_idempotencia (local)

PK(E,S,principal_id,comando,clave varchar(80)); hash_peticion char(64); estado (EN_CURSO, COMPLETADA); resultado jsonb; creada_en; deduplicación conservada mientras existan los hechos financieros; no expira a los 30 días.

ENT-33 infra.nodo_config (local)

nodo char(1) PK CHECK IN (‘A’,‘B’); empresa_id; descripcion. Una fila. App sólo SELECT.

ENT-34 infra.entrega_outbox (local)

PK(evento_id, destino); intentos int DEFAULT 0; proximo_intento timestamptz; confirmado_en NULL; ultimo_error text NULL.

ENT-35 infra.importacion_fuente (local)

fuente varchar(20) (CSV, OCR, FDW); corte_en; sha256; filas_leidas, filas_error int; actor_id. UK(empresa_id, fuente, sha256) evita reimportar el mismo archivo.

ENT-37 periodo_cierre

desde, hasta date (hasta ≥ desde); estado (ABIERTO, CERRADO); version_lock. `EXCLUDEUSINGgist(empresa_idWITH=,sucursal_idWITH=,daterange(desde,hasta,'[]')WITH&&)` (extensión btree_gist, justificada: no solapamiento declarativo).

ENT-38 cierre_periodo_version

PK(E,S,periodo_id,numero_version); tipo (CIERRE, REAPERTURA); actor_id; motivo; corte_en; hash_conciliacion char(64); totales jsonb (inventario, costo de venta, pérdida, cartera). Inmutable.

ENT-39 asiento_valoracion

clase (ADQUISICION, VENTA, COSTO_TARDIO_INVENTARIO, COSTO_TARDIO_VENTA, COSTO_TARDIO_PERDIDA, BAJA, ANULACION_VENTA); sentido (AUMENTO, DISMINUCION) - DISMINUCION sólo para bonificaciones de costo tardío; importe numeric(18,2) > 0; columnas `GENERATEDALWAYSAS(...)STORED`: delta_inventario, delta_costo_venta, delta_perdida según la tabla de `states-and-calculations.md` §3; asignacion_id, operacion_id NULL; fecha_economica; periodo_id. Índice `(E,S,posicion_id)`.

ENT-40 revision_administrativa

operacion_id XOR identificacion_id; criterio varchar(200); resultado (SIN_OBSERVACIONES, CON_OBSERVACIONES, RECHAZADA); revision_anterior_id NULL; responsable_id; revisado_en. La UI rotula “Revisión administrativa registrada”, nunca “autorizado”.

ENT-41 revision_documento

PK(E,S,revision_id,documento_id); resultado; observacion varchar(500).

Soporte: folio_contador (local del dueño, L↔)

PK(E,S,tipo,anio); ultimo integer. Bloqueado FOR UPDATE al asignar folio.

Concordancia física durante implementación

Autoridad ejecutable: `db/migrations/`. Se incorporaron `periodo_id/fecha_reconocimiento` en gasto, pago y aplicación, y fecha de reconocimiento en ajuste (ADR-015). Pago admite ambos nulos mientras está REGISTRADO; validación exige ambos y comprueba periodo abierto. Gasto y aplicación los conservan no nulos. Son tres FK de periodo adicionales: metadata 42 tablas/83 FK. Sesiones JDBC son dos tablas técnicas locales, fuera del MER de negocio y de réplica.

`python3tools/esquema/comparar_catalogo.py` contrasta tipos, columnas, nullabilidad, PK, UK declaradas y FK contra PostgreSQL. No sustituye pruebas de CHECK, índices parciales, triggers, permisos ni revisión conceptual.

18.4 Dependencias funcionales y normalización

Fuente: `docs/database/normalization.md`.

Referencia: F15. Estado: análisis de las 42 tablas lógicas, contrastadas en catálogo A/B; restricciones no expresadas como DF se prueban por separado. Las tablas técnicas adicionales JDBC/Flyway/captura no son entidades comerciales.

1. Dependencias funcionales y claves candidatas por tabla

Notación: X → Y. “CK” = claves candidatas. Las claves técnicas conservan el scope E/S de su UNIQUE o PK: una UUID aislada no se presume única. Se listan además las claves comerciales.

- **Tabla:** empresa. **CK:** {id}. **DF no triviales relevantes:** id → razon_social, rfc, estado. **Forma:** BCNF.
- **Tabla:** sucursal. **CK:** {E,id}, {E,codigo}. **DF no triviales relevantes:** E,id → codigo, nombre, nodo_dueno; E,codigo → id. **Forma:** BCNF (todo determinante es CK).
- **Tabla:** parte. **CK:** {E,id}, {E,rfc} (si rfc no nulo). **DF no triviales relevantes:** E,id → nombre, rfc, …. **Forma:** BCNF.
- **Tabla:** predio. **CK:** {E,id}. **DF no triviales relevantes:** E,id → nombre, parte_responsable_id. **Forma:** BCNF.
- **Tabla:** usuario. **CK:** {E,id}, {E,login_normalizado}. **DF no triviales relevantes:** -. **Forma:** BCNF.
- **Tabla:** membresia_usuario. **CK:** {E,S,usuario_id,rol_codigo}. **DF no triviales relevantes:** → vigente. **Forma:** BCNF.
- **Tabla:** animal. **CK:** {E,id} (UK(E,sede,id) es superclave usada para FK). **DF no triviales relevantes:** E,id → sede_operadora_id, sexo, situacion. **Forma:** BCNF.
- **Tabla:** animal_identificacion. **CK:** {E,id}, {E,sistema,codigo_normalizado}. **DF no triviales relevantes:** codigo_declarado → codigo_normalizado (función determinista, ver §4).

Forma: Desnormalización derivada declarada; no afirmar 3FN/BCNF sin tratar codigo_de clarado → codigo_normalizado.

- **Tabla:** operacion. **CK:** {E,S,id}, {E,S,tipo,folio}. **DF no triviales relevantes:** E,S,id → tipo, parte_id, estado, …; E,S,tipo,folio → id. **Forma:** BCNF.
- **Tabla:** lote_operacion. **CK:** {E,S,operacion_id,numero_lote}. **DF no triviales relevantes:** → descripcion. **Forma:** BCNF.
- **Tabla:** contrato_version. **CK:** {E,S,operacion_id,numero_version}. **DF no triviales relevantes:** → estado, politica_redondeo. **Forma:** BCNF.
- **Tabla:** condicion_lote. **CK:** {E,S,operacion_id,numero_version,numero_lote}. **DF no triviales relevantes:** → precio_unitario, unidad_precio, …. **Forma:** BCNF.
- **Tabla:** seleccion_compra. **CK:** {E,S,op,animal}; PK física {E,S,op,lote,animal} no mínima. **DF no triviales relevantes:** E,S,op,animal → lote. **Forma:** BCNF respecto a la DF declarada: el determinante mínimo es {E,S,op,animal}.
- **Tabla:** posicion_inventario. **CK:** {E,S,id}, {E,S,animal_id,numero_ciclo}, {E,S,compra_op, compra_lote,animal_id}. **DF no triviales relevantes:** id → estado, …; animal_id,numero_c iclo → id. **Forma:** BCNF.
- **Tabla:** reserva_venta. **CK:** {E,S,id}. **DF no triviales relevantes:** id → posicion_id, estado, vence_en. **Forma:** BCNF.
- **Tabla:** pesaje. **CK:** {E,S,id}. **DF no triviales relevantes:** id → modo, bruto, tara, neto; (bruto,tara) → neto **cuando modo = BRUTO_TARA**. **Forma:** Dependencia condicional por modo; neto conservado con CHECK, no prueba automática de 3FN.
- **Tabla:** pesaje_animal. **CK:** {E,S,pesaje_id,animal_id}. **DF no triviales relevantes:** → pes o_individual_kg. **Forma:** BCNF.
- **Tabla:** liquidacion. **CK:** {E,S,id}, {E,S,operacion_id}. **DF no triviales relevantes:** id → total, snapshots. **Forma:** BCNF (snapshots dependen de la liquidación, ver §5).
- **Tabla:** concepto_liquidacion. **CK:** {E,S,liquidacion_id,numero_concepto}. **DF no triviales relevantes:** → tipo, cantidad, precio, importe; (cantidad, precio) → importe salvo redondeo. **Forma:** Importe derivado materializado: excepción a normalización estricta, hecho redondeado inmutable.
- **Tabla:** pago. **CK:** {E,S,id}. **DF no triviales relevantes:** id → parte_id, importe, tipo. **Forma:** BCNF.
- **Tabla:** aplicacion_pago. **CK:** {E,S,id}, {E,S,reverso_de_id} (si no nulo). **DF no triviales relevantes:** id → pago_id, liquidacion_id, importe. **Forma:** BCNF.
- **Tabla:** asignacion_costo. **CK:** {E,S,id}. **DF no triviales relevantes:** id → posicion, fuente, importe. **Forma:** BCNF.
- **Tabla:** asiento_valoracion. **CK:** {E,S,id}. **DF no triviales relevantes:** (clase, sentido, importe) → delta_*. **Forma:** deltas generados (no redundancia editable).
- **Tabla:** documento. **CK:** {E,S,id}, {clave_almacenamiento}. **DF no triviales relevantes:** id → sha256, tamano, …. **Forma:** BCNF.
- **Tabla:** periodo_cierre. **CK:** {E,S,id}; además (E,S,fecha) determina el periodo por la exclu sión. **DF no triviales relevantes:** -. **Forma:** BCNF.

2. Ejemplo A - operación / contraparte

No normalizado (planilla típica de un comercializador):

- **folio:** C-01. **fecha:** 2026-10-02. **contraparte:** Productor Demo 01. **rfc:** DEMO010101AAA. **teléfono:** 555-0101. **lote:** 1. **arete:** DEMO-1, DEMO-2. **precio_kg:** 48.

- **1FN**: “DEMO-1, DEMO-2” es no atómico → una fila por arete: (folio, lote, arete, …). La clave pasa a {folio, arete}.
- **2FN**: con clave {folio, arete}, `fecha`, `contraparte`, `rfc`, `teléfono` dependen sólo de `folio` (dependencia parcial) → se separa `operacion(folio,fecha,parte)` de `seleccion(folio,lote,arete)`.
- **3FN**: en `operacion`, `rfc` y `teléfono` dependen de `contraparte`, no del folio (transitiva folio → contraparte → rfc) → `parte(id,nombre,rfc,telefono)` y `operacion.parte_id`.
- `precio_kg` depende de (folio, lote, **versión** aceptada), no del arete → `condicion_lote` (ejemplo B).
- Resultado: `parte`, `operacion`, `lote_operacion`, `seleccion_compra`, `animal_identificacion` (el arete identifica al animal, no a la selección).

3. Ejemplo B - versión / condición por lote

Planteamiento ingenuo: `operacion(id,precio_kg,deduccion,version)` con UPDATE al renegociar.

- Viola el hecho histórico: la versión 1 deja de existir (no es un problema de forma normal, es de modelado temporal).
- Al versionar: `condicion(operacion,version,lote,precio,unidad,deduccion,politica_redondeo)`. DF: (op, version) → politica_redondeo; (op, version, lote) → precio, unidad, deduccion. `politica_redondeo` depende de parte de la clave → **viola 2FN** → se mueve a `contrato_version(op,version,politica_redondeo,estado)`.
- `condicion_lote(op,version,lote,precio,unidad,base_pesaje,deduccion)`: toda DF tiene como determinante la clave completa → 3FN y BCNF.
- Preservación de dependencias: las dos DF quedan cada una en una tabla; la unión `contrato_version` ⋈ `condicion_lote` sobre (op, version) es sin pérdida porque (op, version) es clave de `contrato_version`.

4. Ejemplo C - liquidación / aplicaciones

Planteamiento ingenuo: `liquidacion(id,total,pagado,saldo,pago1_ref,pago1_importe,pago2_ref,…)`.

- **1FN**: grupo repetido pago1/pago2 → tabla aparte.
- `saldo` = total + ajustes − aplicaciones: atributo derivado editable en paralelo → riesgo de inconsistencia; se elimina y se calcula en vista (`v_saldo_liquidacion`).
- Un pago cubre varias liquidaciones y una liquidación recibe varios pagos (N:M) → `pago(id,parte,importe,…)` + `aplicacion_pago(id,pago_id,liquidacion_id,importe,naturaleza,reverso_de_id)`.
- DF en `aplicacion_pago`: id → (pago_id, liquidacion_id, importe). No hay DF (pago_id, liquidacion_id) → importe porque puede haber varias aplicaciones y reversos entre el mismo par: por eso la aplicación tiene identidad propia.
- Restricción entre filas Σ aplicaciones ≤ importe del pago: no es una DF ni puede ser CHECK (F20); se garantiza en el comando AplicarPago con locks.
- Resultado: BCNF; ninguna sumatoria almacenada.

5. Excepciones deliberadas

Caso	Por qué no es redundancia a eliminar
`liquidacion.snapshot_*`	Hecho temporal “así se aceptó”; depende de la liquidación, no de la parte actual (RN-024)
`concepto_liquidacion.importe`	Valor redondeado HALF_UP que se acordó; recalcular podría diferir por política futura

Caso	Por qué no es redundancia a eliminar
`seleccion_compra.tipo_operacion`	Constante para FK compuesta que garantiza "sólo COMPRA" declarativamente
`pesaje.neto_kg` con bruto/tara	En NETO_REPORTADO es dato primario; en BRUTO_TARA se fuerza igualdad con CHECK
`animal_identificacion.codigo_normalizado`	Necesario para la UNIQUE; determinado por función inmutable sobre `codigo_declarado` (se podría generar; se mantiene CHECK de igualdad)
`asiento_valoracion.delta_*`	Columnas generadas: no editables

6. BCNF y preservación de dependencias

La selección de compra tiene la clave candidata mínima {E,S,op,animal}; añadir lote produce una superclave no mínima. Su DF determina lote y no viola BCNF. La tabla anterior analiza las dependencias declaradas de las tablas enumeradas; no demuestra BCNF del modelo completo. El inventario completo siguiente explicita determinantes mínimos conocidos y excepciones; las restricciones físicas se contrastaron con el catálogo ejecutado, sin confundir una comparación con una demostración exhaustiva de reglas (TEST-30). Los totales materializados y snapshots conservan autoridad e inmutabilidad explícitas; no son campos editables paralelos.

7. Inventario completo de determinantes conocidos

Para cada fila, K → todos los atributos propios del registro. Las claves de abajo provienen de PK/UK completas declaradas, eliminando superclaves que contienen otra clave más pequeña. UK con NULL y unicidad parcial por estado no son claves candidatas del conjunto completo. Todas las DF abreviadas id → … de la tabla inicial significan la clave E[,S],id correspondiente, nunca UUID global.

Los datos escalares/composición separados sostienen 1FN; los atributos comerciales de contr aparte/animal se separan de selecciones y hechos (2FN/3FN). Las constantes para FK de tipo, valores calculados, JSON snapshots y efectos históricos son excepciones explícitas. Una comprobación de claves no descubre automáticamente todas las DF semánticas; este inventario analiza las reconocidas por el dominio, sin una proclamación global falsa de BCNF.

- **Tabla:** empresa. **Determinantes mínimos NOT NULL:** {id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** sucursal. **Determinantes mínimos NOT NULL:** {empresa_id,id}, {empresa_id,codigo}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** rol_aplicacion. **Determinantes mínimos NOT NULL:** {codigo}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** usuario. **Determinantes mínimos NOT NULL:** {empresa_id,id}, {empresa_id,login_normalizado}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** membresia_usuario. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,usuario_id,rol_codigo}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.

- **Tabla:** parte. **Determinantes mínimos NOT NULL:** {empresa_id,id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** predio. **Determinantes mínimos NOT NULL:** {empresa_id,id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** animal. **Determinantes mínimos NOT NULL:** {empresa_id,id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** animal_identificacion. **Determinantes mínimos NOT NULL:** {empresa_id,id}, {empresa_id,sistema,codigo_normalizado}. **UK condicional conocida:** -. **Análisis / excepción:** normalización derivada del código; véase §5, no BCNF estricta.
- **Tabla:** operacion. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,id}, {empresa_id,sucursal_id,tipo,folio}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** folio_contador. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,tipo,anio}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** lote_operacion. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,operacion_id,numero_lote}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** contrato_version. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,operacion_id,numero_version}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** condicion_lote. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,operacion_id,numero_version,numero_lote}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** pesaje. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,id}. **UK condicional conocida:** -. **Análisis / excepción:** neto derivado sólo en BRUTO_TARA; dependencia condicional con CHECK.
- **Tabla:** pesaje_animal. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,pesaje_id,animal_id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** seleccion_compra. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,operacion_id,animal_id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** posicion_inventario. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,id}, {empresa_id,sucursal_id,animal_id,numero_ciclo}, {empresa_id,sucursal_id,compra_operacion_id,compra_numero_lote,animal_id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** seleccion_venta. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,operacion_id,posicion_id}. **UK condicional conocida:** -. **Análisis / excepción:** selección de posición por operación mínima; estado derivado de reserva/posición, no atributo repetido.

- **Tabla:** reserva_venta. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** movimiento_inventario. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** periodo_cierre. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** cierre_periodo_version. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,periodo_id,numero_version}. **UK condicional conocida:** -. **Análisis / excepción:** totales/versiones y hash de corte materializados e inmutables.
- **Tabla:** liquidacion. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,id}, {empresa_id,sucursal_id,operacion_id}. **UK condicional conocida:** -. **Análisis / excepción:** snapshots temporales inmutables, no copia del maestro actual.
- **Tabla:** concepto_liquidacion. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,liquidacion_id,numero_concepto}. **UK condicional conocida:** -. **Análisis / excepción:** importe redondeado materializado; no 3FN estricta respecto a fórmula.
- **Tabla:** ajuste_liquidacion. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** gasto_operacion. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** asignacion_costo. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** asiento_valoracion. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,id}. **UK condicional conocida:** -. **Análisis / excepción:** deltas generados, fórmulas no editables.
- **Tabla:** pago. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** aplicacion_pago. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** documento. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,id}. **UK condicional conocida:** -. **Análisis / excepción:** sha256 no determina nombre original/versión/scope; múltiples metadatos de mismos bytes son válidos.
- **Tabla:** operacion_documento. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,operacion_id,documento_id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** revision_administrativa. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** revision_documento. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,revision_id,documento_id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/

2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.

- **Tabla:** auditoria_evento. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** evento_outbox. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** clave_idempotencia. **Determinantes mínimos NOT NULL:** {empresa_id,sucursal_id,principal_id,comando,clave}. **UK condicional conocida:** -. **Análisis / excepción:** resultado inmutable de intención; contenido/hash/resultado forman hecho temporal.
- **Tabla:** mensaje_inbox. **Determinantes mínimos NOT NULL:** {consumidor,fuente_id,event_id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** infra.nodo_config. **Determinantes mínimos NOT NULL:** {nodo}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** infra.entrega_outbox. **Determinantes mínimos NOT NULL:** {evento_id,destino}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.
- **Tabla:** infra.importacion_fuente. **Determinantes mínimos NOT NULL:** {id}. **UK condicional conocida:** -. **Análisis / excepción:** 1FN/2FN/3FN y BCNF respecto a DF declaradas: determinantes son claves; no se presume otra unicidad comercial.

Tablas técnicas del catálogo SpringSession (clave PRIMARY_ID; SESSION_ID única) y attributes (PRIMARY_ID,ATTRIBUTE_NAME) separan el grupo repetido; bytes de atributo son serialización técnica, no una nueva entidad comercial. Flyway conserva historia de versión/checksum. `infra.tarifa_snapshot.id` determina captura/actor/scope/contenido; sólo aceptada_en puede transitar de NULL a timestamp una vez. Precio y hash son derivados materializados deliberados; el snapshot aceptado del gasto conserva hechos aunque cambie la fuente FDW. Ninguna se publica como maestro comercial ni implica normalización de contenido binario/JSON.

18.5 Casos de uso

Fuente: `docs/domain/use-cases.md`.

Actores: Administrador (ADM), Operador (OPE), Caja (CAJ), Auditor (AUD), Sistema (SIS). Todos los comandos con efecto exigen `Idempotency-Key` y se ejecutan en el nodo dueño (OwnerDatabaseRouter).

- **UC:** UC-01. **Nombre:** Iniciar sesión y elegir sede. **Actor:** Todos. **Flujo normal:** Login → lista de sedes autorizadas → selección. **Alternativos / errores:** Credenciales inválidas 401; sede no autorizada 403; A caído: todas las nuevas solicitudes autenticadas A/B fallan503. **Invariantes:** Sesión de servidor, cookie HttpOnly. **RN:** RN-001. **Comando:** -.
- **UC:** UC-02. **Nombre:** Registrar contraparte/predio. **Actor:** OPE. **Flujo normal:** Captura en A, replicación a B. **Alternativos / errores:** Duplicado de RFC/código → 409. **Invariantes:** Maestros sólo en A. **RN:** RN-001, RN-010. **Comando:** CRUD maestro.
- **UC:** UC-03. **Nombre:** Registrar identidad animal. **Actor:** OPE. **Flujo normal:** Alta en A con código normalizado; si es para B, esperar visibilidad en B (estado PENDIENTE_CONVERGENCIA). **Alternativos / errores:** Colisión → revisión central, no fusión automática.

Invariantes: UK(E,sistema,codigo). **RN:** RN-002..004. **Comando:** RegistrarIdentidadAnimal.

- **UC:** UC-04. **Nombre:** Capturar compra. **Actor:** OPE. **Flujo normal:** Borrador → lotes → selección de animales → versión y condiciones → pesajes → revisión. **Alternativos / errores:** Edición concurrente →412 por If-Match obsoleto; falta →428. **Invariantes:** Un animal una vez por operación. **RN:** RN-007..011, RN-017. **Comando:** edición de borrador.
- **UC:** UC-05. **Nombre:** Aprobar pesaje. **Actor:** OPE. **Flujo normal:** Adjuntar ticket → modo BRUTO_TARA/NETO_REPORTADO → composición → aprobar. **Alternativos / errores:** Composición distinta del lote → advertencia; corrección = nuevo pesaje. **Invariantes:** Composición congelada. **RN:** RN-012..016. **Comando:** AprobarPesaje, RegistrarCorreccionPesaje.
- **UC:** UC-06. **Nombre:** Confirmar compra. **Actor:** OPE. **Flujo normal:** Valida precondiciones → crea posiciones, liquidación, asignaciones, asientos, auditoría, outbox. **Alternativos / errores:** Precondición faltante 422; periodo cerrado 422; reintento devuelve mismo resultado. **Invariantes:** Todo o nada. **RN:** RN-021, RN-026, RN-043. **Comando:** ConfirmarCompra.
- **UC:** UC-07. **Nombre:** Registrar recepción/entrega. **Actor:** OPE. **Flujo normal:** Movimiento por posición; recepción → DISPONIBLE; entrega → CERRADA_VENTA. **Alternativos / errores:** Posición en otro estado 422. **Invariantes:** No genera venta ni costo. **RN:** RN-020. **Comando:** RegistrarRecepcion/Entrega.
- **UC:** UC-08. **Nombre:** Reservar y confirmar venta. **Actor:** OPE. **Flujo normal:** Selección → ReservarPosiciones → condiciones → ConfirmarVenta. **Alternativos / errores:** Posición reservada por otra venta 409; reserva vencida → revalida. **Invariantes:** Una reserva activa por posición. **RN:** RN-018, RN-019, RN-022. **Comando:** ReservarPosiciones, ConfirmarVenta.
- **UC:** UC-09. **Nombre:** Registrar y aplicar pago. **Actor:** CAJ. **Flujo normal:** Registrar pago → validar → aplicar a liquidaciones compatibles. **Alternativos / errores:** Excede saldo/disponible 422; carrera → 409. **Invariantes:** Σ aplicaciones ≤ pago y ≤ saldo. **RN:** RN-028..032. **Comando:** Registrar/ConfirmarPago, AplicarPago.
- **UC:** UC-10. **Nombre:** Revertir aplicación / reembolsar. **Actor:** CAJ+ADM. **Flujo normal:** Reverso completo; reembolso consume disponible. **Alternativos / errores:** Doble reverso 409. **Invariantes:** Original conservado. **RN:** RN-033. **Comando:** RevertirAplicacion, RegistrarReembolso.
- **UC:** UC-11. **Nombre:** Ajustar liquidación. **Actor:** ADM. **Flujo normal:** Ajuste CARGO/ABONO con motivo y fuente. **Alternativos / errores:** Bonificación sobre pagada → guía GOLD-07. **Invariantes:** Liquidación inmutable. **RN:** RN-023, RN-027. **Comando:** AjustarLiquidacion.
- **UC:** UC-12. **Nombre:** Registrar costo tardío. **Actor:** OPE. **Flujo normal:** Gasto → previsualizar reparto → aprobar → asientos por destino económico. **Alternativos / errores:** Periodo cerrado → BV025 sin efecto; seleccionar explícitamente periodo abierto con nueva intención. **Invariantes:** Σ asignaciones = gasto. **RN:** RN-025..027. **Comando:** DistribuirCosto, RegistrarCostoTardio.
- **UC:** UC-13. **Nombre:** Dar de baja posición. **Actor:** ADM. **Flujo normal:** Motivo (PERDIDA/MUERTE) + evidencia → BAJA. **Alternativos / errores:** Posición vendida 422. **Invariantes:** No borrar. **RN:** RN-038. **Comando:** RegistrarBajaInventario.
- **UC:** UC-14. **Nombre:** Cerrar/reabrir periodo. **Actor:** ADM. **Flujo normal:** Lock periodo → conciliación → versión de cierre con hash. **Alternativos / errores:** Hechos concurrentes esperan el lock. **Invariantes:** Cierre publicado inmutable. **RN:** RN-039. **Comando:** CerrarPeriodo, ReabrirPeriodo.
- **UC:** UC-15. **Nombre:** Consultar reportes. **Actor:** Todos/AUD. **Flujo normal:** Cartera, margen, inventario con corte y antigüedad. **Alternativos / errores:** Réplica atrasada → marca

de antigüedad. **Invariantes:** Margen provisional etiquetado. **RN:** RN-035, RN-036. **Comando:** consultas.

- **UC:** UC-16. **Nombre:** Consultar proveedor FDW y aceptar tarifa. **Actor:** OPE. **Flujo normal:** Consulta fuera de transacción con locks → aceptar → snapshot local. **Alternativos / errores:** Fuente caída: mensaje y reintento. **Invariantes:** No media liquidación. **RN:** RN-042. **Comando:** AceptarTarifaExterna.
- **UC:** UC-17. **Nombre:** Revisión administrativa de documentos. **Actor:** ADM. **Flujo normal:** Revisión de versión exacta con resultado y observación. **Alternativos / errores:** Nueva versión no altera revisión previa. **Invariantes:** “Revisión administrativa registrada”, nunca “autorizado”. **RN:** RN-037. **Comando:** RegistrarRevision.
- **UC:** UC-18. **Nombre:** Validar conciliación del propietario. **Actor:** ADM/SIS. **Flujo normal:** Conteos por dueño, huérfanos, sumatorias. **Alternativos / errores:** Diferencias → incidente. **Invariantes:** -. **RN:** RN-040, RN-043. **Comando:** ValidarConciliacionDelPropietario.
- **UC:** UC-19. **Nombre:** Simular margen. **Actor:** OPE. **Flujo normal:** Usa la función canónica en modo preview sin mutar. **Alternativos / errores:** -. **Invariantes:** No persiste. **RN:** RN-035. **Comando:** preview.

18.6 ADR-001: Monolito modular frente a microservicios

Fuente: `docs/architecture/adr/ADR-001-monolito-modular.md`.

- Estado: propuesto · 2026-10-10 · Requisito: AC-08 #### Contexto Equipo de 2 personas, 21 días, transacciones locales que deben cubrir operación + liquidación + inventario + auditoría. #### Alternativas

1. Microservicios por dominio - transacciones distribuidas, más despliegue y observabilidad.
2. Monolito sin módulos - rápido al inicio, acoplamiento alto.
3. **Monolito modular** con paquetes por dominio y puertos. #### Decisión Opción 3: un backend Spring Boot desplegable con 10 módulos (`overview.md` §3). #### Consecuencias

- Una transacción local por caso de uso; despliegue simple. − Disciplina de dependencias entre módulos sin herramienta que la imponga (se revisará con ArchUnit si hay tiempo). Sin Kafka/Kubernetes.

18.7 ADR-002: SQL explícito, cálculo canónico y migraciones versionadas

Fuente: `docs/architecture/adr/ADR-002-sql-explicito-calculo-canonico.md`.

- Estado: propuesto · 2026-10-10 · Requisitos: AC-03, AC-04, AC-05 #### Contexto La materia evalúa SQL, PL/pgSQL y transacciones; los importes deben ser reproducibles al centavo. #### Alternativas

1. JPA/Hibernate con `ddl-auto` - oculta SQL y no sirve como fuente del diseño.
2. **JdbcTemplate + Flyway + funciones PL/pgSQL canónicas.**
3. jOOQ - buena opción, añade generación de código. #### Decisión Opción 2. Importes confirmados se calculan en `fn_calcular_liquidacion` (SQL) y se congelan; la previsualización llama la misma función sin persistir. Java usa BigDecimal y valida; no recalcula con otra fórmula. #### Consecuencias

- SQL visible y evaluable; una sola fórmula. − Más SQL a mano; mapeos explícitos. Pruebas con PostgreSQL real (Testcontainers), nunca H2.

18.8 ADR-003: Animal permanente y posición de inventario por ciclo

Fuente: `docs/architecture/adr/ADR-003-animal-y-posicion.md`.

- Estado: propuesto · 2026-10-10 · Reglas: RN-002, RN-005, RN-006 #### Contexto Un animal puede comprarse, venderse y recomprarse; cada tenencia tiene su propio costo. #### Alternativas

1. Estado y costo en `animal` - pierde historia en recompra.
2. **`animal` (identidad) + `posicion_inventario` (ciclo)** con índice único parcial de ciclo activo. #### Decisión Opción 2. Estados de posición: PENDIENTE_RECEPCION, DISPONIBLE, VENDIDA_PENDIENTE_ENTREGA, CERRADA_VENTA, BAJA; "reservada" derivada de reserva ACTIVA (DEC-01..03). `animal.situacion` bloquea recompra de MUERTO (DEC-04). #### Consecuencias

- GOLD-05/09 naturales. − Consultas de ficha requieren join animal-posiciones.

18.9 ADR-004: Reconocimiento gerencial y costo por posición

Fuente: `docs/architecture/adr/ADR-004-reconocimiento-y-costo.md`.

- Estado: propuesto · 2026-10-10 · Reglas: RN-021, RN-022, RN-035 #### Contexto Entrega, cobro y PDF no deben generar una segunda venta, deuda o costo. #### Alternativas

1. Reconocer al entregar.
2. Reconocer al cobrar.
3. **Reconocer al CONFIRMAR** la operación (animales, base, versión y costos determinados). #### Decisión Opción 3 como política operativa versionada (no dictamen contable). Costo identificado por posición mediante `asignacion_costo` (soporte) y `asiento_valoracion` (efecto), con la tabla de destino económico de `states-and-calculations.md` §3. #### Consecuencias

- Un único evento económico; margen y cartera conciliables. − Cambiar la política exige migración de modelo y reportes.

18.10 ADR-005: Maestro de identidad en A y propiedad de escritura inmutable

Fuente: `docs/architecture/adr/ADR-005-maestro-y-propiedad.md`.

- Estado: propuesto · 2026-10-10 · Requisito: AC-06 · Reglas: RN-003, RN-004, RN-040, RN-041 #### Contexto Dos escritores independientes podrían dar dos UUID al mismo arete; la replicación lógica no garantiza unicidad global. #### Alternativas

1. Unicidad por nodo - permite duplicados entre nodos.
2. Escritura libre en ambos + resolución de conflictos - no existe resolución de negocio segura para inventario/dinero.
3. **Maestros sólo en A; agregados locales sólo en el nodo de su sede; sede del animal inmutable.** #### Decisión Opción 3, con `nodo_origen` en cada fila y `trg_guarda_propietario`. Sin failover automático de dueño. Transferir un animal de sede queda fuera (requiere fencing y otro ADR). #### Consecuencias

- Sin conflictos de negocio por diseño. − Si A cae no hay maestros nuevos ni login nuevo.

18.11 ADR-006: Replicación bidireccional, bootstrap y recuperación

Fuente: `docs/architecture/adr/ADR-006-replicacion-bidireccional.md`.

- Estado: propuesto · 2026-10-10 · Requisito: AC-06 #### Contexto Se requiere M-M con dos nodos escritores; PostgreSQL 18 ofrece replicación lógica con `origin=none`. #### Alternativas

1. Streaming físico (un solo escritor) - no cumple M-M.
2. Extensiones de terceros (pglogical, BDR/PGD) - licencia/soporte, más complejidad.
3. **Replicación lógica nativa bidireccional con publicaciones explícitas y `origin=none`.** #### Decisión Opción 3 con el bootstrap de `docs/database/replication-and-fdw.md` §3 (`copy_data=false` sólo en bases vacías). Recuperación: slots monitoreados, `max_slot_wal_keep_size`, runbook de resincronización. #### Consecuencias

- Nativo y demostrable. − DDL coordinado; triggers/FK no se verifican en apply → conciliación obligatoria.

18.12 ADR-007: Pgpool-II en modo raw y presupuesto de conexiones

Fuente: `docs/architecture/adr/ADR-007-pgpool-raw.md`.

- Estado: propuesto · 2026-10-10 · Requisito: AC-09 #### Contexto La rúbrica exige Pgpool-II en el dibujo; los modos de balanceo asumen un primario único. #### Alternativas

1. Pgpool streaming/replication mode delante de ambos - enrutaría escrituras a un nodo equivocado.
2. **Una instancia Pgpool raw por nodo**, la app elige el Pgpool del dueño.
3. Sin Pgpool - no cumple AC-09. #### Decisión Opción 2 (Pgpool 4.7.x). Presupuesto por nodo: Hikari 8, `num_init_children` 16, `max_pool` 1, + replicación y admin ≤ `max_connections` 50. #### Consecuencias

- Cumple AC-09 sin atribuirle sincronización. − Pgpool añade un salto sin balanceo real.

18.13 ADR-008: Autenticación, permisos y aislamiento del piloto

Fuente: `docs/architecture/adr/ADR-008-autenticacion-y-aislamiento.md`.

- Estado: propuesto · 2026-10-10 · Reglas: RN-001 #### Alternativas

1. JWT en localStorage - expuesto a XSS, revocación difícil.
2. **Sesión de servidor (Spring Security) con cookie HttpOnly/Secure/SameSite=Lax + CSRF, mismo origen.**
3. OIDC externo - no necesario para el piloto. #### Decisión Opción 2. Sesiones locales no replicadas: si cae el nodo de sesiones se reautentica. Contraseñas con `DelegatingPasswordEncoder`. Piloto: una empresa por despliegue; empresa_id se conserva para crecer, sin vender multiempresa compartido hasta probar casos negativos. #### Consecuencias

- Revocación inmediata. − Escritura en B requiere sesión válida (no se promete disponibilidad total si A cae).

18.14 ADR-009: Documentos privados, evidencia y revisión administrativa

Fuente: `docs/architecture/adr/ADR-009-documentos-y-revision.md`.

- Estado: propuesto · 2026-10-10 · Reglas: RN-037 #### Alternativas

1. Bytes en la BD (bytea) - infla respaldos y réplica.
2. **Puerto `DocumentStorage` con adaptador de volumen local privado** (dev/academic) y almacenamiento de objetos privado en piloto; metadatos + SHA-256 en BD. #### Decisión Opción 2. Nombres internos aleatorios, validación de tamaño y MIME real, descarga autorizada; nueva versión = nuevo registro con `documento_previo_id`. Revisión apunta a versión exacta; rótulo "revisión administrativa registrada", nunca "autorizado por SENASICA". #### Consecuencias

- Integridad verificable. − El respaldo debe incluir el volumen de documentos además de la BD.

18.15 ADR-010: Despliegue, respaldos y límites de disponibilidad

Fuente: `docs/architecture/adr/ADR-010-despliegue-respaldos.md`.

- Estado: propuesto · 2026-10-10 #### Alternativas

1. Kubernetes - sobredimensionado.
2. **Docker Compose + Caddy en un VPS**, respaldos fuera del host. #### Decisión Opción 2. Demo: dumps lógicos con corte y conciliación + restauración aislada medida. Piloto: backup base + archivo WAL por nodo antes de prometer RPO 15 min / RTO 2 h. #### Consecuencias

- Reproducible y barato. − Un solo host es un único dominio de fallo; no hay alta disponibilidad.

18.16 ADR-011: Graphify y mantenimiento del contexto

Fuente: `docs/architecture/adr/ADR-011-graphify.md`.

- Estado: propuesto · 2026-10-10 #### Contexto El equipo quiere contexto navegable para Claude y personas. Instalado: paquete `graphifyy` 0.9.49. #### Decisión Grafo de desarrollo (no dependencia del runtime) con `/graphify.` y `/graphify.--update` tras cambios de documentos; `graphifyupdate.` sólo para cambios de código. `.graphifyignore` excluye secretos, datos, dumps, dependencias y `graphify-out/`. Salidas por checkout; `graphify-out/` no se versiona; sin Graphify Cloud ni servidor público. Estado en `docs/knowledge/graph-status.md`. #### Consecuencias
- Navegación de reglas↔tablas↔pruebas. − Aristas inferidas deben verificarse contra fuentes; el procesamiento semántico puede usar el modelo configurado (no es offline garantizado).

18.17 ADR-012: Alcance aplazado y condiciones para comercializar

Fuente: `docs/architecture/adr/ADR-012-alcance-aplazado.md`.

- Estado: propuesto · 2026-10-10 #### Fuera del MVP Exportación, subastas, sacrificio/cortes, leche, reproducción, nutrición, diagnósticos, prescripciones, crédito, custodia de dinero,

timbrado CFDI, Carta Porte, básculas en tiempo real, cambio de sede escritora, transacciones entre sedes, edición concurrente de la misma entidad en ambos nodos, devoluciones físicas posteriores a la entrega, migración de inventario histórico, cuentas por pagar a tra nsportistas. #### Condiciones para comercializar

1. Problema y precio validados (validation-plan). 2. Contrato con cancelación, exportación, retención y aviso de privacidad. 3. Respaldo con WAL y restauración ensayada. 4. Facturación del servicio vía PAC. 5. Soporte dimensionado. #### Consecuencias Las funciones aplazadas se bloquean explícitamente en API/UI (no botones genéricos como “restaurar inventario”).

18.18 ADR-013: Sesiones centrales y disponibilidad

Fuente: `docs/architecture/adr/ADR-013-sesiones-disponibilidad.md`.

- Estado: aceptado para implementación local por encargo del usuario · 2026-10-11

Decisión

Spring Session JDBC y autorización vigente residen en A. Toda nueva petición autenticada de A/B depende de A; su caída produce 503 controlado. Una transacción B ya autorizada puede finalizar. Sustituye cualquier promesa de continuidad de sesiones B del ADR-008. Cookie Http Only/SameSite=Lax, CSRF también en login/logout; Secure obligatorio fuera del perfil HTTP local.

Validación

Implementación y pruebas pendientes al registrar esta decisión; ver evidencia (referencia del repositorio: `../../testing/evidence.md`). La revisión humana permanece pendiente.

18.19 ADR-014: Muerte en B e identidad en A

Fuente: `docs/architecture/adr/ADR-014-muerte-entre-duenos.md`.

- Estado: aceptado para implementación local por encargo del usuario · 2026-10-11

Decisión

B reconoce baja/pérdida y MuerteAnimalSolicitada.v1 atómicamente. Worker reclama lease local corto, termina la transacción y entrega a inbox idempotente A. A actualiza identidad sin volver a reconocer pérdida. Acuse perdido recupera resultado inbox; estado aplicado en A y visibilidad en B son distintos. Sustituye la actualización directa del maestro desde el comando B.

Validación

Implementación y pruebas pendientes al registrar esta decisión; ver evidencia (referencia del repositorio: `../../testing/evidence.md`). La revisión humana permanece pendiente.

18.20 ADR-015: Reconocimiento y cierres históricos

Fuente: `docs/architecture/adr/ADR-015-periodos-cartera.md`.

- Estado: aceptado para implementación local por encargo del usuario · 2026-10-11

Decisión

Periodos mensuales calendario por empresa/sede America/Mexico_City, uno abierto y rangos consecutivos sin solapamiento. Todo hecho que cambia valoración/cartera conserva periodo y fecha de reconocimiento, además de fecha documental y registro UTC. Lock común antes de recursos financieros. Cierre/versiones inmutables. Reapertura sólo último cerrado sin periodo posterior ni abierto. Perdedor frente al cierre recibe BV025 sin efecto, cambia periodo sólo mediante intención explícita nueva.

Validación

Implementación y pruebas pendientes al registrar esta decisión; ver evidencia (referencia del repositorio: `../../testing/evidence.md`). La revisión humana permanece pendiente.

18.21 ADR-016: Autoridad del esquema físico

Fuente: `docs/architecture/adr/ADR-016-autoridad-esquema.md`.

- Estado: aceptado para implementación local por encargo del usuario · 2026-10-11

Decisión

db/migrations es autoridad física ejecutable. tools/esquema/esquema.py es representación lógica documental que debe contrastarse con catálogo PostgreSQL; no se genera DDL ciegamente. Migrador separado, runtime noDDL. Tablas técnicas locales y excepciones de publicación explícitas. P0001 DETAIL JSON validado transporta códigos BV, no SQLSTATE inventado.

Validación

Implementación y pruebas pendientes al registrar esta decisión; ver evidencia (referencia del repositorio: `../../testing/evidence.md`). La revisión humana permanece pendiente.

18.22 ADR-017: Despliegue inicial preparado

Fuente: `docs/architecture/adr/ADR-017-despliegue-local.md`.

- Estado: aceptado para implementación local por encargo del usuario · 2026-10-11

Decisión

Una empresa por despliegue, VM Linux DockerCompose+Caddy preparada; ejecución actual localhost. Dos nodos en una máquina son topología académica, no redundancia física. Sin publicación/push/contratación. Secretos externos privados, puertosloopback, archivos fuerawebroot. Restauración aislada ejecutada antes de afirmar recuperación.

Validación

Implementación y pruebas pendientes al registrar esta decisión; ver evidencia (referencia del repositorio: `../../testing/evidence.md`). La revisión humana permanece pendiente.

Capítulo 19

Estado de tareas y alcance de la entrega

Este estado proviene del backlog canónico del corte del software. Conserva responsables humanos aunque los agentes hayan ejecutado trabajo. Las52 tareas verificadas localmente no están aprobadas automáticamente por Aldo/Jared: todas las55 revisiones humanas permanecen pendientes. A028 contiene preparación local pero depende de staging/TLS/CI externos; A031/J020 son OCR aplazado. La publicación GitHub main se realizó después del corte de algunos documentos y se confirmó en92 d3 d41.

Tarea y responsable	Entregable y estado técnico
`TASK-A-001` - Aldo	Inventario del repo/herramientas y matriz AC. **verificado_local**. Revisión humana pendiente.
`TASK-A-002` - Aldo	Negocio, glosario, reglas y GOLD. **verificado_local**. Revisión humana pendiente.
`TASK-A-003` - Aldo	MER Chen y matriz de relaciones. **verificado_local**. Revisión humana pendiente.
`TASK-A-004` - Aldo	Modelo relacional, diccionario y normalización. **verificado_local**. Revisión humana pendiente.
`TASK-A-005` - Aldo	Arquitectura, ADR y plan de despliegue. **verificado_local**. Revisión humana pendiente.
`TASK-A-006` - Aldo	Contexto Markdown, README y grafo inicial. **verificado_local**. Revisión humana pendiente.
`TASK-A-007` - Aldo	OpenAPI, errores y ejemplos para Jared. **verificado_local**. Revisión humana pendiente.
`TASK-A-008` - Aldo	Backend modular y build reproducible. **verificado_local**. Revisión humana pendiente.
`TASK-A-009` - Aldo	DDL/Flyway, claves, índices y seed sintético. **verificado_local**. Revisión humana pendiente.
`TASK-A-010` - Aldo	Roles PostgreSQL, grants y guardas de dueño. **verificado_local**. Revisión humana pendiente.
`TASK-A-011` - Aldo	Compose A/B + Pgpool y experimento M-M mínimo. **verificado_local**. Revisión humana pendiente.
`TASK-A-012` - Aldo	Tercero simulado y FDW sólo lectura. **verificado_local**. Revisión humana pendiente.
`TASK-A-013` - Aldo	Autenticación y permisos por sede. **verificado_local**. Revisión humana pendiente.

Tarea y responsable	Entregable y estado técnico
TASK-A-014 - Aldo	Maestros, animal e identificaciones centrales. **verificado_local**. Revisión humana pendiente.
TASK-A-015 - Aldo	Operaciones, lotes y versiones contractuales. **verificado_local**. Revisión humana pendiente.
TASK-A-016 - Aldo	Documentos, revisión y pesajes. **verificado_local**. Revisión humana pendiente.
TASK-A-017 - Aldo	ConfirmarCompra y asignar costo inicial. **verificado_local**. Revisión humana pendiente.
TASK-A-018 - Aldo	Movimientos, recepción, entrega y baja. **verificado_local**. Revisión humana pendiente.
TASK-A-019 - Aldo	Reservas, expiración y ConfirmarVenta. **verificado_local**. Revisión humana pendiente.
TASK-A-020 - Aldo	Pagos, anticipos y aplicaciones. **verificado_local**. Revisión humana pendiente.
TASK-A-021 - Aldo	Ajustes, reversos y reembolsos. **verificado_local**. Revisión humana pendiente.
TASK-A-022 - Aldo	Costo tardío, valoración y periodos. **verificado_local**. Revisión humana pendiente.
TASK-A-023 - Aldo	Auditoría, outbox/inbox y reintentos. **verificado_local**. Revisión humana pendiente.
TASK-A-024 - Aldo	Consultas de cartera, margen e inventario. **verificado_local**. Revisión humana pendiente.
TASK-A-025 - Aldo	Integrar aceptación de fuente FDW y QR. **verificado_local**. Revisión humana pendiente.
TASK-A-026 - Aldo	Pruebas M-M completas, fallas y conciliación. **verificado_local**. Revisión humana pendiente.
TASK-A-027 - Aldo	Pruebas acumulativas de dominio/SQL/API y concurrencia. **verificado_local**. Revisión humana pendiente.
TASK-A-028 - Aldo	Pipeline, staging, TLS y configuración. **pendiente_externo**. Revisión humana pendiente.
TASK-A-029 - Aldo	Respaldo, restauración y runbooks. **verificado_local**. Revisión humana pendiente.
TASK-A-030 - Aldo	Rendimiento, conexiones y observabilidad. **verificado_local**. Revisión humana pendiente.
TASK-A-031 - Aldo	OCR supervisado si se aprueba el núcleo. **diferido**. Revisión humana pendiente.
TASK-A-032 - Aldo	Dossier académico, manual SQL y exposición. **verificado_local**. Revisión humana pendiente.
TASK-A-033 - Aldo	Fuentes, README y Graphify final consolidado. **verificado_local**. Revisión humana pendiente.
TASK-J-001 - Jared	Entender negocio, actores y términos. **verificado_local**. Revisión humana pendiente.
TASK-J-002 - Jared	Wireframes y arquitectura de información. **verificado_local**. Revisión humana pendiente.
TASK-J-003 - Jared	Tokens y componentes visuales accesibles. **verificado_local**. Revisión humana pendiente.
TASK-J-004 - Jared	React/TS/Vite, router y build. **verificado_local**. Revisión humana pendiente.

Tarea y responsable	Entregable y estado técnico
TASK-J-005 - Jared	Cliente tipado, mocks y errores del contrato. **verificado_local**. Revisión humana pendiente.
TASK-J-006 - Jared	Sesión, navegación y sede autorizada. **verificado_local**. Revisión humana pendiente.
TASK-J-007 - Jared	Contrapartes y predios. **verificado_local**. Revisión humana pendiente.
TASK-J-008 - Jared	Ficha animal, ciclos e inventario. **verificado_local**. Revisión humana pendiente.
TASK-J-009 - Jared	Captura de compra y condiciones por lote. **verificado_local**. Revisión humana pendiente.
TASK-J-010 - Jared	Pesajes, archivos y revisión. **verificado_local**. Revisión humana pendiente.
TASK-J-011 - Jared	Selección de venta y reservas. **verificado_local**. Revisión humana pendiente.
TASK-J-012 - Jared	Simulador y detalle explicable de liquidación. **verificado_local**. Revisión humana pendiente.
TASK-J-013 - Jared	Cobros/pagos y aplicación guiada. **verificado_local**. Revisión humana pendiente.
TASK-J-014 - Jared	Ajustes, devoluciones de dinero y periodos. **verificado_local**. Revisión humana pendiente.
TASK-J-015 - Jared	Recepciones, entregas y expediente temporal. **verificado_local**. Revisión humana pendiente.
TASK-J-016 - Jared	Cartera, margen y QR interno. **verificado_local**. Revisión humana pendiente.
TASK-J-017 - Jared	Estados de fuente caída/sincronización/conflicto. **verificado_local**. Revisión humana pendiente.
TASK-J-018 - Jared	Responsive, teclado y accesibilidad. **verificado_local**. Revisión humana pendiente.
TASK-J-019 - Jared	Pruebas E2E integradas con Aldo. **verificado_local**. Revisión humana pendiente.
TASK-J-020 - Jared	UI de OCR sólo si el extra está implementado. **diferido**. Revisión humana pendiente.
TASK-J-021 - Jared	Manual, capturas auténticas y demo. **verificado_local**. Revisión humana pendiente.
TASK-J-022 - Jared	README, contexto frontend y actualización de grafo. **verificado_local**. Revisión humana pendiente.

Capítulo 20

Matriz de pruebas por escenario

El siguiente registro enlaza cada TEST con evidencia SQL/API/UI y conserva sus límites. Es el alcance de las ejecuciones registradas, no pruebas nuevas lanzadas por generar el manual.

20.0.1 Matriz de pruebas SQL e infraestructura ejecutadas

Esta matriz distingue aserciones SQL, infraestructura y solicitudes HTTP. `PASS_SQL` nunca certifica por sí solo API/UI, capacidad, despliegue o35/35 completos. Las mutaciones SQL usan namespaces QA; GOLD revierte su transacción. Main conserva sus hechos y sólo se consultó/autenticó durante fallos.

- **ID:** TEST-01. **Estado:** PASS_SQL. **Evidencia y resultado:** GOLD01 costo252000, margen34000, saldo244000; variación100kg/2 % sin descuento automático gold-final-all.log (referencia del repositorio: `../../../db/tests/gold-final-all.log`). **Límite:** HTTP/UI se certifican con evidencia de sus ejecutores..
- **ID:** TEST-02. **Estado:** PASS_SQL. **Evidencia y resultado:** GOLD02 costo4400, margen1800, inventario6600. gold-final-all.log (referencia del repositorio: `../../../db/tests/gold-final-all.log`). **Límite:** -.
- **ID:** TEST-03. **Estado:** PASS_SQL. **Evidencia y resultado:** GOLD03 reparto exacto0.34/0.33/0.33; NUMERIC real. gold-final-all.log (referencia del repositorio: `../../../db/tests/gold-final-all.log`). **Límite:** No sustituye prueba de unidad Java del transporte decimal..
- **ID:** TEST-04. **Estado:** PASS_SQL. **Evidencia y resultado:** GOLD04A deuda1000/pago600/aplicaciones400+200 y replayexacto; GOLD04B deuda1000/pago1200/aplicado1000/anticipo200. gold-final-all.log (referencia del repositorio: `../../../db/tests/gold-final-all.log`). **Límite:** -.
- **ID:** TEST-05. **Estado:** PASS_SQL. **Evidencia y resultado:** Entrega cierra primer ciclo; recompra crea ciclo2; costo histórico1200 conservado. gold-final-all.log (referencia del repositorio: `../../../db/tests/gold-final-all.log`). **Límite:** -.
- **ID:** TEST-06. **Estado:** PASS_SQL. **Evidencia y resultado:** GOLD06 tardío4600/6900/1600; bonificación4520/6780/1680; deuda inicial inmutable. gold-final-all.log (referencia del repositorio: `../../../db/tests/gold-final-all.log`). **Límite:** -.
- **ID:** TEST-07. **Estado:** PASS_SQL. **Evidencia y resultado:** GOLD07 ajuste atómico: reverso1000/reaplicación900/abono100; disponible100, reembolso100→0. gold-final-all.log (referencia del repositorio: `../../../db/tests/gold-final-all.log`). **Límite:** -.
- **ID:** TEST-08. **Estado:** PASS_SQL. **Evidencia y resultado:** GOLD08 baja1100 y costo tardío50 producen pérdida1150/inventario0. gold-final-all.log (referencia del repositorio: `../../../db/tests/gold-final-all.log`). **Límite:** -.
- **ID:** TEST-09. **Estado:** PASS_SQL. **Evidencia y resultado:** Ambos órdenes: carrera cierre primero y coste primero forzado con barrera/espera Lock observada. Inventario200→207/

cierre207 con documental2027-02-01/reconocimiento2026-10-11; versión anterior intacta trasreabrir. academic-result.json (referencia del repositorio: `../../../db/tests/academic-result.json`), academic.log (referencia del repositorio: `../../../db/tests/academic.log`), closure-order-result.json (referencia del repositorio: `../../../db/tests/closure-order-result.json`), closure-order.log (referencia del repositorio: `../../../db/tests/closure-order.log`). **Límite:** -.

- **ID:** TEST-10. **Estado:** PASS_SQL. **Evidencia y resultado:** Dos conexiones/barrera: una reserva/venta gana; otra23505; sin segunda liquidación. academic-result.json (referencia del repositorio: `../../../db/tests/academic-result.json`), academic.log (referencia del repositorio: `../../../db/tests/academic.log`). **Límite:** Mapeo HTTP409 pertenece a API..
- **ID:** TEST-11. **Estado:** PASS_SQL. **Evidencia y resultado:** Dos conexiones/barrera compiten por pago600: sólo aplica400, otraBV020, disponible200. academic-result.json (referencia del repositorio: `../../../db/tests/academic-result.json`), academic.log (referencia del repositorio: `../../../db/tests/academic.log`). **Límite:** -.
- **ID:** TEST-12. **Estado:** PASS_SQL. **Evidencia y resultado:** Misma clave/hash replayexacto; payload distintoBV405 y ETag/version obsoletaBV412. Cancelar/anular/acordar/reabrir: stale412sinidem/audit; replayexacto trascommit antesguardversionactual. gold-final-all.log (referencia del repositorio: `../../../db/tests/gold-final-all.log`), version-guards-result.json (referencia del repositorio: `../../../db/tests/version-guards-result.json`). **Límite:** SQL no certifica por sí solo HTTP409/412..
- **ID:** TEST-13. **Estado:** PASS_SQL. **Evidencia y resultado:** Dos conexiones/barrera, scopes de sedesA/B hacia escritor canónicoA: un identificador normalizado único; otro23505. negatives-result.json (referencia del repositorio: `../../../db/tests/negatives-result.json`), negatives.log (referencia del repositorio: `../../../db/tests/negatives.log`). **Límite:** -.
- **ID:** TEST-14. **Estado:** PASS_SQL. **Evidencia y resultado:** FK otra sede23503; empresa ajena42501; ningún hecho cruzado. negatives-result.json (referencia del repositorio: `../../../db/tests/negatives-result.json`), negatives.log (referencia del repositorio: `../../../db/tests/negatives.log`). **Límite:** -.
- **ID:** TEST-15. **Estado:** PASS_SQL_API_UI. **Evidencia y resultado:** Scope empresa ajena42501 y token de otro actor/sede404; control canónico. negatives-result.json (referencia del repositorio: `../../../db/tests/negatives-result.json`), negatives.log (referencia del repositorio: `../../../db/tests/negatives.log`), owner-capture-result.json (referencia del repositorio: `../../../db/tests/owner-capture-result.json`). **Límite:** API valida scope y AUDITOR403; UI crea auditor/membresía reales y comprueba rechazo. Backend (referencia del repositorio: `../../../backend/verification-result.json`), UI (referencia del repositorio: `ui/bovicierre-ui-final-run.json`)..
- **ID:** TEST-16. **Estado:** PASS_SQL. **Evidencia y resultado:** Rol app sinDML/DDL/SETsession_replication_role; intento desactivar trigger42501; PUBLIC EXECUTE revocado. DML real como rolbovi_auditor también42501. negatives-result.json (referencia del repositorio: `../../../db/tests/negatives-result.json`), negatives.log (referencia del repositorio: `../../../db/tests/negatives.log`), security.sql (referencia del repositorio: `../../../db/tests/security.sql`), security-final.log (referencia del repositorio: `../../../db/tests/security-final.log`). **Límite:** La prueba usa roles técnicos; políticas de auditor humano se verifican en API/UI..
- **ID:** TEST-17. **Estado:** PASS_SQL. **Evidencia y resultado:** Como rol dueño de funciones, UPDATE a liquidación/contrato aceptado/condición/pesaje aprobado produceBV016: protección independiente del ACL app. negatives-result.json (referencia del repositorio: `../../../db/tests/negatives-result.json`), negatives.log (referencia del repositorio: `../../../db/tests/negatives.log`). **Límite:** -.
- **ID:** TEST-18. **Estado:** PASS_SQL. **Evidencia y resultado:** Pesaje grupal no fabrica pesos individuales; reparto por peso ausenteBV018. gold-final-all.log (referencia del repositorio: `../../../db/tests/gold-final-all.log`). **Límite:** -.

- **ID:** TEST-19. **Estado:** PASS_SQL. **Evidencia y resultado:** DobleEXCEPT de composiciones distintas produceNO_COMPARABLE; variación comparable exacta enGOLD01. gold-final-all.log (referencia del repositorio: `../../../db/tests/gold-final-all.log`). **Límite:** -.
- **ID:** TEST-20. **Estado:** PASS_SQL. **Evidencia y resultado:** Dos conexiones/barrera, esperaLock observada, tx empezó antes de vencimiento y desbloqueó después:BV403, sin liquidación/idempotencia parcial. expiry-result.json (referencia del repositorio: `../../../db/tests/expiry-result.json`), expiry.log (referencia del repositorio: `../../../db/tests/expiry.log`). **Límite:** -.
- **ID:** TEST-21. **Estado:** PASS_INFRA_QA. **Evidencia y resultado:** Publicaciones bidireccionales reales; DISABLE/ENABLE de suscripción QA y convergencia; conteos audit/outbox sin duplicación. academic-result.json (referencia del repositorio: `../../../db/tests/academic-result.json`), academic.log (referencia del repositorio: `../../../db/tests/academic.log`). **Límite:** Partición se simuló en stream apply QA; no se interrumpió red física..
- **ID:** TEST-22. **Estado:** PASS_SQL_INFRA_HTTP. **Evidencia y resultado:** A source parado real: HTTP autenticado nuevas solicitudesA503/B503; B transacción autorizada antes de caída confirma hechos propios; A reanudado enfinally. outages-result.json (referencia del repositorio: `../../../db/tests/outages-result.json`), outages.log (referencia del repositorio: `../../../db/tests/outages.log`). **Límite:** SQL financiero de prueba usa namespace QA; solicitudesHTTPmain fueron consultas/auth. No promoción..
- **ID:** TEST-23. **Estado:** PASS_SQL. **Evidencia y resultado:** Conciliación explícita en ambos dueños:0 huérfanos y0 liquidaciones desbalanceadas sobreFK comerciales. academic-result.json (referencia del repositorio: `../../../db/tests/academic-result.json`), academic.log (referencia del repositorio: `../../../db/tests/academic.log`). **Límite:** -.
- **ID:** TEST-24. **Estado:** PASS_SQL_INFRA. **Evidencia y resultado:** A caída real: fallo entrega guarda intento/backoff y limpia lease; lease abandonada30s recuperada por worker nuevo; ACKantiguaBV402. ConsumeA dos veces/inbox1, acuseB; perdida1100 +costetardío1. academic-result.json (referencia del repositorio: `../../../db/tests/academic-result.json`), outages-result.json (referencia del repositorio: `../../../db/tests/outages-result.json`), outages.log (referencia del repositorio: `../../../db/tests/outages.log`). **Límite:** Worker abandonado simulado dejando lease vencer; no se mata un procesoJava. ACKperdido simulado omitiendo primer acuse..
- **ID:** TEST-25. **Estado:** PASS_SQL_INFRA_HTTP. **Evidencia y resultado:** FDW realA viaPgpool SELECT12000/UPDATE42501; B sin servidor proveedor. Proveedor source parado de vuelveHTTP503 y gastoB usa captura aceptada local sin red. Frontera12000.0050→Money 12000.01 aceptada, raw4/hash preservados. academic.log (referencia del repositorio: `../../../db/tests/academic.log`), provider.log (referencia del repositorio: `../../../db/tests/provider.log`), owner-capture-result.json (referencia del repositorio: `../../../db/tests/owner-capture-result.json`), provider-precision-result.json (referencia del repositorio: `../../../db/tests/provider-precision-result.json`), outages-result.json (referencia del repositorio: `../../../db/tests/outages-result.json`). **Límite:** FuenteQA modificada para fronteras y restauradafinally; stop real afecta sólo container propio proveedor..
- **ID:** TEST-26. **Estado:** PASS_RECUPERACION_HTTP. **Evidencia y resultado:** Coordinador: restauración final V13 aislada28,616s, datos/hash/archivos/dedup/inbox,0sesiones; compra1000/replay/lookup/descarga HTTP. Registro (referencia del repositorio: `recovery/restore-v13.json`). **Límite:** Mismo host; no PITR ni backup fuera del host..
- **ID:** TEST-27. **Estado:** PASS_INFRA. **Evidencia y resultado:** Recreación real sólo servicio pg-a, IDnuevo, mismo volumen; firmas completas datos main+QA antes/después idénticas, incluyehistory/localcapturas/dedup/inbox/leases. persistence-result.json (referencia del repositorio: `../../../db/tests/persistence-result.json`), persistence.log (referencia del repositorio: `../../../db/tests/persistence.log`). **Límite:** Se recrea nodoA; se excluyen sesiones auth efímeras de firma. Tiempo1.744s, sin pérdida de datos..

- **ID:** TEST-28. **Estado:** PASS_MEDICION_LOCAL. **Evidencia y resultado:** 4EXPLAINANALYZEBUFFERS y100muestras/consulta tras5warmups: inventario/cartera/liquidación /expediente, dataset declarado. performance-result.json (referencia del repositorio: `../../../db/tests/performance-result.json`), performance.log (referencia del repositorio: `../../../db/tests/performance.log`). **Límite:** Dataset mayor:1901comandos/replay,800consultas canónicas mediantePgpool con1/4clientes; benchmark (referencia del repositorio: `../../../db/tests/benchmark-result.json`). Caché caliente, sinHTTP/carga escritora; no certificaciónSLO/capacidad..
- **ID:** TEST-29. **Estado:** PASS_UI_ACOTADO. **Evidencia y resultado:** Dieciséis E2E:15integrados más caída física de proveedor reales,13unit/build;390/768/1440 y teclado, Intel real. Registro (referencia del repositorio: `ui/bovicierre-ui-final-run.json`), capturas/alcance (referencia del repositorio: `ui/frontend-ui-tests.md`). **Límite:** No auditoría WCAG completa ni teléfono físico..
- **ID:** TEST-30. **Estado:** PASS_CATALOGO_VISUAL. **Evidencia y resultado:** Baseline3 comparado antes adopción; V13, metadatos42tablas83FK vsA/B sin diferencias; cinco tests de diagramas y PDF/SVG MER/relacional revisados. Adopción (referencia del repositorio: `../../../db/tests/flyway-adoption.json`), diagramas (referencia del repositorio: `../../diagrams/README.md`). **Límite:** No demuestra todas las RN; revisión semántica humana pendiente..
- **ID:** TEST-31. **Estado:** PASS_GRAFO_FUENTES_CAMBIADAS. **Evidencia y resultado:** AST real y53documentos canónicos reinterpretados; rutas Pantalla8/API, RN029/comandoV13/TEST11 y tabla/FK/REL20 contrastadas. Registro (referencia del repositorio: `graph-final.json`). **Límite:** Históricos sin cambios conservan extracción anterior; aristas no sustituyen SQL ni revisión humana..
- **ID:** TEST-32. **Estado:** PASS_SQL. **Evidencia y resultado:** Snapshot de conteo/liquidaciones/obligaciones/conceptos/asientos/inventario/costo sin cambio tras registrar/validar/aplicar pago, documento y entrega. gold-final-all.log (referencia del repositorio: `../../../db/tests/gold-final-all.log`). **Límite:** Documento SQL usa metadatos sintéticos; descarga/generaciónPDF real se certifica aparte en API/UI..
- **ID:** TEST-33. **Estado:** PASS_SQL. **Evidencia y resultado:** Muerte centralA converge aB; recompra del animal muertoBV011 sin liquidación. academic-result.json (referencia del repositorio: `../../../db/tests/academic-result.json`), academic.log (referencia del repositorio: `../../../db/tests/academic.log`). **Límite:** -.
- **ID:** TEST-34. **Estado:** PASS_SQL. **Evidencia y resultado:** Documento v2 no altera v1/hash ni revisión anterior; revisiones vinculadas a versión exacta. gold-final-all.log (referencia del repositorio: `../../../db/tests/gold-final-all.log`). **Límite:** SQL metadata only; bytes privados/versionado archivo probados por API/UI..
- **ID:** TEST-35. **Estado:** PASS_SQL. **Evidencia y resultado:** Anulación antes entrega con tardío restaura1200 a inventario; costo venta neto0; ciclo histórico conservado. gold-final-all.log (referencia del repositorio: `../../../db/tests/gold-final-all.log`). **Límite:** -.

Ejecuciones y alcance

GOLD último:157 llamadas a comandos en PostgreSQL18 real/NUMERIC; rollback completo. Carrera coste-primero:7 comandos/0.107 s. Guardas versiones:22 comandos/0.140 s. Outages:9 comandos/52.177 s; fuentesA/B/proveedor reanudadas enfinally. RecreaciónPG A:1.744 s y firmasmain/QA iguales. Estas cifras no expresan cobertura niSLO.

TEST15/26/29/30 incorporan evidencia definitiva de coordinador/backend/frontend; TEST31 integra los53 documentos canónicos cambiados, conservando extracciones históricas sin cambios. TEST28 incorpora cuatro planes y800 consultas canónicas con uno y cuatro clientes en QA; caché caliente, sin HTTP ni escrituras concurrentes y sin capacidad certificada. La restauración final V13 fue ejecutada por el coordinador y concilió datos y archivos; origen reanudado antes del ensayo.

Fallos/recreación se ejecutaron en ventana coordinada sin otros escritores, verificando labels ex actasbovicierre-academic. El harness no administra contenedores de otros proyectos. Capturas y metadatos documentales SQL son sintéticos; bytes/documentos reales se acreditan con API/UI.

Capítulo 21

Recorrido visual de la aplicación

Las capturas siguientes proceden de las pruebas de la aplicación integrada con datos sintéticos persistidos. Son evidencia del corte anterior, no maquetas ni una visita nueva a una empresa. La explicación detallada del frontend identifica sus campos, eventos y llamadas.

21.1 Bandeja de operaciones

Permite localizar operaciones y pendientes. El estado de una venta se distingue del saldo y del estado físico.

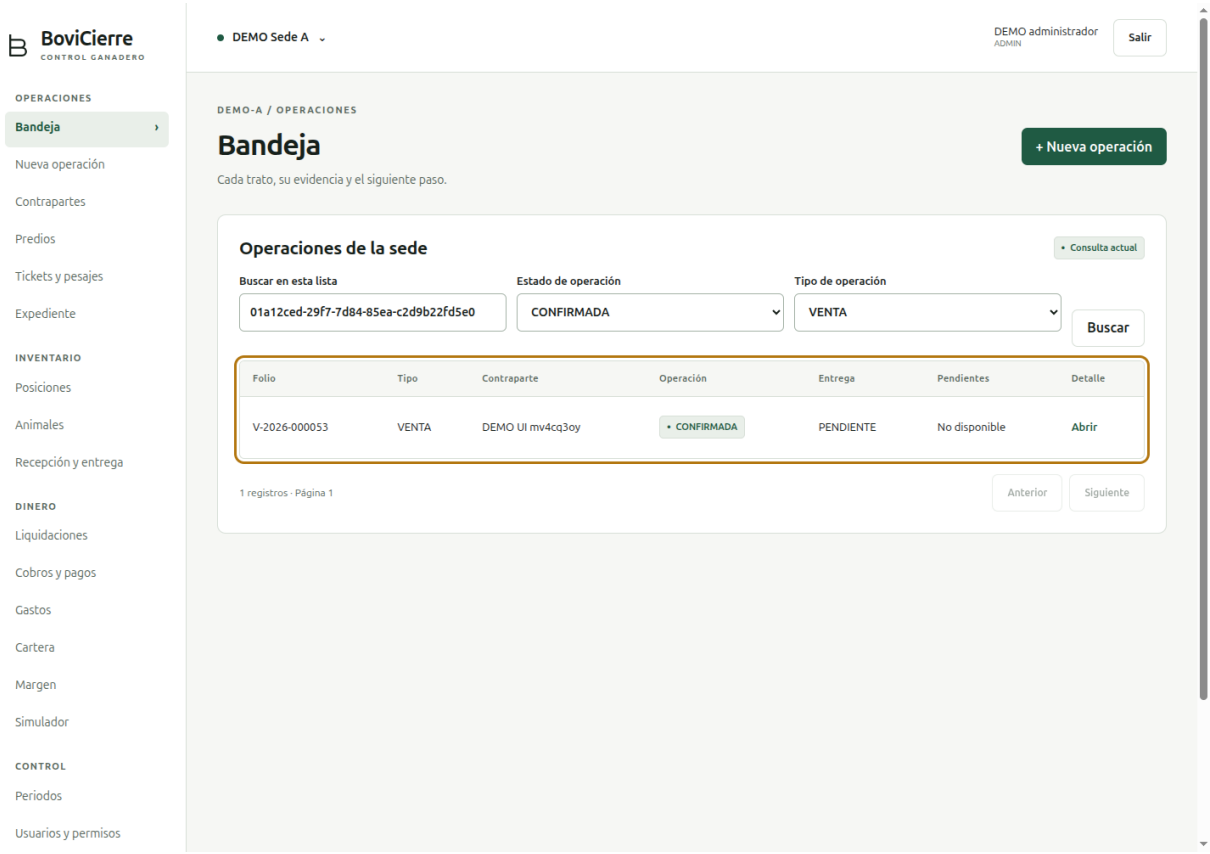


Figura 21.1: Bandeja de operaciones con datos DEMO.

21.2 Recorrido de operación

Los datos provienen de backend y PostgreSQL. Las condiciones por lote y sus selecciones deben revisarse antes de confirmar.

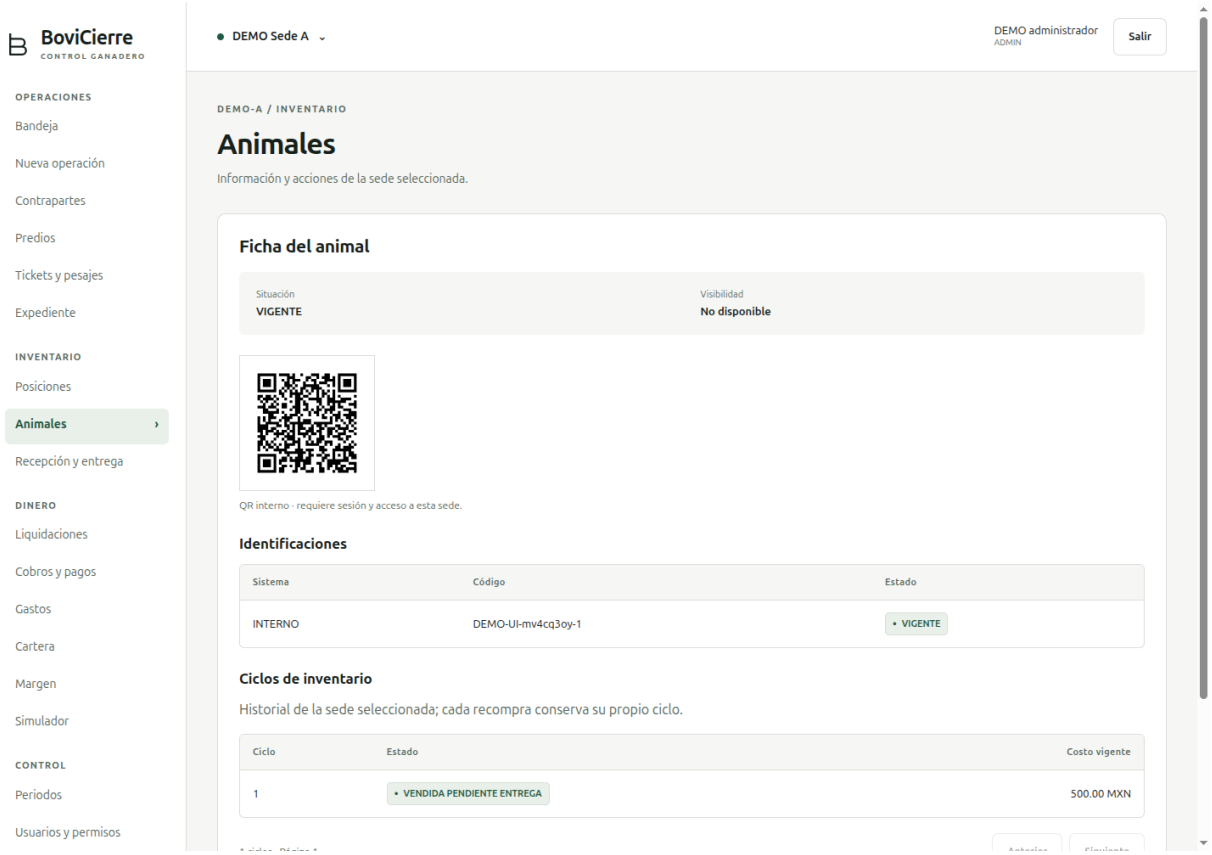


Figura 21.2: Recorrido de operación con datos DEMO.

21.3 Corrección de pesajes

La corrección conserva el conjunto exacto y exige motivo. Los pesos individuales desconocidos permanecen ausentes.

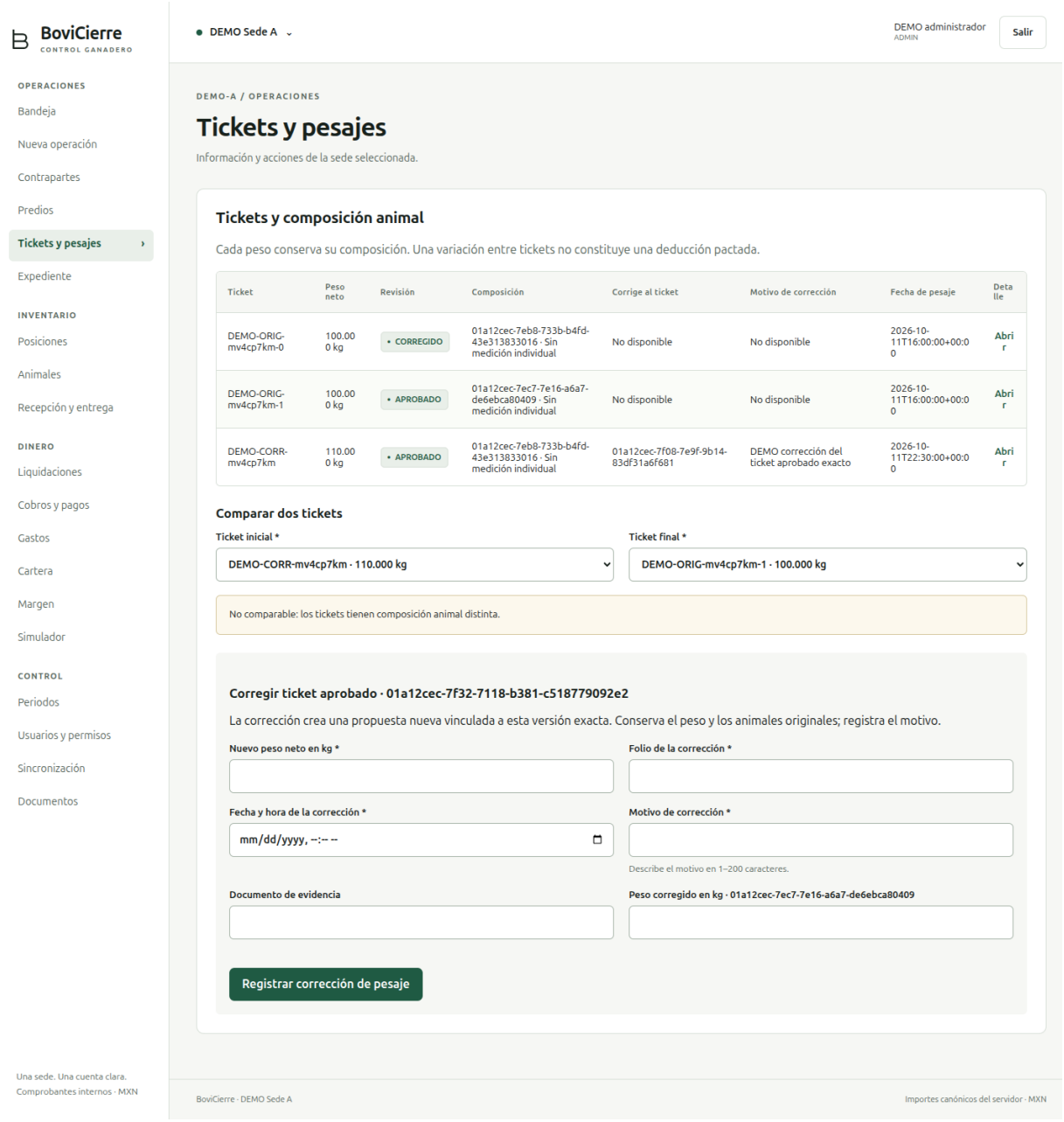


Figura 21.3: Corrección de pesajes con datos DEMO.

21.4 Cartera

La consulta informa corte y fuente. Saldo, vencimiento y disponible de pago tienen significados distintos.

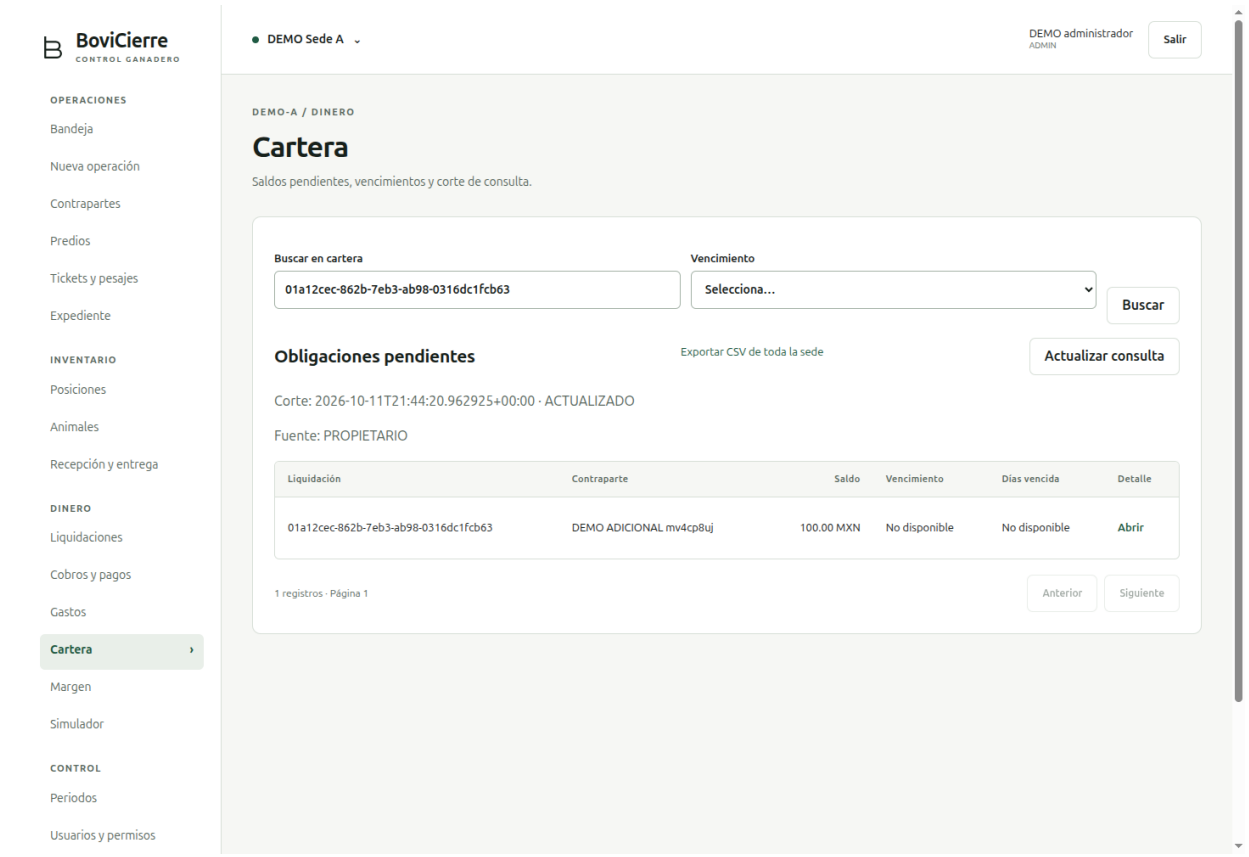


Figura 21.4: Cartera con datos DEMO.

21.5 Margen

El resultado contiene venta, costo, gastos y pérdida con procedencia. Un cierre histórico se distingue del resultado actual.

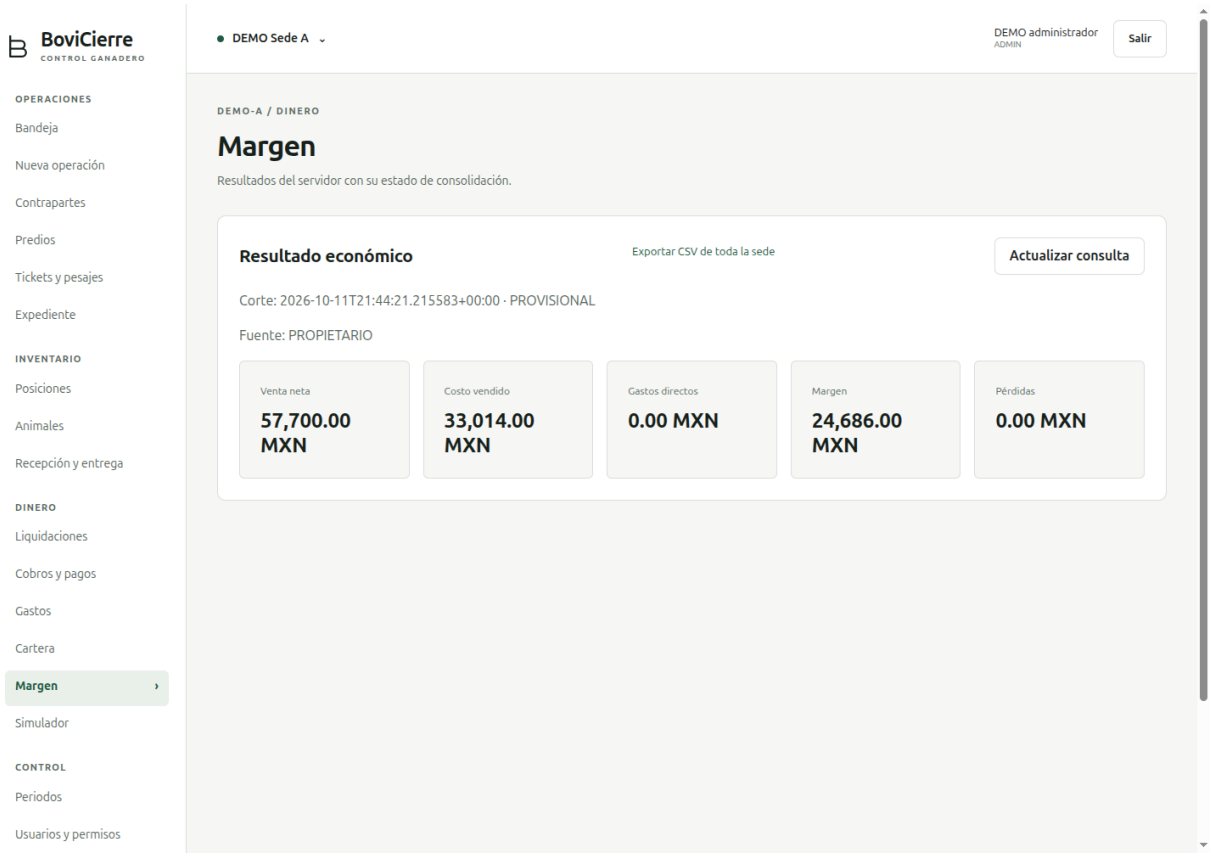


Figura 21.5: Margen con datos DEMO.

21.6 Periodos y cierres

El reconocimiento utiliza un periodo abierto explícito. Las versiones de cierre conservan el resultado histórico.

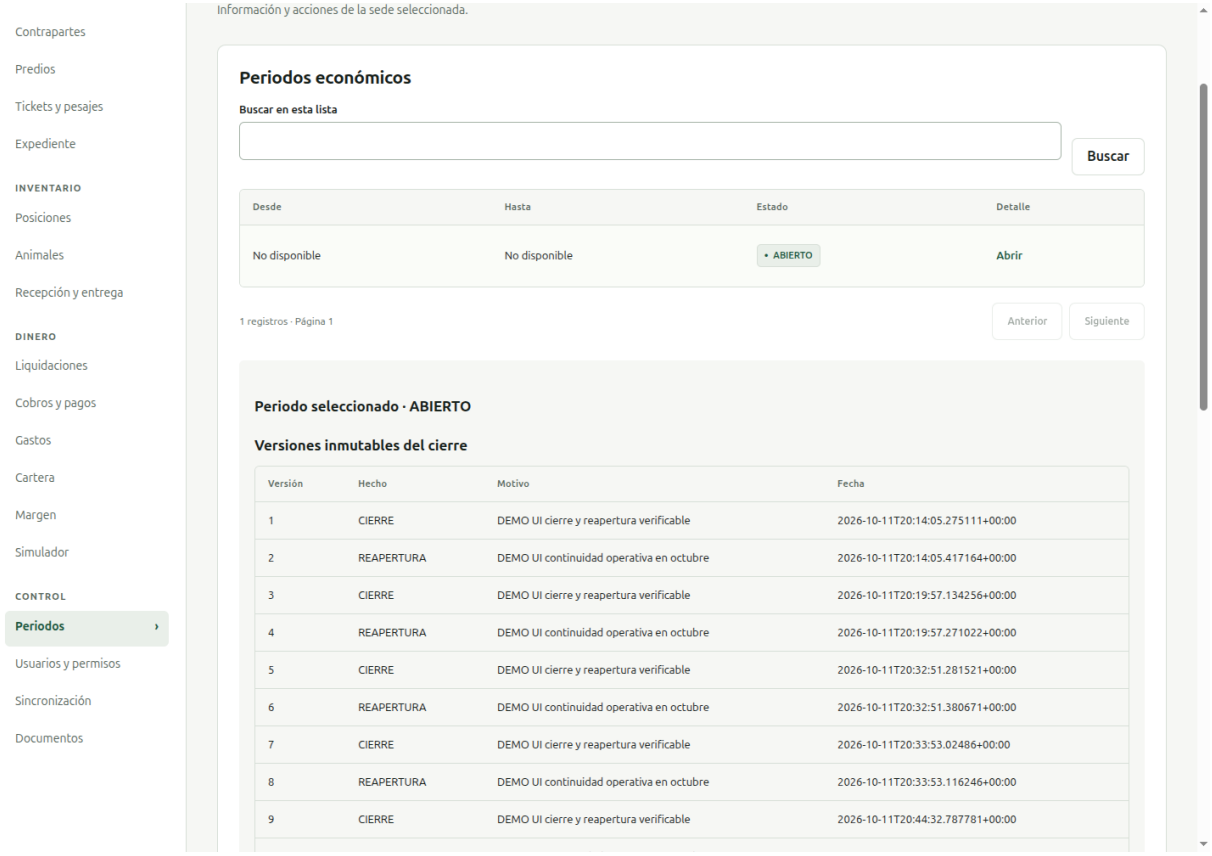


Figura 21.6: Periodos y cierres con datos DEMO.

21.7 Operación a390 px con teclado

El flujo de compra, venta y aplicación se comprobó con teclado. La captura no certifica lector de pantalla ni dispositivo físico.

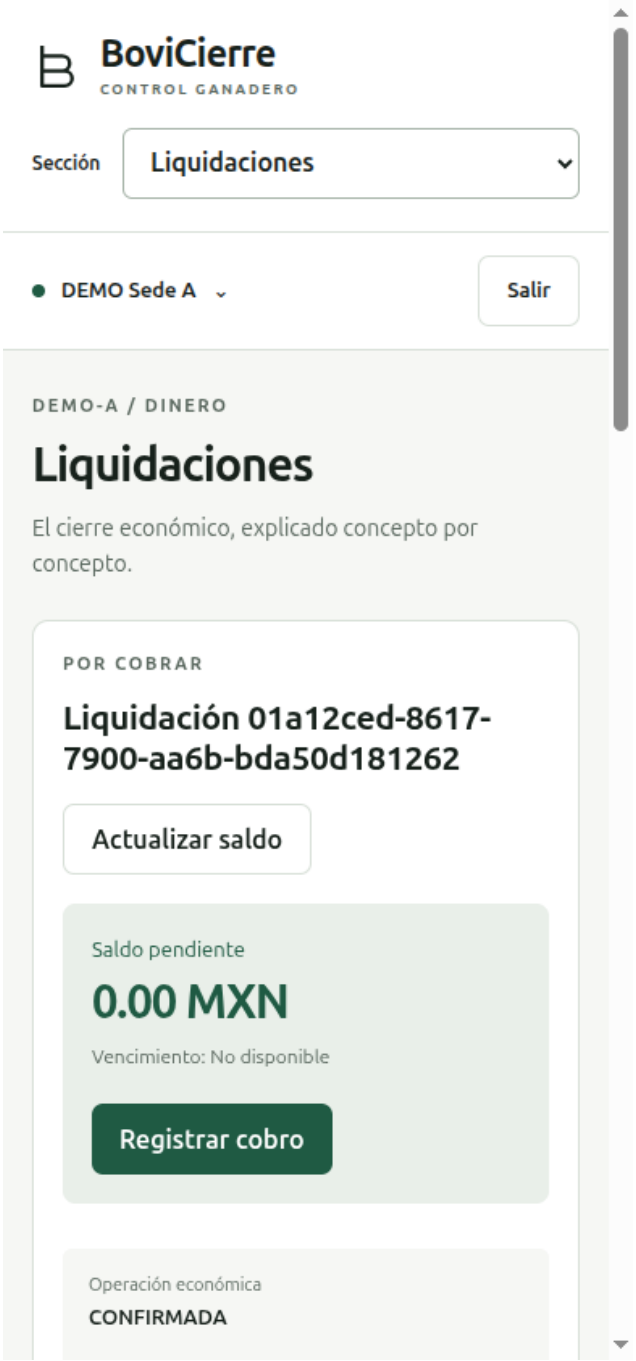


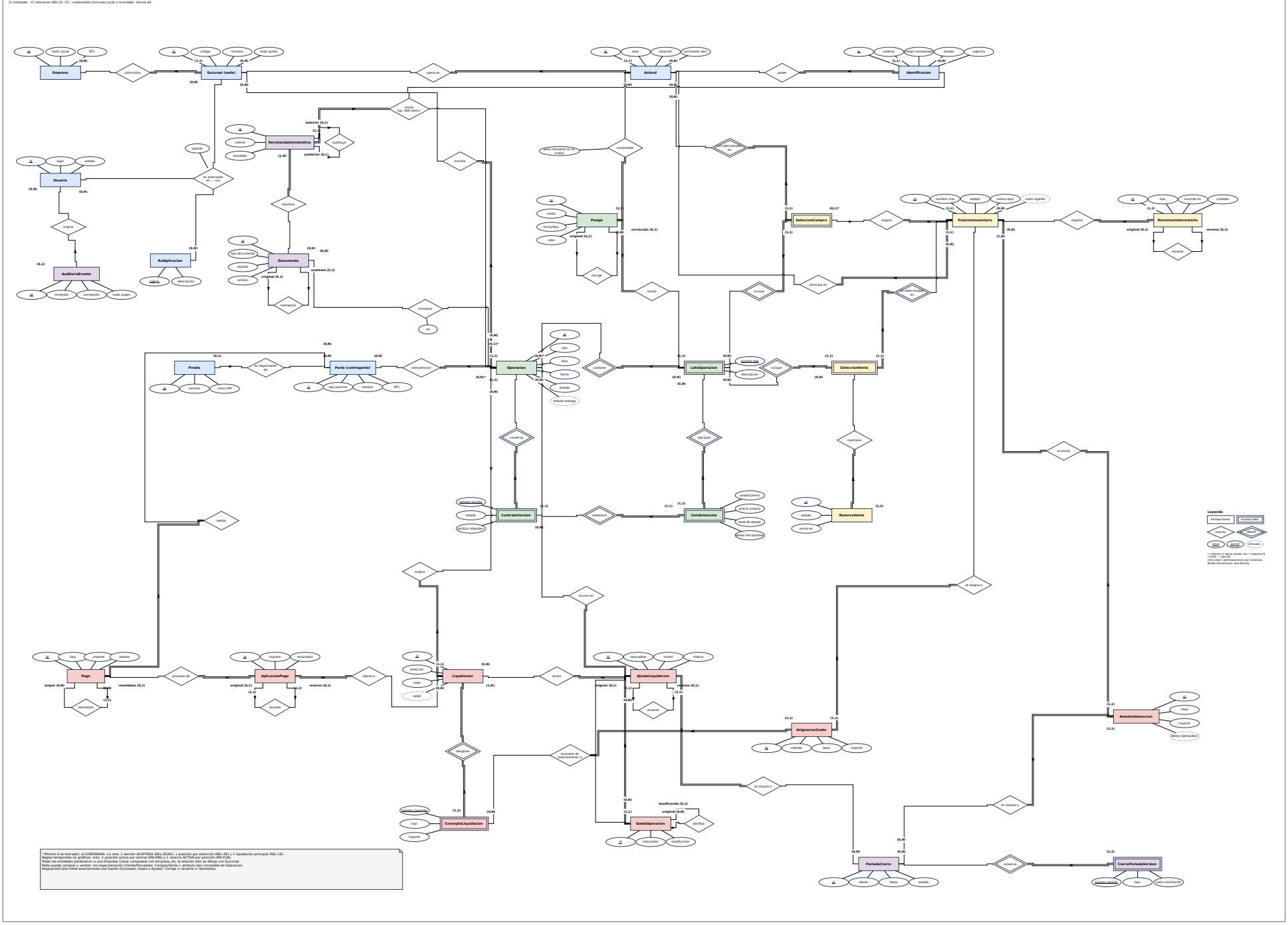
Figura 21.7: Operación a390px con teclado con datos DEMO.

Capítulo 22

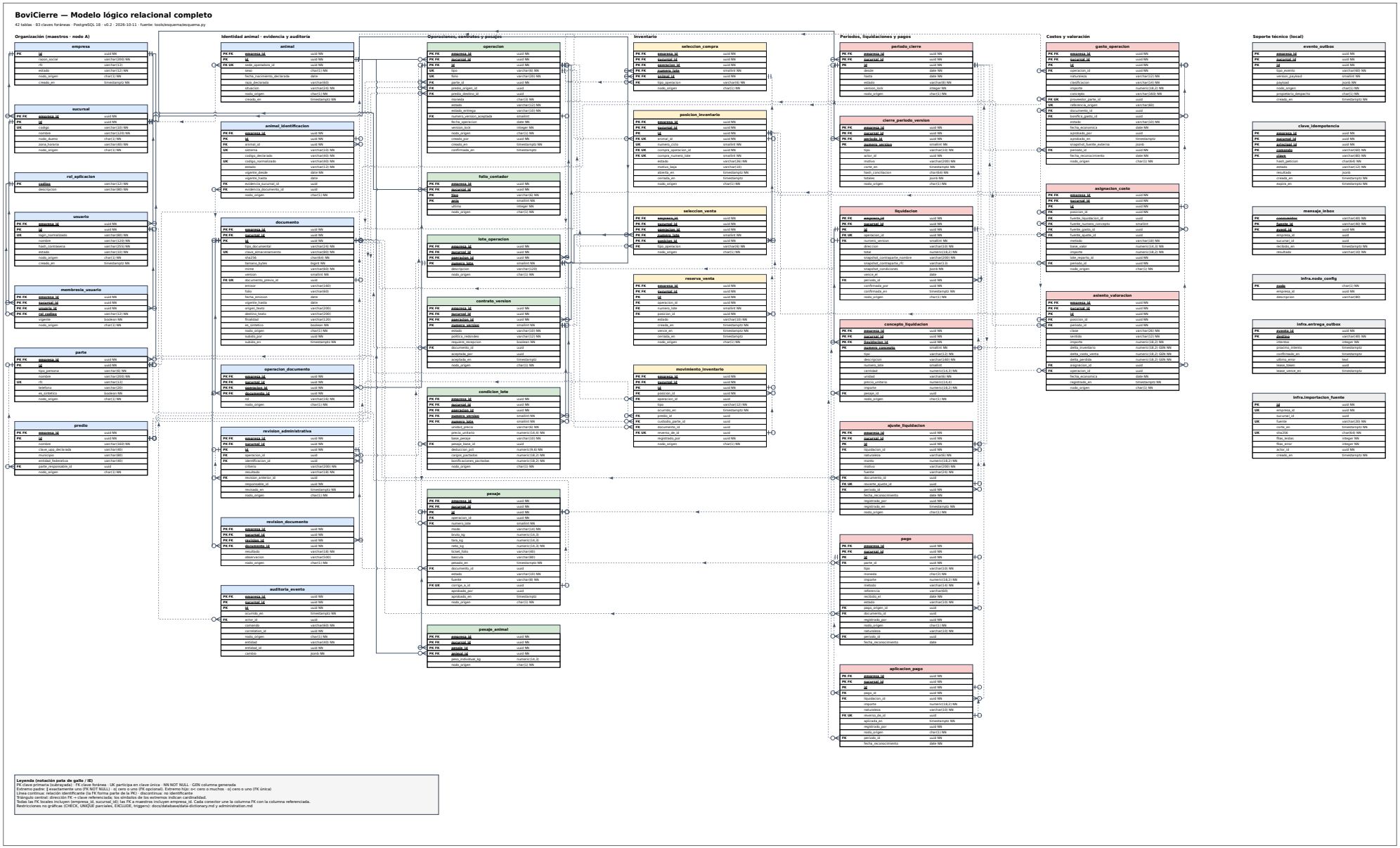
Láminas de arquitectura de datos

Las láminas vectoriales siguientes reúnen el MER Chen y el modelo relacional completo, seguidos de sus vistas por dominio. El dibujo completo permite comprender el conjunto; las vistas permiten leer las relaciones y atributos. Puede ampliarse el PDF sin convertir el texto del diagrama en una captura borrosa. La matriz y el diccionario anexos proporcionan la referencia textual. Revisión humana de cardinalidades y aceptación del profesor siguen pendientes.

BoviCierre — MER conceptual completo
31 entidades · 37 relaciones (REL-01..37) · cardinalidad (min,max) junto a la entidad · lámina A0



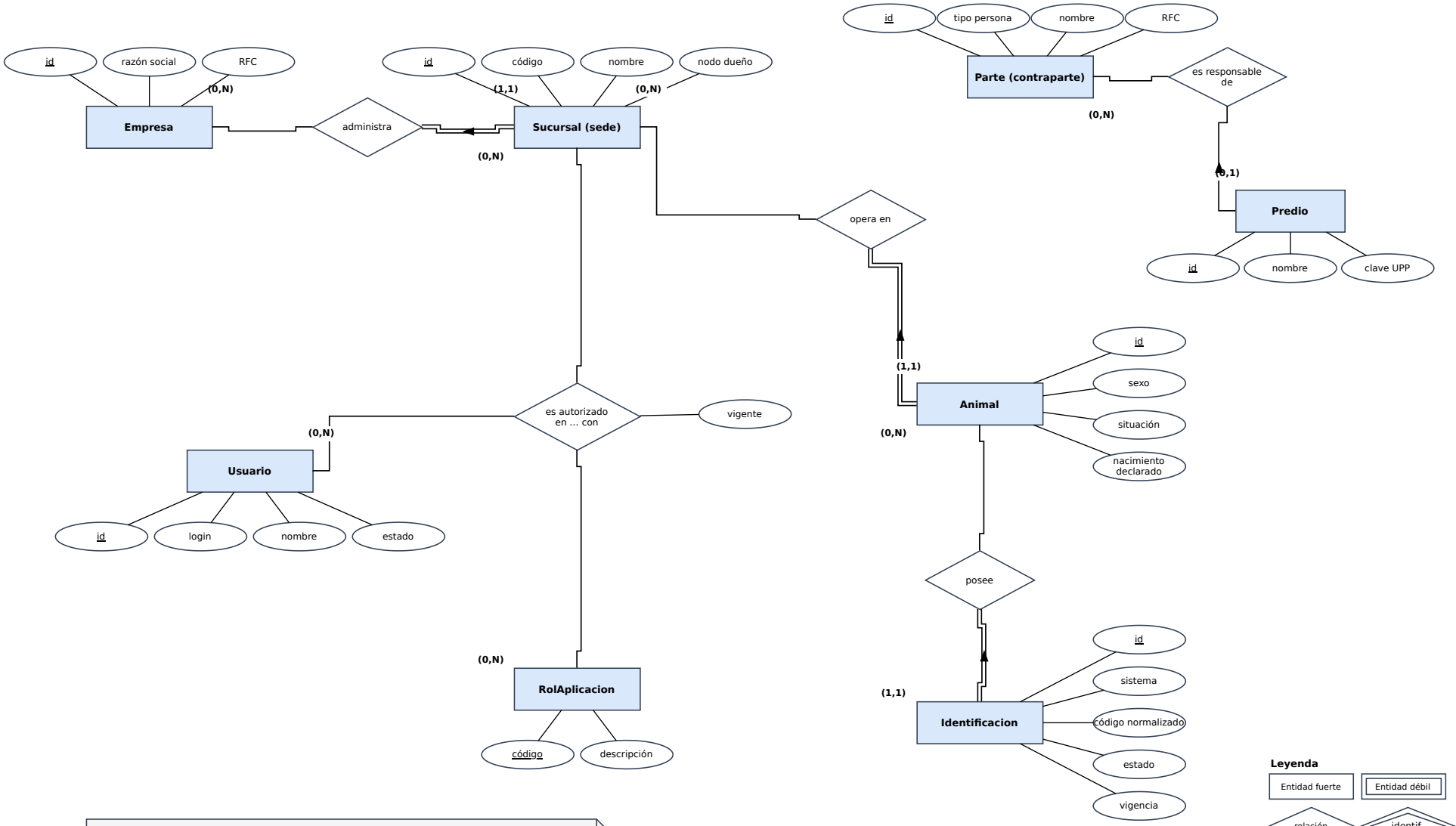
* Mínimo 0 en borrador; al CONFIRMAR: ≥1 lote, 1 versión ACEPTADA (REL-05/06), 1 posición por selección (REL-09) y 1 liquidación principal (REL-14).
Reglas temporales no gráficas: máx. 1 posición activa por animal (RN-056) y 1 reserva ACTIVA por posición (RN-018).
Todas las entidades pertenecen a una Empresa (clave compuesta con empresa_id); la relación sólo se dibuja con Sucursal.
Parte puede comprar y vender: sin especialización Cliente/Proveedor. Compra/Venta = atributo tipo inmutable de Operacion.
AsignacionCosto tiene exactamente una fuente (Concepto, Gasto o Ajuste). Corrige ≠ revierte ≠ reembolsa.


BoviCierre — Modelo lógico relacional completo
42 tablas · 83 claves foráneas · PostgreSQL 18 · v0.2 · 2026-10-11 · fuente: tools/esquema/esquema.py
Organización (maestros · nodo A)
empresa
sucursal
rol_aplicacion
usuario
membresia_usuario
parte
predio
Identidad animal · evidencia y auditoría
animal
animal_identificacion
documento
operacion_documento
revision_administrativa
revision_documento
auditoria_evento
Operaciones, contratos y pesajes
operacion
folio_contador
lote_operacion
contrato_version
condicion_lote
pesaje
pesaje_animal
Inventario
seleccion_compra
posicion_inventario
seleccion_venta
reserva_venta
movimiento_inventario
Períodos, liquidaciones y pagos
periodo_cierre
cierre_periodo_version
liquidacion
concepto_liquidacion
ajuste_liquidacion
pago
aplicacion_pago
Costos y valoración
gasto_operacion
asignacion_costo
asiento_valoracion
Soporte técnico (local)
evento_outbox
clave_idempotencia
mensaje_inbox
infra.nodo_config
infra.entrega_outbox
infra.importacion_fuente
Leyenda (notación pata de gallo / IE)
PK clave primaria (subrayada) · FK clave foránea · UK participa en clave única · NN NOT NULL · GEN columna generada
Extremo padre: || exactamente uno (PK NOT NULL) · o| cero o uno (FK opcional). Extremo hijo: o< cero o muchos · |< uno o muchos
Línea continua: relación identificante (la FK forma parte de la PK) · discontinua: no identificante
Triángulo central: dirección FK → clave referenciada; los símbolos de los extremos indican cardinalidad.
Todas las FK locales incluyen (empresa_id, sucursal_id); las FK a maestros incluyen empresa_id. Cada conector une la columna FK con la columna referenciada.
Restricciones no gráficas (CHECK, UNIQUE parciales, EXCLUDE, triggers): docs/database/data-dictionary.md y administration.md

BoviCierre — Lámina 2 · Identidad, contrapartes y sedes

Maestros escritos sólo en el nodo A (RN-003, ADR-005) · REL-01, 04, 26, 27, 28

MER conceptual · notación Chen/EER (mín,máx) junto a la entidad
v0.2 · 2026-10-11 · Fuente: docs/database/

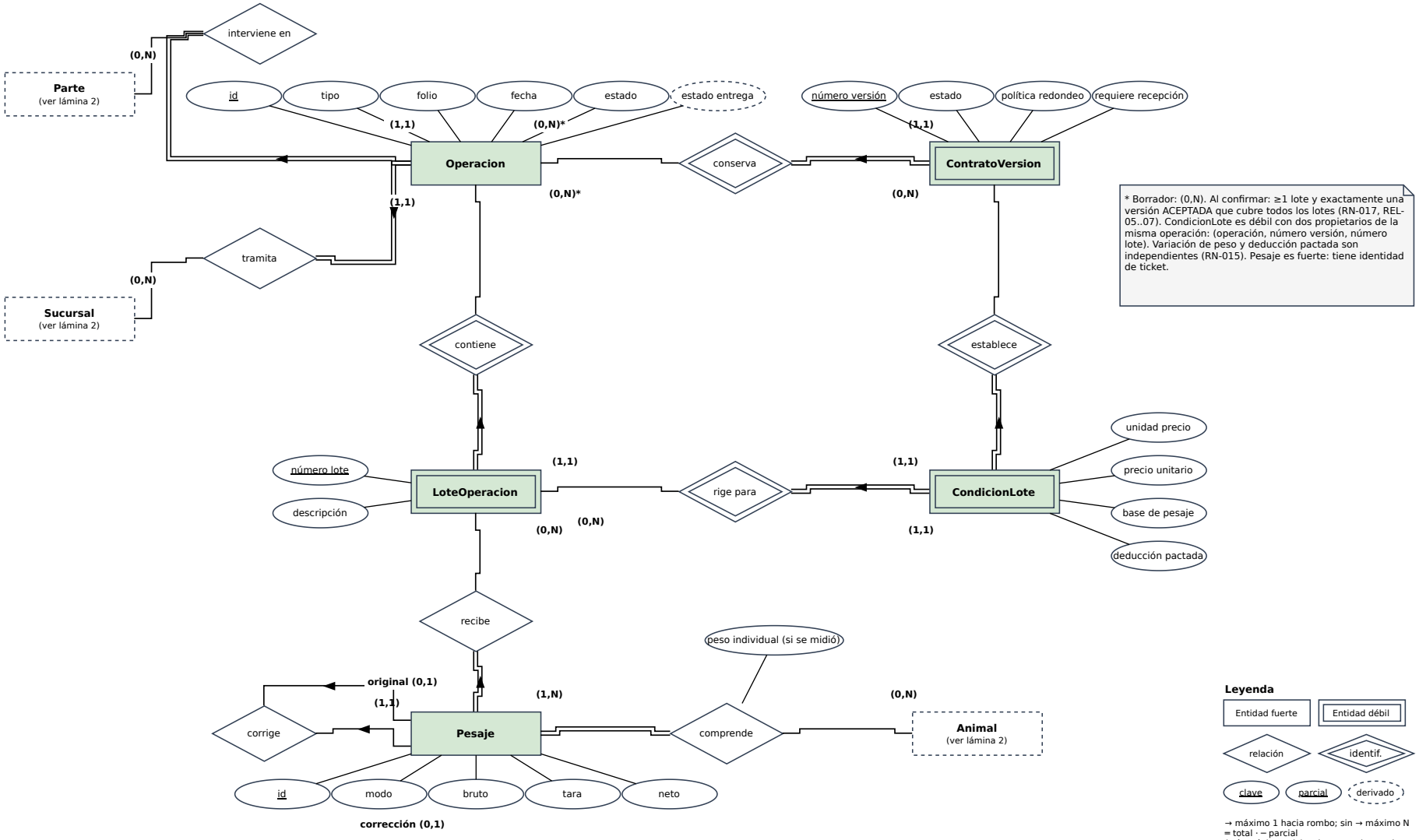


Todas las entidades de la lámina pertenecen a una Empresa (clave compuesta con empresa_id, RN-001); la relación se dibuja sólo para Sucursal. Parte puede comprar y vender: no hay especialización Cliente/Proveedor. Identificacion modela el código como entidad con historia (no atributo multivaluado). La sede operadora del animal es inmutable (RN-004).

BoviCierre — Lámina 3 · Operaciones, lotes, condiciones y pesajes

Escritas en el nodo dueño de la sede · REL-02, 03, 05, 06, 07, 14, 15, 31

MER conceptual · notación Chen/EER (mín,máx) junto a la entidad
v0.2 · 2026-10-11 · Fuente: docs/database/

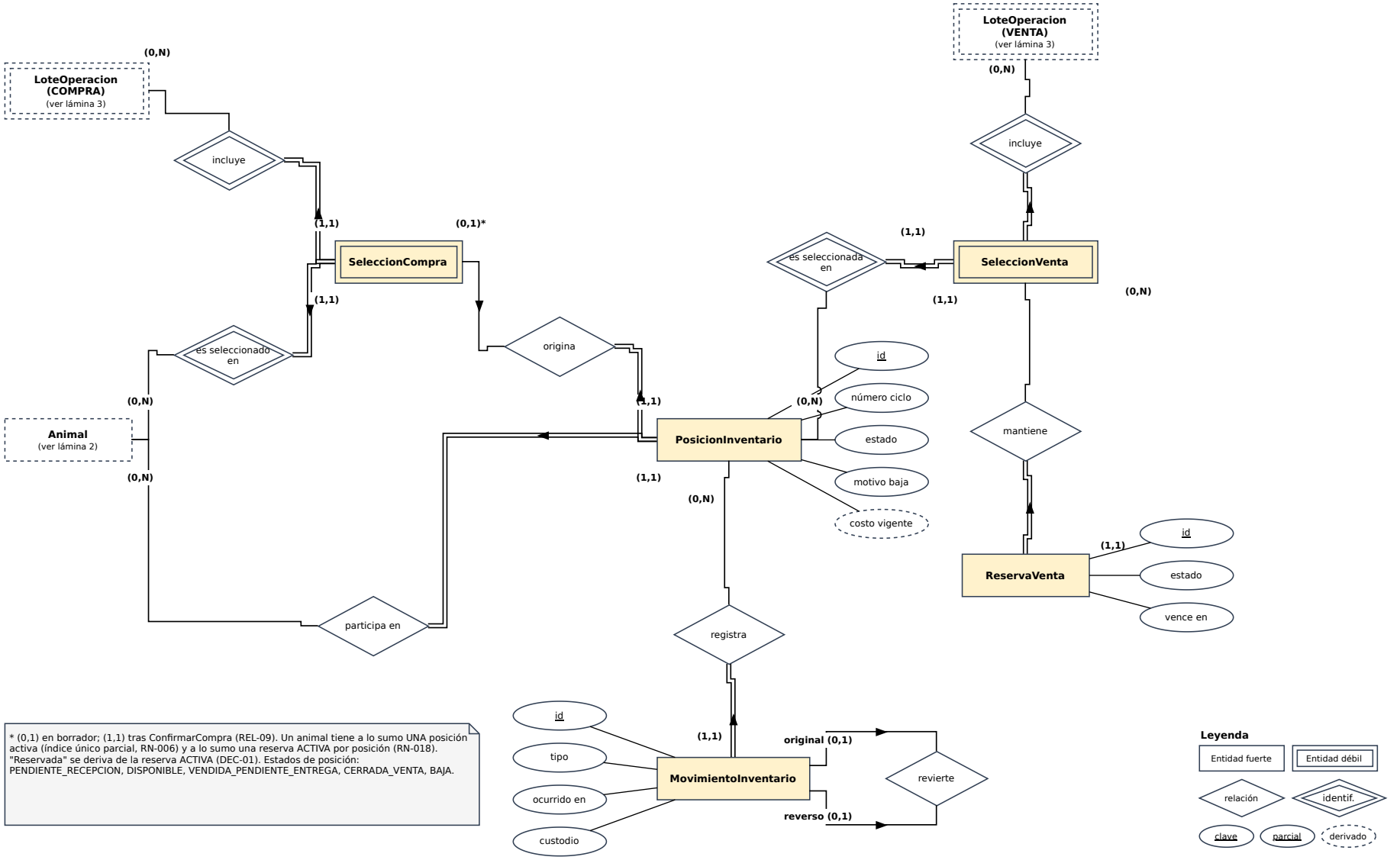


→ máximo 1 hacia rombo; sin → máximo N
═ total · ─ parcial
(mín,máx): participaciones por instancia.
Borde discontinuo: otra lámina.

BoviCierre — Lámina 4 · Inventario, selección de venta, reserva y movimientos

Posición = ciclo de inventario de un animal (ADR-003) · REL-08..13

MER conceptual · notación Chen/EER (mín,máx) junto a la entidad
v0.2 · 2026-10-11 · Fuente: docs/database/



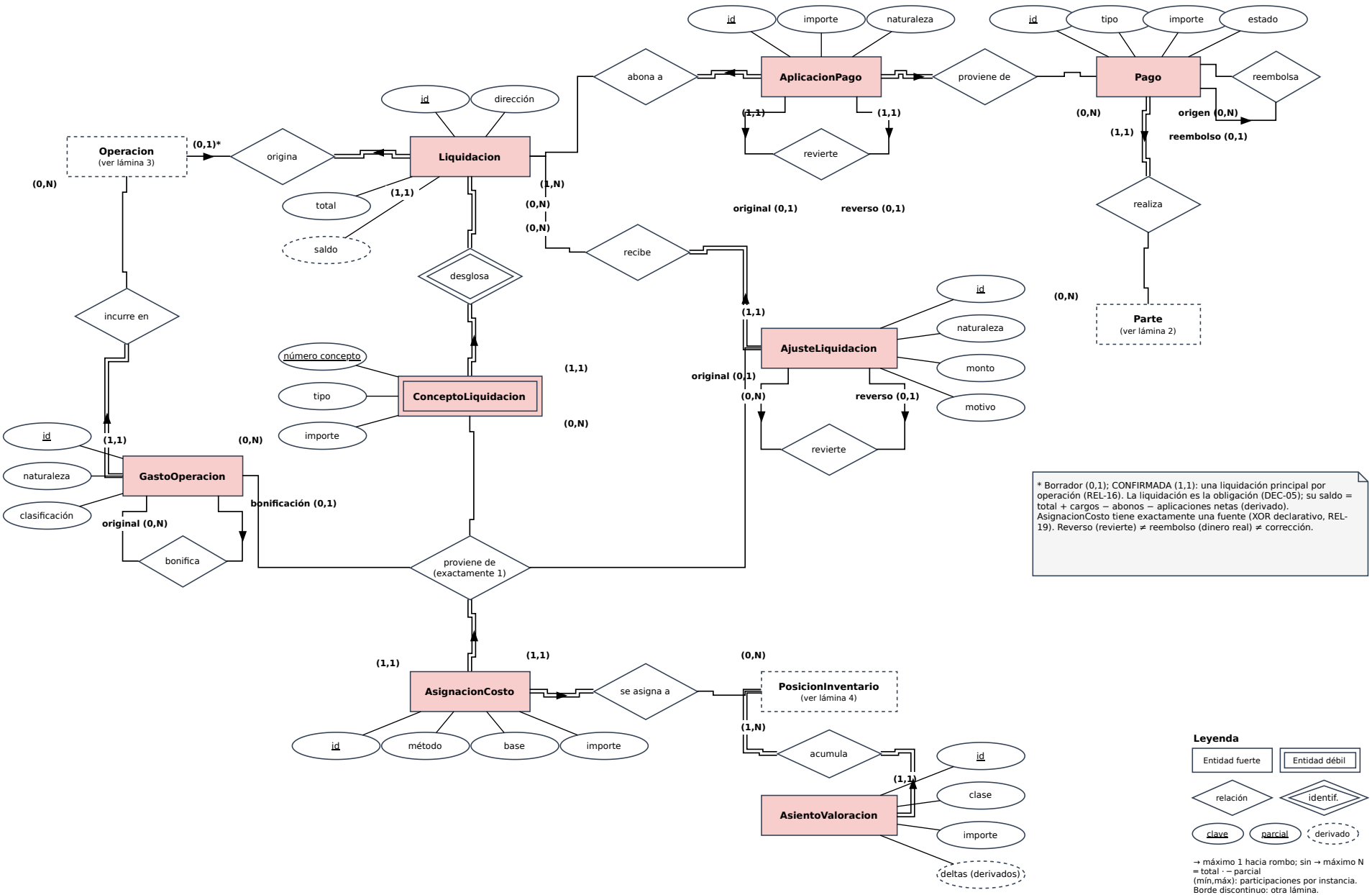
* (0,1) en borrador; (1,1) tras ConfirmarCompra (REL-09). Un animal tiene a lo sumo UNA posición activa (índice único parcial, RN-006) y a lo sumo una reserva ACTIVA por posición (RN-018). "Reservada" se deriva de la reserva ACTIVA (DEC-01). Estados de posición: PENDIENTE_RECEPCION, DISPONIBLE, VENDIDA_PENDIENTE_ENTREGA, CERRADA_VENTA, BAJA.

→ máximo 1 hacia rombo; sin → máximo N
= total · ~ parcial
(mín,máx): participaciones por instancia.
Borde discontinuo: otra lámina.

BoviCierre — Lámina 5 · Liquidaciones, costos, pagos y aplicaciones

Un único evento económico: CONFIRMADA (ADR-004) · REL-16..21, 25, 30, 32, 33

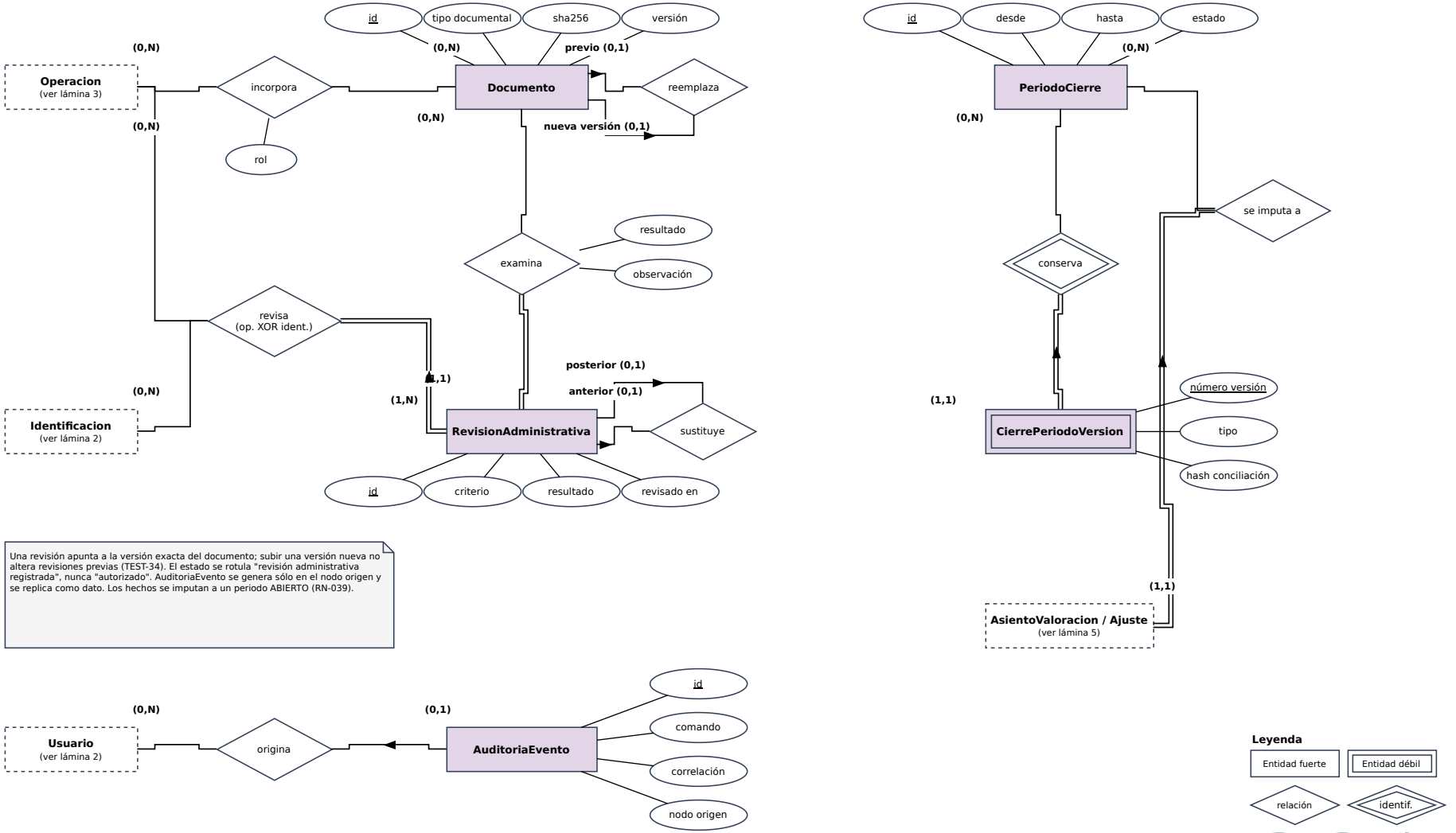
MER conceptual · notación Chen/EER (mín,máx) junto a la entidad
v0.2 · 2026-10-11 · Fuente: docs/database/



BoviCierre — Lámina 6 · Evidencia, revisiones, periodos y auditoría

Revisión administrativa ≠ autorización sanitaria (RN-037) · REL-22..24, 29, 35..37

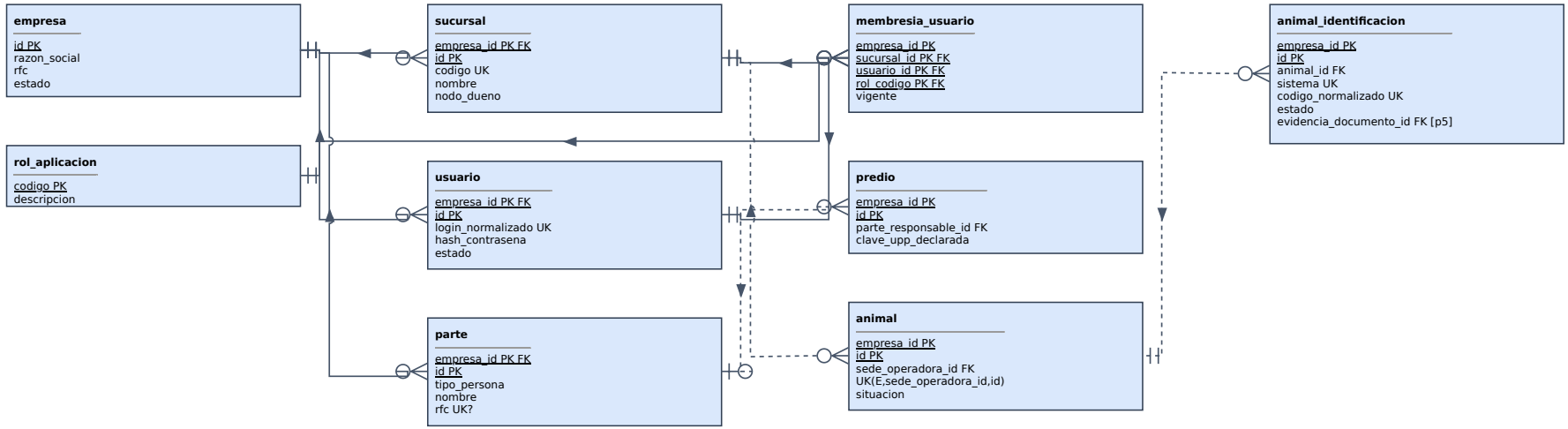
MER conceptual · notación Chen/EER (mín,máx) junto a la entidad
v0.2 · 2026-10-11 · Fuente: docs/database/



BoviCierre — Lámina 7 · Modelo lógico relacional · Organización e identidad (nodo A)

PK subrayada · FK → tabla · [pN] = FK hacia otra página · E,S = empresa_id, sucursal_id

Modelo lógico relacional (no Chen)
v0.2 · 2026-10-11 · Fuente: docs/database/

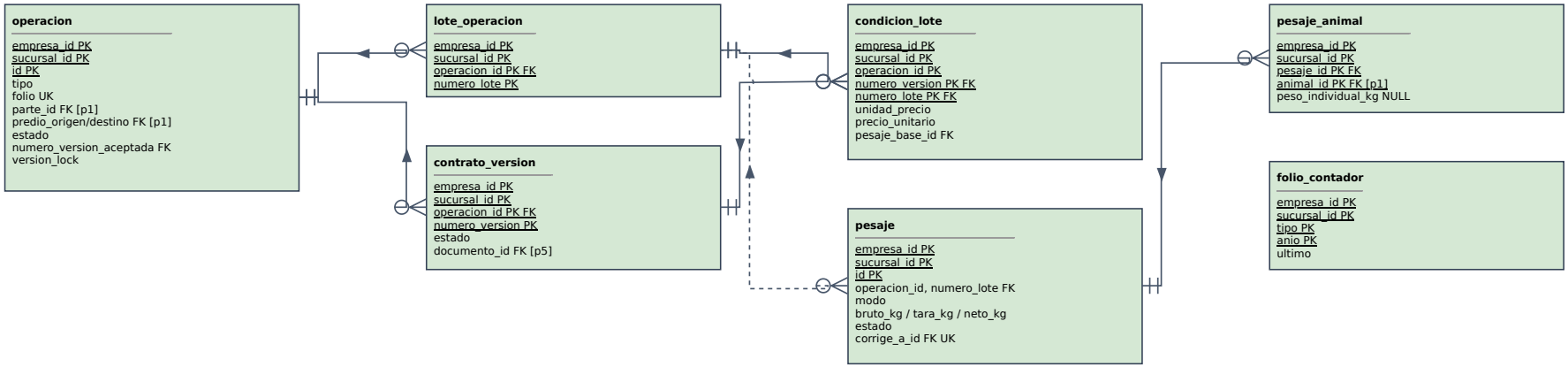


Triángulo central: FK → tabla referenciada. Extremos: ‖ uno obligatorio; o| opcional único; o< muchos. Todas las FK locales incluyen (empresa_id, sucursal_id); las FK a maestros incluyen empresa_id. Columnas completas, tipos y restricciones: docs/database/data-dictionary.md.

BoviCierre — Lámina 7 · Modelo lógico relacional · Operaciones, contratos y pesajes

PK subrayada · FK → tabla · [pN] = FK hacia otra página · E,S = empresa_id, sucursal_id

Modelo lógico relacional (no Chen)
v0.2 · 2026-10-11 · Fuente: docs/database/

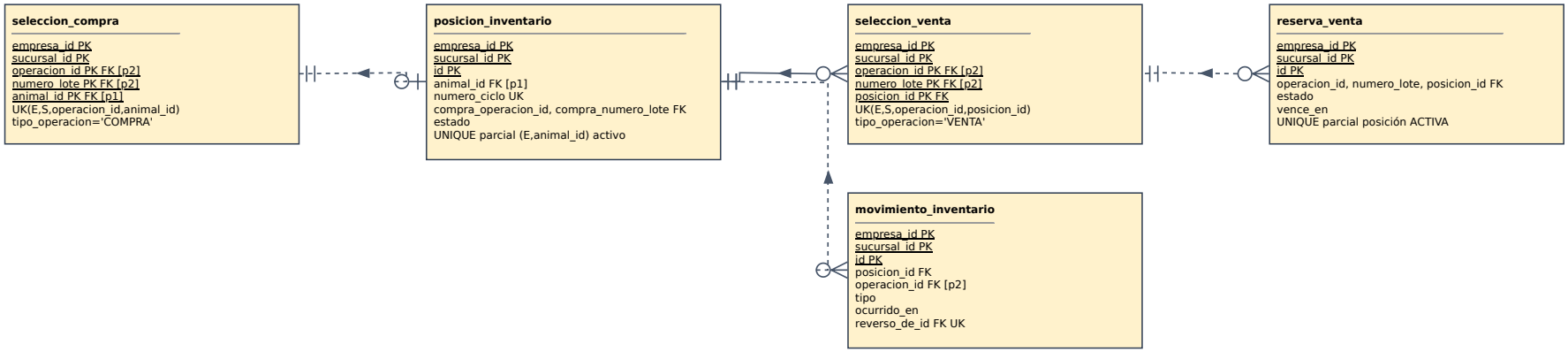


Triángulo central: FK → tabla referenciada. Extremos: ‖ uno obligatorio; o| opcional único; o< muchos. Todas las FK locales incluyen (empresa_id, sucursal_id); las FK a maestros incluyen empresa_id. Columnas completas, tipos y restricciones: docs/database/data-dictionary.md.

BoviCierre — Lámina 7 · Modelo lógico relacional · Inventario

PK subrayada · FK → tabla · [pN] = FK hacia otra página · E,S = empresa_id, sucursal_id

Modelo lógico relacional (no Chen)
v0.2 · 2026-10-11 · Fuente: docs/database/

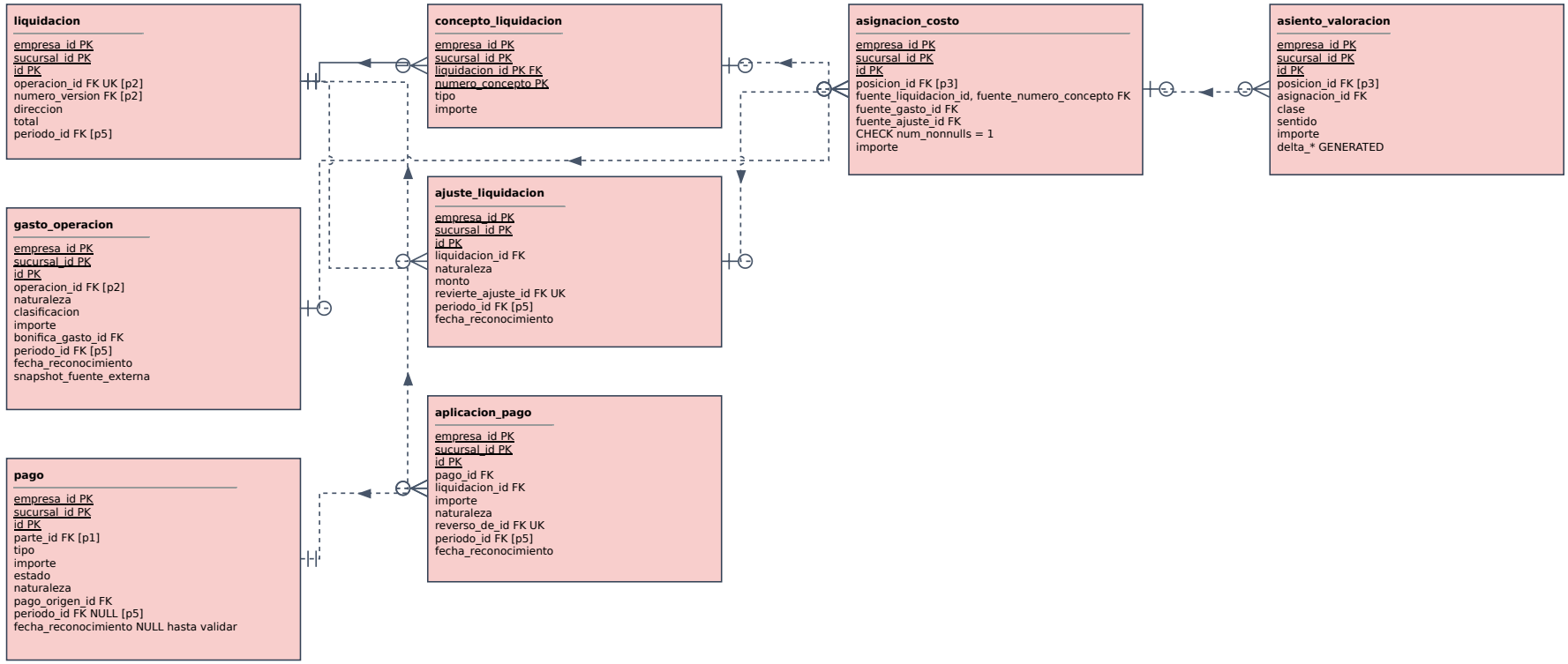


Triángulo central: FK → tabla referenciada. Extremos: ‖ uno obligatorio; o| opcional único; o< muchos. Todas las FK locales incluyen (empresa_id, sucursal_id); las FK a maestros incluyen empresa_id. Columnas completas, tipos y restricciones: docs/database/data-dictionary.md.

BoviCierre — Lámina 7 · Modelo lógico relacional · Liquidaciones, costos y pagos

Modelo lógico relacional (no Chen)
v0.2 · 2026-10-11 · Fuente: docs/database/

PK subrayada · FK → tabla · [pN] = FK hacia otra página · E,S = empresa_id, sucursal_id

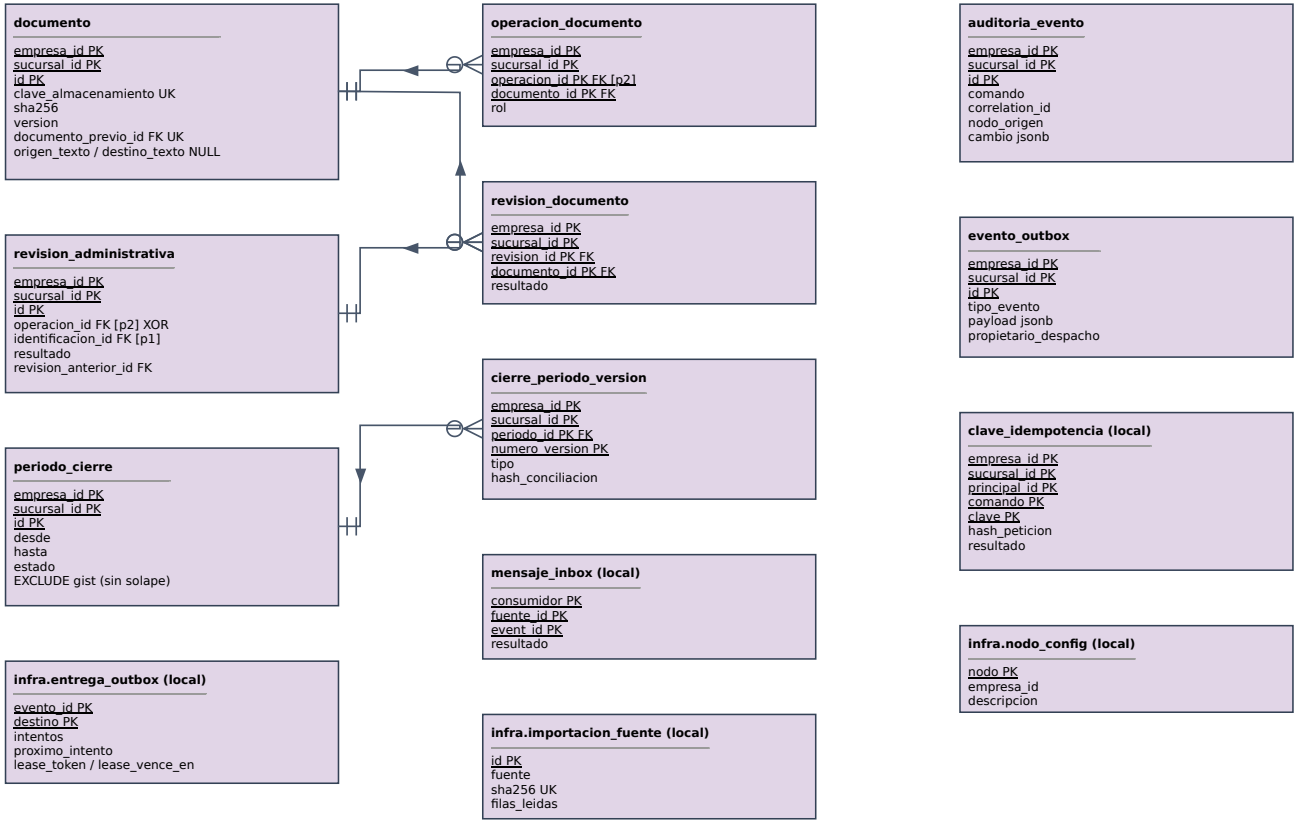


Triángulo central: FK → tabla referenciada. Extremos: ‖ uno obligatorio; o| opcional único; o< muchos. Todas las FK locales incluyen (empresa_id, sucursal_id); las FK a maestros incluyen empresa_id. Columnas completas, tipos y restricciones: docs/database/data-dictionary.md.

BoviCierre — Lámina 7 · Modelo lógico relacional · Evidencia, periodos, auditoría y soporte técnico

Modelo lógico relacional (no Chen)
v0.2 · 2026-10-11 · Fuente: docs/database/

PK subrayada · FK → tabla · [pN] = FK hacia otra página · E,S = empresa_id, sucursal_id



Triángulo central: FK → tabla referenciada. Extremos: ‖ uno obligatorio; o| opcional único; o< muchos. Todas las FK locales incluyen (empresa_id, sucursal_id); las FK a maestros incluyen empresa_id. Columnas completas, tipos y restricciones: docs/database/data-dictionary.md.

Capítulo 23

Fuentes y trazabilidad del manual

Revisión de código `92d3d4103a174eb11f4ccf21eff2deed8cc77682`. Los hashes identifican los archivos públicos utilizados, no incluyen directorios privados, contraseñas, cookies ni dumps. La documentación de capítulos aporta referencias adicionales de evidencia e infraestructura.

Archivo	SHA-256
backend/src/main/java/mx/bovicierre/BoviCierreApplication.java	d980c168ba1cdfcaf12a34ff85e5c3ac9fbac7d4dbcfb5a9ffb4bd3b334d2fd8
backend/src/main/java/mx/bovicierre/acceso/AccessService.java	0304ac667578d051ff55484e9972c438d99d1ce82ec77d0fb6a419adf0f021a8
backend/src/main/java/mx/bovicierre/acceso/AvailabilityFilter.java	5806a5edd039a20d529c07331985ccdd8b1cf14a4d911684f5db25262e08718c
backend/src/main/java/mx/bovicierre/acceso/SecurityConfiguration.java	10c96e1cae65befebfeacb9f3bc539464cfaec4fc93e4af6503c3dbee8b5ac22
backend/src/main/java/mx/bovicierre/acceso/SessionController.java	5754cfe69035116b05a3dc46abbd0ddfa8a8c5b69867c19e0342bb09fb8c64f7
backend/src/main/java/mx/bovicierre/acceso/UserController.java	c12a7a0cf0baaf2c1aa9ee0ded3bf45979109c8d2e6e8d08b163a84aacfd6896
backend/src/main/java/mx/bovicierre/evidencia/AnimalQrController.java	0b6a6e3e58b074f1f5a824fc9230322c00d11750e97f35889fb54c0cf2f9ff47
backend/src/main/java/mx/bovicierre/evidencia/CleanupMain.java	0397be2c5e180c65bebe8a040abd58b03619909d4d3f0068f7928bca62268772
backend/src/main/java/mx/bovicierre/evidencia/DocumentController.java	bb3ea4e99dd3cbd3ee1fb3cb0dc568d26f6929a27390f66e3f9a154cc29ce11d
backend/src/main/java/mx/bovicierre/evidencia/PrivateDocuments.java	1ba2a68c44b82f52befbd895646e6abd9dd30b62475f29455c0229854cbaf75f
backend/src/main/java/mx/bovicierre/evidencia/SettlementController.java	7c052b65f2a025c8939a7a1b4be095ebb21732b58993107425a0af94e41a31f5
backend/src/main/java/mx/bovicierre/evidencia/SettlementPdf.java	8dbbcb11d87bc00b9e74e50d3d99d8664d3af111be0066cf44463dd7f21ef698
backend/src/main/java/mx/bovicierre/evidencia/application/DocumentStorage.java	5ac197356bde15fc6ce659d6440e2a039a651c5cd9b3b4c09ba5448b1630a810
backend/src/main/java/mx/bovicierre/integraciones/DiagnosticsController.java	50f1358ceb808e0c8e4ef871196a5b4c9441052840efb734b09278ca08679b87
backend/src/main/java/mx/bovicierre/integraciones/OutboxWorker.java	342557eef2b4ee80ae5a8a4abe93e649aa905a7bf5fb5071a8df4bf0a144b5a6
backend/src/main/java/mx/bovicierre/integraciones/ProviderController.java	178136f71e4f0195f8d8a8fd84007f0bb9d306e7b33ec729b04dd91791aa39a1
backend/src/main/java/mx/bovicierre/operaciones/CoreController.java	b1dfb33002e08028c46057fcfe8cc2ef8103c6a0fe72c13f0a528f347ab5c261

Archivo	SHA-256
backend/src/main/java/mx/bovicierre/reportes/CsvController.java	3e47b279a3e96034c186d92ac8cb9943acbca898cce440a2662a0c5e49646195
backend/src/main/java/mx/bovicierre/shared/ApiErrors.java	412a052933aa19717d87c29d5339e4d40a8633e1998f211528aa5d2352f1a77b
backend/src/main/java/mx/bovicierre/shared/ApiException.java	9ec9372a5a0cbd0b76540fa371038a71afaba3705f3dae75e691867f2b53dfc4
backend/src/main/java/mx/bovicierre/shared/CommandService.java	1155245ac57caf95b6c85e014711ab91b3168c7b550ed9526f27b65fe3a2b171
backend/src/main/java/mx/bovicierre/shared/DatabaseConfiguration.java	a3836302471740162a1274873080ad1e93b2c36db20ac14b00065898fdbc174f
backend/src/main/java/mx/bovicierre/shared/JsonSupport.java	2080fcd9843efadb72229c4fccc29ae5e8e0766f0b7b14ae87d130812f7ade20
backend/src/main/java/mx/bovicierre/shared/NodeHealth.java	84ffffa377961c6113844d45666b8ca66ad2ed870c987fdfe523dc2843f7f48d
backend/src/main/resources/application-production.yml	bfcc733378cf611c3ac96f728c53e7931fbddcfb0b8a8da91eb02271f9d92124
backend/src/main/resources/application.yml	4c4d625e70f97278baff22b1d560dae926aa14ce34488380ac23feb5c1da2987
db/bootstrap/bootstrap.py	c848592d82e067a1eb6462219ec194b4487870bbffb15576aea7914199a63fed
db/migrations/V10__filtro_posiciones_vendibles.sql	fa88f9c089a4cc52ae031aa99b55999315be2151031cef23f202531bf3b2b1ab
db/migrations/V11__cancelacion_anulacion_version.sql	85cb7f45ac83d6df429fcaa5f8362294dcceb632b94378cb662a1dd536aeedff
db/migrations/V12__pesajes_reportes_canonicos.sql	ba0af087d801fafaf088f7d38012412cd4dd1b1a2cb513728b624341890ccdf9
db/migrations/V13__correccion_composicion_exacta.sql	1c58fc42da30d1cdea67b339f8ece408764fa7f0103eda20d9792eb2b30d743c
db/migrations/V1__esquema.sql	8412272f53a2aaeb0453c4acfd8af9d36a6fe359ee9fb663da05a674cf637e5e
db/migrations/V2__comandos.sql	7022f6430a72603d6a1781407705631ad884ef7430f8b6f2db961461d597412c
db/migrations/V3__consultas_sesiones_grants.sql	e26bc752bccd1dea5a22ba7776ef3d5869ab85856f46dac0412192c52a97ae3b
db/migrations/V4__capturas_proveedor_dto.sql	32f075db76d99245c8bb48e373c4e300ced46ad473bd3f1fad2d76c1a3d3f378
db/migrations/V5__corte_por_periodo_reconocimiento.sql	7e2c049246ba3ecb0f2e1c1b5cfec621d79cddbd86d6d4763e884f71bfdfcbfc
db/migrations/V6__tarifa_money_paginacion.sql	9fe7e7e27c008a0ffeb585ea32a317344f60656105bc496cc23f0ab619329759
db/migrations/V7__filtros_consultas.sql	7f6f3226a72d23ea7964783f717267524a3bf998037fe7b625d457730b502479
db/migrations/V8__borrador_kg_y_expiracion.sql	cb9d7b36eead9672bcf60c3b9a341442a959e0fdea7be1ed856443cee90f5367
db/migrations/V9__lector_a_captura_owner_fecha.sql	a7dc89b0656e14dc62c28f6653dad4954005b7750cd6b37954483adea208bb1d
db/seed/demo.py	d1c5aac0d7f173e67e2ae856565d45f13669ff13f6d54477ab8b2cc7cdb7d3b9
db/tests/benchmark.py	567196ae28024ca38a8516260cfcd306b73b1b42a8cb92aeba2bcec6e741d02b
docs/api/openapi.yaml	d635fb00d126833d3bd9a6f243ab2ec21a59488f304ce3bac1a9cdc412fddad7
docs/architecture/adr/ADR-001-monolito-modular.md	4d72eeec9291a6073ae1a631fb88b42bed004218039e6e40033ecc93ecfdc6c3

Archivo	SHA-256
docs/architecture/adr/ADR-002-sql-explicito-calculo-canonico.md	550f9d2167b1e62adc92a46a12b0a36d43498dc65cfadc26cfcf1d97b054ecf1
docs/architecture/adr/ADR-003-animal-y-posicion.md	2152cf4798f1142471f3fcb7bf059563ae2af341e6b1593844ada8f0fb96f202
docs/architecture/adr/ADR-004-reconocimiento-y-costo.md	fc7d95d6edb5423e9ebf71f54e0fdc74dca379c23fcd405f5e088a7de0a0326d
docs/architecture/adr/ADR-005-maestro-y-propiedad.md	ea5ba9327d9ec434d552644dea391509837bfd9cf303dcc97119e569e7dd5d71
docs/architecture/adr/ADR-006-replicacion-bidireccional.md	6a80728dd848a2a01838c1f013f3101e317725234733da840d5f80297d1fbab1
docs/architecture/adr/ADR-007-pgpool-raw.md	6728367b5a398bd9b200532d4d978d3d92c6c1e44451d7f37d9f5fe72b25714c
docs/architecture/adr/ADR-008-autenticacion-y-aislamiento.md	20804ffaba0c97de9d7492a53c911f1e186f40f1a2d9c66f7ab4dbae519cb370
docs/architecture/adr/ADR-009-documentos-y-revision.md	b06908e8e11d08c1da88a957171e152311120784b89e5fadc0ad3c8b0294ceb7
docs/architecture/adr/ADR-010-despliegue-respaldos.md	afc5ca3b528d63864d3631e45515dcd71b08d638616acc92269f1710f7fe51b3
docs/architecture/adr/ADR-011-graphify.md	540ff40c28a3301be41a4319ea3d6273166f63aa6c6f6023a305523984073545
docs/architecture/adr/ADR-012-alcance-aplazado.md	3ffbb4312018355ef5e26ecc336067461fa70417da59be4bd67a2f8a8663ef55
docs/architecture/adr/ADR-013-sesiones-disponibilidad.md	ca5ea5e8ad10f271cf797fe6c2eefb9ac51249314a19e8c4094d817c38148a2f
docs/architecture/adr/ADR-014-muerte-entre-duenos.md	018c9c1055690b7974ed0bd410f4e1258311316ce8659d75717e40868e861f41
docs/architecture/adr/ADR-015-periodos-cartera.md	5f804df10834dad175b406d4561fdd47bb34c48398a19a0244ffe17ba8f6bbb1
docs/architecture/adr/ADR-016-autoridad-esquema.md	d7e7585da3ad5dd1748d9196aea81876e35231a2ebbe29752c2afb596424d9c9
docs/architecture/adr/ADR-017-despliegue-local.md	a14fb7d1a048369cec2bed892417fdde5d817838ca3b6f0edf97f59a1b5fcede
docs/database/data-dictionary.md	4062189a8d17e4c04837507e045d317fc1ea6f4e18fcbb0ffd57bec2a6406202
docs/database/normalization.md	41e9b66e398822eb5196e69e75f2e768c6b3d91f9500a3ccd52e42370b763ee6
docs/domain/business-rules.md	bcda2f84af489ec68fd5c7712dffbf3c79433b7dc3a61dd87c68255dd3f2796a
docs/domain/glossary.md	32af005f9ba92301617325c1ffc68b2970c9f92e9bdd3e3d3921c47fc93bf7a0
docs/domain/use-cases.md	43f4a1a0f3f92dcc80cf480794a5c6565d04b34fb98926960f914f85f0b76f7f
frontend/src/App.tsx	f0325f17a2e5bc541be695a59c1afd7211f19be1de3738c7b550ba1b57225367
frontend/src/api.ts	33a8ed401e986157f05586bcb769360dd5da57046bc4eddbfe9e3ccb8234d787
frontend/src/domain.ts	7410af2f7a5db646f020ab5bd93f45d520b62eb1fde48f0a020e7ce26781b5d6
frontend/src/generated/api-schema.ts	24f01aa65cecf591cb8adcf83352d6ae859da28c1e465227b8305c83a933f97b
frontend/src/main.tsx	e34b50223ac942759bc00c81a3d886fa77fc39645c31a32bd14841cbaceb2300
frontend/src/screens.tsx	db38246a91b46b34c692cbbb256537a8c9e33306a8ef611bb5c9a1f927d92eae

Archivo	SHA-256
frontend/src/styles.css	d59390ef19345dedab9c2f9c517f728e47d26c3f0bd045863d2ca9e67231dd1a
frontend/src/ui.tsx	18619ae8fcc27bffd235ca8734be12ba92d22acdd44b1538a7823d311e1a1a0a
frontend/src/workflow.ts	20e030ed1ddf17df4604576c053adef3517fbfe14bc82f9210f6c614205403b7
scripts/academic.py	c88e9852db350f5476e070d08f88602168606ca69c7e39fc490e577c58839b4e
scripts/dev.py	3747b730bb044601a8be5f14e6253e7fe3a91d98288ea5366de02b9ed699020b
scripts/doctor.py	68bc5b701db50d83b4ac04dcf3f38cddfb6151f57bade4f92f3ac5ed52414f97
scripts/recovery.py	f965704aea82f2fb262054b769f8a5d42cfd1b74bdcc575c12ec856356d00900
scripts/sync-task-tables.py	efac9ff97c57233fb41b39fa70b544ff9af5cd59e403b574cfd55504a8ea29b0
tools/dev/check_api_contract.py	58a7df485501aaf23cf8ceaeba610addbea2c5602799d5f443f31e34d68053ef
tools/esquema/comparar_catalogo.py	2ce2511a2ef240a8e3bb8797799eb159f93ef85ba963b399fbd7e410d31e2698